

CONTROL METHODS FOR ROBOTIC APPLICATIONS

Lecture notes

Leonid B. Freidovich

(June 9, 2017)



UMEÅ UNIVERSITY

Contents

I	Introduction and Kinematics	7
1	Introduction and Preview	9
1.1	Disclosure and Acknowledgments	9
1.2	Industrial Robotic Manipulators	10
1.3	Terminology and Examples	10
1.4	Introduction of the Conceptual Problems	13
1.4.1	Forward Kinematics	15
1.4.2	Inverse Kinematics	17
1.4.3	Velocity Kinematics	18
1.4.4	Dynamics of Motions	20
1.4.5	Path Planning and Trajectory Generation	20
1.4.6	Motion Control	21
1.4.7	Force Feedback	21
1.4.8	Computer Vision and Vision Based Control	21
1.5	Conclusions	22
2	Kinematics	23
2.1	Rigid Motions & Homogeneous Transformations	23
2.1.1	Frames, Points, and Vectors	24
2.1.2	Rotations in 2D and 3D	27
2.1.3	Composition of Rotations	33
2.1.4	Parametrization of Rotation Matrices	40
2.1.5	Homogeneous Transformations	45
2.2	Forward Kinematics	47
2.2.1	Kinematic Chains	47
2.2.2	The Denavit-Hartenberg (DH) Convention	50
2.2.3	Examples of using DH convention	57
2.3	Inverse Kinematics	58
2.3.1	Problem Formulation	58
2.3.2	Kinematic Decoupling	59
3	Velocity Kinematics and Jacobian	65
3.1	Angular Velocity for Characterizing Rotations	65

3.1.1	Rotation around a Fixed Axis	65
3.1.2	Matrix Representation of Cross Products	66
3.1.3	Derivatives of Rotation Matrices	68
3.1.4	Linear and Angular Velocities of a Moving Frame	70
3.1.5	Linear Velocities under Rigid Motions	73
3.2	Concept of the Manipulator Jacobian	74
3.2.1	Velocities of Points in the End-effector Frame	74
3.2.2	Definition of the Jacobian	75
3.2.3	Computing $J_\omega(q)$ Using DH Frames	76
3.2.4	Computing $J_v(q)$ Using DH Frames	77
3.2.5	Analytic Jacobian	81
3.2.6	Singularities	82
3.3	Inverse Velocity Kinematics	83
3.3.1	Assigning Joint Velocities	83
3.3.2	Inverse Kinematics based on Jacobian	84
3.3.3	Manipulability	85
II	Path Planning and Dynamics	86
4	Path and Trajectory Planning	88
4.1	Space Description	88
4.1.1	Configuration Space	88
4.1.2	Workspace	89
4.2	Path Planning	91
4.2.1	Path Planning: Potential Field Approach	91
4.2.2	Path Planning: Probabilistic Road Map Method	92
4.3	Trajectory Planning	93
4.3.1	From path to trajectory	93
4.3.2	Trajectories for Point to Point Motions	94
5	Dynamics: Euler-Lagrange Equations	97
5.1	Example: from Newton's Law to Euler-Lagrange Equations	97
5.2	Example: Dynamics of a Single Link Robot	99
5.3	Holonomic Constraints and Virtual Work	102
5.3.1	Degrees of Freedom and Holonomic Constraints	102
5.3.2	Generalized Coordinates, Virtual Displacements	105
5.3.3	Principle of Virtual Work and Constraint Forces	108
5.4	D'Alembert Principle and Dynamics	110
5.4.1	D'Alembert Principle	110
5.4.2	Generalized Forces	111
5.4.3	Virtual Work of the Inertial Forces	113
5.4.4	Equations of Motion for Systems of Particles	114

5.5	Lagrangian for an n -Link Robot	116
5.5.1	Kinetic Energy for a Rigid Body	116
5.5.2	Kinetic Energy for an n -Link Robot	119
5.5.3	Potential Energy for an n -Link Robot	120
5.6	Equations for Dynamics an n -Link Robot	120
5.6.1	General form of equations of motion	120
5.6.2	Structure of the Equations of Motion	122
5.7	Examples for Equations of Motion	124
5.7.1	Two-Link Cartesian Manipulator	124
5.7.2	Planar Elbow Manipulator	126
5.7.3	Double Link Pendulum with Absolute Angles	129
5.7.4	Euler-Lagrange equations in excessive coordinates	132
5.7.5	Mechanism of a Closed Kinematic Chain	134
5.8	Properties of Equations of Motion	139
5.8.1	Passivity Relation	139
5.8.2	Bounds on Matrix of Inertia	141
5.8.3	Linearity of Dynamics in Parameters	142
III	Robotics Control	145
6	Multivariable Control of Manipulators	147
6.1	Dynamics with Geared DC-Motors	147
6.2	PD-Controller Design	149
6.2.1	The Case of Absent Gravity	149
6.2.2	PD-Controller with Gravity Compensation	150
6.3	Feedback Linearization	151
6.3.1	Feedback Linearization Under No Uncertainty	151
6.3.2	Robust and Adaptive Motion Control	153
6.3.3	Robust Control Based on Feedback Linearization	154
6.3.4	Adaptive Control Based on Feedback Linearization	161
6.4	Passivity Based Motion Control	163
6.4.1	Passivity Based Control Under No Uncertainty	163
6.4.2	Adaptive Passivity Based Motion Control	165
7	Independent Joint Control	167
7.1	Development of Control Systems for Robots	167
7.2	Conceptual Control Loop for a Single Joint	168
7.3	Actuator Dynamics	169
7.4	Dynamical Model for a Single Joint	170
7.5	PD and PID Tuning Based on a Simplified Linear Model	173
7.6	Simplified Feedforward Control Design	177
7.7	Dealing with Flexible Transmission	177

8	Geometric Nonlinear Control	182
8.1	Introduction to Differential Geometry	182
8.1.1	Smooth Embedded Manifold in $\mathbb{R}^n$	182
8.1.2	Smooth Vector and Co-vector Fields	184
8.1.3	Distributions and Co-distributions	185
8.1.4	Lie Bracket of Vector Fields	186
8.1.5	Lie Derivative	186
8.2	Frobenius Theorem	187
8.2.1	Manifolds as Solutions of PDEs	187
8.2.2	Tangent Space to a Solution of a PDE	188
8.2.3	Integral Manifolds	189
8.2.4	Lie Derivative Representation for PDEs	189
8.2.5	Integrability and Involutivity for Distributions	190
8.2.6	Frobenius Theorem: integrability $\Leftrightarrow$ involutivity	191
8.3	Feedback Linearization	191
8.3.1	Concept of Feedback Linearization	191
8.3.2	How to find $T(\cdot)$, $\alpha(\cdot)$ and $\beta(\cdot)$?	192
8.3.3	Feedback Linearization for Single Link Flexible Joint Robot	194
A	Elements of Force Control	203
A.1	Coordinate Frames and Constraints	204
A.2	Natural and Artificial Constraints	205
A.2.1	Inserting Task	205
A.2.2	Turning a Crank Task	206
A.3	Network Models and Impedance	207
A.3.1	Network Models	208
A.3.2	Mechanical Impedance	210
A.4	Task Space Dynamics and Impedance Control	211
A.4.1	Dynamics of a Rigid Manipulator in Contact with an Environment	211
A.4.2	Task Space Dynamics and Impedance Control	213
B	Elements of Computer Vision	215
B.1	Digital Image	215
B.1.1	Geometry of Image Formation	216
B.1.2	Perspective Projection	216
B.1.3	Coordinates on Image Plane vs Sensor Array	217
B.2	Camera Calibration	218
B.2.1	Extrinsic Camera Parameters	218
B.2.2	Intrinsic Camera Parameters	219
B.2.3	Steps in Camera Calibration	219
B.3	Image Processing: Segmentation	220

B.3.1	Segmentation of Image	221
B.3.2	Tools for Image Processing	221
B.4	Estimating Positions and Orientations	223
C	Elements of Vision Based Control	225
C.1	Position <i>vs</i> Imaged-Based Control	225
C.2	Camera Motion and Interaction Matrix	226
C.2.1	Camera Motion Description	226
C.2.2	Definition of the Interaction Matrix	226
C.2.3	Velocities of the Camera Frame	227
C.2.4	Constructing the Interaction Matrix	228
C.2.5	Properties of the Interaction Matrix	229
C.3	Reconstructing a Camera Motion	230
C.4	An Idea for Image-Based Control Design	231
Index		233

Part I

Introduction and
Kinematics

Chapter 1

Introduction and Preview

We start with a brief introduction and a preview of all the other chapters using a simple academic example.

1.1 Disclosure and Acknowledgments

Below we present mostly a compendium of an excellent comprehensive introduction to mathematical methods for (model-based) control of industrial robotic manipulators based on the following textbook:

M.W. Spong, S. Hutchinson, and M. Vidyasagar, **Robot Modeling and Control**, *John Wiley & Sons*, Inc. ISBN 0-471-64990-2.

To the best of our knowledge, nowadays, this book and

Introduction to robotics: Mechanics and Control by J.J. Craig, *Pearson Education International*, ISBN 0-33-123629-6

are the most commonly used for university engineering education in English on robot control.

It should also be acknowledged that for the preparation of this notes, we have used lecture slides prepared by Professor Anton S. Shiriaev for an advanced course on robot dynamics and control. Using these slides and the textbook by Spong et al., the course has been taught several times by A.S. Shiriaev, L.B. Freidovich, and U. Mettin at Umeå university, Sweden and at the Norwegian University of Science and Technology (NTNU), Norway. This notes have been also used by A. Robertsson at Lund University, Sweden. The current presentation has been shaped by feedback from the instructors and many students. Finally, we would like to acknowledge that many figures have been prepared by Oleg Borisov and Vladislav Gromov with some inspiration from the book by M.W. Spong et al. using Ipe by O. Cheong (<http://ipe.otfried.org/>).

The first version of these notes has been published by NRO ITMO and therefore this text should be cited as

Leonid B. Freidovich, *Control Methods for Robotic Applications: Lecture Notes*, Saint Petersburg National Research University of Information Technologies Mechanics and Optics, St. Petersburg, Russia, 2013.

1.2 Industrial Robotic Manipulators

Here, the notion **industrial robot** is understood, following ISO 8373, as

“an automatically controlled, reprogrammable, multipurpose manipulator programmable in three or more axes, which may be either fixed in place or mobile for use in industrial automation applications,”

where, according to the International Federation of Robotics (IFR),

- *Reprogrammable* means that its programmed executable motions or auxiliary functions may be changed without physical alterations;
- *Multipurpose* means capable of being adapted to a different application with physical alterations;
- *Axis* refer to direction used to specify the robot motion in a linear or rotary mode.

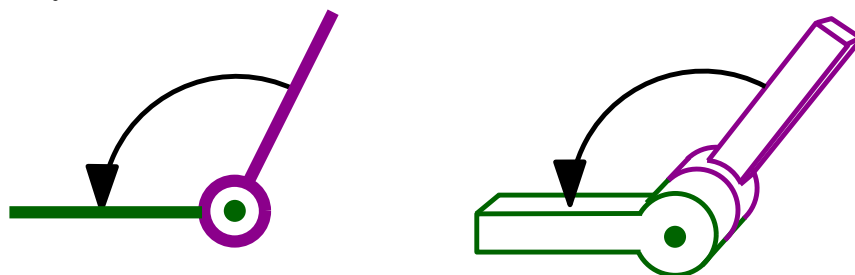
In essence, an industrial robot is an externally powered mechanical device, motion of which is controlled by a computer following a changeable program. **The motions must be planned in advance for a particular industrial task**, such as **an automated assembly**. They should be executed repeatably and typically with high precision. To achieve this, one needs to develop a reliable mathematical model for the electro-/hydraulic-/pneumatico-mechanical systems.

Below, we mostly concentrate on modeling the **mechanical part**, review basic terminology and useful concepts, describe elements of sensing the environment and self configuration, as well as present basic approaches for control design and most common methods for relevant analysis.

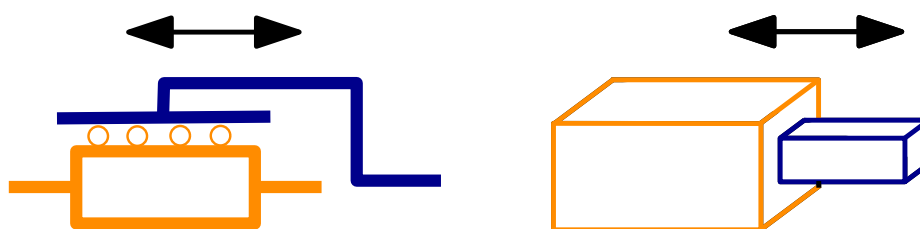
1.3 Terminology and Examples

Mechanical part of a manipulating robot is a kinematic chain, which is called **an arm**. It consists of several (rigid) bodies, called **links**, that are connected by various **joints** and move relative to each other.

Below (in these notes), without loss of generality, we consider only so-called **single-degree-of-freedom (1-DOF) joints** that are either **revolute** or **prismatic**. Revolute joint means that one link rotates around an axis fixed on the other one and a single angular variable fully defines their relative orientation. Prismatic joint means that one link moves along a straight line fixed on the other one and a single distance variable fully defines their relative displacement that corresponds either to an extension or a retraction. They are schematically shown below.



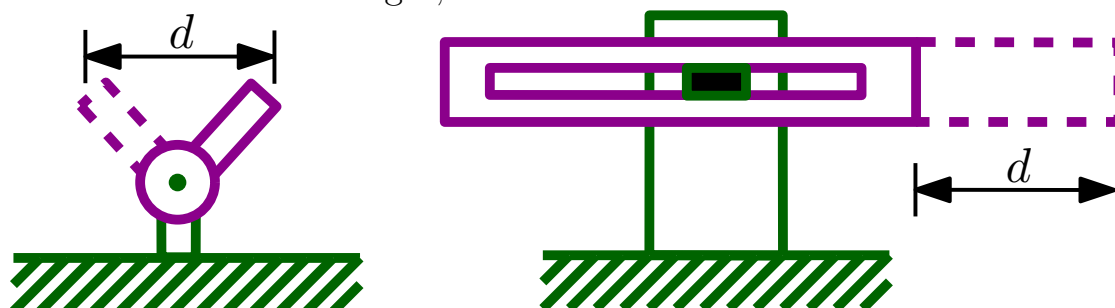
Revolute



Prismatic

The other types of joints can be easily modeled by combinations of these two by introducing **auxiliary zero-mass zero-length** links.

It should be noticed that prismatic joints are much easier for analysis and control design to achieve certain goals of manipulation. However, revolute joints have certain advantages; one of them is illustrated next.



It is clear from the above picture that much larger links are necessary to allow covering the same distance d when they are connected with a revolute joint instead of a prismatic one. As a result, most of the currently used industrial robotic manipulators possess some revolute joints. Note that there are hardly any prismatic joints in nature! Manipulators with only revolute

joints are called **articulated**.

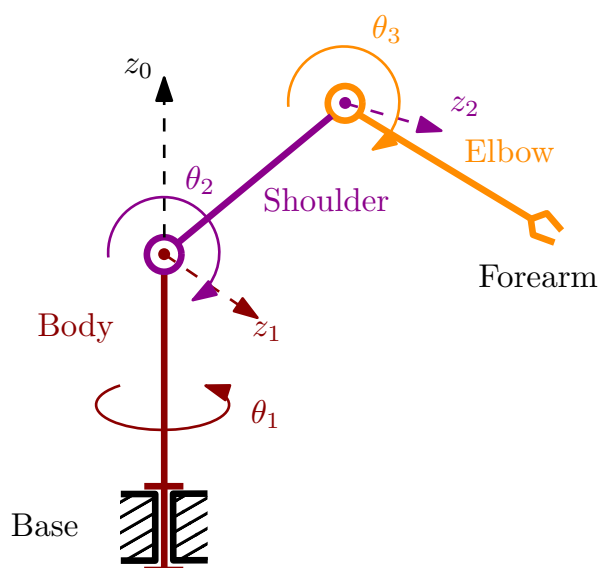
Typically, manipulators are classified by the number of single-degree-of-freedom (1-DOF) joints, that is by the number of mechanical degrees of freedom (DOF), and by the type of joints connecting the first three links counting from the base. For example, an RRP manipulator means that the first joint and the second one are revolute (R), while the third is prismatic (P) and the type of the other joints, if any, is not specified.

At the end of the arm, an industrial manipulator has an **end-effector** that can be a tool or an active (or passive) gripper. Often, the end-effectors can be changed, either automatically or manually, according to the current task. A few examples of tools and end-effectors are shown below (courtesy of Schunk)

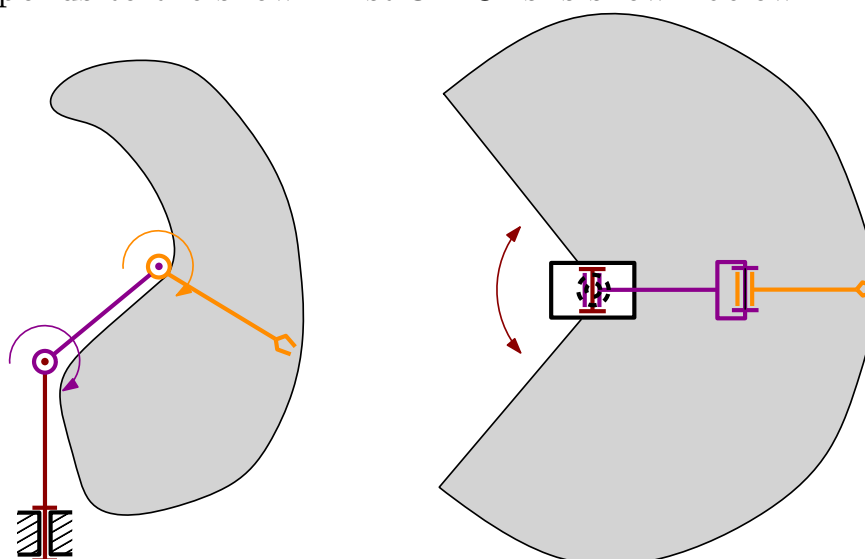


To use a tool, manipulator needs to reach a target object. The region, within which it can operate, is called **workspace**.

Consider, for example, the PBA manipulator by Schunk that is shown below (courtesy of Schunk) together with a symbolic representation for the first 3 DOF.



This is an articulated robotic manipulator, which has 6-DOF and is of RRR-type. The workspace for a hypothetical **3-DOF** (“elbow”) manipulator that corresponds to the shown first **3** DOFs is shown below.



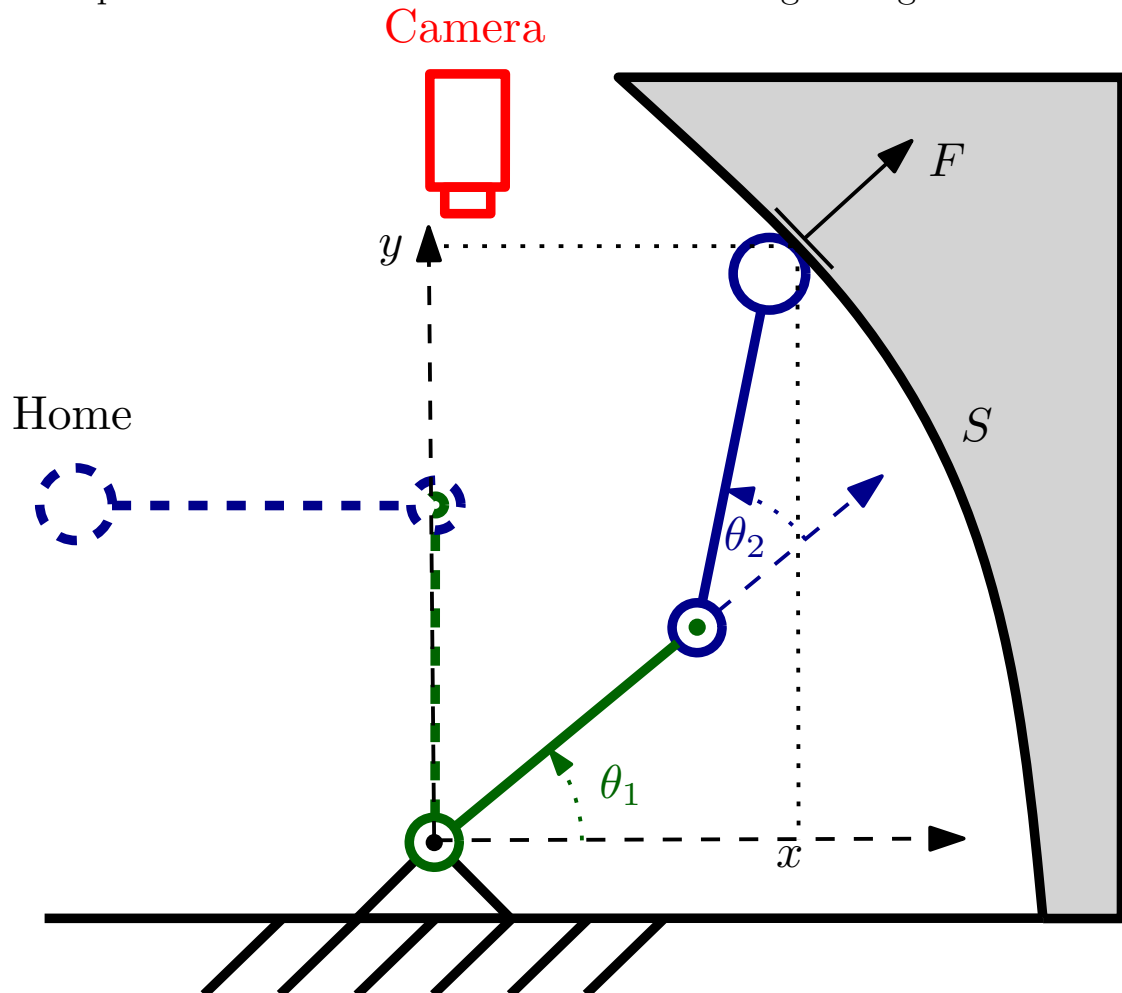
Our goal for the rest of this text is to understand how to force an industrial robotic manipulator to perform a required task. In the next section we introduce a few conceptual problems that should be solved in order to achieve this on a simple example. Each chapter after that is to cover one of these and a few other problems in details.

1.4 Introduction of the Conceptual Problems

The following conceptual problems must be resolved to make a robot succeed in performing a typical task:

- Forward Kinematics (see Chapter 2),
- Inverse Kinematics (see Chapter 2),
- Velocity Kinematics (see Chapter 3),
- Dynamics (see Chapter 5),
- Path Planning and Trajectory Generation (see Chapter 4),
- Motion Control (see Chapters 6 and 7),
- Force Control (see Chapter A),
- Computer Vision and Vision Based Control (see Chapters B and C).

Let us briefly introduce these tasks on the following example: Suppose a two-link planar articulated robot is to be used for grinding a wall.



Let us assume that we can assign the torques applied to the links by the actuators at the two joints. If we manage to produce these torques appropriately, we should be able to control the two corresponding angles. How do we choose these angles to perform grinding?

Obviously, to answer this question, we need to know

- A parametrization of the wall in the plane,
- Configurations of the robot (i.e. the values of the angles) that correspond to touching the wall.

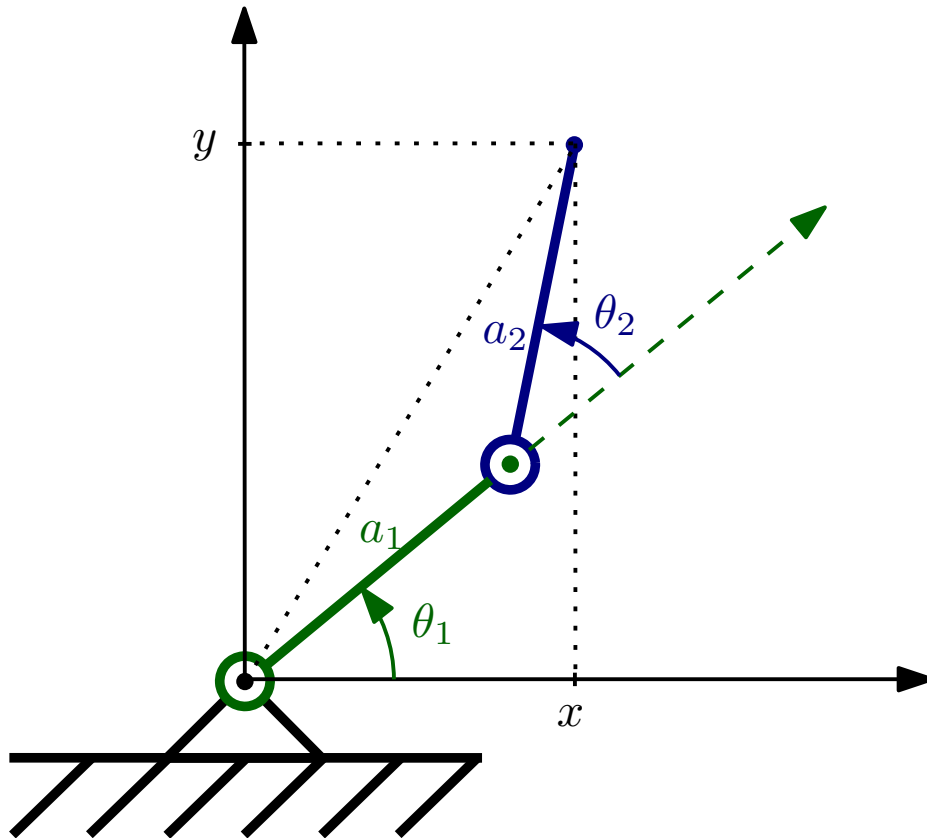
To know the result of assigning the joint angles in terms of the position and the orientation of the end-effector one must solve the **forward kinematics** problem: Find the position (x, y) of the end-effector and the orientation of the tool as functions of joint angles (θ_1, θ_2) .

At the same time, to define the control task, we need to know the required values of the joint angles to touch the wall, i.e. we must solve the **inverse kinematics** problem: Find the angles (θ_1, θ_2) as functions of coordinates of the end-effector (x, y) .

Let us proceed with these two tasks.

1.4.1 Forward Kinematics

Denote by a_1 and a_2 the lengths of the links; the drawings needed to compute the position of the end-effector is given next.



Let us denote by (x_1, y_1) and (x_2, y_2) coordinates of the endings of the links with respect to their beginnings. Then, using elementary geometry

$$\begin{aligned} x &= x_1 + x_2 = a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2) \\ y &= y_1 + y_2 = a_1 \sin \theta_1 + a_2 \sin(\theta_1 + \theta_2) \end{aligned}$$

It is also quite easy to find the orientation of the tool assuming it is attached to the ending of the last link and directed along it. Clearly, it should make the angle

$$\theta = \theta_1 + \theta_2$$

with the horizontal line. However, we will soon discover that it is more convenient to define the tool orientation (especially in 3D) as a direction of an appropriate unit vector, $\vec{x}_2$, together with a couple of the other vectors, $\vec{y}_2$ and $\vec{z}_2$, defining an (orthogonal) coordinate frame rigidly attached to the tool. In our case, we can easily compute a coordinate representation of the direction vector $\vec{x}_2$ with respect to the fixed frame, which is rigidly attached to the base:

$$x_2^0 = [\cos(\theta), \sin(\theta), 0]^T$$

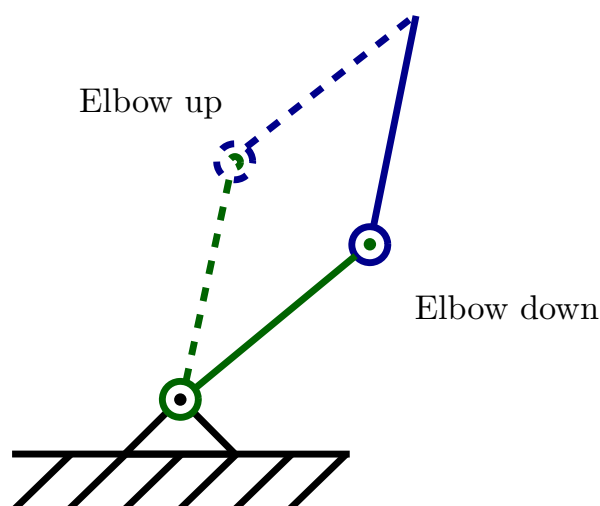
Similarly, we can define coordinate representations the other two vectors:

$$\begin{aligned} y_2^0 &= [-\sin(\theta), \cos(\theta), 0]^T \\ z_2^0 &= [0, 0, 1]^T \end{aligned}$$

The components of these three representations of direction vectors define a special 3×3 matrix, called **rotation matrix**, that is even more useful for specification of the end-effector orientation.

For a more complex multi-link not planar robot finding appropriate triangles in the 3D-space in order to use similar arguments is hardly possible. This is why we will adopt a more systematic approach in Chapter 2: We will attach to each link a coordinate system (frame) and compute pairwise transformations between them. After that, we will be able to inductively compute the position and the orientation of the end-effector moving step by step from the base in a straightforward way.

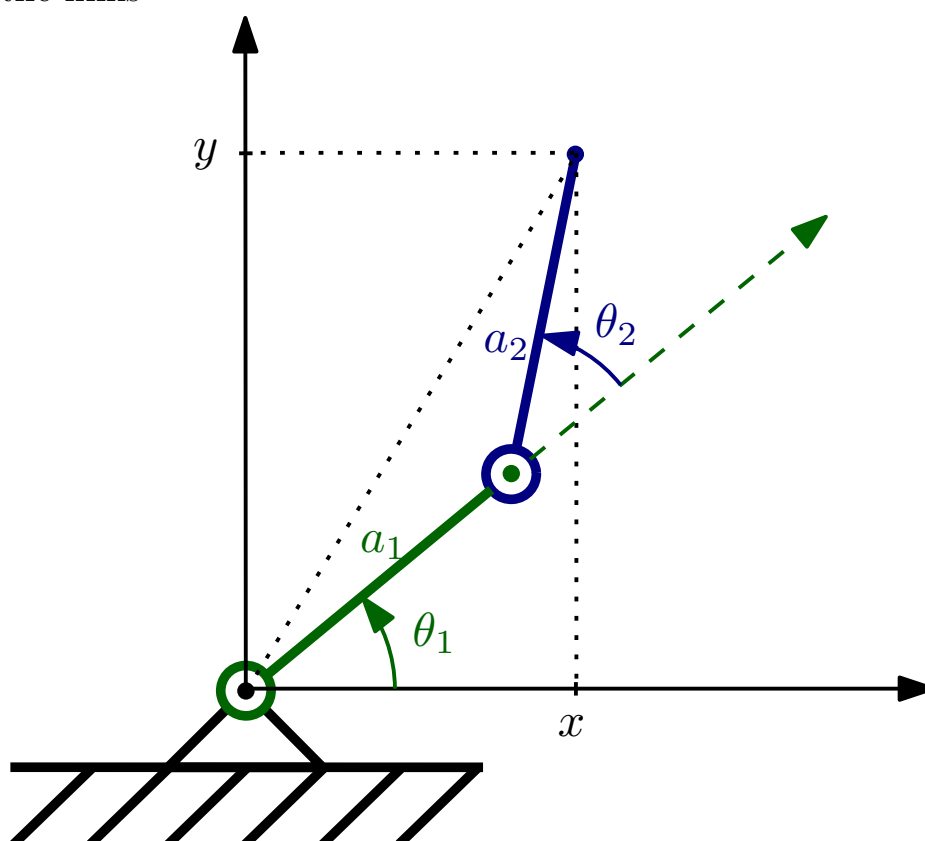
Returning to our example, it is useful to note that if the tool position (x, y) is given, but the orientation is not defined, then, except for a couple of particular cases, there are two possible configurations, called “Elbow Up” and “Elbow Down”. They are shown below.



Now, from the solution of the forward kinematics problem we know the result of assigning particular values for the joint angles on the position and orientation of the tool for wall grinding in our example. To find the required angles from a description of the wall we need to solve the inverse problem, which is often harder.

1.4.2 Inverse Kinematics

In our simple case, instead of trying to solve the computed above equations for the forward kinematics we can just use the cosine theorem for the triangle made by the links



We immediately obtain

$$\cos \theta_2 = \frac{x^2 + y^2 - a_1^2 - a_2^2}{2a_1a_2} =: \textcolor{red}{D}, \quad \left\{ \Rightarrow \sin \theta_2 = \pm \sqrt{1 - \textcolor{red}{D}^2} \right\}$$

$$\Rightarrow \textcolor{red}{\theta}_2 = \tan^{-1} \left(\frac{\sin \theta_2}{\cos \theta_2} \right) = \tan^{-1} \left(\frac{\pm \sqrt{1 - \textcolor{red}{D}^2}}{\textcolor{red}{D}} \right)$$

$$\Rightarrow \textcolor{red}{\theta}_1 = \tan^{-1} \left(\frac{y}{x} \right) - \tan^{-1} \left(\frac{a_2 \sin \textcolor{red}{\theta}_2}{a_1 + a_2 \cos \textcolor{red}{\theta}_2} \right)$$

In a more complex case, finding a solution of the inverse kinematics problem may be challenging and may require using numerical methods. We will discuss this in Chapter 2.

1.4.3 Velocity Kinematics

In many situations it is necessary to compute (and to assign via an appropriate feedback action) not only the position and orientation but also the velocity of the tool. The corresponding forward problem and, surprisingly, the inverse problem are simpler.

Clearly, the geometrical relations between (x, y) and (θ_1, θ_2)

$$x = a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2), \quad y = a_1 \sin \theta_1 + a_2 \sin(\theta_1 + \theta_2)$$

immediately imply the following relations between the velocities

$$\begin{aligned} \frac{d}{dt}x(t) &= -a_1 \sin \theta_1 \frac{d}{dt}\theta_1 - a_2 \sin(\theta_1 + \theta_2) \left(\frac{d}{dt}\theta_1 + \frac{d}{dt}\theta_2 \right) \\ \frac{d}{dt}y(t) &= a_1 \cos \theta_1 \frac{d}{dt}\theta_1 + a_2 \cos(\theta_1 + \theta_2) \left(\frac{d}{dt}\theta_1 + \frac{d}{dt}\theta_2 \right) \end{aligned}$$

In a more compact (matrix) form this relation can be written as follows

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \underbrace{\begin{bmatrix} -a_1 \sin \theta_1 - a_2 \sin(\theta_1 + \theta_2) & -a_2 \sin(\theta_1 + \theta_2) \\ a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2) & a_2 \cos(\theta_1 + \theta_2) \end{bmatrix}}_{=: \textcolor{red}{J}(\theta_1, \theta_2)} \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}$$

The matrix $\textcolor{red}{J}(\cdot)$ above is called **Jacobian** of the manipulator. It is introduced for a general case in Chapter 3. Whenever this matrix is not singular, the computed relation between the joint velocities and the tool velocity

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \textcolor{red}{J}(\theta_1, \theta_2) \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}$$

allows to compute the joint velocities $\dot{\theta}_1(t)$ and $\dot{\theta}_2(t)$ that are required to

achieve a particular velocity of the tool.

Indeed, $\dot{\theta}_1(t)$ and $\dot{\theta}_2(t)$ are found by

$$\begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} = \mathbf{J}^{-1}(\theta_1, \theta_2) \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix}$$

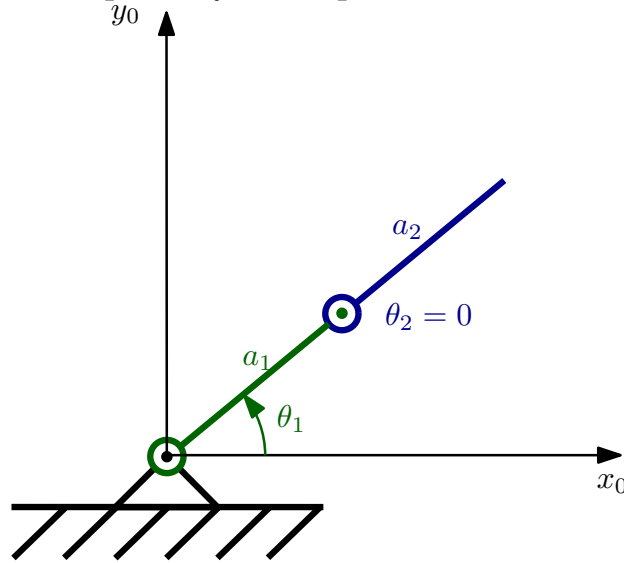
with

$$\mathbf{J}^{-1}(\theta_1, \theta_2) = \frac{-1}{a_1 a_2 \sin(\theta_2)} \left[\begin{array}{c|c} a_2 \cos(\theta_1 + \theta_2) & a_2 \sin(\theta_1 + \theta_2) \\ a_1 \cos \theta_1 + a_2 \cos(\theta_1 + \theta_2) & a_1 \sin \theta_1 + a_2 \sin(\theta_1 + \theta_2) \end{array} \right]$$

It is clear that the inverse of the Jacobian is not defined if and only if

$$\sin \theta_2 = 0 \quad \{ \Rightarrow \theta_2 = \pi \cdot k, \quad k = \dots - 1, 0, 1, \dots \}$$

The corresponding postures of the manipulator are called **Singular Configurations**; in our example they correspond to the case of full arm extension:



If $\theta_2 = 0$, then the Jacobian $\mathbf{J}(\theta_1, \theta_2)$ loses the rank and cannot be inverted. It means that the tool velocity, defining direction of motion of the end-effector, cannot be any, but should belong to a particular line irrespective of the joint velocities. Since near the singularity very large joint velocities are needed to direct the motions of the tool, it is common practice to avoid motions in their vicinity.

Now, with a solution for kinematics at hand, one can define a task of achieving certain time evolution of the joint angles. In order to understand how to induce the corresponding motion with the help of the actuators, equations of dynamics should be considered.

1.4.4 Dynamics of Motions

Clearly, position, orientation, and velocity kinematics define motions of the manipulator irrespective of an actuation and laws of mechanics. However, in order to understand how to introduce the effect of control action and to describe feasible trajectories, equations of motion should be derived. In Chapter 5 we will introduce a systematic approach for deriving such equations using formalism of the Euler-Lagrange equations.

With their help, under certain assumptions, one would easily derive for example the following equations describing dynamics of our double pendulum when it moves towards the wall

$$\begin{bmatrix} p_1 + p_2 + 2p_3 \cos \theta_2 & p_2 + p_3 \cos \theta_2 \\ p_2 + p_3 \cos \theta_2 & p_2 \end{bmatrix} \begin{bmatrix} \ddot{\theta}_1 \\ \ddot{\theta}_2 \end{bmatrix} + p_3 \sin \theta_2 \begin{bmatrix} -\dot{\theta}_2 & -\dot{\theta}_2 - \dot{\theta}_1 \\ \dot{\theta}_1 & 0 \end{bmatrix} \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} + \begin{bmatrix} p_4 g \cos \theta_1 + p_5 g \cos(\theta_1 + \theta_2) \\ p_5 g \cos(\theta_1 + \theta_2) \end{bmatrix} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

where

- p_1 - p_5 are constants defined by physical parameters of the links (masses, moments of inertia, lengths, and locations of the centers of masses);
- u_1 , u_2 are control torques applied at the joints.

These equations can be used to plan a required time evolution of the joint angles necessary for the tool to follow the wall together with the required torques. Also, they can be used to propose and to rigorously justify an appropriate feedback control law to apply in order to achieve the target motion using the sensors and the actuators installed on the robot.

It should be noticed, however, that just barely following the wall is not enough for grinding. One needs to introduce specifications and to plan the motion with certain care. It is also important to realize that the equations above are derived ignoring the interaction forces that may include impact forces, appearing when the tool touches the wall, as well as friction and resistance forces, appearing during execution of grinding. So, appropriate modifications may be required.

1.4.5 Path Planning and Trajectory Generation

Planning a motion to perform a task, such as grinding a wall, is typically done in two steps. First a path is planned and after that it is enhanced with certain velocities to follow. The next should be noticed.

- **Path** is a curve in the configuration space of the robot: It defines geometric evolution of the tool (to follow the wall) without timing.

- **Trajectory** is the path augmented with the information on velocities, with which the tool and the links will travel along the path.
- Planning trajectory might also include specifications on the accelerations and the induced interaction forces with the wall.

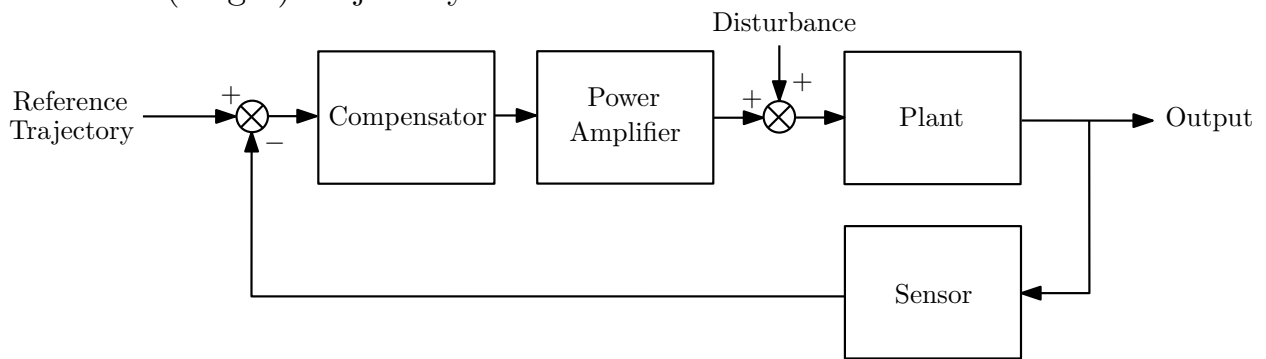
Chapter 4 is devoted to an introductory discussion on the issue, covering the basic approach relying only on kinematics. We will not discuss here modern approaches based on dynamic models that are currently under active development.

As soon as a target trajectory is generated, one needs to suggest a feedback control law to implement it.

1.4.6 Motion Control

Several mathematics-based and engineering-intuition-based approaches are discussed in Chapter 6 and Chapter 7 respectively.

Possible basic structure of a feedback control system assuming a generated reference (target) trajectory is shown below



1.4.7 Force Feedback

It is clear that as soon as the tool touches the wall the equations of motion for the free robotic arm presented above are invalid. They should be modified by introducing the wall reaction forces acting on the tool. These forces should be controlled in order to prevent the tool from breaking and to achieve necessary quality of grinding. An introduction to the current commonly used approaches is presented in Chapter A.

1.4.8 Computer Vision and Vision Based Control

Performing the task of grinding the wall requires the control system to react on various factors detectable by cameras. Also, the wall profile may be unknown in advance and detected by a camera. In this case, a trajectory and an appropriate feedback action should be generated “on fly” or using

camera signals for feedback. Signals from cameras cannot be used in feedback directly as, say, signals from encoders giving the values of the joint angles. The obtained video images must be processed. The issue is touched in Chapters [B](#) and [C](#).

1.5 Conclusions

In this Chapter we have introduced some basic terminology used in robotics and have briefly given a preview of the challenges and of the contents of the next Chapters. We proceed next with introducing necessary mathematical tools and discussing the way of solving the problem of control design for industrial robotic manipulators.

Chapter 2

Kinematics

In order to describe kinematics of a robotic arm of arbitrary complexity we proceed as follows. The arm is a chain of links, which are assumed to be rigid bodies, connected at (1-DOF) joints. Starting from the base of the manipulator, at each joint two parts of the arm are moving with respect to each other. The problem of kinematics is essentially to describe positions and velocities of every point of each link. As will be clear later, we are mostly interested in the mutual orientations of the links and positions of the points associated with the joints, with the centers of masses of the links, and with the end point of the tool and the orientation of the end-effector.

A systematic procedure that we will follow is to attach to each rigid body a coordinate system, called a **frame**. Each frame consists of an origin and **2** axes (for planar motions) or **3** axes (for general motions). Every point of each rigid body is uniquely defined by the coordinates in the attached frame that do not change irrespective to whether the body is moving or not. However, whenever the arm is set in motion, the positions of the origins of the frames and their pairwise orientations are changing. As soon as we define such relations between each couple of the frames, we will be able to easily describe positions and velocities of all the points.

So, let us start with introducing necessary notation and appropriate mathematical description of so-called rigid motions and homogeneous transformations in **2D** and **3D**.

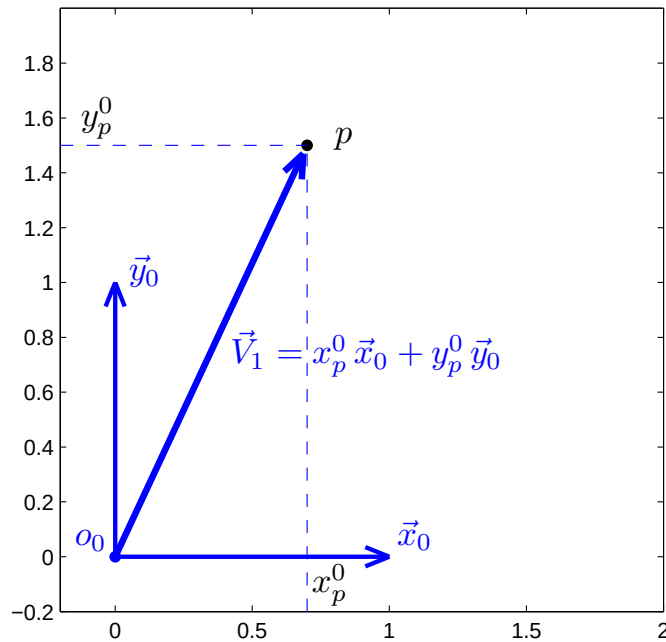
2.1 Rigid Motions & Homogeneous Transformations

We need to agree on how to describe and to denote frames (coordinate systems), points, and (free) vectors.

2.1.1 Frames, Points, and Vectors

Coordinates of points and vectors (in 2D)

Consider a point $\mathbf{p}$ in plane (in $\mathbb{R}^2$) and introduce a frame, that is another special point called the origin $\mathbf{o}_0$ and two mutually orthogonal axes $\mathbf{x}_0$ and $\mathbf{y}_0$ intersecting at $\mathbf{o}_0$. Orthogonal projection of the point into the axes define its coordinates.



The point $\mathbf{p}$ can be associated with the vector $\vec{V}_1$ from $\mathbf{o}_0$ to $\mathbf{p}$, as shown above. Let x_p^0 denote $\mathbf{x}$ -coordinate and y_p^0 denote $\mathbf{y}$ -coordinate of the point $\mathbf{p}$ respectively in $(\mathbf{o}_0, \mathbf{x}_0, \mathbf{y}_0)$ -frame, called below for brevity the $\mathbf{0}$ -frame. Note that if we denote by $\vec{x}_0$ and $\vec{y}_0$ the unite vectors defining directions of the axes, we have

$$\vec{V}_1 = \overrightarrow{\mathbf{o}_0 \mathbf{p}} = x_p^0 \vec{x}_0 + y_p^0 \vec{y}_0$$

that means that one should move from the point $\mathbf{o}_0$ exactly x_p^0 units in the direction of $\mathbf{x}_0$ axis and y_p^0 units in the direction of $\mathbf{y}_0$ axis to reach the point $\mathbf{p}$. Therefore, the two numbers x_p^0 and y_p^0 can be considered as coordinates of both, the point $\mathbf{p}$ and of the vector $\vec{V}_1$ with respect to $\mathbf{0}$ -frame. Note that vectors literally define displacement of the number of units defined by their magnitude in the specified direction.

Magnitude of the vector can be computed using the coordinates in any frame

$$\text{magnitude of } \vec{V}_1 = |\vec{V}_1| = \sqrt{(x_p^0)^2 + (y_p^0)^2}$$

but it is a specification of the vector that is frame-independent as well as the

direction of the vector that can be computed as

$$\text{direction of } \vec{V}_1 = \frac{\vec{V}_1}{|\vec{V}_1|} = \frac{x_p^0}{|\vec{V}_1|} \vec{x}_0 + \frac{y_p^0}{|\vec{V}_1|} \vec{y}_0$$

Analogous definitions are valid in **3D**.

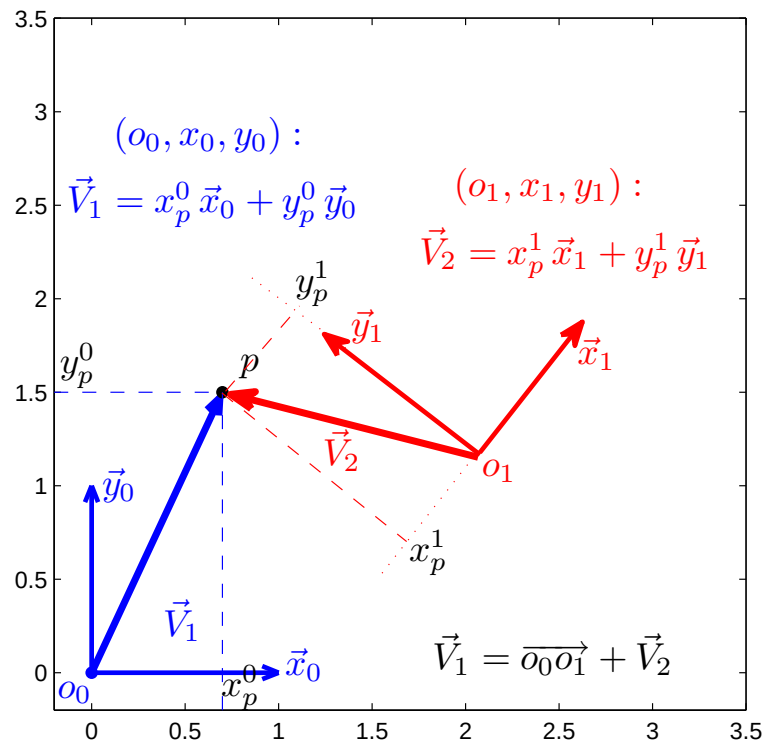
Vectors with magnitude equal to **1** are called unit vectors and are used to specify directions.

Let us summarize our notation:

- $\mathbf{o}_0$ is the origin of the $(\mathbf{o}_0, \mathbf{x}_0, \mathbf{y}_0)$ -frame, to be called the **0-frame**
- $\vec{x}_0$ denotes the unit vector defining the direction of the $\mathbf{x}_0$ axis
- $\vec{y}_0$ denotes the unit vector defining the direction of the $\mathbf{y}_0$ axis
- x_p^0 denotes the $\mathbf{x}$ -coordinate of point $\mathbf{p}$ in the $(\mathbf{o}_0, \mathbf{x}_0, \mathbf{y}_0)$ -frame
- y_p^0 denotes the $\mathbf{y}$ -coordinate of point $\mathbf{p}$ in the $(\mathbf{o}_0, \mathbf{x}_0, \mathbf{y}_0)$ -frame
- $\vec{V}_1 = \overrightarrow{o_0 p}$ denotes the vector indicating the displacement from $\mathbf{o}_0$ to $\mathbf{p}$

Representations in different frames

Suppose now there are **0-frame** $(\mathbf{o}_0, \mathbf{x}_0, \mathbf{y}_0)$ and **1-frame** $(\mathbf{o}_1, \mathbf{x}_1, \mathbf{y}_1)$:



The point $\mathbf{p}$ can now be associated not only with vector $\vec{\mathbf{V}}_1 = \overrightarrow{\mathbf{o}_0\mathbf{p}}$ but also with vector $\vec{\mathbf{V}}_2 = \overrightarrow{\mathbf{o}_1\mathbf{p}}$ in a similar way. Now, we have

$$\vec{\mathbf{V}}_2 = x_p^1 \vec{\mathbf{x}}_1 + y_p^1 \vec{\mathbf{y}}_1$$

It is clear than that it is wise to use notation identifying frames. From now on, we will use the following notation for the two representations of $\mathbf{p}$

$$\mathbf{p}^0 = \mathbf{V}_1^0 = \begin{bmatrix} x_p^0 \\ y_p^0 \end{bmatrix} \quad \text{and} \quad \mathbf{p}^1 = \mathbf{V}_2^1 = \begin{bmatrix} x_p^1 \\ y_p^1 \end{bmatrix}$$

associated with the respective frames ($\mathbf{0}$ and $\mathbf{1}$).

Let us remind the reader the following.

- **A point** (material point) corresponds to a particular location in the space or within a rigid body.
- **A point** has different representations (coordinates) in different frames that can be associated with certain vectors.
- A **vector** (free vector) is completely defined by **magnitude** $|\vec{\mathbf{V}}|$ and **direction** $\frac{\vec{\mathbf{V}}}{|\vec{\mathbf{V}}|}$ and can be also represented by sets of coordinates combined into columns $\mathbf{V}^0$ or $\mathbf{V}^1$, e.g. in 2D: $\mathbf{x}_0^0 = \mathbf{x}_1^1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\mathbf{y}_0^0 = \mathbf{y}_1^1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$.. It can be interpreted as a directed displacement.
- **Vectors** with the same direction and magnitude are identical (they are not attached to particular points that may be used to define them!)

Vectors $\vec{\mathbf{V}}_1 = \overrightarrow{\mathbf{o}_0\mathbf{p}}$ and $\vec{\mathbf{V}}_2 = \overrightarrow{\mathbf{o}_1\mathbf{p}}$ represent the same point $\mathbf{p}$ with respect to different frames ($\mathbf{V}_1^0$ and $\mathbf{V}_2^1$ defined above). It is clear that $\vec{\mathbf{V}}_1 = \overrightarrow{\mathbf{o}_0\mathbf{o}_1} + \vec{\mathbf{V}}_2$; however, to define a relation between $\mathbf{V}_1^0$ and $\mathbf{V}_2^1$, we need to take into account relative orientation of the two frames. Hence, it is clear that we need

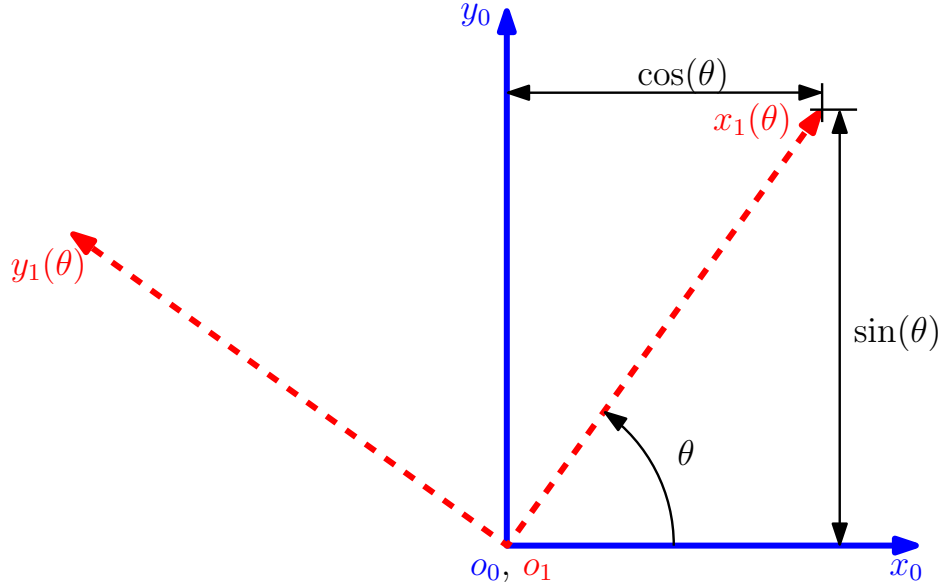
1. To find representations of the unite vectors defining one frame with respect to the other, i.e. to specify a rotation defining relative orientation of the two frames;
2. To find the relation between the origins $\mathbf{o}_0$ and $\mathbf{o}_1$ defining a relative translation of the frames, associated with the vector $\overrightarrow{\mathbf{o}_0\mathbf{o}_1}$.

Rotations are defined next.

2.1.2 Rotations in 2D and 3D

Rotations in 2D

Obviously, rotations in plane can be defined by a single angle. Suppose we have a fixed **0**-frame and a rotating **1**-frame with coinciding origins.



To find an appropriate way to parametrize rotations in **2D**, we can use simple trigonometry to find coordinates of the points defined by $\vec{x}_1(\theta)$ and $\vec{y}_1(\theta)$ giving directions of the moving axes $\mathbf{x}_1$ and $\mathbf{y}_1$ with respect to **0**-frame for an arbitrary value of the angle θ defining relative rotation of the two frames.

It is clear from the picture that $\vec{x}_1(\theta) = \cos(\theta)\vec{x}_0 + \sin(\theta)\vec{y}_0$ and therefore in **0**-frame $\vec{x}_1(\theta)$ and $\vec{y}_1(\theta)$ represented as

$$\mathbf{x}_1^0(\theta) = \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix}, \quad \mathbf{y}_1^0(\theta) = \mathbf{x}_1^0\left(\theta + \frac{\pi}{2}\right) = \begin{bmatrix} \cos\left(\theta + \frac{\pi}{2}\right) \\ \sin\left(\theta + \frac{\pi}{2}\right) \end{bmatrix} = \begin{bmatrix} -\sin(\theta) \\ \cos(\theta) \end{bmatrix}$$

Now, for two representations of a point $\mathbf{p}$, rigidly attached to the moving **1**-frame, i.e. its coordinates are changing in **0**-frame and are fixed in **1**-frame

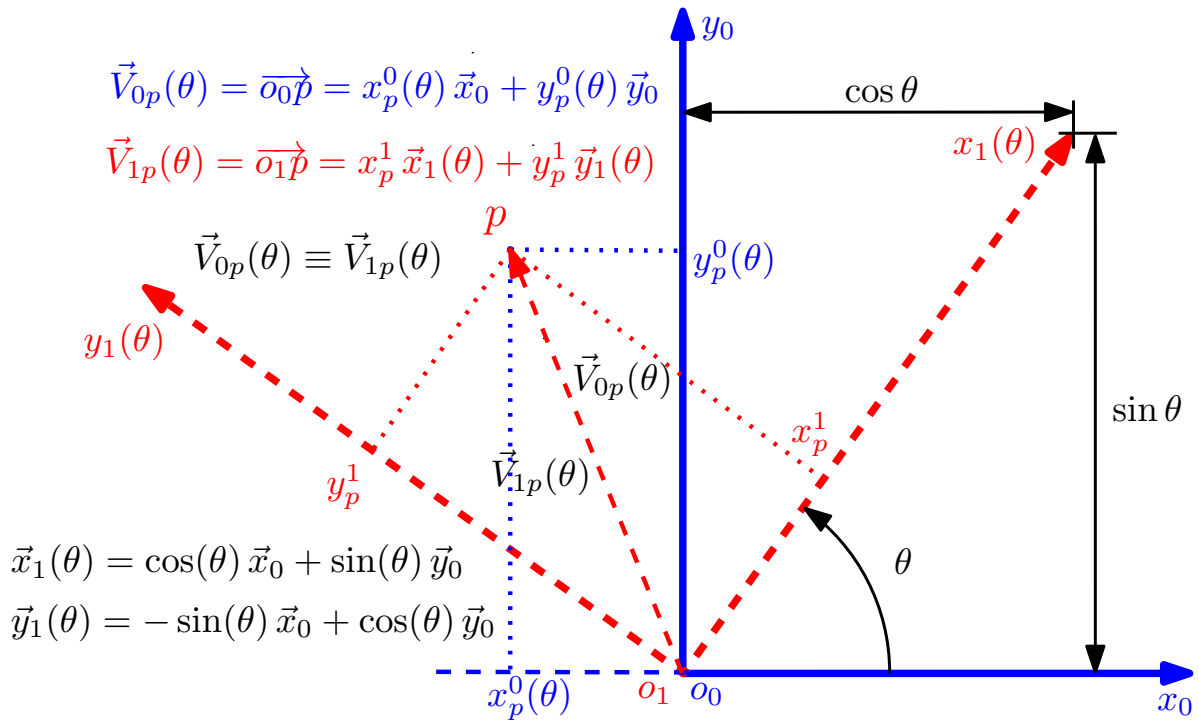
$$\mathbf{p}^0(\theta) = \mathbf{V}_{0p}^0(\theta) = \begin{bmatrix} x_p^0(\theta) \\ y_p^0(\theta) \end{bmatrix} \quad \text{and} \quad \mathbf{p}^1 = \mathbf{V}_{1p}^1 = \begin{bmatrix} x_p^1 \\ y_p^1 \end{bmatrix}$$

defined by vectors

$$\vec{V}_{0p} = \overrightarrow{o_0 p} = x_p^0(\theta) \vec{x}_0 + y_p^0(\theta) \vec{y}_0 \quad \text{and} \quad \vec{V}_{1p} = \overrightarrow{o_1 p} = x_p^1 \vec{x}_1(\theta) + y_p^1 \vec{y}_1(\theta)$$

that must be identical since the origins coincide ($\overrightarrow{o_0 o_1} = \vec{0}$), we have, after substituting the expressions for $\vec{x}_1(\theta)$ and $\vec{y}_1(\theta)$, the following

$$\vec{V}_{1p} = x_p^1 (\cos(\theta) \vec{x}_0 + \sin(\theta) \vec{y}_0) + y_p^1 (-\sin(\theta) \vec{x}_0 + \cos(\theta) \vec{y}_0) = \vec{V}_{0p}$$



It follows that

$$x_p^0(\theta) = x_p^1 \cos(\theta) - y_p^1 \sin(\theta) \quad \text{and} \quad y_p^0(\theta) = x_p^1 \sin(\theta) + y_p^1 \cos(\theta)$$

and writing this in a matrix form one obtains

$$\begin{bmatrix} x_p^0(\theta) \\ y_p^0(\theta) \end{bmatrix} = R_1^0(\theta) \begin{bmatrix} x_p^1 \\ y_p^1 \end{bmatrix} \quad \Leftrightarrow \quad p^0(\theta) = R_1^0(\theta) p^1$$

where the matrix

$$R_1^0(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

is called a **rotation matrix**, which is a coordinate representation of a **rotation tensor** $R(\theta)$. The latter can be interpreted as a transformation of vectors by rotation on angle θ in counterclockwise direction. In particular, it defines **1-frame** as a **0-frame** rotated on angle θ . We have computed its representation in **0-frame** as an action on (linear transformation of) coordinates of vectors with respect to **1-frame**. This representation (the matrix) allows us to compute coordinates with respect to **0-frame** from coordinates with respect to **1-frame**.

Rotation matrices in 2D and 3D

The next properties of R_1^0 (and other rotation matrices) are easy to verify

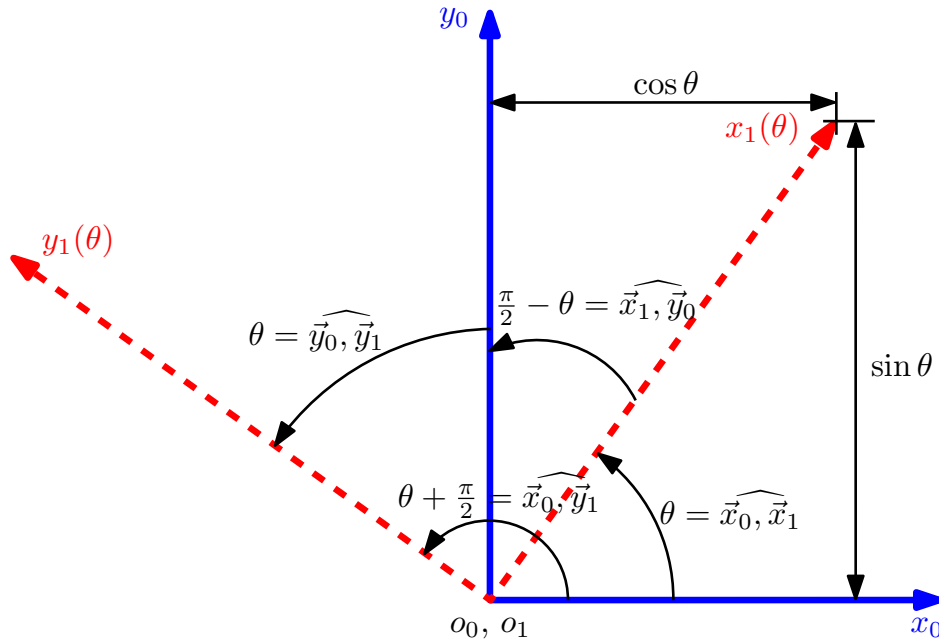
- $\det R_1^0(\theta) = \cos^2(\theta) + \sin^2(\theta) = 1$

$$\bullet R_0^1(\theta) = [R_1^0(\theta)]^{-1} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} = [R_1^0(\theta)]^T = R_1^0(-\theta)$$

Let us consider now a way of defining the physical action of rotation by angle θ , i.e. the tensor $\mathbf{R}(\theta)$, in a coordinate-free way. To this end, we notice that the columns of the rotation matrix

$$R_1^0(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \cos(\theta + \frac{\pi}{2}) \\ \cos(\frac{\pi}{2} - \theta) & \cos(\theta) \end{bmatrix} = [x_1^0(\theta) | y_1^0(\theta)]$$

are defined by projections of the direction vectors of $\mathbf{1}$ -frame onto the axes of $\mathbf{0}$ -frame or by the cosines of the angles between the respective axes.



Vector projections can be defined using the notion of scalar product between two vectors, which can be computed either using a representation in an arbitrary frame or using the physical coordinate-free notions of magnitude of a vector and angle between two vectors:

$$\vec{V}_1 \cdot \vec{V}_2 = (V_1^*)^T V_2^* = |\vec{V}_1| |\vec{V}_2| \cos(\widehat{\vec{V}_1, \vec{V}_2})$$

where $(\cdot)^*$ denotes a coordinate representation in an arbitrary frame ($* = 0, * = 1, \dots$).

It is easy to see that

$$x_1^0(\theta) = \begin{bmatrix} (x_1^0(\theta))^T x_0^0 \\ (x_1^0(\theta))^T y_0^0 \end{bmatrix}, \quad y_1^0(\theta) = \begin{bmatrix} (y_1^0(\theta))^T x_0^0 \\ (y_1^0(\theta))^T y_0^0 \end{bmatrix}$$

where, obviously,

$$\mathbf{x}_0^0 = \begin{bmatrix} \mathbf{1} \\ \mathbf{0} \end{bmatrix} \quad \text{and} \quad \mathbf{y}_0^0 = \begin{bmatrix} \mathbf{0} \\ \mathbf{1} \end{bmatrix} \quad \left(\Leftarrow \quad \vec{x}_0 = \mathbf{1} \vec{x}_0 + \mathbf{0} \vec{y}_0, \quad \vec{y}_0 = \mathbf{0} \vec{x}_0 + \mathbf{1} \vec{y}_0 \right)$$

and, therefore, rotation of **1**-frame with respect to **0**-frame by angle θ can be defined as

$$\mathbf{R}_1^0(\theta) = [\mathbf{x}_1^0(\theta) | \mathbf{y}_1^0(\theta)] = \begin{bmatrix} \vec{x}_1(\theta) \cdot \vec{x}_0 & \vec{y}_1(\theta) \cdot \vec{x}_0 \\ \vec{x}_1(\theta) \cdot \vec{y}_0 & \vec{y}_1(\theta) \cdot \vec{y}_0 \end{bmatrix}$$

In a similar way, it can be shown that the matrix defining rotation of **1**-frame with respect to **0**-frame with the same origin in **3**-dimensions can be computed as $\mathbf{R}_1^0 = [\mathbf{x}_1^0 | \mathbf{y}_1^0 | \mathbf{z}_1^0]$, i.e.

$$\mathbf{R}_1^0 = \begin{bmatrix} (\mathbf{x}_1^0)^T \mathbf{x}_0^0 & (\mathbf{y}_1^0)^T \mathbf{x}_0^0 & (\mathbf{z}_1^0)^T \mathbf{x}_0^0 \\ (\mathbf{x}_1^0)^T \mathbf{y}_0^0 & (\mathbf{y}_1^0)^T \mathbf{y}_0^0 & (\mathbf{z}_1^0)^T \mathbf{y}_0^0 \\ (\mathbf{x}_1^0)^T \mathbf{z}_0^0 & (\mathbf{y}_1^0)^T \mathbf{z}_0^0 & (\mathbf{z}_1^0)^T \mathbf{z}_0^0 \end{bmatrix} = \begin{bmatrix} \vec{x}_1 \cdot \vec{x}_0 & \vec{y}_1 \cdot \vec{x}_0 & \vec{z}_1 \cdot \vec{x}_0 \\ \vec{x}_1 \cdot \vec{y}_0 & \vec{y}_1 \cdot \vec{y}_0 & \vec{z}_1 \cdot \vec{y}_0 \\ \vec{x}_1 \cdot \vec{z}_0 & \vec{y}_1 \cdot \vec{z}_0 & \vec{z}_1 \cdot \vec{z}_0 \end{bmatrix}$$

It can be verified that similar to the **2D**-case in **3D** for every θ

- Columns of $\mathbf{R}_1^0$ are mutually orthogonal;
- $\mathbf{R}_1^0 [\mathbf{R}_1^0]^T = \mathbf{I}_3$ and $\mathbf{R}_0^1 = [\mathbf{R}_1^0]^T$;
- $\det \mathbf{R}_1^0 = 1$.

It is also known that if a rotation between two frames in **3D** is done around a fixed (stationary) axes through the origin, the constant direction of which is defined by a unit vector $\vec{k}$, on the angle θ in counterclockwise direction as observed from its arrow, than its matrix representation in the **0**-frame ($\mathbf{o}_0 \mathbf{x}_0 \mathbf{y}_0 \mathbf{z}_0$) can be computed as follows:

$$\mathbf{R}^0 = \mathbf{k}^0 (\mathbf{k}^0)^T + \cos(\theta) (\mathbf{I} - \mathbf{k}^0 (\mathbf{k}^0)^T) + \sin(\theta) \mathbf{S}^0(\vec{k})$$

where $\mathbf{I} = \mathbf{x}_0 \mathbf{x}_0^T + \mathbf{y}_0 \mathbf{y}_0^T + \mathbf{z}_0 \mathbf{z}_0^T$ is the identity matrix, while with the **0**-frame-coordinate representation $(\mathbf{k}^0)^T = [\mathbf{k}_x^0, \mathbf{k}_y^0, \mathbf{k}_z^0]$ we use the notation

$$\mathbf{S}^0(\vec{k}) = \begin{bmatrix} \mathbf{0} & -\mathbf{k}_z^0 & \mathbf{k}_y^0 \\ \mathbf{k}_z^0 & \mathbf{0} & -\mathbf{k}_x^0 \\ -\mathbf{k}_y^0 & \mathbf{k}_x^0 & \mathbf{0} \end{bmatrix} \quad \text{to be introduced later when we talk about}$$

matrix representations of vector cross products. In coordinate-free form, using the standard notation for tensor and cross products between vectors, the formula above, which is known as Rodrigues' formula, can be written as

$$\mathbf{R}(\theta) = \vec{k} \otimes \vec{k} + \cos(\theta) (\mathbf{I} - \vec{k} \otimes \vec{k}) + \sin(\theta) \vec{k} \times \mathbf{I}$$

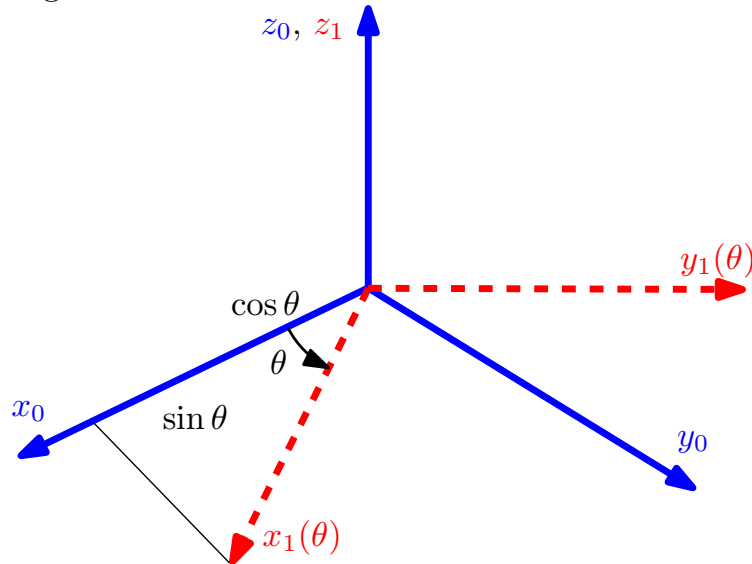
where $\vec{k} \otimes \vec{k}$, $I = \vec{x}_0 \otimes \vec{x}_0 + \vec{y}_0 \otimes \vec{y}_0 + \vec{z}_0 \otimes \vec{z}_0$ and $S(\vec{k}) \equiv \vec{k} \times I$ denote actions on vectors that correspond

- to creating vector projections onto $\vec{k}$: $(\vec{k} \otimes \vec{k}) \cdot \vec{v} = (\vec{k} \cdot \vec{v}) \vec{k}$,
- to keeping the vectors unchanged: $I \cdot \vec{v} = \vec{v}$, and
- to making a cross product with $\vec{k}$: $(\vec{k} \times I) \cdot \vec{v} = \vec{k} \times \vec{v}$,

respectively. We will discuss later an alternative formula for computation of a coordinate representation of a rotation around a given direction.

Basic rotations in 3D

Let us consider now the most important for the development of our targeted solution of the forward kinematics problem examples of rotation in 3D. Consider the following rotation from 0-frame to 1-frame



Here we have 1-frame rotated by θ -angle around the z_0 -axis that coincides with the z_1 -axis.

It is easy to see that

$$\begin{aligned} \vec{x}_1(\theta) \cdot \vec{x}_0 &= \cos \theta, & \vec{y}_1(\theta) \cdot \vec{x}_0 &= -\sin \theta, & \vec{z}_1(\theta) \cdot \vec{x}_0 &= 0 \\ \vec{x}_1(\theta) \cdot \vec{y}_0 &= \sin \theta, & \vec{y}_1(\theta) \cdot \vec{y}_0 &= \cos \theta, & \vec{z}_1(\theta) \cdot \vec{y}_0 &= 0 \\ \vec{x}_1(\theta) \cdot \vec{z}_0 &= 0, & \vec{y}_1(\theta) \cdot \vec{z}_0 &= 0, & \vec{z}_1(\theta) \cdot \vec{z}_0 &= 1 \end{aligned}$$

Therefore, we obtain

$$R_1^0(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \{=: R_{z,\theta}\}$$

Same expression can be obtained from the Euler formula with $\vec{k} = \vec{z}_0 = \vec{z}_1$, or in, say, 0-frame coordinates with $(\vec{k}^0)^T = [0, 0, 1]$.

This matrix, called a **basic rotation matrix**, clearly satisfies the following properties

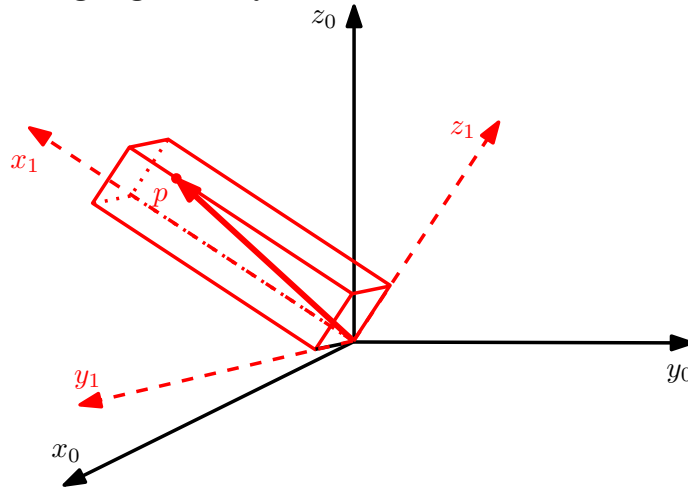
$$\mathbf{R}_{z,0} = \mathbf{I}_3, \quad \mathbf{R}_{z,\theta}\mathbf{R}_{z,\phi} = \mathbf{R}_{z,\theta+\phi}, \quad [\mathbf{R}_{z,\theta}]^{-1} = \mathbf{R}_{z,-\theta}$$

There are two other basic rotations in 3D (around $\vec{k} = \vec{x}_0$ and $\vec{k} = \vec{y}_0$):

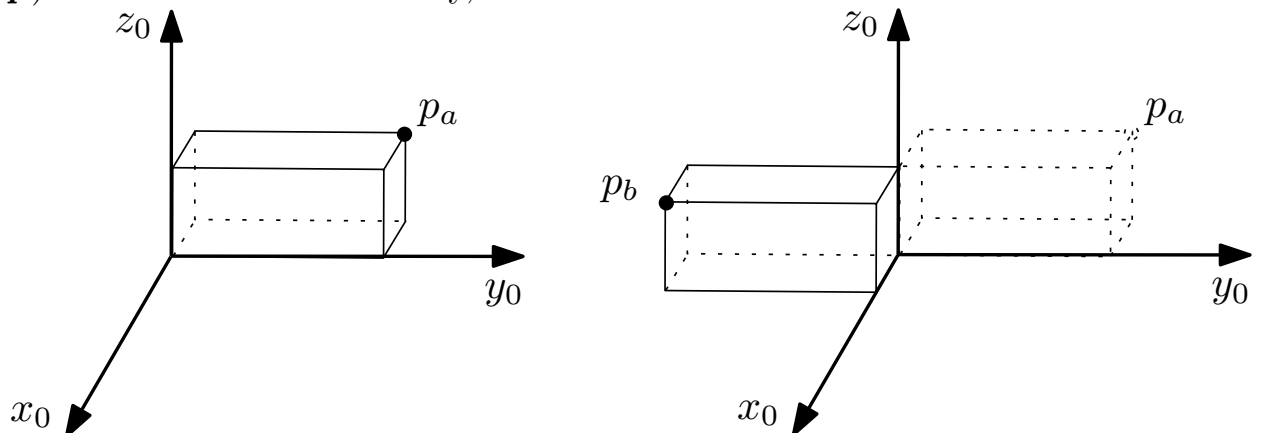
$$\mathbf{R}_{x,\theta} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}, \quad \mathbf{R}_{y,\theta} = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

Rotation of a rigid body

We are now ready to present an application of the just introduced rotations in 3-D. Let the **0**-frame define our world (fixed) frame, while the **1**-frame be attached to a rotating rigid body:



What will happen with points of the body (let say with a particular point p) when we rotate the body, i.e. the **1**-frame?



How to trace the change of position (coordinates) of a particular point of the body with respect to the **0**-frame?

Suppose we are interested in coordinates of a particular point p of the

body. Clearly, in the **1**-frame they are constant, but in the **0**-frame they are changing during the rotation!

Let the coordinates of $\mathbf{p}$ in the **1**-frame be $\mathbf{p}^1 == [\mathbf{x}_p^1, \mathbf{y}_p^1, \mathbf{z}_p^1]^T$, i.e.

$$\mathbf{p}^1 = \mathbf{x}_p^1 \mathbf{x}_1^1 + \mathbf{y}_p^1 \mathbf{y}_1^1 + \mathbf{z}_p^1 \mathbf{z}_1^1 = \mathbf{x}_p^1 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + \mathbf{y}_p^1 \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + \mathbf{z}_p^1 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{x}_p^1 \\ \mathbf{y}_p^1 \\ \mathbf{z}_p^1 \end{bmatrix}$$

and the coordinates $\mathbf{x}_p^1, \mathbf{y}_p^1, \mathbf{z}_p^1$ are not changing when the **1**-frame is rotated. We need to find the coordinates of $\mathbf{p}$ in the **0**-frame that are changing.

Clearly, with the rotation the basis of the **1**-frame is moving with respect to **0**-frame and we know how it does that: according to a rotation matrix $\mathbf{R}_1^0$ relating the two frames!

We have for the basis vectors of the **1**-frame:

$$\mathbf{x}_1^0 = \mathbf{R}_1^0 \mathbf{x}_1^1 = \mathbf{R}_1^0 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{y}_1^0 = \mathbf{R}_1^0 \mathbf{y}_1^1 = \mathbf{R}_1^0 \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{z}_1^0 = \mathbf{R}_1^0 \mathbf{z}_1^1 = \mathbf{R}_1^0 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Therefore the coordinates of the point $\mathbf{p}$ in **0**-frames are computed as follows

$$\mathbf{p}^0 = \mathbf{x}_p^1 \mathbf{x}_1^0 + \mathbf{y}_p^1 \mathbf{y}_1^0 + \mathbf{z}_p^1 \mathbf{z}_1^0 = \mathbf{R}_1^0 \begin{bmatrix} \mathbf{x}_p^1 \\ 0 \\ 0 \end{bmatrix} + \mathbf{R}_1^0 \begin{bmatrix} 0 \\ \mathbf{y}_p^1 \\ 0 \end{bmatrix} + \mathbf{R}_1^0 \begin{bmatrix} 0 \\ 0 \\ \mathbf{z}_p^1 \end{bmatrix} = \underbrace{\mathbf{R}_1^0 \begin{bmatrix} \mathbf{x}_p^1 \\ \mathbf{y}_p^1 \\ \mathbf{z}_p^1 \end{bmatrix}}_{=\mathbf{R}_1^0 \mathbf{p}^1}$$

Now, to handle in the future rotation of the tool as a result of rotation of all the links, we need to understand how to combine multiple rotations.

2.1.3 Composition of Rotations

We will see soon that when a body is involved in several (an ordered sequence of) rotations, it is important to distinguish between two situations:

- Rotations with Respect to the Current Frame (that is, except for the first one, every rotation is performed around one of the axis obtained after previous rotation)
- Rotations with Respect to the Fixed Frame (that is every rotation is performed around a not rotated axis)

However, before dealing with this, let us derive a simple (algebraic) composition rule for rotations.

Sequence of two rotations

Suppose that we have **3** different frames in $3D$:

$$(\mathbf{o}_0, x_0, y_0, z_0), \quad (\mathbf{o}_1, x_1, y_1, z_1), \quad (\mathbf{o}_2, x_2, y_2, z_2).$$

with coinciding origins, i.e. $\mathbf{o}_0 = \mathbf{o}_1 = \mathbf{o}_2$ or $\overrightarrow{\mathbf{o}_0\mathbf{o}_1} = \overrightarrow{\mathbf{o}_1\mathbf{o}_2} = \overrightarrow{\mathbf{o}_2\mathbf{o}_0} = \vec{0}$.

Any point $\mathbf{p}$ will have three coordinate representations:

$$\mathbf{p}^0 = [u_0, v_0, w_0]^T, \quad \mathbf{p}^1 = [u_1, v_1, w_1]^T, \quad \mathbf{p}^2 = [u_2, v_2, w_2]^T$$

defined by three identical vectors ($\vec{V}_0 = \vec{V}_1 = \vec{V}_2$):

$$\begin{aligned} \vec{V}_0 &= u_0 \vec{x}_0 + v_0 \vec{y}_0 + w_0 \vec{z}_0, & \vec{V}_1 &= u_1 \vec{x}_1 + v_1 \vec{y}_1 + w_1 \vec{z}_1, \\ \vec{V}_2 &= u_2 \vec{x}_2 + v_2 \vec{y}_2 + w_2 \vec{z}_2 \end{aligned}$$

Orientation of each pair of frames can be defined by appropriate rotations that automatically define the transformations between the coordinate representations:

$$\mathbf{p}^0 = \mathbf{R}_1^0 \mathbf{p}^1, \quad \mathbf{p}^1 = \mathbf{R}_2^1 \mathbf{p}^2, \quad \mathbf{p}^0 = \mathbf{R}_2^0 \mathbf{p}^2$$

Let us answer the question: How are the matrices $\mathbf{R}_1^0$, $\mathbf{R}_2^1$, and $\mathbf{R}_2^0$ related? One can compute $\mathbf{p}^0$ in two different ways

$$\mathbf{p}^0 = \mathbf{R}_1^0 \mathbf{p}^1 = \mathbf{R}_1^0 \mathbf{R}_2^1 \mathbf{p}^2, \quad \mathbf{p}^0 = \mathbf{R}_2^0 \mathbf{p}^2$$

As a result, we conclude

$$\boxed{\mathbf{R}_1^0 \mathbf{R}_2^1 \equiv \mathbf{R}_2^0}$$

which simply means that a rotation from **0**-frame to **2**-frame is equivalent to a composition of two rotations: from **0**-frame to **1**-frame and from **1**-frame to **2**-frame. To memorize this, notice that the lower index **1** and the upper index **1** symbolically cancel each other.

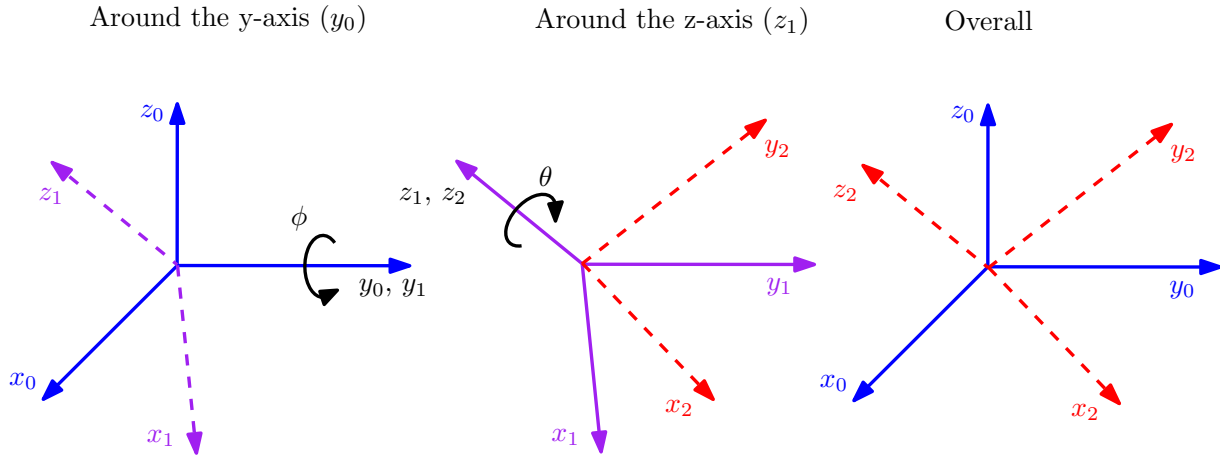
Combining rotations with respect to current axes

Let us use this rule to derive a composition of the following two rotations:

- First, **0**-frame is rotated by angle ϕ around the current **y**-axis, i.e. $\mathbf{y}_0$, to obtain **1**-frame,
- Second, **1**-frame is rotate by angle θ around the current **z**-axis, i.e. $\mathbf{z}_1$, to obtain the final **2**-frame.

defining the change of coordinates from **2**-frame to **0**-frame.

This is illustrated below



The rotations around the current **y**-axis and **z**-axis are basic rotations

$$\mathbf{R}_{y_0, \phi} = \begin{bmatrix} \cos \phi & 0 & \sin \phi \\ 0 & 1 & 0 \\ -\sin \phi & 0 & \cos \phi \end{bmatrix} = \mathbf{R}_1^0, \quad \mathbf{R}_{z_1, \theta} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} = \mathbf{R}_2^1$$

Therefore, the overall rotation is $\mathbf{R}_2^0 = \mathbf{R}_1^0 \mathbf{R}_2^1$, i.e.

$$\mathbf{R}_2^0 = \underbrace{\mathbf{R}_{y_0, \phi}}_{\text{first}} \underbrace{\mathbf{R}_{z_1, \theta}}_{\text{second}} = \begin{bmatrix} \cos \phi & 0 & \sin \phi \\ 0 & 1 & 0 \\ -\sin \phi & 0 & \cos \phi \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

and after the matrix multiplication

$$\mathbf{R}_2^0 = \mathbf{R}_{y_0, \phi} \mathbf{R}_{z_1, \theta} = \begin{bmatrix} c_\phi c_\theta & -c_\phi s_\theta & s_\phi \\ s_\theta & c_\theta & 0 \\ -s_\phi c_\theta & s_\phi s_\theta & c_\phi \end{bmatrix}, \quad \{\Rightarrow p^0 = \mathbf{R}_2^0 p^2\}$$

where the standard short notation is used for the trigonometric functions.

Order of rotations is important

Here, we should make an important observation: Rotations do not commute:

$$\mathbf{R}_{y, \phi} \mathbf{R}_{z, \theta} \neq \mathbf{R}_{z, \theta} \mathbf{R}_{y, \phi}$$

So, the order of rotations is important!

Indeed,

$$\underbrace{\mathbf{R}_{y_0, \phi}}_{\text{first}} \underbrace{\mathbf{R}_{z_1, \theta}}_{\text{second}} = \mathbf{R}_{y, \phi} \mathbf{R}_{z, \theta} = \begin{bmatrix} c_\phi c_\theta & -c_\phi s_\theta & s_\phi \\ s_\theta & c_\theta & 0 \\ -s_\phi c_\theta & s_\phi s_\theta & c_\phi \end{bmatrix}$$

and

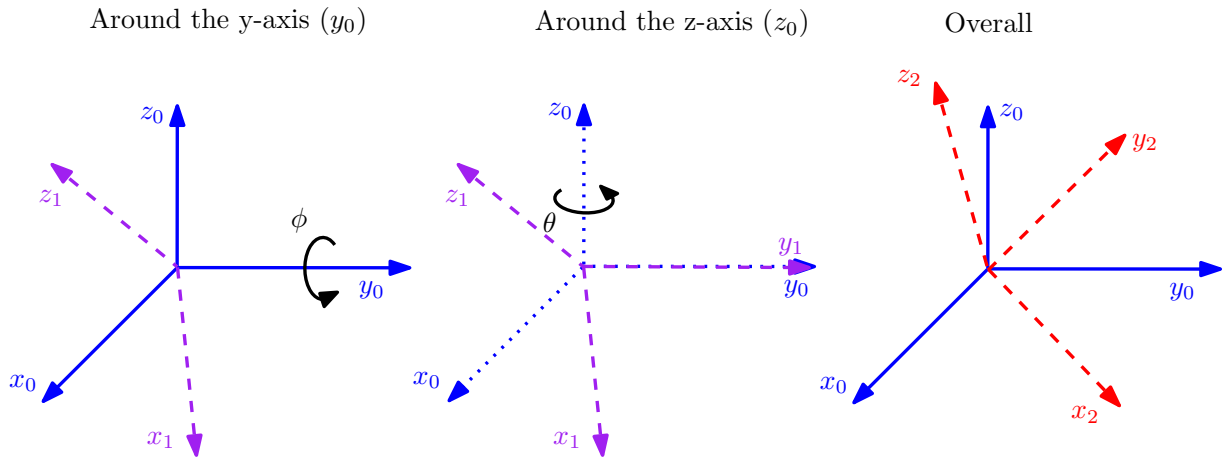
$$\underbrace{R_{z_0, \theta}}_{\text{first}} \underbrace{R_{y_1, \phi}}_{\text{second}} = \mathbf{R}_{z, \theta} \mathbf{R}_{y, \phi} = \begin{bmatrix} c_\phi c_\theta & -s_\theta & c_\theta s_\phi \\ s_\theta c_\phi & c_\theta & s_\phi s_\theta \\ -s_\phi & 0 & c_\phi \end{bmatrix}$$

Combining rotations with respect to fixed axes

Let us consider now consecutive rotations with respect to a fixed frame:

- First, **0**-frame is rotated by angle ϕ around the current **y**-axis, i.e. y_0 , to obtain **1**-frame,
- Second, **1**-frame is rotate by angle θ around the original **z**-axis, i.e. z_0 , to obtain the final **2**-frame.

They are illustrated below.



Obviously, the second rotation is not around one of the current coordinate axes. So, we either should use the Euler formula or a simple trick, described next, that allows to use coordinate representations of basic rotations in this case also. To use the trick, we should derive first a useful formula relating coordinate representations of a linear transformation, such as rotations, in two different frames.

Change of coordinates for representation of rotations

Consider two frames with a given relative rotation (and the same origin):

$$(o_0, x_0, y_0, z_0), \quad (o_1, x_1, y_1, z_1), \quad p^0 = \mathbf{R}_1^0 p^1$$

Given a linear transform $\mathbf{A}$, that acts on vectors:

$$\vec{a} = \mathbf{A} \cdot \vec{b}$$

for which we know a coordinate representation in the **0**-frame

$$\mathbf{a}^0 = \mathbf{A}^0 \mathbf{b}^0$$

How to compute its representation $\mathbf{a}^1 = \mathbf{A}^1 \mathbf{b}^1$ in the **1**-frame? Obviously, we just need to use the coordinate representations in the two frames:

- For a given $\mathbf{a}^0$, one can compute $\mathbf{a}^1 = \mathbf{R}_0^1 \mathbf{a}^0 = [\mathbf{R}_1^0]^{-1} \mathbf{a}^0$
- For a given $\mathbf{b}^1$, one can compute $\mathbf{b}^0 = \mathbf{R}_1^0 \mathbf{b}^1$

To define $\mathbf{A}^1$, the representation of $\mathbf{A}$ in the **1**-frame, one proceeds as follows

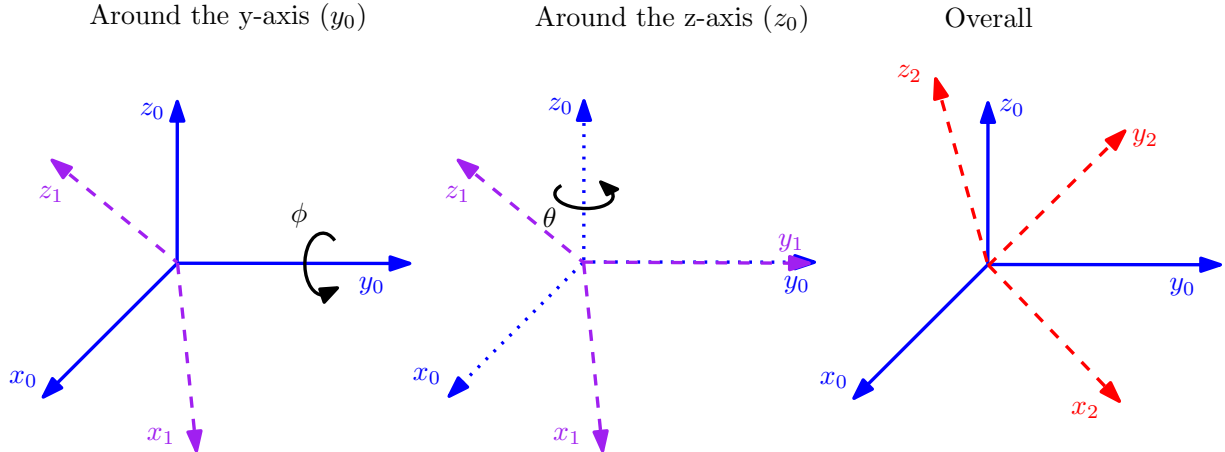
$$\mathbf{a}^1 = [\mathbf{R}_1^0]^{-1} \mathbf{a}^0 = [\mathbf{R}_1^0]^{-1} (\mathbf{A}^0 \mathbf{b}^0) = [\mathbf{R}_1^0]^{-1} \mathbf{A}^0 (\mathbf{R}_1^0 \mathbf{b}^1) = \mathbf{A}^1 \mathbf{b}^1$$

Since this relation is true for arbitrary $\mathbf{a}^1$ and $\mathbf{b}^1$, we conclude

$$\boxed{\mathbf{A}^1 = [\mathbf{R}_1^0]^{-1} \mathbf{A}^0 \mathbf{R}_1^0} = \mathbf{R}_0^1 \mathbf{A}^0 \mathbf{R}_1^0 = \mathbf{R}_0^1 \mathbf{A}^0 [\mathbf{R}_0^1]^{-1}$$

Combining rotations with respect to fixed axes (continued)

Now, let us go back to our composition of rotations:



We have two rotations

- the basic rotation $\mathbf{R}_1^0 = \mathbf{R}_{y_0, \phi}$: by angle ϕ around y_0 -axis
- the rotation $\mathbf{R}_2^1$ defined as the rotation by angle θ around z_0 -axis (not z_1 -axis)

The combined rotation will be

$$\mathbf{R}_2^0 = \mathbf{R}_1^0 \mathbf{R}_2^1 = \mathbf{R}_{y_0, \phi} \mathbf{R}_2^1 = \mathbf{R}_{y_0, \phi} \underbrace{([\mathbf{R}_{y_0, \phi}]^{-1} \mathbf{R}_{z_0, \theta} \mathbf{R}_{y_0, \phi})}_{\mathbf{A}^1 = [\mathbf{R}_1^0]^{-1} \mathbf{A}^0 \mathbf{R}_1^0}$$

where we use the just derived formula for transformation of the basic rotation representation $\mathbf{A}^0 = \mathbf{R}_{z_0, \theta}$ in another frame, the **1**-frame, which is related to the **0**-frame by another rotation matrix $\mathbf{R}_1^0 = \mathbf{R}_{y_0, \phi}$.

After cancellations of the product of a matrix and its inverse, we have

$$\mathbf{R}_2^0 = \underbrace{\mathbf{R}_{z_0, \theta}}_{\text{second}} \underbrace{\mathbf{R}_{y_0, \phi}}_{\text{first}} = \mathbf{R}_{z, \theta} \mathbf{R}_{y, \phi}$$

and comparing with composition of rotations with respect to current axes, we see that the order of multiplications of the coordinate representations is reversed! Actually, some authors consider this order as more natural since it coincides with the order of action on the coordinates of a vector.

Although the order of rotations is important as well as the order of matrix multiplications: $\mathbf{R}_{z, \theta} \mathbf{R}_{y, \phi} \neq \mathbf{R}_{y, \phi} \mathbf{R}_{z, \theta}$, it can be easily noticed that

$$\underbrace{\mathbf{R}_{z_0, \theta}}_{\text{second}} \underbrace{\mathbf{R}_{y_0, \phi}}_{\text{first}} = \mathbf{R}_{z, \theta} \mathbf{R}_{y, \phi} = \underbrace{\mathbf{R}_{z_0, \theta}}_{\text{first}} \underbrace{\mathbf{R}_{y_1, \phi}}_{\text{second}}$$

i.e. the same basic rotations can be performed in reversed order if we switch to the appropriate current axis instead of the fixed one.

Let us consider a few simple examples of combining rotations.

Example with two rotations

Find a representation of the rotation $\mathbf{R}_2^0$ defined by the following ordered sequence of basic rotations:

0 $\rightarrow$ **1** By θ about the current axis $\mathbf{x}$, i.e. $\mathbf{x}_0$;

1 $\rightarrow$ **2** By ϕ about the current axis $\mathbf{z}$, i.e. $\mathbf{z}_1$.

Since the rotations are performed around the current axes, we multiply the basic rotation matrices in the same order, to obtain

$$\mathbf{R}_2^0 = \mathbf{R}_{x, \theta} \mathbf{R}_{z, \phi}$$

The obtained transformation defines for any point of the **2**-frame with coordinates $\mathbf{p}^2 = [x^2, y^2, z^2]^T$, its coordinates in the **0**-frame as

$$\mathbf{p}^0 = \mathbf{R}_2^0 \mathbf{p}^2 = \mathbf{R}_{x, \theta} \mathbf{R}_{z, \phi} \mathbf{p}^2$$

Example with three rotations

Find a representation of the rotation R_3^0 defined by the following ordered sequence of basic rotations:

$0 \rightarrow 1$ By θ about the current axis $\mathbf{x}$, i.e. $\mathbf{x}_0$;

$1 \rightarrow 2$ By ϕ about the current axis $\mathbf{z}$, i.e. $\mathbf{z}_1$;

$2 \rightarrow 3$ By α about the **fixed axis $\mathbf{z}$** , i.e. $\mathbf{z}_0$.

Following the rule discovered above: “every new rotation with respect to the current axis implies multiplication by the corresponding matrix from the right (direct order), while every new rotation with respect to the fixed (initial) axis implies multiplication by the corresponding matrix from the left (reverse order)” we obtain the answer

$$R_3^0 = R_{z,\alpha} R_2^0 = R_{z,\alpha} R_{x,\theta} R_{z,\phi}$$

Example with four rotations

Find a representation of the rotation R_4^0 defined by the following ordered sequence of basic rotations:

$0 \rightarrow 1$ By θ about the current axis $\mathbf{x}$, i.e. $\mathbf{x}_0$;

$1 \rightarrow 2$ By ϕ about the current axis $\mathbf{z}$, i.e. $\mathbf{z}_1$;

$2 \rightarrow 3$ By α about the **fixed axis $\mathbf{z}$** , i.e. $\mathbf{z}_0$;

$3 \rightarrow 4$ By β about the current axis $\mathbf{y}$, i.e. $\mathbf{y}_3$.

The result of three rotations, obtained above, should be multiplied by the last basic rotation matrix from the right

$$R_4^0 = R_3^0 R_{y,\beta} = R_{z,\alpha} R_{x,\theta} R_{z,\phi} R_{y,\beta}$$

Examples with five rotations

Find a representation of the rotation R_5^0 defined by the following ordered sequence of basic rotations:

$0 \rightarrow 1$ By θ about the current axis $\mathbf{x}$, i.e. $\mathbf{x}_0$;

$1 \rightarrow 2$ By ϕ about the current axis $\mathbf{z}$, i.e. $\mathbf{z}_1$;

2 $\rightarrow$ **3** By α about the **fixed axis z** , i.e. z_0 ;

3 $\rightarrow$ **4** By β about the current axis y , i.e. y_3 ;

4 $\rightarrow$ **5** By δ about the **fixed axis x** , i.e. x_0 .

By now, the answer should be obvious:

$$R_5^0 = R_{x,\delta} R_4^0 = R_{x,\delta} R_{z,\alpha} R_{x,\theta} R_{z,\phi} R_{y,\beta}$$

But what should we do if we would like to compute $\tilde{R}_5^0$ with, by some reasons, changed last rotation as follows:

4 $\rightarrow$ **5** By δ about the **axis x in 1-frame**, i.e. x_1 ?

We can now think about the last rotation equivalently performed as a basic rotation around x -axis but needed to be represented in another frame. So, we have

$$\tilde{R}_5^0 = R_4^0 R_5 = R_{z,\alpha} R_{x,\theta} R_{z,\phi} R_{y,\beta} R_5$$

where

$$R_5 = [R_4^1]^{-1} R_{x,\delta} R_4^1, \quad \text{with} \quad R_4^1 = R_{z,\phi} \{ [R_2^0]^{-1} R_{z,\alpha} R_2^0 \} R_{y,\beta}$$

and $R_2^0 = R_{x,\theta} R_{z,\phi}$. The final answer can be obtained by a straightforward substitution.

Let us discuss now possible parameterizations of coordinate representations of rotations.

2.1.4 Parametrization of Rotation Matrices

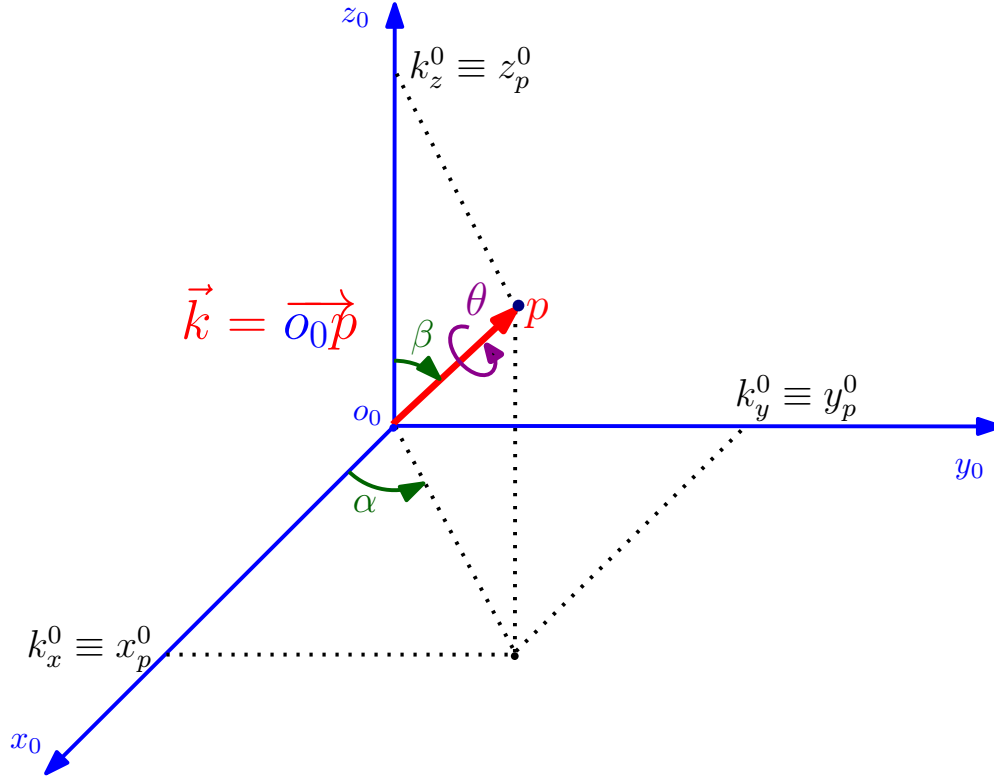
Any coordinate representation of a rotation matrix

- is of dimension **3** $\times$ **3**, i.e. it is defined by **9**-numbers;
- belongs to **SO(3)**, i.e. its **3** columns define vectors
 - of magnitude **1** (**3** equations)
 - that are orthogonal to each other (**3** equations)

Since there are **6** relations on **9** numbers, it should be sufficient to specify only **3** numbers to completely define a rotation matrix!

Angle-axis representation

A possible choice of such numbers follows from Leonard Euler's observation: Every rotation is defined by two numbers α and β , defining the orientation of a unit vector $\vec{k}$ along the axis of rotation, and the angle of rotation θ :



From this picture we see that

$$\alpha = \text{atan2}(k_y^0, k_x^0), \quad \beta = \text{atan2}\left(\sqrt{(k_x^0)^2 + (k_y^0)^2}, k_z^0\right)$$

where the Matlab-inspired function “**atan2**” stands for the four-quadrant inverse tangent, i.e. $\alpha \in (-\pi, \pi]$ is such that $\tan(\alpha) = k_y^0/k_x^0$.

To derive a formula for the corresponding rotation matrix (an alternative formula have been presented earlier), we can perform rotation by angle θ about an axis defined by $\vec{k}$ as follows. Imagine a change of coordinates, defined by a matrix $\bar{R}^0$ that makes the vector $\vec{k}$ coincide with the frame's z -axis. It follows from our derivations above, that we just need to use the basic rotation around z in combination with the change of coordinates formula. Note that with the angles α and β shown in the figure above, we can use rotations around z -axis by α followed by rotation around the current y -axis by β to define the orientation of $\vec{k}$. As a result one obtains the following product of basic rotation matrices

$$R_{\vec{k}, \theta}^0 = \bar{R}^0 R_{z, \theta}^0 (\bar{R}^0)^{-1}, \quad \bar{R}^0 = R_{z, \alpha} R_{y, \beta}$$

We should note that finding θ and k^0 (or α and β) from a give expres-

sion for $\mathbf{R}_{\vec{k},\theta}^0$ may be not trivial. However, it follows immediately from the Rodrigues's formula for $\mathbf{R}_{\vec{k},\theta}^0$, discussed above,

$$\begin{bmatrix} (k_x^0)^2 & k_x^0 k_y^0 & k_x^0 k_z^0 \\ k_y^0 k_x^0 & (k_y^0)^2 & k_y^0 k_z^0 \\ k_z^0 k_x^0 & k_z^0 k_y^0 & (k_z^0)^2 \end{bmatrix} + c_\theta \begin{bmatrix} 1 - (k_x^0)^2 & -k_x^0 k_y^0 & -k_x^0 k_z^0 \\ -k_y^0 k_x^0 & 1 - (k_y^0)^2 & -k_y^0 k_z^0 \\ -k_z^0 k_x^0 & -k_z^0 k_y^0 & 1 - (k_z^0)^2 \end{bmatrix} + s_\theta \begin{bmatrix} 0 & -k_z^0 & k_y^0 \\ k_z^0 & 0 & -k_x^0 \\ -k_y^0 & k_x^0 & 0 \end{bmatrix},$$

where $c_\theta = \cos(\theta)$ and $s_\theta = \sin(\theta)$, which will not prove here, that

$$\cos(\theta) = \frac{1}{2} \left(\text{trace} \left\{ \mathbf{R}_{\vec{k},\theta}^0 \right\} - 1 \right)$$

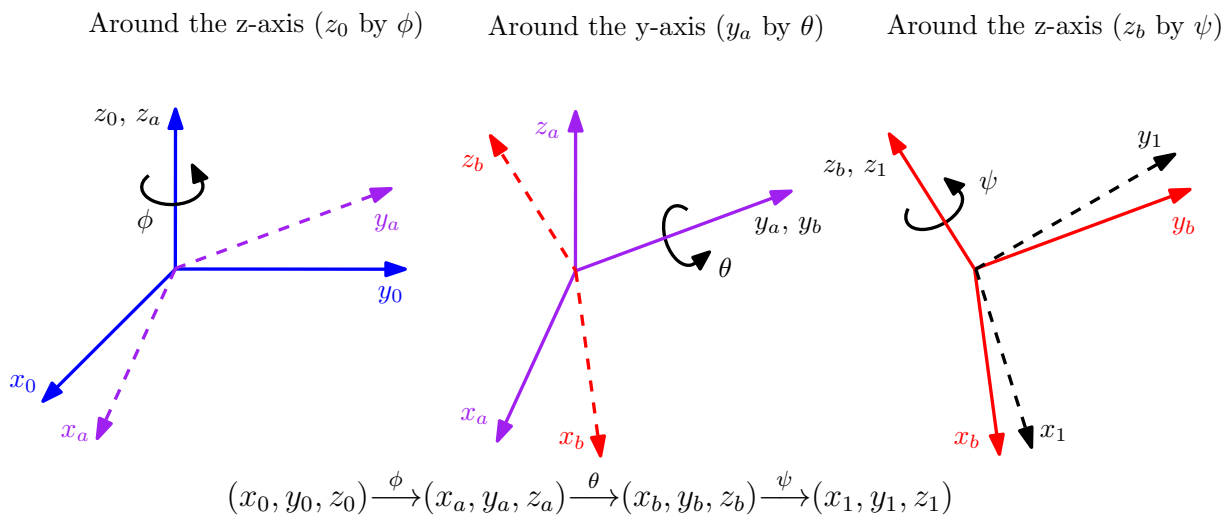
while the coordinates of $\vec{k}$ in $\mathbf{0}$ -frame, $\mathbf{k}^0$, can be computed from

$$\left[\begin{array}{c|c|c} 0 & -k_z^0 & k_y^0 \\ k_z^0 & 0 & -k_x^0 \\ -k_y^0 & k_x^0 & 0 \end{array} \right] = \frac{1}{2 \sin(\theta)} \left(\mathbf{R}_{\vec{k},\theta}^0 - \left(\mathbf{R}_{\vec{k},\theta}^0 \right)^T \right)$$

Let us introduce now other standard parametrizations of rotations in **3D**.

Euler angles

One of the most frequently used one is so called Euler Angles:



Euler **ZYZ**-angles (ϕ , θ , and ψ) specify combination of **3** basic rotations about current axes (z_0 , $y_1 = y_a$, and $z_2 = z_b$), so that

$$\mathbf{R}_{\text{ZYZ}} := \mathbf{R}_{z,\phi} \mathbf{R}_{y,\theta} \mathbf{R}_{z,\psi}$$

According to our previous derivations, we have

$$R_{ZYZ} := \begin{bmatrix} c_\phi & -s_\phi & 0 \\ s_\phi & c_\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix} \cdot \begin{bmatrix} c_\psi & -s_\psi & 0 \\ s_\psi & c_\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

that results in

$$R_{ZYZ} := \begin{bmatrix} c_\phi c_\theta c_\psi - s_\phi s_\psi & -c_\phi c_\theta s_\psi - s_\phi c_\psi & c_\phi s_\theta \\ s_\phi c_\theta c_\psi + c_\phi s_\psi & -s_\phi c_\theta s_\psi + c_\phi c_\psi & s_\phi s_\theta \\ -s_\theta c_\psi & s_\theta s_\psi & c_\theta \end{bmatrix}$$

Let us now solve the inverse problem, i.e. start with an arbitrary representation of a rotation matrix and compute the corresponding Euler angles:

Given the rotation matrix $\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$,

how to find angles ϕ, θ, ψ ?

It is enough to concentrate on the last row and the last column:

$$\mathbf{R} := \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} = \begin{bmatrix} c_\phi c_\theta c_\psi - s_\phi s_\psi & -c_\phi c_\theta s_\psi - s_\phi c_\psi & c_\phi s_\theta \\ s_\phi c_\theta c_\psi + c_\phi s_\psi & -s_\phi c_\theta s_\psi + c_\phi c_\psi & s_\phi s_\theta \\ -s_\theta c_\psi & s_\theta s_\psi & c_\theta \end{bmatrix}$$

We know that

$$\begin{aligned} r_{13}^2 + r_{23}^2 + r_{33}^2 &= 1 \\ r_{31}^2 + r_{32}^2 + r_{33}^2 &= 1 \end{aligned}$$

and it is useful to consider two cases: $|r_{33}| = 1$ and $|r_{33}| \neq 1$.

Case 1: $r_{33} = \pm 1 \Rightarrow r_{13} = r_{23} = r_{31} = r_{32} = 0$

$$r_{33} = 1 \Rightarrow \cos \theta = 1, \quad \sin \theta = 0$$

$$\Rightarrow \begin{bmatrix} r_{11} & r_{12} & 0 \\ r_{21} & r_{22} & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos(\phi + \psi) & -\sin(\phi + \psi) & 0 \\ \sin(\phi + \psi) & \cos(\phi + \psi) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\Rightarrow \theta = 0, \quad \phi + \psi = \text{atan2}(r_{21}, r_{11})$$

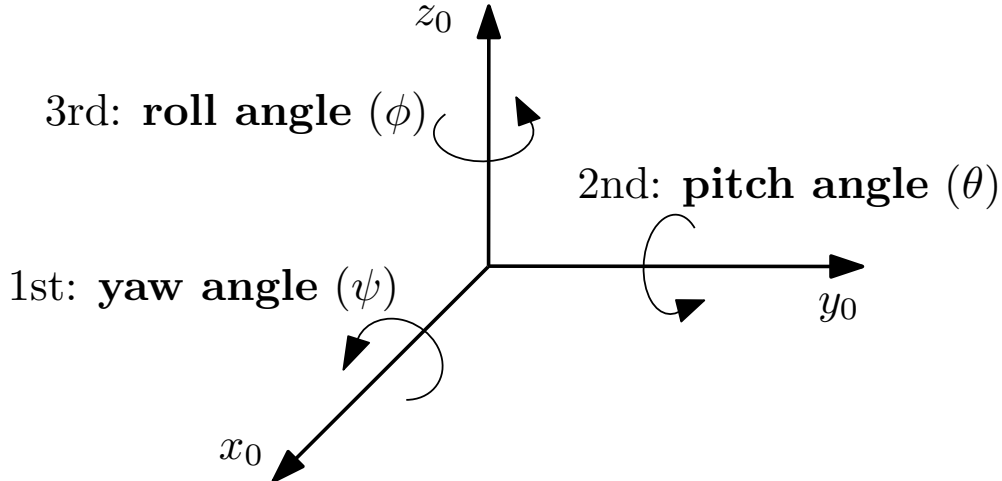
Case 2: $r_{33}^2 < 1 \Rightarrow \{r_{13}^2 + r_{23}^2 \neq 0\}, \{r_{31}^2 + r_{32}^2 \neq 0\}$

$$r_{33} = \cos \theta, \quad \sin \theta = \pm \sqrt{1 - r_{33}^2}$$

$$\phi = \text{atan2}(-\mathbf{r}_{23}, -\mathbf{r}_{13}), \quad \psi = \text{atan2}(\mathbf{r}_{32}, -\mathbf{r}_{31})$$

Roll, Pitch, and Yaw Angles

Let us introduce another representation, that is associated with decomposition into three rotations with respect to the fixed frame:



Here, so called, roll, pitch, and yaw angles are angles of **3** rotations about the fixed axes $\mathbf{x}$, $\mathbf{y}$ and $\mathbf{z}$ and so

$$\mathbf{R}_{xyz} := \mathbf{R}_{z,\phi} \cdot \mathbf{R}_{y,\theta} \cdot \mathbf{R}_{x,\psi}$$

that is

$$\mathbf{R}_{xyz} := \begin{bmatrix} c_\phi & -s_\phi & 0 \\ s_\phi & c_\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\psi & -s_\psi \\ 0 & s_\psi & c_\psi \end{bmatrix}$$

or after performing multiplications

$$\mathbf{R}_{xyz} := \begin{bmatrix} c_\phi c_\theta & -s_\phi c_\psi + c_\phi s_\theta s_\psi & s_\phi s_\psi + c_\phi s_\theta c_\psi \\ s_\phi c_\theta & c_\phi c_\psi + s_\phi s_\theta s_\psi & -c_\phi s_\psi + s_\phi s_\theta c_\psi \\ -s_\theta & c_\theta s_\psi & c_\theta c_\psi \end{bmatrix}$$

It should be noticed that finding the angles from a given matrix representation can be done similarly to the case of Euler angles concentrating on the first row and the first column:

$$\mathbf{R}_{xyz} := \begin{bmatrix} \mathbf{c}_\phi \mathbf{c}_\theta & -s_\phi c_\psi + c_\phi s_\theta s_\psi & s_\phi s_\psi + c_\phi s_\theta c_\psi \\ \mathbf{s}_\phi \mathbf{c}_\theta & c_\phi c_\psi + s_\phi s_\theta s_\psi & -c_\phi s_\psi + s_\phi s_\theta c_\psi \\ -s_\theta & \mathbf{c}_\theta \mathbf{s}_\psi & \mathbf{c}_\theta \mathbf{c}_\psi \end{bmatrix}$$

Let us discuss now the case of representations of motions using frames

that not necessary have coinciding origins.

2.1.5 Homogeneous Transformations

Let us remind that our goal is to have a simple to use mathematical tool (a machinery) for description of motions of rigid bodies representing links of a robotic manipulator. We will have frames attached (systematically assigned) to each link and it is obvious that in a typical situation they not only change relative orientations but also relative positions of the origins. Such changes are called rigid motions and they are introduced next.

Rigid Motions

Given two frames, a rigid motion is an ordered pair $(\mathbf{R}, \vec{d})$, where a rotation tensor $\mathbf{R}$ represented in a particular frame by a rotation matrix from $\mathbf{SO}(3)$ describes relative rotation, while $\vec{d}$ is a vector describing translation that can be associated with the directed displacement between the origins of the two corresponding frames. The group of representations of all rigid motions is known as **Special Euclidean Group** denoted by $\mathbf{SE}(3)$.

In general, as clear from the previous discussions, coordinates of a point $\mathbf{p}$ in **0**-frame and **1**-frame are related as follows

$$\mathbf{p}^0 = \mathbf{R}_1^0 \mathbf{p}^1 + \mathbf{o}_1^0$$

where matrix $\mathbf{R}_1^0$ has been introduced above and $\vec{d}_1 = \overrightarrow{o_0 o_1}$ is defined by $\mathbf{d}_1^0 \equiv \mathbf{o}_1^0$.

Correspondingly, if there are three frames: **0**-frame and **1**-frame are related by the rigid motions $(\mathbf{R}_1, \vec{d}_1)$ while **1**-frame and **2**-frame are related by the rigid motion $(\mathbf{R}_2, \vec{d}_2)$, we have

$$\mathbf{p}^0 = \mathbf{R}_1^0 \mathbf{p}^1 + \mathbf{d}_1^0, \quad \mathbf{p}^1 = \mathbf{R}_2^1 \mathbf{p}^2 + \mathbf{d}_2^1$$

so that the composite rigid motion between **0**-frame and **2**-frame is defined as

$$\mathbf{p}^0 = \mathbf{R}_1^0 \mathbf{R}_2^1 \mathbf{p}^2 + \mathbf{R}_1^0 \mathbf{d}_2^1 + \mathbf{d}_1^0$$

Matrix representation of rigid motions

Homogeneous transformation, which is to be introduced next, is just a more convenient way to rewrite the formula

$$\mathbf{p}^0 = \mathbf{R}_2^0 \mathbf{p}^2 + \mathbf{d}_2^0, \quad \mathbf{R}_2^0 = \mathbf{R}_1^0 \mathbf{R}_2^1, \quad \mathbf{d}_2^0 = \mathbf{R}_1^0 \mathbf{d}_2^1 + \mathbf{d}_1^0$$

as a product of matrices even in the case when all the origins are different.

The following identity can be easily verified

$$\begin{bmatrix} \mathbf{R}_1^0 & \mathbf{d}_1^0 \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R}_2^1 & \mathbf{d}_2^1 \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_1^0 \mathbf{R}_2^1 & \mathbf{R}_1^0 \mathbf{d}_2^1 + \mathbf{d}_1^0 \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \equiv \begin{bmatrix} \mathbf{R}_2^0 & \mathbf{d}_2^0 \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}$$

To use it, for a rigid motion defined in $\mathbf{0}$ -frame by $(\mathbf{R}^0, \mathbf{d}^0) \in \mathcal{SE}(3)$, we introduce the following 4×4 -matrix

$$\mathbf{H}^0 = \begin{bmatrix} \mathbf{R}^0 & \mathbf{d}^0 \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}$$

which is called **homogeneous transformation** associated with $(\mathbf{R}^0, \mathbf{d}^0)$.

Obviously, with such matrices we have composition of two rigid motions represented as a product

$$\mathbf{H}_1^0 \mathbf{H}_2^1 = \mathbf{H}_2^0$$

Now, to directly use homogeneous transformations for computing coordinates of points, we need to extend the coordinates of a point $\mathbf{p}$ in $\mathbf{0}$ -frame, $\mathbf{p}^0$, and in $\mathbf{1}$ -frame, $\mathbf{p}^1$, by the auxiliary fourth coordinate taken as $\mathbf{1}$:

$$\mathbf{P}^0 = \begin{bmatrix} \mathbf{p}^0 \\ 1 \end{bmatrix}, \quad \mathbf{P}^1 = \begin{bmatrix} \mathbf{p}^1 \\ 1 \end{bmatrix}$$

Then,

$$\mathbf{P}^0 = \begin{bmatrix} \mathbf{p}^0 \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_1^0 \mathbf{p}^1 + \mathbf{d}^0 \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{R}_1^0 & \mathbf{d}^0 \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}}_{\mathbf{H}_1^0} \underbrace{\begin{bmatrix} \mathbf{p}^1 \\ 1 \end{bmatrix}}_{\mathbf{P}^1}$$

and the change of coordinates formula

$$\mathbf{p}^0 = \mathbf{R}_1^0 \mathbf{p}^1 + \mathbf{d}^0 \quad \text{becomes} \quad \boxed{\mathbf{P}^0 = \mathbf{H}_1^0 \mathbf{P}^1}$$

or, in more details,

$$\underbrace{\begin{bmatrix} \mathbf{p}^0 \\ 1 \end{bmatrix}}_{\mathbf{P}^0} = \begin{bmatrix} \begin{bmatrix} x_p^0 \\ y_p^0 \\ z_p^0 \\ 1 \end{bmatrix} \\ \begin{bmatrix} (x_1^0)_x & (y_1^0)_x & (z_1^0)_x & (o_1^0)_x \\ (x_1^0)_y & (y_1^0)_y & (z_1^0)_y & (o_1^0)_y \\ (x_1^0)_z & (y_1^0)_z & (z_1^0)_z & (o_1^0)_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_p^1 \\ y_p^1 \\ z_p^1 \\ 1 \end{bmatrix} \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{R}_1^0 & \mathbf{o}_1^0 \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}}_{\mathbf{H}_1^0} \underbrace{\begin{bmatrix} \mathbf{p}^1 \\ 1 \end{bmatrix}}_{\mathbf{P}^1}$$

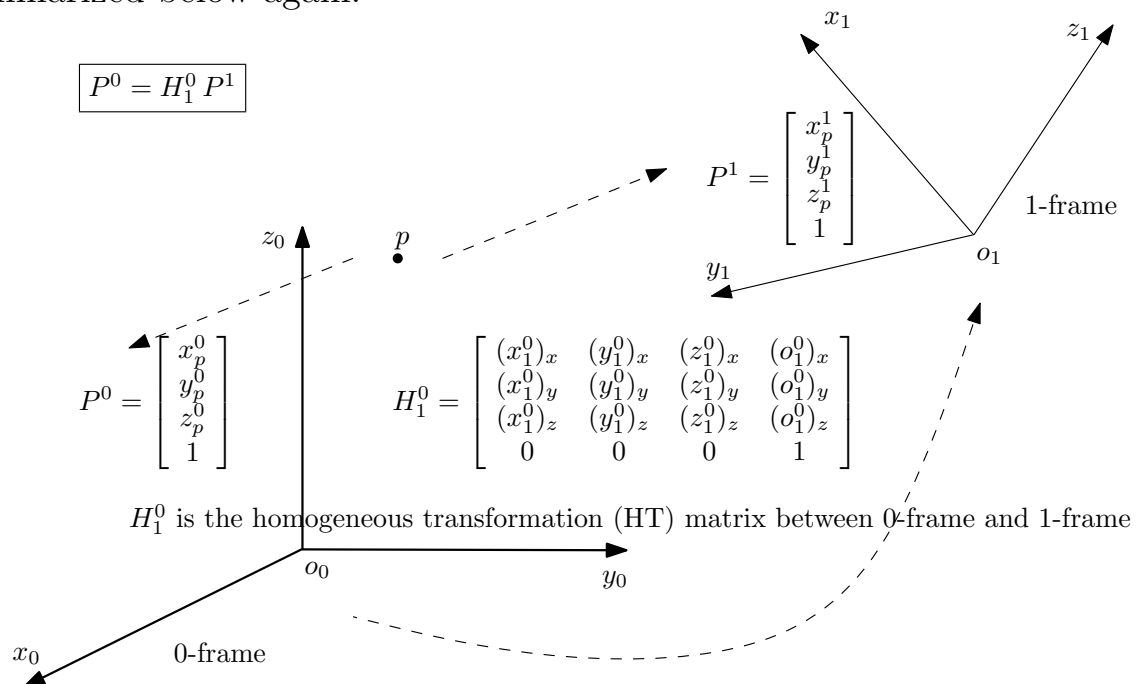
Note that the whole point of introducing an extra auxiliary (dummy) row

above is the following: The formula for a composition of two rigid motions

$$p^0 = R_1^0 R_2^1 p^2 + R_1^0 d_2^1 + d_1^0 \quad \text{becomes} \quad \boxed{P^0 = H_1^0 H_2^1 P^2}$$

and now it can be performed (encoded) using multiplication of two (4×4) matrices.

From now on, we will use homogeneous transformation (HT) matrices to represent relations among various frames and to describe rigid motions. The corresponding change of the coordinates of a point formula above is summarized below again.



2.2 Forward Kinematics

With the mathematical description of rigid motions through homogeneous transformation (HT), we are ready to approach the problems of kinematics in a systematic way.

2.2.1 Kinematic Chains

Let us recall our basic assumptions and terminology:

- A robot manipulator is composed of a set of **links** (rigid bodies) connected together by **joints**;
- Each joint can be either

- a **revolute joint** (a rotation of a link by an angle about an axis fixed on another link) or
- a **prismatic joint** (a displacement of a link along a single axis fixed on another link)

and cannot be

- a more complex **2-DOF** or **3-DOF** joint, which assumed to be represented as a **combinations of the ones above** with introduction of fictitious mass-less and length-less links.
- Each joint connects two links. A robot manipulator with n joints will have $(n + 1)$ links;
- We number joints from **1** to n , and links from **0** to n . So that **joint i connects links $(i - 1)$ and i** ;
- The location of joint i is fixed with respect to the link $(i - 1)$.

For a robotic manipulator with n joints, we attach $n + 1$ -frames. Each i -frame is attached to the i -th link. So, we should have the moving frames

$$(o_0, x_0, y_0, z_0), \quad (o_1, x_1, y_1, z_1), \quad \dots, \quad (o_n, x_n, y_n, z_n)$$

Each point of i -th link will be defined by fixed coordinates in the corresponding i -frame. When manipulator is in motion, coordinates of all the points of various links in the fixed to the base of the robot **0**-frame will be changing. They will be defined by the relative rigid motions of the links.

Relative orientations of the $n + 1$ frames is to be defined by n rotation matrices

$$R_1^0, \quad R_2^1, \quad \dots, \quad R_{(n)}^{(n-1)}$$

that represent consecutive rotation between the current frames:

$$\left\{ \text{from } (x_0 y_0 z_0) \text{ to } (x_1 y_1 z_1) \right\}, \quad \left\{ \text{from } (x_1 y_1 z_1) \text{ to } (x_2 y_2 z_2) \right\}, \quad \dots, \\ \left\{ \text{from } (x_{n-1} y_{n-1} z_{n-1}) \text{ to } (x_n y_n z_n) \right\} \quad \text{respectively,}$$

while relative positions of the origins of the frames should be defined by

$$o_1^0, \quad o_2^1, \quad \dots, \quad o_n^{n-1}$$

giving coordinates of the origin of each frame with respect to the previous one. This information is to be used to define the HT between various couples of frames.

We will use the next basic assumptions and terminology:

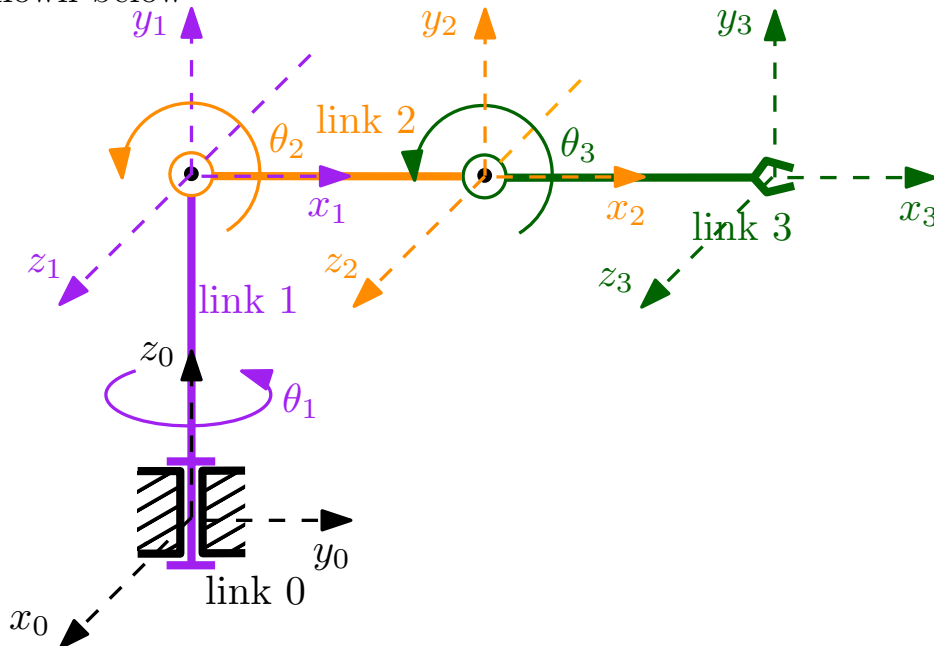
- The link **0** (the base of the robot) is fixed;
- With the i^{th} joint we associate a **joint variable**

$$q_i = \begin{cases} \theta_i & \text{if joint } i \text{ is revolute} \\ d_i & \text{if joint } i \text{ is prismatic} \end{cases}$$

defining the angle of rotation or the linear displacement respectively.

- For each **link i** (with $i = 0, 1, \dots, n$) we have a rigidly attached coordinate frame **$(o_i x_i y_i z_i)$** described above;
- When joint i is actuated, the **link i** and its frame experience a relative **rigid motion** defined by the corresponding HT;
- The frame **$(o_0 x_0 y_0 z_0)$** attached to the base is referred to as the **inertial frame**.

An example of a set of coordinate frames attached to an elbow manipulator is shown below



We will use the next basic assumptions and terminology:

- Suppose $A_i \equiv H_i^{i-1}$ is a homogeneous transformation that gives
 - the relative **position** of the origin and
 - the relative **orientation** of the axes

of the frame **$(o_i x_i y_i z_i)$** with respect to the frame **$(o_{i-1} x_{i-1} y_{i-1} z_{i-1})$** ;

- The HT matrix $A_i \equiv H_i^{i-1} = \begin{bmatrix} R_i^{i-1} & o_i^{i-1} \\ 0 & 1 \end{bmatrix}$ is changing as the configuration of the robot is changing;

- $\mathbf{A}_i$ is the function of a scalar variable q_i , which is the joint variable, i.e. $\mathbf{A}_i = \mathbf{A}_i(q_i)$;
- Homogeneous transformation that expresses the position and orientation of $(o_j x_j y_j z_j)$ with respect to $(o_i x_i y_i z_i)$, i.e. $\mathbf{H}_j^i$, can be computed as follows

$$\mathbf{H}_j^i = \begin{cases} \mathbf{A}_{i+1} \mathbf{A}_{i+2} \dots \mathbf{A}_{j-1} \mathbf{A}_j, & \text{if } i < j \\ \mathbf{I}, & \text{if } i = j \end{cases}, \quad \mathbf{H}_j^i = \left(\mathbf{H}_i^j \right)^{-1}, \text{ if } i > j$$

and is called a **transformation matrix**.

To solve the forward kinematics problem, we need to compute the position and the orientation of the end-effector with respect to the inertia frame, i.e.

$$\mathbf{o}_n^0 \quad \text{and} \quad \mathbf{R}_n^0$$

Obviously, they can be extracted from the homogeneous transformation

$$\mathbf{H}_n^0 = \mathbf{A}_1(q_1) \mathbf{A}_2(q_2) \dots \mathbf{A}_{n-1}(q_{n-1}) \mathbf{A}_n(q_n) = \begin{bmatrix} \mathbf{R}_n^0 & \mathbf{o}_n^0 \\ \mathbf{0} & \mathbf{1} \end{bmatrix}$$

Recall that for $j > i$ we have

$$\mathbf{H}_j^i = \mathbf{A}_{i+1} \mathbf{A}_{i+2} \dots \mathbf{A}_{j-1} \mathbf{A}_j = \begin{bmatrix} \mathbf{R}_j^i & \mathbf{o}_j^i \\ \mathbf{0} & \mathbf{1} \end{bmatrix}$$

where the following recursive definition can be verified

$$\mathbf{R}_j^i = \mathbf{R}_{i+1}^i \dots \mathbf{R}_j^{j-1}, \quad \mathbf{o}_j^i = \mathbf{o}_{j-1}^i + \mathbf{R}_{j-1}^i \mathbf{o}_j^{j-1} \quad \text{for } j > i + 1$$

In order to solve the forward kinematics problem, one can start by assigning frames to all the links. This can be done in a variety of ways. However, arbitrary assignment will lead to the need of introducing the transformation between the frames depending on the joint variable in a complex way. This is not convenient and may lead to unnecessarily lengthy calculations. To avoid such complexity and to standardize the frame assignment, a convention introduced next is commonly used in robotics.

2.2.2 The Denavit-Hartenberg (DH) Convention

First of all, it should be noticed that a transformation between two frames can be defined by 6 parameters (3 for the relative position of the origin and

3 for the relative orientation of the axes). However, the necessary number of parameters may be reduced using specific relation between the consecutive frames if they are assigned in a certain systematic way.

Second of all, it seems useful to have a procedure for frame assignment that ensures that:

- one of the parameters is the joint variable $\mathbf{q}$ (the angle θ for every revolute joint and the position $\mathbf{d}$ for every prismatic joint),
- the other parameters are constant,
- the HT between each pair of frames is computed in a unified systematic way (for example as a product of basic rotations or translations),
- the number of parameters is as small as possible.

From these desirable requirements, it appears reasonable to have one axis of each frame, say $\mathbf{z}$, to be directed along the action of rotation or translation. In this case, two of the basic HT should be the rotation around $\mathbf{z}$ by θ (to be denoted as $\text{Rot}_{\mathbf{z},\theta_i}$) and the translation along $\mathbf{z}$ by $\mathbf{d}$ (to be denoted as $\text{Trans}_{\mathbf{z},\mathbf{d}_i}$). How many other basic HT we would need to include into a product representation and which one we should choose?

It can be shown that, using the structure of kinematic chains, it is possible to define transformations between the frames by just **4** parameters: θ_i , $\mathbf{d}_i$, $\mathbf{a}_i$, and α_i and to represent each HT matrix $\mathbf{A}_i$ as a product of the following **4** special elementary HT matrices (with $\mathbf{q}_i = \theta_i$ or $\mathbf{q}_i = \mathbf{d}_i$)

$$\mathbf{A}_i \equiv \mathbf{H}_i^{i-1}(\mathbf{q}_i) = \text{Rot}(\mathbf{z}, \theta_i) \text{Trans}(\mathbf{z}, \mathbf{d}_i) \text{Trans}(\mathbf{x}, \mathbf{a}_i) \text{Rot}(\mathbf{x}, \alpha_i)$$

$$\mathbf{A}_i = \left[\begin{array}{ccc|c} c_{\theta_i} & -s_{\theta_i} & 0 & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array} \right] \left[\begin{array}{ccc|c} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \mathbf{d}_i \\ 0 & 0 & 0 & 1 \end{array} \right] \left[\begin{array}{ccc|c} 1 & 0 & 0 & \mathbf{a}_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array} \right] \left[\begin{array}{ccc|c} 1 & 0 & 0 & 0 \\ 0 & c_{\alpha_i} & -s_{\alpha_i} & 0 \\ 0 & s_{\alpha_i} & c_{\alpha_i} & 0 \\ 0 & 0 & 0 & 1 \end{array} \right]$$

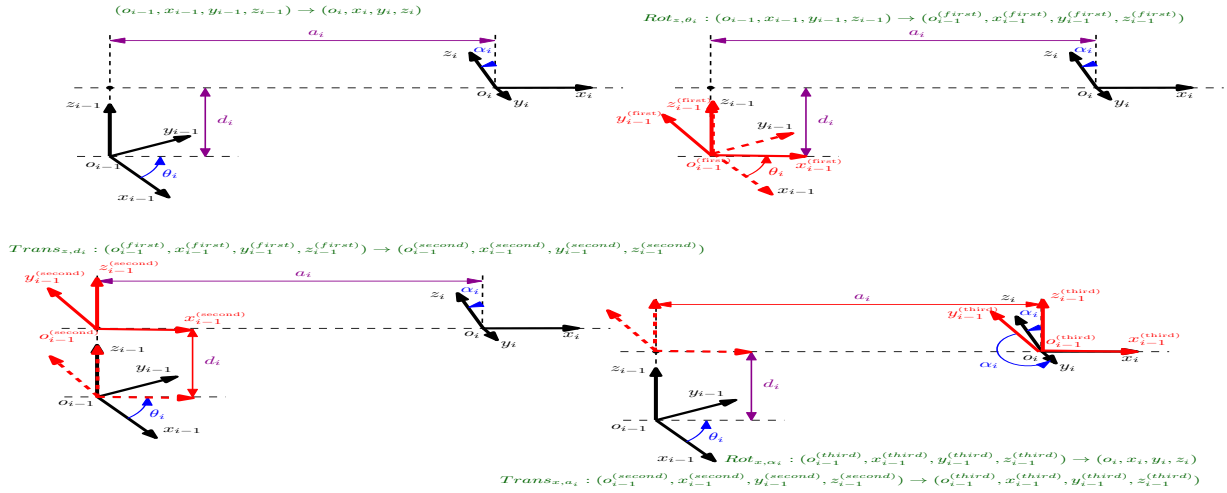
parameters of which are defined next.

The product is

$$\mathbf{A}_i = \left[\begin{array}{ccc|c} \cos(\theta_i) & -\sin(\theta_i) \cos(\alpha_i) & \sin(\theta_i) \sin(\alpha_i) & \mathbf{a}_i \cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i) \cos(\alpha_i) & -\cos(\theta_i) \sin(\alpha_i) & \mathbf{a}_i \sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & \mathbf{d}_i \\ 0 & 0 & 0 & 1 \end{array} \right]$$

Essentially, we are also to formulate **2** conditions on the pairwise relation between the assigned frames to remove the need to perform **2** actions when

changing between coordinates expressed in them: translation along and rotation around the current $\mathbf{y}$ -axis. The sequence of transformations from frame $i - 1$ to frame i is illustrated below.



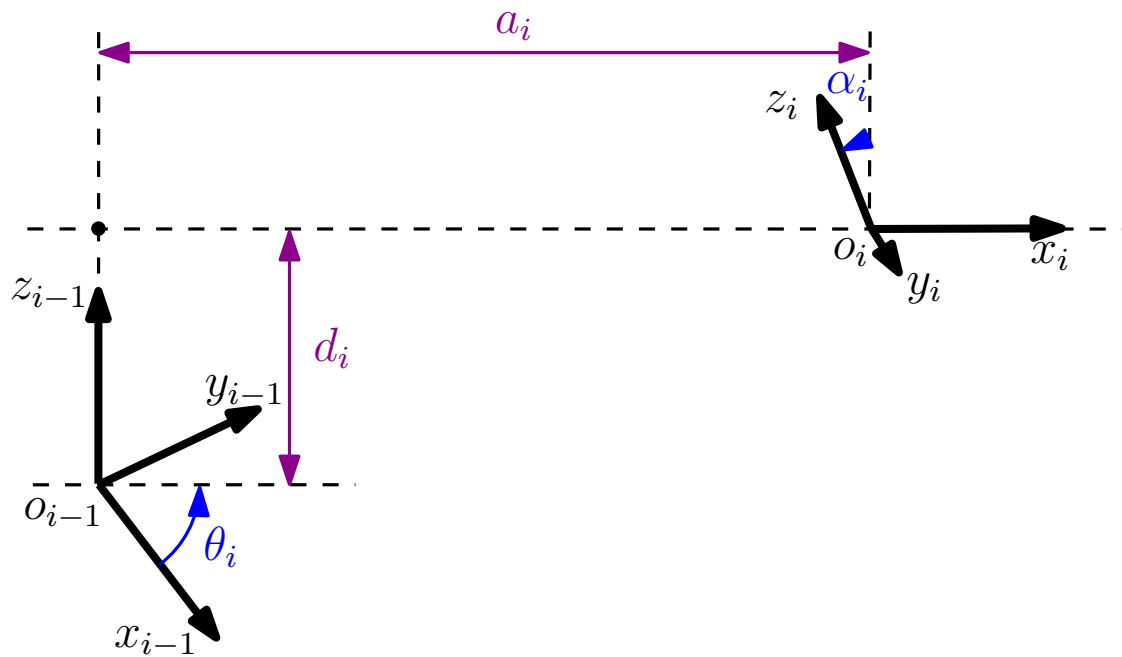
DH parameters and DH conditions

The 4 parameters of the DH transformation **from frame $i - 1$ to frame i** , for a particular assignment of the axes to be discussed shortly, are known as

- θ_i : the **link angle** (that is constant for prismatic joints), which is the constant angle between the unit vectors $\vec{x}_{i-1}$ and $\vec{x}_i$, i.e. it is the angle of the **rotation around $\vec{z}_{i-1}$** (the first transformation: Rot_{z, θ_i});
- d_i : the **link offset** (that is constant for revolute joints), which is the constant distance from the previous origin $\mathbf{o}_{i-1}$ to the current axis $\mathbf{x}_i$, i.e. it is the **displacement along $\vec{z}_{i-1}$** (the second: Trans_{z, d_i});
- a_i : the constant **link length**, which is the constant distance from the current origin $\mathbf{o}_i$ to the action axis of the previous frame $\mathbf{z}_{i-1}$, i.e. it is the **displacement along $\vec{x}_i$** (the third: Trans_{x, a_i});
- α_i : the constant **link twist**, which is the constant angle between the unit vectors defining the previous and current action axes, i.e. $\vec{z}_{i-1}$ and $\vec{z}_i$, i.e. it is the angle of the **rotation around $\vec{x}_i$** (the last: Rot_{x, α_i}).

See also <https://www.youtube.com/watch?v=rA9tm0gTln8>.

The parameters $a_i, \alpha_i, d_i, \theta_i$ are identified on the next picture together with the two conditions on the way the frames $i - 1$ and i are assigned to allow reduction of the number of parameters by 2.



The two conditions are the following:

DH1: The axis $\mathbf{x}_i$ is perpendicular to the axis $\mathbf{z}_{i-1}$

DH2: The axis $\mathbf{x}_i$ intersects the axis $\mathbf{z}_{i-1}$

Interestingly enough, these two conditions, can be satisfied for every open kinematic chain.

Assigning Frames Following the DH Convention

Given a robot manipulator with

- $\mathbf{n}$ revolute and/or prismatic joints and, correspondingly,
- $(\mathbf{n} + 1)$ links, numbered sequentially from $\mathbf{0}$ to $\mathbf{n} + 1$, starting from the inertial base link and up to the last one holding the end-effector.

We describe below the commonly used **procedure** for systematically assigning coordinate frames to each link that ensures that the transformation between the frames can be written using the DH convention.

The algorithm of assigning $(\mathbf{n} + 1)$ frames for $(\mathbf{n} + 1)$ links

- treats separately first $\mathbf{n}$ -frames and the last one (the end-effector frame),
- is recursive in first part, so that it is generic.

The $\mathbf{i}^{th}$ frame, i.e. the origin $\mathbf{o}_i$ and the three mutually orthogonal axes $\mathbf{x}_i$, $\mathbf{y}_i$, and $\mathbf{z}_i$, are to be attached to the $\mathbf{i}^{th}$ link and move together with it completely defining the motion of link $\mathbf{i}$ relative to link $\mathbf{i} - 1$.

Assigning First n -Frames ($i = 0, 1, \dots, n - 1$) can be done in the next three steps.

Step 1 (Choice of z -axes):

- Choose **z_0 -axis** along the **actuation line of the 1st-link**;
- Choose **z_1 -axis** along the **actuation line of the 2nd-link**;
- ...
- Choose **z_{n-1} -axis** along the **actuation line of the n^{th} -link**;
- Leave **z_n -axis** not assigned for now.

After this step is done, it is left to assign:

- A **point** on each of z_i -axis that will be **the origin o_i** of the i^{th} -frame for $i = 1, \dots, n - 1$ (Step 2),
- An **x_i -axis** for each frame (Step 2) so that the two DH-conditions hold, i.e. so that
 - DH1:** The axis x_i is perpendicular to the axis z_{i-1} ,
 - DH2:** The axis x_i intersects the axis z_{i-1} .
- An **y_i -axis** for each frame to have them right-oriented (Step 3)
- The last n^{th} (tool) frame in a convenient way.

Step 2 (Choice of x -axes and the origins): The axis x_0 and the origin of the base frame (o_0) are chosen arbitrary. For the other x -axes ($i = 1, 2, \dots, n - 1$) this step is done recursively as follows.

- Suppose that we have chosen the $(i - 1)^{\text{th}}$ -frame and need to proceed with the i^{th} -frame.
- Note that for the i^{th} -frame, the z_i axis is already fixed.
- We need to ensure that:
 - (1) **x_i intersects z_{i-1}** ,
 - (2) **$x_i \perp z_{i-1}$** ,
 - (3) **$x_i \perp z_i$** .

There are **3** possible cases:

- (a) z_i and z_{i-1} are not coplanar, i.e. there is no plane that contain both axis

(b) $\mathbf{z}_i$ and $\mathbf{z}_{i-1}$ are parallel,

(c) $\mathbf{z}_i$ and $\mathbf{z}_{i-1}$ intersect,

that should be treated differently, as described below.

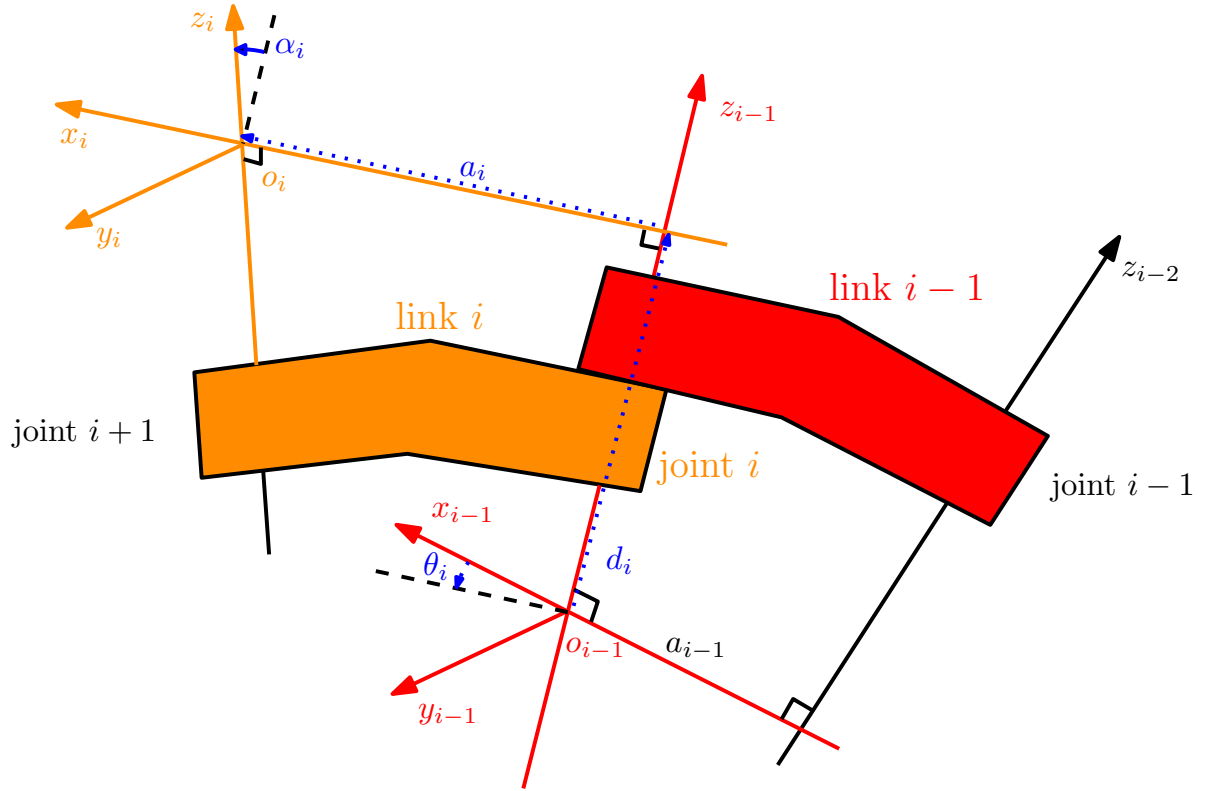
- (a) **If $\mathbf{z}_i$ and $\mathbf{z}_{i-1}$ are not coplanar,** then there is the unique common perpendicular (direction of which is defined by $\vec{\mathbf{z}}_i \times \vec{\mathbf{z}}_{i-1}$) for both lines! Let us assign the $\mathbf{x}_i$ -axis along this common perpendicular and the origin $\mathbf{o}_i$ for the i^{th} -frame at the point of intersection of $\mathbf{z}_i$ and $\mathbf{x}_i$.
- (b) **If $\mathbf{z}_i$ and $\mathbf{z}_{i-1}$ are parallel,** then there are many common perpendiculars for both lines! We can choose any one of them to define the origin $\mathbf{o}_i$ and the $\mathbf{x}_i$ -axis for the i^{th} -frame.
- (c) **If $\mathbf{z}_i$ and $\mathbf{z}_{i-1}$ intersect,** then there is a vector orthogonal to the plane formed by $\mathbf{z}_i$ and $\mathbf{z}_{i-1}$! Let us take the point of intersection to be the new origin $\mathbf{o}_i$, while the $\mathbf{x}_i$ -axis should follow this common perpendicular (defined by $\vec{\mathbf{z}}_i \times \vec{\mathbf{z}}_{i-1}$).

Note that, whenever possible, we just take either $\vec{\mathbf{x}}_i = \vec{\mathbf{z}}_i \times \vec{\mathbf{z}}_{i-1}$ or $\vec{\mathbf{x}}_i = \vec{\mathbf{z}}_{i-1} \times \vec{\mathbf{z}}_i$ to ensure (2) and (3) above.

Step 3 (Choice of $\mathbf{y}$ -axes):

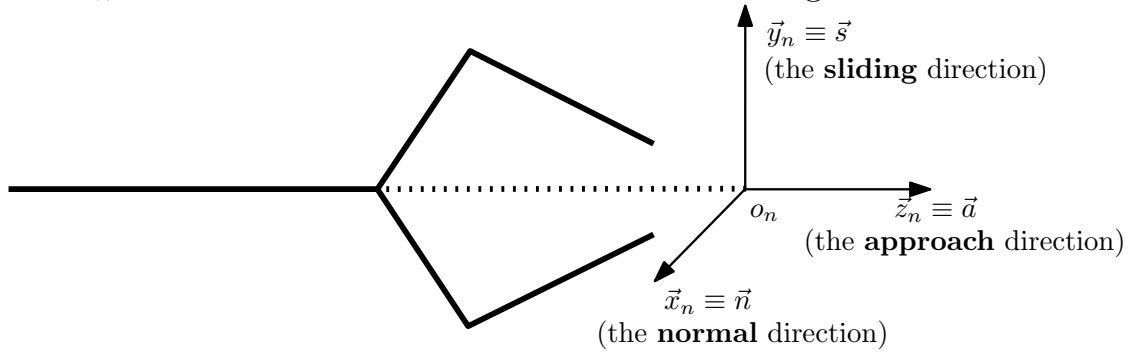
Suppose we have already chosen the axes $\mathbf{z}_i$ and $\mathbf{x}_i$ as well as the point $\mathbf{o}_i$ for the i^{th} -frame. Then, we have the only possibility to obtain the right-handed coordinate system (i.e. the system with $\vec{\mathbf{x}}_i \times \vec{\mathbf{y}}_i = \vec{\mathbf{z}}_i$, $\vec{\mathbf{y}}_i \times \vec{\mathbf{z}}_i = \vec{\mathbf{x}}_i$, and $\vec{\mathbf{z}}_i \times \vec{\mathbf{x}}_i = \vec{\mathbf{y}}_i$). We simply take $\mathbf{y}_i$ along $\vec{\mathbf{y}}_i$ defined by the right-hand rule of the cross-product operation: **$\vec{\mathbf{y}}_i = \vec{\mathbf{z}}_i \times \vec{\mathbf{x}}_i$.**

Note that as soon as all the frames are assigned, we can compute the DH parameters as described earlier and illustrated below.



Note, however, that we cannot use the procedure above for the last end-effector frame since we can not start assigning z_n along the actuation line of the not existing next link.

Assigning the Last Frame ($i = n$) for the End-Effector. For most robots it is common to take the z_n axis coinciding with the z_{n-1} axis, while y_n and x_n axes can be taken as shown in the next figure



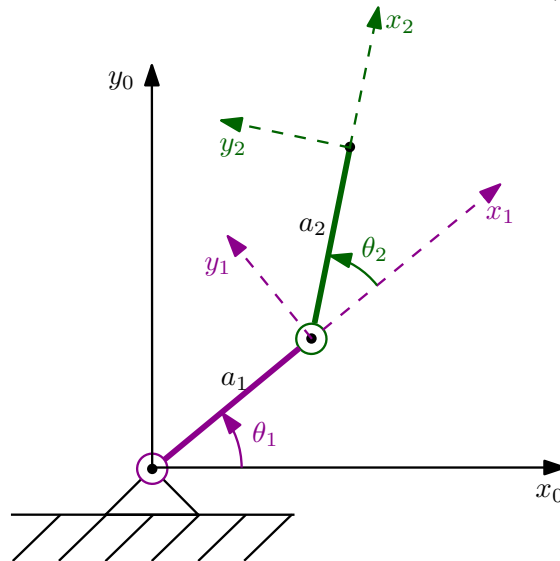
Then, the transformation between the $(n - 1)$ -frame and the n -frame corresponds to

- translation by d_n along z_{n-1} -axis and
- rotation by θ_n about z_n -axis.

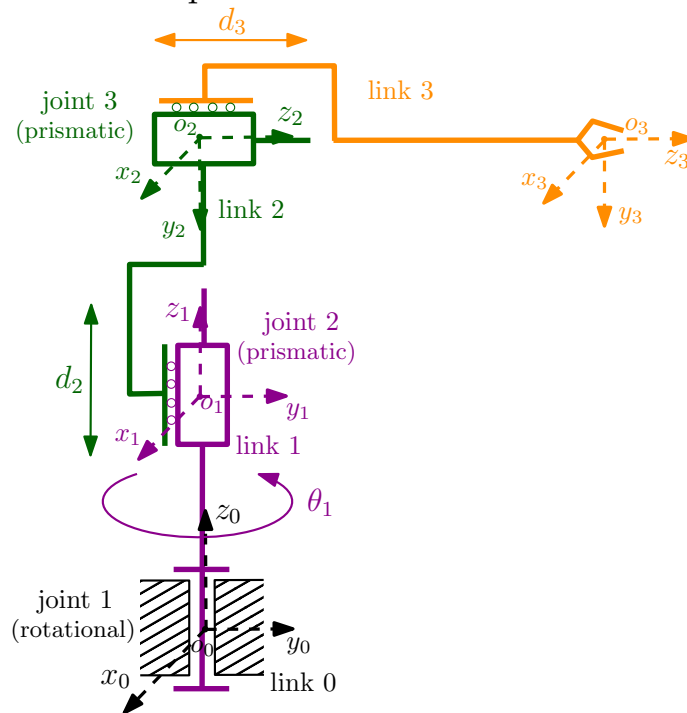
In general, whenever possible, one should make a choice that would lead to **simplifying the description of the transition between the frames**, i.e. if the actuation axis are parallel, their directions should be chosen identical, i.e. $\vec{z}_i = \vec{z}_{i-1}$, and if it is possible, without violating the DH conditions, one should assign $\vec{x}_i = \vec{x}_{i-1}$.

2.2.3 Examples of using DH convention

An example of the result of assigning the frames following our procedure for the planar two-link manipulator, considered in the introduction, is shown below (the axes z_1 and z_2 are orthogonal to the plane).



An example of the result of assigning the frames following our procedure for a **3**-link cylindrical manipulator is shown below.



From the picture above, one can extract the following set of

DH-parameters:

Link	θ_i	d_i	a_i	α_i
1	θ_1	d_1	0	0
2	0	d_2	0	$-\frac{\pi}{2}$
3	0	d_3	0	0

As soon as we know the DH parameters, we can proceed using the formu-

lae above for computation of the HT matrices relating the frames. For the **3**-link cylindrical manipulator we obtain:

$$\begin{aligned} \mathbf{A}_1 &= \left[\begin{array}{ccc|c} \cos \theta_1 & -\sin \theta_1 & 0 & 0 \\ \sin \theta_1 & \cos \theta_1 & 0 & 0 \\ 0 & 0 & 1 & d_1 \\ 0 & 0 & 0 & 1 \end{array} \right], & \mathbf{A}_3 &= \left[\begin{array}{ccc|c} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_3 \\ 0 & 0 & 0 & 1 \end{array} \right], \\ \mathbf{A}_2 &= \left[\begin{array}{ccc|c} 1 & 0 & 0 & 0 \\ 0 & \cos(-\frac{\pi}{2}) & -\sin(-\frac{\pi}{2}) & 0 \\ 0 & \sin(-\frac{\pi}{2}) & \cos(-\frac{\pi}{2}) & d_2 \\ 0 & 0 & 0 & 1 \end{array} \right], \end{aligned}$$

and finally,

$$\mathbf{T}_3^0 = \mathbf{A}_1 \mathbf{A}_2 \mathbf{A}_3.$$

In analogous way, one can assign frames for any manipulator and compute the transformations among the frames. As soon as this is done, we have a solution of the forward kinematics problem.

2.3 Inverse Kinematics

Let us consider now the inverse problem.

2.3.1 Problem Formulation

Given a 4×4 matrix of homogeneous transformation

$$\mathbf{H} = \left[\begin{array}{c|c} \mathbf{R} & \mathbf{o} \\ \hline \mathbf{0}_{3 \times 1} & 1 \end{array} \right] \in \mathcal{SE}(3)$$

obtained as a solution of the forward kinematics problem (i.e. $\mathbf{R} = \mathbf{R}_n^0$ and $\mathbf{o} = \mathbf{o}_n^0$), our goal is to find a solution (possibly one of many) of the equation

$$\mathbf{H}_n^0(\mathbf{q}_1, \dots, \mathbf{q}_n) = \mathbf{A}_1(\mathbf{q}_1) \mathbf{A}_2(\mathbf{q}_2) \cdots \mathbf{A}_n(\mathbf{q}_n) = \mathbf{H}$$

So, we have **12** equations with respect to n variables: $\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_n$. However, these equations are not independent since **9** entries of the rotation matrix satisfy certain equations and can be defined by just **3** independent variables, such as Euler angles. Hence, if $n < 6$ either not every target position or not every orientation can be achieved; while if $n > 6$ there may be several solutions for particular target position and orientation.

Note that it is often advantageous to find a solution of

$$\mathbf{H}_n^0(\mathbf{q}_1, \dots, \mathbf{q}_n) = \mathbf{H} = \left[\begin{array}{ccc|c} h_{11} & h_{12} & h_{13} & h_{14} \\ h_{21} & h_{22} & h_{23} & h_{24} \\ h_{31} & h_{32} & h_{33} & h_{34} \\ \hline 0 & 0 & 0 & 1 \end{array} \right]$$

in an analytical form

$$\mathbf{q}_k = \mathbf{f}_k(h_{11}, h_{12}, \dots, h_{34}), \quad k = 1, \dots, n$$

since they should be often computed as fast as possible.

In fact, $\mathbf{H}$, as a function of time, defines the required for a particular task position of the end-effector, specified by $\mathbf{o}$, and the orientation of the frame attached to the end-effector, specified by $\mathbf{R}$. Therefore, the computed from the solution $\mathbf{q}_k(t)$ become references to be enforced by actuators controlling the joint variables. Therefore, if there are several solutions, then an on-line numerical procedure may result in switching among different solutions and consequently in **possible jumps in reference signals** that lead to a challenging task for control design since no control action can result in discontinuous trajectories.

However one should keep in mind that, although highly desirable, finding an analytic solution is not always possible. An important situation when this can be done is considered next.

2.3.2 Kinematic Decoupling

Sometimes, it is possible to separate the joint variables that are responsible for defining the orientation of the end-effector, i.e. $\mathbf{R} \in \mathbf{SO}(3)$, and the ones that are responsible for defining the position of the end-effector, i.e. $\mathbf{o} \in \mathbf{R}^3$.

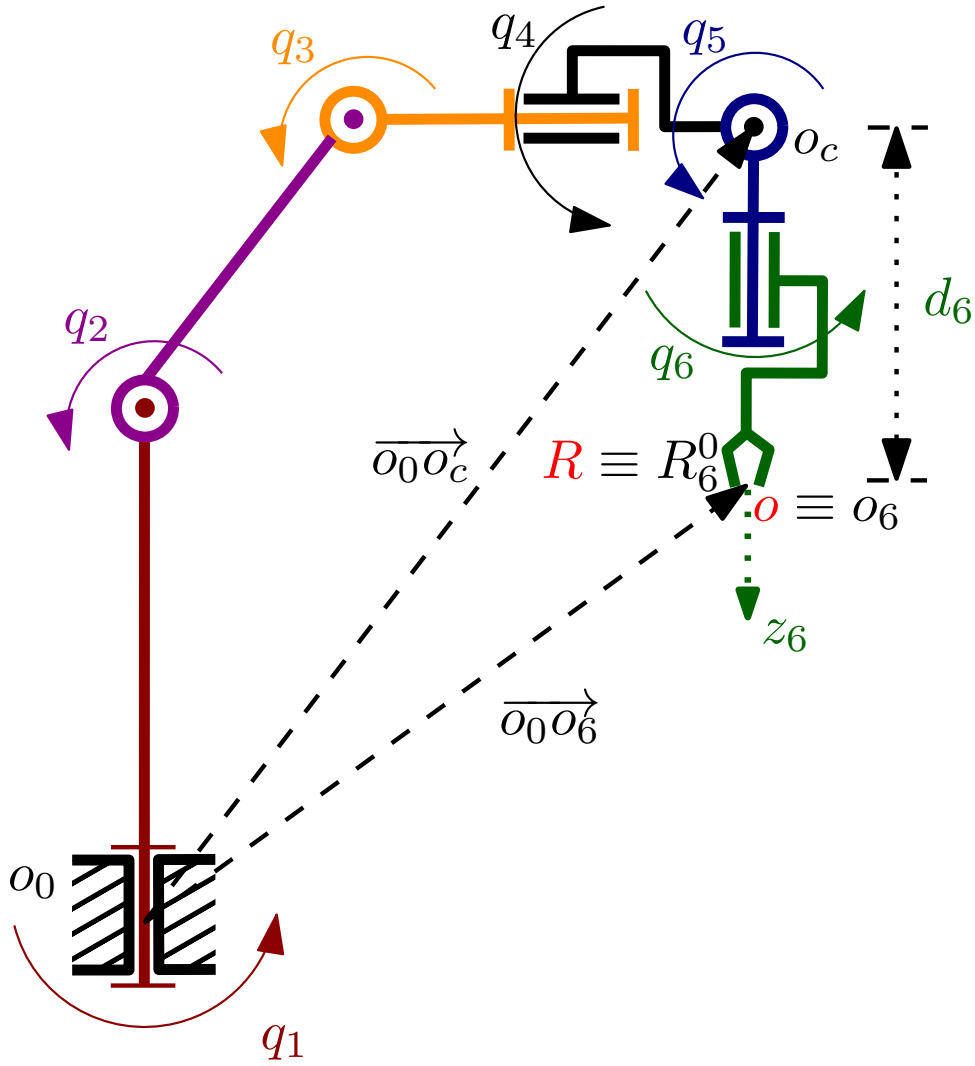
Suppose our manipulator has

- at least **6** joints (otherwise, either not all orientations or not all positions of the tool would be possible),
- the last **3** joint axes intersecting in one point (this is not a rear case).

then, the problem is decoupled into two sub-problems:

- inverse **position** kinematics and
- inverse **orientation** kinematics.

For an illustration, let us consider a **6**-DOF articulated manipulator with last **3** axis of rotation intersecting at the single point $\mathbf{o}_c$ as shown in the scheme below



Here, (although it is not clear from the picture) the first **3** rotational joints correspond to an anthropomorphic arm with $\mathbf{a}_1 = \alpha_2 = \alpha_3 = \mathbf{d}_2 = \mathbf{d}_3 = \mathbf{0}$, $\alpha_1 = \pi/2$ and the last **3** rotational joints model a spherical wrist with $\mathbf{a}_4 = \mathbf{a}_5 = \mathbf{a}_6 = \alpha_6 = \mathbf{d}_4 = \mathbf{d}_5 = \mathbf{0}$, $\alpha_4 = -\alpha_5 = -\pi/2$.

Let us show that we can formulate the inverse position kinematics based on computation of $\mathbf{o}_c^0$ and $\mathbf{R}_3^0$ and inverse orientation kinematics based on computation of $\mathbf{R}_6^3$.

Since the last three joint axes intersect in one point, we have

$$\mathbf{o} = \mathbf{o}_c^0 + d_6 \mathbf{z}_6^0 = \mathbf{o}_c^0 + d_6 \mathbf{R} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \mathbf{o}_c^0 + d_6 \begin{bmatrix} \mathbf{r}_{11} & \mathbf{r}_{12} & \mathbf{r}_{13} \\ \mathbf{r}_{21} & \mathbf{r}_{22} & \mathbf{r}_{23} \\ \mathbf{r}_{31} & \mathbf{r}_{32} & \mathbf{r}_{33} \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

where $\mathbf{o} = \mathbf{o}_6^0$, $\mathbf{R} = \mathbf{R}_6^0$, $\mathbf{o}_c^0 = \mathbf{o}_3^0$, and $\mathbf{z}_6^0 = \mathbf{R} \mathbf{z}_6^6 = [\mathbf{r}_{13}, \mathbf{r}_{23}, \mathbf{r}_{33}]^T$ is computed from $\mathbf{z}_6^6 = [0, 0, 1]^T$.

As a result, from given $\mathbf{o}$ and $\mathbf{R}$ one can compute $\mathbf{o}_c^0$

$$\mathbf{o}_c^0 = \begin{bmatrix} \mathbf{x}_c \\ \mathbf{y}_c \\ \mathbf{z}_c \end{bmatrix} = \mathbf{o} - d_6 \mathbf{R} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{o}_x - d_6 \mathbf{r}_{13} \\ \mathbf{o}_y - d_6 \mathbf{r}_{23} \\ \mathbf{o}_z - d_6 \mathbf{r}_{33} \end{bmatrix}$$

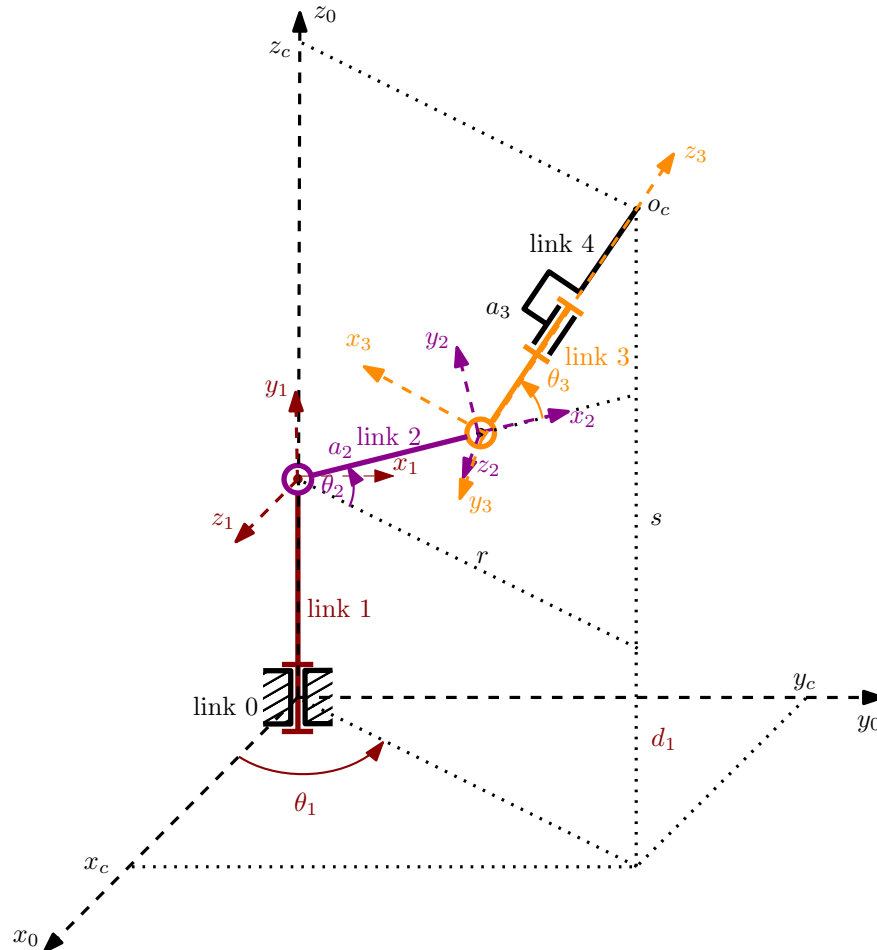
and formulate the **inverse position kinematics problem** as finding the three angles of the first three joints θ_1 , θ_2 , and θ_3 that produce the just computed values of $\mathbf{x}_c$, $\mathbf{y}_c$, and $\mathbf{z}_c$.

As soon as the inverse position kinematics problem is solved, we have a computed value for $\mathbf{R}_3^0$. Now, since

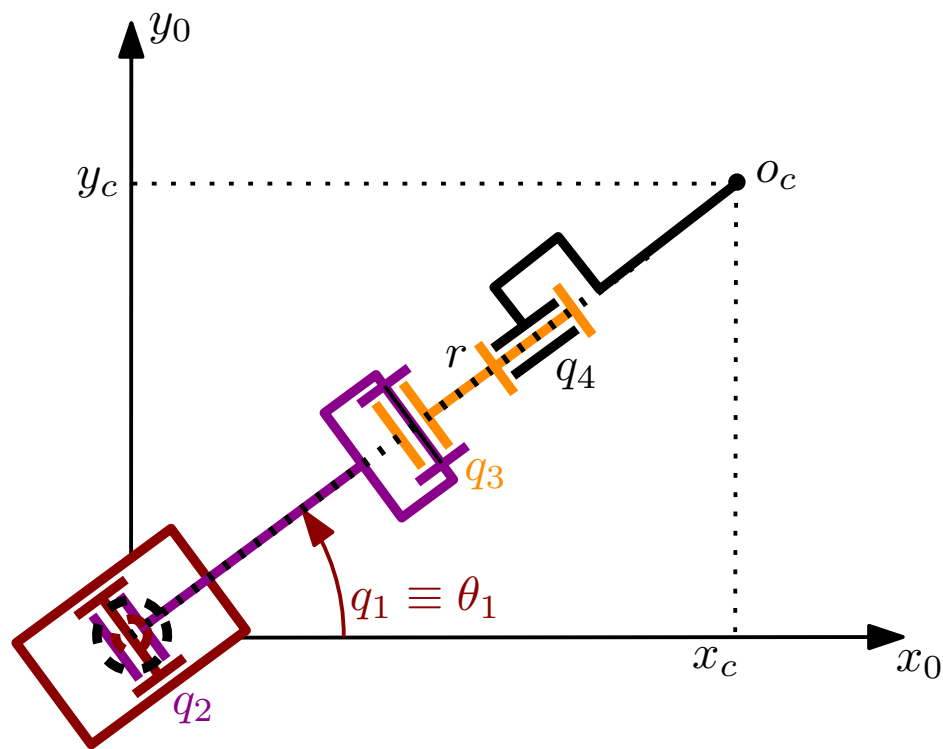
$$\mathbf{R} = \mathbf{R}_3^0 \cdot \mathbf{R}_6^3 \Rightarrow \mathbf{R}_6^3 = [\mathbf{R}_3^0]^{-1} \mathbf{R} = [\mathbf{R}_3^0]^T \mathbf{R}$$

we can formulate the **inverse orientation kinematics problem** as finding the three angles of the last three joints θ_4 , θ_5 , and θ_6 that result in just computed values of $\mathbf{R}_6^3$.

The following picture helps to find a solution of the first problem, i.e. of computing θ_1 , θ_2 , and θ_3 from $\mathbf{x}_c$, $\mathbf{y}_c$, and $\mathbf{z}_c$:



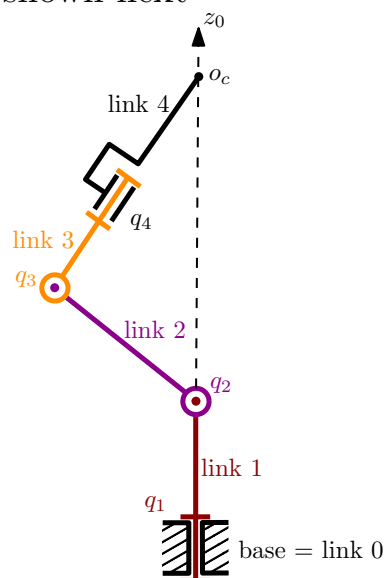
Now, looking at the projection onto the $(\mathbf{x}_c, \mathbf{y}_c)$ -plane, we see how to compute θ_1 :



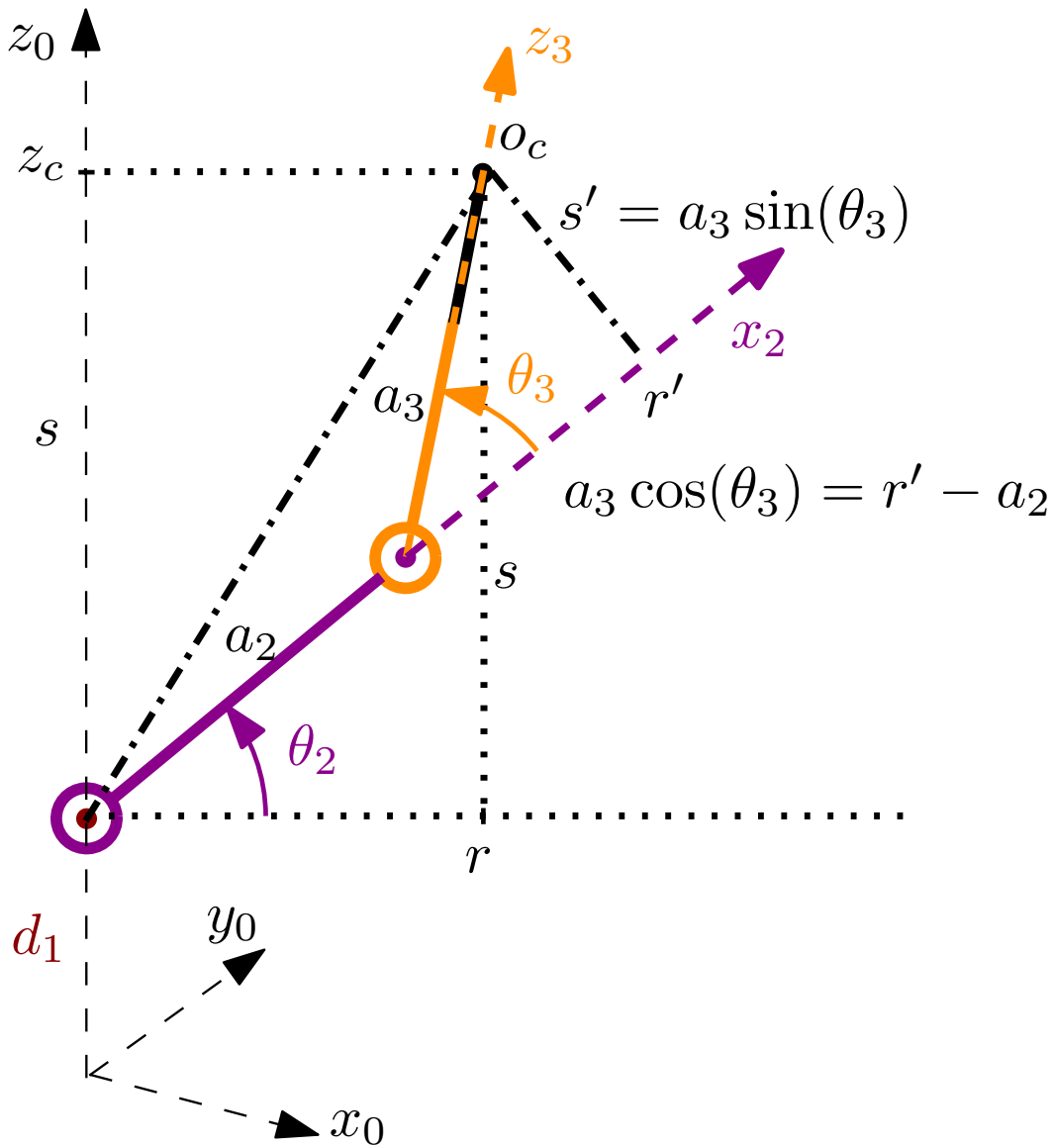
Obviously,

$$\theta_1 = \text{atan2}(x_c, y_c)$$

provided $(x_c, y_c) \neq (0, 0)$, i.e. provided the robot is not in a singular configuration like the one shown next



Now, in order to compute θ_2 and θ_3 we should look at the projection onto the appropriate vertical plane



This is the plane formed by the 2nd and the 3rd links. Now, it is not hard to see that

$$\cos \theta_3 = \frac{(r^2 + s^2) - a_2^2 - a_3^2}{2a_2a_3}$$

where $s = z_c - d_1$ and $r = \sqrt{x_c^2 + y_c^2}$. Hence,

$$\cos \theta_3 = \frac{x_c^2 + y_c^2 + (z_c - d_1)^2 - a_2^2 - a_3^2}{2a_2a_3}$$

Other triangles help as to realize that as soon as θ_3 is found, we can easily compute θ_2 :

$$\theta_2 = \text{atan2}(\sqrt{x_c^2 + y_c^2}, z_c - d_1) - \text{atan2}(a_2 + a_3 \cos \theta_3, a_3 \sin \theta_3).$$

This completes finding an analytical solution for the inverse position kinematics.

With the computed values of the first three angles at hand, we use the DH parameters of the links to compute the corresponding **3** HT:

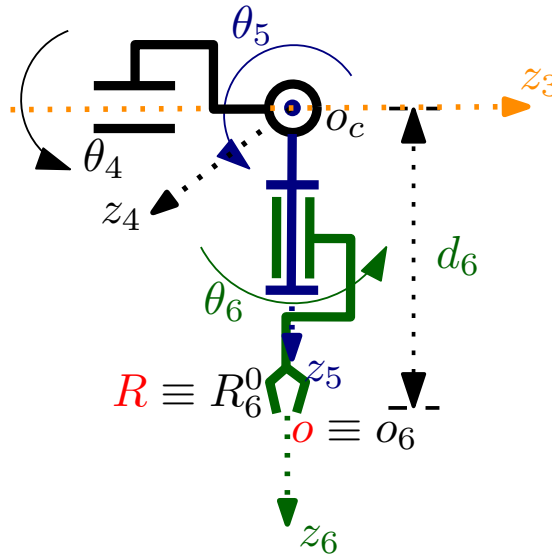
- Link 1: $\theta = \theta_1$, $d = d_1$, $a = 0$, $\alpha = \frac{\pi}{2} \Rightarrow H_1^0$
- Link 2: $\theta = \theta_2$, $d = 0$, $a = a_2$, $\alpha = 0 \Rightarrow H_2^1$
- Link 3: $\theta = \theta_3 + \frac{\pi}{2}$, $d = 0$, $a = 0$, $\alpha = \frac{\pi}{2} \Rightarrow H_3^2$

Let us see what needs to be done to solve the inverse orientation kinematics problem.

We have computed $H_3^0 = H_1^0 H_2^1 H_3^2 \Rightarrow R_3^0$, $o_3^0 = o_c^0$. Now, we can compute the rotation matrix

$$R_6^3 = [R_3^0]^{-1} \cdot R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

In order to find the required analytical expressions for θ_4 , θ_5 , and θ_6 , we may notice that they can be associated with the Euler angles (ϕ, θ, ψ) :



Hence, we just need to compute the Euler angle representation for a given rotation matrix:

$$R_6^3 = R_{z,\phi} \cdot R_{y,\theta} \cdot R_{z,\psi}.$$

and the overall inverse kinematics problem for an articulated arm with a spherical wrist is solved.

This completes our brief overview of the forward and inverse position and orientation kinematics.

Next we talk about defining velocities and introduce a very important characteristics of a manipulator known as Jacobian.

Chapter 3

Velocity Kinematics and Jacobian

We are interested here to describe velocities of the various points of a robotic manipulator.

Let us recall that motions of rigid bodies, such as links of a manipulator, can be classified as:

- a (pure) **translation**, when all the points move along parallel straight lines passing the same distances during the same time;
- a **rotation around a fix axis**, when all the points move along circles centered at a particular not moving axis;
- a general **rigid motion**, which at every instant of time can be understood as a combination of an instantaneous translation and a rotation around an instantaneous axis.

Translations are easy to describe since all the velocities of the body are the same, while rotations can be described by rotation matrices, change in time of which defines so called angular velocity.

3.1 Angular Velocity for Characterizing Rotations

Let us begin with a discussion of rotations of rigid bodies around a fix axis and introduce the necessary mathematical apparatus.

3.1.1 Rotation around a Fixed Axis

When a rigid body rotates around a fixed axis the following is true.

- Every point of the body moves around a circle with a center at the axis of rotation.
- As the body rotates, every perpendicular to the axis line through any point of the body sweeps the same angle during the same time period. It is customary to use time evolution of such an angle, say θ , to characterize the rotation.
- If $\vec{k}$ is a unit vector in the direction of the axis of rotation, then one can introduce the rigid body's axial vector of **angular velocity** given by

$$\vec{\omega} = \left(\frac{d}{dt}\theta\right) \vec{k} \quad \text{and} \quad |\vec{\omega}| = |\dot{\theta}|$$

- The linear velocity of any point p of the body can be computed as

$$\vec{v}_p = \dot{\vec{r}}_p = \vec{v}_o + \vec{\omega} \times \vec{r}_p$$

where $\vec{r}_p = \overrightarrow{op}$ is the rotating vector to p from the origin o of an attached frame and $\vec{v}_o$ is the linear velocity of the origin, which is equal to zero if o lies on the axis. In the later case of $\vec{v}_o = \vec{0}$, the formula for the speed is

$$|\vec{v}_p| = \dot{\theta} \underbrace{\left(|\vec{r}_p| \sin(\widehat{\vec{k}, \vec{r}_p}) \right)}_{\text{distance from } p \text{ to the axis}}.$$

The aim of this section is to derive the formula above for $\vec{v}_p$ for the case of rotation of a rigid body not necessary along an axis with a fixed direction and to show how to compute the angular velocity for this formula.

3.1.2 Matrix Representation of Cross Products

We have mentioned above that description of velocities of points of a rotating rigid body can be done using the notion of a cross product between two vectors. Let us remind that the obtained (axial) vector can be computed with respect to a given (fixed) frame as follows.

Suppose coordinates of two vectors $\vec{a}$ and $\vec{b}$ in a $\mathbf{0}$ -frame are

$$\mathbf{a}^0 = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}, \quad \mathbf{b}^0 = \begin{bmatrix} b_x \\ b_y \\ b_z \end{bmatrix}$$

then the coordinates of the vector of their cross product

$$\vec{c} = \vec{a} \times \vec{b}$$

in the $\mathbf{0}$ -frame are

$$\mathbf{c}^0 = \begin{bmatrix} c_x \\ c_y \\ c_z \end{bmatrix} = \begin{bmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{bmatrix}$$

To simplify some derivations, for a given $\vec{a}$, it is useful to introduce a linear transformation (a tensor) $\mathbf{S}(\vec{a}) = \vec{a} \times \mathbf{I}$ acting on vectors by making a cross product with $\vec{a}$, i.e. for every $\vec{b}$

$$\boxed{\mathbf{S}(\vec{a}) \cdot \vec{b} = (\vec{a} \times \mathbf{I}) \cdot \vec{b} = \vec{a} \times \vec{b}}$$

in the $\mathbf{0}$ -frame such transformation can be represented by the following matrix

$$\mathbf{S}^0(\vec{a}) = (\vec{a} \times \mathbf{I})^0 = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix}$$

since it can be easily verified that

$$\mathbf{S}^0(\vec{a}) \mathbf{b}^0 = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix} \begin{bmatrix} b_x \\ b_y \\ b_z \end{bmatrix} = \begin{bmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{bmatrix} = \mathbf{c}^0$$

As can be seen by inspection,

$$\mathbf{S}^0(\vec{a}) = -(\mathbf{S}^0(\vec{a}))^T$$

and it is not hard to verify, in a similar way as done for representation of rotations, that the matrices for $\mathbf{0}$ -frame and $\mathbf{1}$ -frame are related by

$$\mathbf{S}^1(\vec{a}) = (\mathbf{R}_1^0)^T \mathbf{S}^0(\vec{a}) \mathbf{R}_1^0, \quad \mathbf{S}^0(\vec{a}) = \mathbf{R}_1^0 \mathbf{S}^1(\vec{a}) (\mathbf{R}_1^0)^T$$

Note that an $n \times n$ matrix $\mathbf{S}^0$ is said to be **skew symmetric** and denoted $\mathbf{S}^0 \in \mathfrak{so}(n)$ if

$$\mathbf{S}^0 + (\mathbf{S}^0)^T = \mathbf{0}$$

or, in terms of the elements

$$\begin{aligned} s_{ij}^0 + s_{ji}^0 &= 0, \quad i = 1, \dots, n, \quad j = 1, \dots, n \\ \Rightarrow \quad &\left\{ \begin{array}{l} s_{ii}^0 = 0 \quad \text{and} \quad s_{ij}^0 = -s_{ji}^0 \quad \forall i \neq j \end{array} \right\} \end{aligned}$$

Another important property of the skew symmetric matrices is the fol-

lowing identity

$$\vec{x} \cdot \mathbf{S} \cdot \vec{x} = (\mathbf{x}^0)^T \mathbf{S}^0 \mathbf{x} = 0 \quad \text{for any } \mathbf{S}^0 \in \mathfrak{so}(3), \mathbf{x}^0 \in \mathbb{R}^3$$

As for every linear transformation, it can be also verified that

$$\mathbf{S}(\alpha \vec{a} + \beta \vec{c}) = \alpha \mathbf{S}(\vec{a}) + \beta \mathbf{S}(\vec{c})$$

for every scalars α and β and every vectors $\vec{a}$ and $\vec{b}$.

For application to rigid body motions, it is important to know what happens with cross products under rotation, say $\mathbf{R} \in \mathcal{SO}(3)$. To this end, the next formula is not hard to check

$$\mathbf{R} \cdot (\mathbf{S}(\vec{a}) \cdot \vec{b}) = \boxed{\mathbf{R} \cdot (\vec{a} \times \vec{b}) = (\mathbf{R} \cdot \vec{a}) \times (\mathbf{R} \cdot \vec{b})} = \mathbf{S}(\mathbf{R} \cdot \vec{a}) \cdot (\mathbf{R} \cdot \vec{b})$$

It just states that rotating a result of cross product of two vectors is the same as making cross product of the rotated vectors.

Another important property is the following

$$\boxed{\mathbf{R} \cdot \mathbf{S}(\vec{a}) \cdot \mathbf{R}^T = \mathbf{S}(\mathbf{R} \cdot \vec{a})} \quad \text{for any } \mathbf{R} \in \mathcal{SO}(3), \vec{a} \in \mathbb{R}^3$$

It states that the result of the sequence of three transformations (rotation, making a cross product with $\vec{a}$, performing inverse rotation) is the same as the result of making a cross product with rotated $\vec{a}$.

To show this identity, take any $\vec{p}$ and apply to it the transformation on the right and the formula above together with $\mathbf{R} \cdot \mathbf{R}^T = \mathbf{I}$:

$$\begin{aligned} [\mathbf{R} \cdot \mathbf{S}(\vec{a}) \cdot \mathbf{R}^T] \cdot \vec{p} &= \mathbf{R} \cdot (\mathbf{S}(\vec{a}) \cdot \underbrace{[\mathbf{R}^T \cdot \vec{p}]}_{\vec{b}}) \\ &= \mathbf{S}(\mathbf{R} \cdot \vec{a}) \cdot (\mathbf{R} \cdot \underbrace{[\mathbf{R}^T \cdot \vec{p}]}_{\vec{b}}) = [\mathbf{S}(\mathbf{R} \cdot \vec{a})] \cdot \vec{p} \end{aligned}$$

To see how a cross product appears in the formula relating linear and angular velocities, we need to know how to compute derivatives of rotation tensors (or of rotation matrices as their coordinate representations).

3.1.3 Derivatives of Rotation Matrices

For an arbitrary rotation $\mathbf{R}(\theta) \in \mathcal{SO}(3)$ defined by an angle θ and an axis, let us compute derivative of its coordinate representation in a fixed $\mathbf{0}$ -frame.

In other words, let us consider a matrix-valued differentiable function

$$\boldsymbol{\theta} \mapsto \mathbf{R}^0(\boldsymbol{\theta})$$

of a scalar variable $\boldsymbol{\theta}$ and compute its instantaneous rate of change $\frac{d}{d\boldsymbol{\theta}} \mathbf{R}^0(\boldsymbol{\theta})$. Since for any $\boldsymbol{\theta}$ the matrix $\mathbf{R}^0(\boldsymbol{\theta})$ is a rotation matrix, we have

$$\mathbf{R}^0(\boldsymbol{\theta}) (\mathbf{R}^0(\boldsymbol{\theta}))^T = (\mathbf{R}^0(\boldsymbol{\theta}))^T \mathbf{R}(\boldsymbol{\theta}) = \mathbf{I}$$

Differentiating this identity according to the product rule we obtain

$$\underbrace{\frac{d}{d\boldsymbol{\theta}} \left[\mathbf{R}^0(\boldsymbol{\theta}) (\mathbf{R}^0(\boldsymbol{\theta}))^T \right]}_{\mathbf{I} = \text{const}} = \frac{d}{d\boldsymbol{\theta}} [\mathbf{R}^0(\boldsymbol{\theta})] (\mathbf{R}^0(\boldsymbol{\theta}))^T + \mathbf{R}^0(\boldsymbol{\theta}) \frac{d}{d\boldsymbol{\theta}} [(\mathbf{R}^0(\boldsymbol{\theta}))^T] = \mathbf{0}$$

Let us introduce the following linear transformation parametrized by $\boldsymbol{\theta}$

$$\mathbf{S}(\boldsymbol{\theta}) = \mathbf{R}'(\boldsymbol{\theta}) \cdot (\mathbf{R}(\boldsymbol{\theta}))^T, \quad \text{where} \quad \mathbf{R}'(\boldsymbol{\theta}) = \frac{d}{d\boldsymbol{\theta}} [\mathbf{R}(\boldsymbol{\theta})]$$

which is called **spin matrix** associated with a varying rotation matrix.

Then, using the following known identities for matrices $(\mathbf{A} \mathbf{B}^T)^T = \mathbf{B} \mathbf{A}^T$ and $\frac{d}{d\boldsymbol{\theta}} [(\mathbf{R}^0(\boldsymbol{\theta}))^T] = [\frac{d}{d\boldsymbol{\theta}} (\mathbf{R}^0(\boldsymbol{\theta}))]^T$, we can rewrite the result of differentiation above as $\mathbf{S}^0(\boldsymbol{\theta}) + (\mathbf{S}^0(\boldsymbol{\theta}))^T = \mathbf{0}$ and conclude

$$\mathbf{S}(\boldsymbol{\theta}) + \mathbf{S}^T(\boldsymbol{\theta}) = \mathbf{0}$$

which means that $\mathbf{S}(\boldsymbol{\theta})$ is skew symmetric; and, therefore, it defines a transformation equivalent to a cross product with a fixed vector. Now, multiplying the definition of $\mathbf{S}$ by $\mathbf{R}(\boldsymbol{\theta})$ on the right we obtain

$$\begin{aligned} \mathbf{S}(\boldsymbol{\theta}) \cdot \mathbf{R}(\boldsymbol{\theta}) &= \mathbf{R}'(\boldsymbol{\theta}) \cdot \underbrace{(\mathbf{R}(\boldsymbol{\theta}))^T \cdot \mathbf{R}(\boldsymbol{\theta})}_{\mathbf{I}} \\ \Rightarrow \quad \boxed{\frac{d}{d\boldsymbol{\theta}} [\mathbf{R}(\boldsymbol{\theta})] = \mathbf{S}(\boldsymbol{\theta}) \cdot \mathbf{R}(\boldsymbol{\theta})} &\quad (\text{Poisson equation for rotations}) \end{aligned}$$

or, equivalently, in $\mathbf{0}$ -frame: $\frac{d}{d\boldsymbol{\theta}} [\mathbf{R}^0(\boldsymbol{\theta})] = \mathbf{S}^0(\boldsymbol{\theta}) \mathbf{R}^0(\boldsymbol{\theta})$, $\mathbf{S}^0(\boldsymbol{\theta}) \in \mathfrak{so}(3)$.

An example: computing $\mathbf{S}^0$ of a basic rotation.

Let $\mathbf{R}^0(\boldsymbol{\theta}) = \mathbf{R}_{z,\theta}$, that is consider the basic rotation around the axis $\mathbf{z}_0$ of the $\mathbf{0}$ -frame, then

$$\mathbf{S}^0 = \frac{d}{d\theta} (\mathbf{R}_{z,\theta}) \mathbf{R}_{z,\theta}^T = \frac{d}{d\theta} \left(\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \right) \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}^T$$

$$\begin{aligned}
&= \begin{bmatrix} -\sin \theta & -\cos \theta & 0 \\ \cos \theta & -\sin \theta & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \\
&= \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{bmatrix} = S^0(\vec{a})
\end{aligned}$$

where

$$\mathbf{a}^0 = [a_x, a_y, a_z]^T = [0, 0, 1]^T \quad \Rightarrow \quad \vec{a} = \vec{z}_0$$

In a similar way it can be shown that

$$\frac{d}{d\theta} [\mathbf{R}_{\vec{k},\theta}] \cdot (\mathbf{R}_{\vec{k},\theta})^T = S(\vec{k}) \quad \left[\text{reminder: } \vec{a} \times \mathbf{B} := S(\vec{a}) \cdot \mathbf{B} \right],$$

which, with any differentiable $\theta(t)$ and a constant $\vec{k}$, implies

$$\frac{d}{dt} [\mathbf{R}_{\vec{k},\theta(t)}] = \dot{\theta} S(\vec{k}) \cdot \mathbf{R}_{\vec{k},\theta(t)} = (\dot{\theta} \vec{k}) \times \mathbf{R}_{\vec{k},\theta(t)}$$

Let us see how the generalization of this formula for an arbitrary rotation derived above can be used for a formal introduction of the notion of angular velocity of a rigid body associated with an attached frame.

3.1.4 Linear and Angular Velocities of a Moving Frame

Angular Velocity

Suppose we have a rotating rigid body and the associated rotation $\mathbf{R}(t) \in \mathbf{SO}(3)$ of the attached frame is a differentiable function of time.

Repeating the general arguments for computation of its derivative, we conclude that there exists a skew-symmetric time-dependent transformation $\mathbf{S}$, known as tensor of spin, represented in coordinates by a spin matrix, defined by

$$\mathbf{S} = \dot{\mathbf{R}}(t) \cdot (\mathbf{R}(t))^T$$

such that

$$\frac{d}{dt} \mathbf{R}(t) = \mathbf{S} \cdot \mathbf{R}(t), \quad \mathbf{S} \in \mathfrak{so}(3)$$

which is known as Poisson equation for rotations.

Since $\mathbf{S} = -\mathbf{S}^T$, there exist a vector $\vec{\omega}(t)$ such that

$$\mathbf{S} = \vec{\omega}(t) \times \mathbf{I} \quad \text{i.e.} \quad \mathbf{S}^0 = S^0(\vec{\omega}(t)) = \begin{bmatrix} 0 & -\omega_z^0(t) & \omega_y^0(t) \\ \omega_z^0(t) & 0 & -\omega_x^0(t) \\ -\omega_y^0(t) & \omega_x^0(t) & 0 \end{bmatrix}$$

The (axial) vector $\vec{\omega}(t)$, defined by $\omega^0(t) = [\omega_x^0(t), \omega_y^0(t), \omega_z^0(t)]^T$, is called the **angular velocity** of a rotating frame with respect to the fixed (inertial) frame at time t .

As a useful example for a rotation parametrized using time-dependent Euler ZYZ -angles $(\phi(t), \theta(t), \psi(t))$

$$R_{ZYZ} := \begin{bmatrix} c_\phi c_\theta c_\psi - s_\phi s_\psi & -c_\phi c_\theta s_\psi - s_\phi c_\psi & c_\phi s_\theta \\ s_\phi c_\theta c_\psi + c_\phi s_\psi & -s_\phi c_\theta s_\psi + c_\phi c_\psi & s_\phi s_\theta \\ -s_\theta c_\psi & s_\theta s_\psi & c_\theta \end{bmatrix}$$

one can verify that

$$\frac{d}{dt} R_{ZYZ} \cdot R_{ZYZ}^T = S(\omega_{ZYZ}(t)), \quad \text{where} \quad \omega_{ZYZ}(t) = \begin{bmatrix} c_\phi s_\theta \dot{\psi} - s_\phi \dot{\theta} \\ s_\phi s_\theta \dot{\psi} + c_\phi \dot{\theta} \\ c_\theta \dot{\psi} + \dot{\phi} \end{bmatrix}$$

We should notice that, as shown above, whenever rotation is performed along a **fixed axis** in the direction of unit vector $\vec{k}$ by an angle $\theta(t)$, we have

$$\vec{\omega}(t) = \dot{\theta} \vec{k}.$$

However, in general, if both the angle, $\theta(t)$, and the direction of the axis of rotation, $\vec{k}(t)$, are time-varying, the following relation is true:

$$\vec{\omega} = \dot{\theta} \vec{k} + \sin(\theta) \dot{\vec{k}} + (1 - \cos(\theta)) \vec{k} \times \dot{\vec{k}}$$

that, in particular, implies $\vec{\omega} = \frac{d}{dt} (\theta \vec{k}) + O(|\theta|^2)$.

Velocity of a point of a rotating rigid body

Consider a point p rigidly attached to a moving 1-frame, then its coordinates in a fixed (inertial) 0-frame (assuming that both origins are on the axis of rotation) are computed as follows

$$p^0(t) = R_1^0(t) p^1$$

Differentiating this expression we obtain

$$\begin{aligned} \frac{d}{dt} [p^0(t)] &= \frac{d}{dt} [R_1^0(t)] p^1 = \mathbf{S}^0(t) R_1^0(t) p^1 \\ &= \omega^0(t) \times R_1^0(t) p^1 = \omega^0(t) \times p^0(t) \end{aligned}$$

Let us see now what happens with angular velocity if we have a combination of several (relative) rotations.

Relative Angular Velocities and Combining Relative Rotations

Suppose we have now a family of frames, such as in the case of frames attached to different links of a manipulator.

With every time-varying (differentiable) rotation matrix $R_j^i(t)$ describing orientations of j -frame moving with respect to i -frame, we can associate the vector of **relative angular velocity** $\vec{\omega}_{i,j}$ that can be expressed in k -frame as $\omega_{i,j}^k(t) = R_i^k(t) \omega_{i,j}^i(t)$ according to the Poisson formula

$$\frac{d}{dt} R_j^i(t) \equiv \dot{R}_j^i = S^i(\vec{\omega}_{i,j}(t)) R_j^i(t) \equiv \omega_{i,j}^i(t) \times R_j^i(t)$$

Let us compute the relation among such coordinate representations.

Consider two moving frames (**1**-frame and **2**-frame) and a fixed **0**-frame, all with the common origin. The combination of rotations formula

$$R_2^0(t) = R_1^0(t) R_2^1(t)$$

expresses rotation of **2**-frame with respect to **0**-frame in terms of rotation of **1**-frame with respect to **0**-frame and of **2**-frame with respect to **1**-frame.

Let us differentiate this expression

$$\begin{aligned} \frac{d}{dt} R_2^0(t) &= \frac{d}{dt} [R_1^0(t)] R_2^1(t) + R_1^0(t) \frac{d}{dt} [R_2^1(t)] \\ &= S^0(\vec{\omega}_{0,1}(t)) \underbrace{R_1^0(t) R_2^1(t)}_{= R_2^0(t)} + R_1^0(t) [S^1(\vec{\omega}_{1,2}(t)) R_2^1(t)] \\ &= S^0(\vec{\omega}_{0,1}(t)) R_2^0(t) + R_1^0(t) S^1(\vec{\omega}_{1,2}(t)) \underbrace{R_1^0(t)^T R_1^0(t)}_{= I} R_2^1(t) \\ &= S^0(\vec{\omega}_{0,1}(t)) R_2^0(t) + \underbrace{R_1^0(t) S^1(\vec{\omega}_{1,2}(t)) R_1^0(t)^T}_{= S^0(\vec{\omega}_{1,2}(t))} R_1^0(t) R_2^1(t) \\ &= S^0(\vec{\omega}_{0,1}(t)) R_2^0(t) + S^0(\vec{\omega}_{1,2}(t)) \underbrace{R_1^0(t) R_2^1(t)}_{= R_2^0(t)} \\ &= [S^0(\vec{\omega}_{0,1}(t)) + S^0(\vec{\omega}_{1,2}(t))] R_2^0(t) \end{aligned}$$

Now, applying again the Poisson formula $\frac{d}{dt} R_2^0(t) = S^0(\vec{\omega}_{0,2}(t)) R_2^0(t)$

$$S^0(\vec{\omega}_{0,2}(t)) = S^0(\vec{\omega}_{0,1}(t)) + S^0(\vec{\omega}_{1,2}(t))$$

This relation, together with the property: $S(a) + S(c) = S(a + c)$, imply that the sum of relative angular velocities gives the angular velocity

$$\omega_{0,2}^0(t) = \omega_{0,1}^0(t) + R_1^0(t) \omega_{1,2}^1(t) = \omega_{0,1}^0(t) + \omega_{1,2}^0(t)$$

This formula can be generalized to the case of arbitrary number, say n , moving frames with the same origins as follows. We have

$$R_n^0(t) = R_1^0(t)R_2^1(t) \cdots R_n^{n-1}(t) \Rightarrow \frac{d}{dt}R_n^0(t) = S^0(\vec{\omega}_{0,n}(t))R_n^0(t)$$

and, therefore, the angular velocity of the n -frame $\vec{\omega}_n$ can be computed adding (rotated) relative angular velocities

$$\begin{aligned} \omega_n^0 = \omega_{0,n}^0(t) &= \omega_{0,1}^0(t) + \omega_{1,2}^0(t) + \omega_{2,3}^0(t) + \cdots + \omega_{n-1,n}^0(t) \\ &= \omega_{0,1}^0 + R_1^0 \omega_{1,2}^1 + R_2^0 \omega_{2,3}^2 + \cdots + R_{n-1}^0 \omega_{n-1,n}^{n-1} \end{aligned}$$

With a clear notions of angular velocity and relative angular velocity, we can proceed with describing velocities under rigid motions.

3.1.5 Linear Velocities under Rigid Motions

Consider a moving **1**-frame and a fixed (inertial) **0**-frame related by a time-varying homogeneous transform

$$H_1^0(t) = \begin{bmatrix} R_1^0(t) & o_1^0(t) \\ 0 & 1 \end{bmatrix}$$

i.e., coordinates of each point p fixed in the **1**-frame can be computed as

$$p^0(t) = R_1^0(t)p^1 + o_1^0(t)$$

Differentiating this equation and applying the Poisson formula we obtain

$$\begin{aligned} v_p^0(t) = \frac{d}{dt}p^0(t) &= \omega_1^0(t) \times (R_1^0(t)p^1) + \frac{d}{dt}[o_1^0(t)] \\ &= \omega_1^0(t) \times \underbrace{(p^0(t) - o_1^0(t))}_{(\bar{o}_1 p)^0} + \underbrace{\frac{d}{dt}[o_1^0(t)]}_{v_{o_1}^0(t)} \end{aligned}$$

where, as above, the **0**-frame representation of the angular velocity vector $\omega_1^0(t) = [\omega_x^0(t), \omega_y^0(t), \omega_z^0(t)]^T$ can be obtained from

$$\omega_1^0(t) \times I = \begin{bmatrix} 0 & -\omega_z^0(t) & \omega_y^0(t) \\ \omega_z^0(t) & 0 & -\omega_x^0(t) \\ -\omega_y^0(t) & \omega_x^0(t) & 0 \end{bmatrix} = \frac{d}{dt}(R_1^0(t))(R_1^0(t))^T$$

We are ready now to proceed with a description of velocities of a manipulator's end-effector as well as various other points.

3.2 Concept of the Manipulator Jacobian

Let us adapt the derivations above to the case of a kinematic chain.

3.2.1 Velocities of Points in the End-effector Frame

For a given n -link manipulator with joint variables $q_1, \dots, q_n$, let us see how to compute the angular velocity of the end-effector frame and the linear velocity of its origin.

Suppose the forward kinematics problem is solved (see Chapter 2), i.e. let $H_n^0(q)$ be the homogeneous transformation between the end-effector frame and the (inertial) base frame:

$$H_n^0(q) = \begin{bmatrix} R_n^0(q) & o_n^0(q) \\ 0 & 1 \end{bmatrix}, \quad q = [q_1, \dots, q_n]^T$$

so that $\forall p$ fixed in the end-effector frame with constant coordinates p^n , its (changing during the motion) coordinates in the base frame are

$$p^0 = R_n^0 p^n + o_n^0$$

Note that, since as the robot moves, the joint variables become functions of time

$$t \rightarrow q(t) = [q_1(t), \dots, q_n(t)]^T$$

so that the formula above takes the following form

$$p^0(t) = R_n^0(q(t)) p^n + o_n^0(q(t))$$

From the discussion above, we know how to compute $\frac{d}{dt}p^0(t)$:

$$\begin{aligned} \frac{d}{dt}p^0(t) &= \frac{d}{dt} [R_n^0(q(t))] p^n + \frac{d}{dt} [o_n^0(q(t))] \\ &= S^0(\vec{\omega}_{0,n}(t)) R_n^0(q(t)) p^n + v_n^0(t) \\ &= \omega_{0,n}^0(t) \times r^0(t) + v_n^0(t) \end{aligned}$$

with $r^0(t) = p^0(t) - o_n^0(q(t)) = R_n^0(q(t)) p^n$ is the $\mathbf{0}$ -frame representation of the vector $\vec{r}$ from the origin of the n -frame to the point p , which is defined uniquely by the solution of the forward kinematics problem.

Therefore, to compute velocity of any point in the end-effector frame, it suffices to know representations in the $\mathbf{0}$ -frame

- of the angular velocity of the end-effector frame $\omega_{0,n}^0(t)$ and

- of the linear velocity of the the end-effector frame origin $\mathbf{v}_n^0(t)$.

In principle, from a given solution of the forward kinematics problem $\mathbf{H}_n^0(\mathbf{q}(t)) = \mathbf{A}_1(\mathbf{q}_1(t)) \cdots \mathbf{A}_n(\mathbf{q}_n(t))$, which is a homogeneous transformation between the end-effector $\mathbf{n}$ -frame and the base $\mathbf{0}$ -frame,

$$\mathbf{H}_n^0(\mathbf{q}(t)) = \begin{bmatrix} \mathbf{R}_n^0(\mathbf{q}(t)) & \mathbf{o}_n^0(\mathbf{q}(t)) \\ \mathbf{0} & 1 \end{bmatrix}$$

one can extract $\mathbf{R}_n^0(\mathbf{q}_1, \dots, \mathbf{q}_n)$ and $\mathbf{o}_n^0(\mathbf{q}_1, \dots, \mathbf{q}_n)$ and use them

- to find the velocity $\dot{\mathbf{v}}_n(t)$ of the the end-effector frame origin:

$$\mathbf{v}_n^0(t) = \frac{d}{dt} \mathbf{o}_n^0(\mathbf{q}(t)) = \sum_{i=1}^n \left[\frac{\partial \mathbf{o}_n^0(\mathbf{q}_1, \dots, \mathbf{q}_n)}{\partial \mathbf{q}_i} \right] \bigg|_{\mathbf{q}=\mathbf{q}(t)} \frac{d}{dt} \mathbf{q}_i(t)$$

- to find the angular velocity $\dot{\boldsymbol{\omega}}_{0,n}(t)$ of the end-effector frame:

$$\begin{aligned} \boldsymbol{\omega}_{0,n}^0 = \begin{bmatrix} \omega_x^0 \\ \omega_y^0 \\ \omega_z^0 \end{bmatrix} &\Leftarrow \begin{bmatrix} 0 & -\omega_z^0 & \omega_y^0 \\ \omega_z^0 & 0 & -\omega_x^0 \\ -\omega_y^0 & \omega_x^0 & 0 \end{bmatrix} = \left(\sum_{i=1}^n \frac{\partial \mathbf{R}_n^0(\mathbf{q})}{\partial \mathbf{q}_i} \frac{d}{dt} \mathbf{q}_i \right) \left(\mathbf{R}_n^0(\mathbf{q}) \right)^T \\ &= \sum_{i=1}^n \left[\frac{\partial \mathbf{R}_n^0(\mathbf{q})}{\partial \mathbf{q}_i} \left(\mathbf{R}_n^0(\mathbf{q}) \right)^T \right] \bigg|_{\mathbf{q}=\mathbf{q}(t)} \dot{\mathbf{q}}_i(t) \end{aligned}$$

3.2.2 Definition of the Jacobian

It is obvious from the expressions above that there are two matrix functions $\mathbf{J}_v$ and $\mathbf{J}_\omega$ such that

$$\mathbf{v}_n^0(t) = \mathbf{J}_v(\mathbf{q}(t)) \frac{d}{dt} \mathbf{q}(t) \quad \text{and} \quad \boldsymbol{\omega}_{0,n}^0(t) = \mathbf{J}_\omega(\mathbf{q}(t)) \frac{d}{dt} \mathbf{q}(t)$$

Concatenation of them leads to a $6 \times n$ -matrix function $\mathbf{J}(\cdot)$ defined by

$$\boldsymbol{\xi}(t) = \begin{bmatrix} \mathbf{v}_n^0(t) \\ \boldsymbol{\omega}_{0,n}^0(t) \end{bmatrix} = \mathbf{J}(\mathbf{q}(t)) \frac{d}{dt} \mathbf{q}(t) = \begin{bmatrix} \mathbf{J}_v(\mathbf{q}(t)) \\ \mathbf{J}_\omega(\mathbf{q}(t)) \end{bmatrix} \frac{d}{dt} \mathbf{q}(t)$$

that is called the **manipulator Jacobian**.

Let us derive a simple formula for computation of the Jacobian.

3.2.3 Computing $J_\omega(q)$ Using DH Frames

Recall that angular velocity of a rigid body is defined by derivative of the representation of the corresponding rotation matrix, i.e.

$$R_n^0(t) = R_1^0(t) R_2^1(t) \cdots R_n^{n-1}(t) \Rightarrow \frac{d}{dt} R_n^0(t) = S^0(\vec{\omega}_{0,n}(t)) R_n^0(t)$$

that imply, as shown above

$$\begin{aligned} \omega_{0,n}^0(t) &= \omega_{0,1}^0(t) + \omega_{1,2}^0(t) + \omega_{2,3}^0(t) + \cdots + \omega_{n-1,n}^0(t) \\ &= \omega_{0,1}^0 + R_1^0 \omega_{1,2}^1 + R_2^0 \omega_{2,3}^2 + \cdots + R_{n-1}^0 \omega_{n-1,n}^{n-1} \end{aligned}$$

To compute each term in the sum above, we should distinguish between two types of joints:

- If i^{th} -joint is revolute, then
 - the axis of rotation coincides with z_{i-1} , fixed in the $(i-1)$ -frame;
 - the relative angular velocity in $(i-1)$ -frame is

$$\omega_{i-1,i}^{i-1} = \dot{q}_i(t) z_{i-1}^{i-1} = \dot{q}_i(t) \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

and in the inertial frame is

$$\omega_{i-1,i}^0 = R_{i-1}^0 \omega_{i-1,i}^{i-1} = \dot{q}_i(t) z_{i-1}^0 = \dot{q}_i(t) R_{i-1}^0 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

- for the future reference we mark this case by $\rho_i = 1$.
- If j^{th} -joint is prismatic, then
 - there is no relative rotation in the $(j-1)$ -frame;
 - the relative angular velocity in $(j-1)$ -frame is

$$\omega_{j-1,j}^{j-1} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

- for the future reference we mark this case by $\rho_j = 0$.

Therefore, $\omega_{0,n}^0(t) = \omega_{0,1}^0(t) + \omega_{1,2}^0(t) + \omega_{2,3}^0(t) + \cdots + \omega_{n-1,n}^0(t)$

$$\begin{aligned}
&= \omega_{0,1}^0 + R_1^0 \omega_{1,2}^1 + R_2^0 \omega_{2,3}^2 + \cdots + R_{n-1}^0 \omega_{n-1,n}^{n-1} \\
&= \rho_1 \dot{q}_1 z_0^0 + \rho_2 \dot{q}_2 z_1^0 + \cdots + \rho_n \dot{q}_n z_n^0
\end{aligned}$$

And, finally, we obtain

$$\omega_{0,n}^0(t) = \sum_{i=1}^n \rho_i \dot{q}_i z_{i-1}^0 = \underbrace{[\rho_1 z_0^0, \rho_2 z_1^0, \dots, \rho_n z_{n-1}^0]}_{= J_\omega(q)} \begin{bmatrix} \dot{q}_1 \\ \vdots \\ \dot{q}_n \end{bmatrix}$$

defining $J_\omega(q(t))$ since z_1^0 is a function of q_1 , z_2^0 depends on q_1 and q_2 , etc.

3.2.4 Computing $J_v(q)$ Using DH Frames

The coordinates of the linear velocity $v_n^0(t)$ of the origin of the end-effector frame can be computed differentiating the expression for $o_n^0(t)$ defined from forward kinematics equations.

In case when all the joint variables are constant, there is no motion of any frame origins and therefore $v_n^0(t) \equiv 0$. In fact, o_n^0 can be written as a function of the joint variables q_i , $i = 1, \dots, n$:

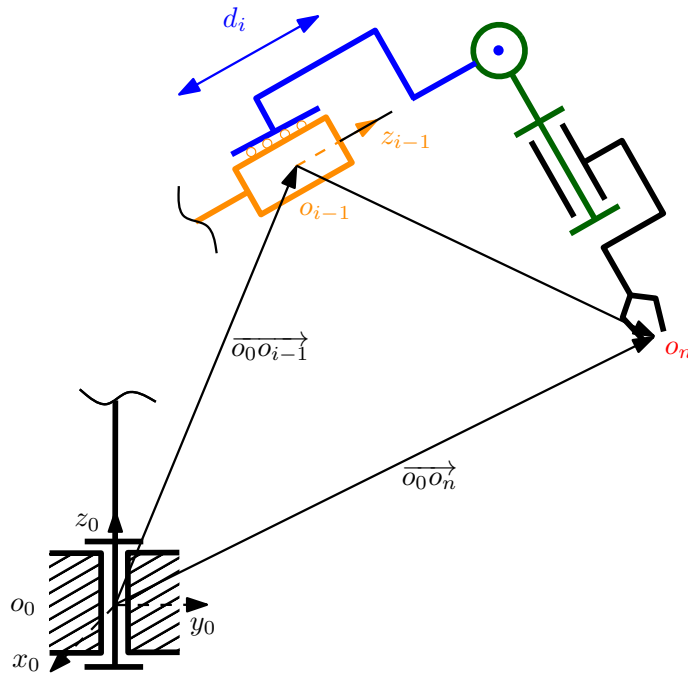
$$o_n^0 = O_n^0(q_1, q_2, \dots, q_n)$$

Therefore, there are functions $J_{v_1}(q), \dots, J_{v_n}(q)$ such that

$$\begin{aligned}
v_n^0(t) &= \frac{d}{dt} o_n^0(t) = J_{v_1}(q) \dot{q}_1 + J_{v_2}(q) \dot{q}_2 + \cdots + J_{v_n}(q) \dot{q}_n \\
&= \left[\frac{\partial}{\partial q_1} O_n^0 \right] \dot{q}_1 + \left[\frac{\partial}{\partial q_2} O_n^0 \right] \dot{q}_2 + \cdots + \left[\frac{\partial}{\partial q_n} O_n^0 \right] \dot{q}_n
\end{aligned}$$

To compute each of the partial derivatives above, we can assume that all the joint variables but one are fixed. As before, we should consider:

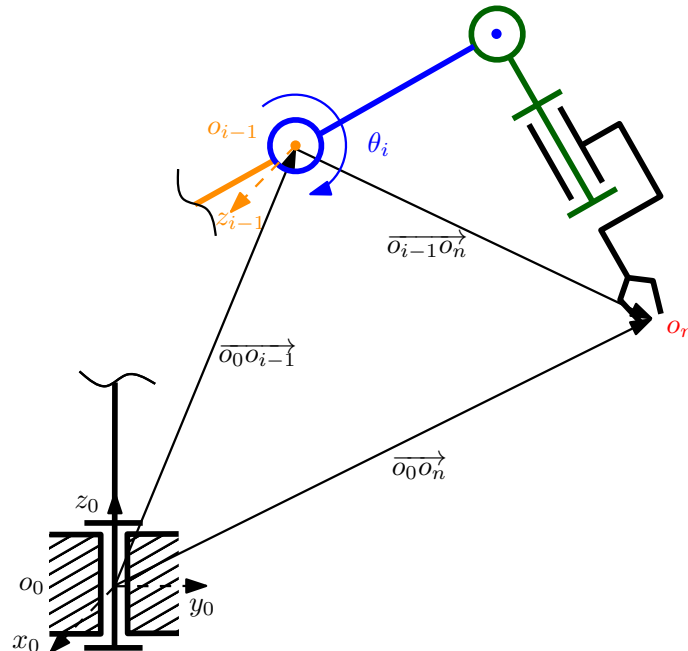
- The case of a prismatic i -th joint is illustrated below



With all other joints frozen, we have translation along the z_{i-1} -axis with the speed $\frac{d}{dt}d_i$, so that

$$\frac{d}{dt}o_n^0(t) = \frac{d}{dt}d_i(t)R_{i-1}^0 z_{i-1}^{i-1} = \frac{d}{dt}d_i(t) z_{i-1}^0 = \mathbf{J}_{v_i} \dot{d}_i$$

- The case of a revolute i -th joint is illustrated below



With all other joints frozen, we have rotation around the z_{i-1} -axis with the angular velocity $\omega_{i-1,i}^0 = \frac{d}{dt}\theta_i z_{i-1}^0$, so that

$$\frac{d}{dt}o_n^0(t) = \omega_{i-1,i}^0 \times r^0 = [\dot{\theta} z_{i-1}^0] \times [o_n^0 - o_{i-1}^0] = \mathbf{J}_{v_i} \dot{\theta}_i$$

Combining the two possible cases we conclude that for $i = 1, \dots, n$

$$\mathbf{J}_{v_i} = \begin{cases} \mathbf{z}_{i-1}^0, & \text{for prismatic joint } (\rho_i = 0) \\ \mathbf{z}_{i-1}^0 \times [\mathbf{o}_n^0 - \mathbf{o}_{i-1}^0], & \text{for revolute joint } (\rho_i = 1) \end{cases}$$

or, in another notation,

$$\mathbf{J}_{v_i} = ((1 - \rho_i)\mathbf{I} - \rho_i \mathbf{S}(\mathbf{o}_n^0 - \mathbf{o}_{i-1}^0)) \mathbf{z}_{i-1}^0$$

and

$$\mathbf{J}_v(q(t)) = [\mathbf{J}_{v_1}, \dots, \mathbf{J}_{v_n}]$$

In summary, we have the following recipe: Each 6×1 column of the $6 \times n$ Jacobian matrix

$$\mathbf{J} = [\mathbf{J}_1, \dots, \mathbf{J}_n] \in \mathbb{R}^{6 \times n}$$

corresponds to a particular joint $i = 1, \dots, n$ of the manipulator and takes the following form:

$$\mathbf{J}_i = \begin{cases} \begin{bmatrix} \mathbf{z}_{i-1}^0 \times (\mathbf{o}_n^0 - \mathbf{o}_{i-1}^0) \\ \mathbf{z}_{i-1}^0 \end{bmatrix} & \text{if joint } i \text{ is revolute } (\rho_i = 1) \\ \begin{bmatrix} \mathbf{z}_{i-1}^0 \\ 0 \\ 0 \\ 0 \end{bmatrix} & \text{if joint } i \text{ is prismatic } (\rho_i = 0) \end{cases}$$

where for $2 \leq i \leq n+1$: $\mathbf{z}_{i-1}^0$ and $\mathbf{o}_{i-1}^0$ can be extracted from the third and fourth columns of the homogeneous transformation matrices:

$$\mathbf{H}_{i-1}^0 = \begin{bmatrix} \mathbf{R}_{i-1}^0 & \mathbf{o}_{i-1}^0 \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} = \begin{bmatrix} x_{i-1}^0 & y_{i-1}^0 & z_{i-1}^0 & o_{i-1}^0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

computed by multiplication of HTs ($\mathbf{q}_j = \mathbf{d}_j$ or $\mathbf{q}_j = \boldsymbol{\theta}_j$) matrices:

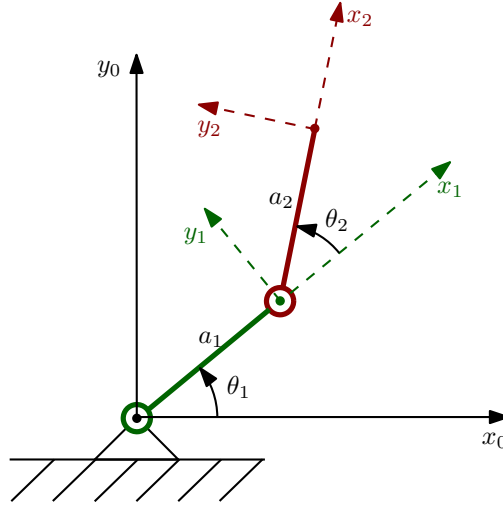
$$\mathbf{H}_{i-1}^0(q_1, \dots, q_i) = \mathbf{H}_1^0(q_1) \mathbf{H}_2^1(q_2) \dots \mathbf{H}_j^{j-1}(q_j) \dots \mathbf{H}_i^{i-1}(q_i)$$

where each $\mathbf{H}_j^{j-1}(q_j)$ can be defined from the DH parameters

$$\begin{aligned} \mathbf{H}_j^{j-1}(q_j) &= \text{Rot}_{z, \boldsymbol{\theta}_j} \text{Trans}_{z, \mathbf{d}_j} \text{Trans}_{x, \mathbf{a}_j} \text{Rot}_{x, \boldsymbol{\alpha}_j} \\ &= \left[\begin{array}{ccc|c} \cos(\boldsymbol{\theta}_j) & -\sin(\boldsymbol{\theta}_j) \cos(\boldsymbol{\alpha}_j) & \sin(\boldsymbol{\theta}_j) \sin(\boldsymbol{\alpha}_j) & \mathbf{a}_j \cos(\boldsymbol{\theta}_j) \\ \sin(\boldsymbol{\theta}_j) & \cos(\boldsymbol{\theta}_j) \cos(\boldsymbol{\alpha}_j) & -\cos(\boldsymbol{\theta}_j) \sin(\boldsymbol{\alpha}_j) & \mathbf{a}_j \sin(\boldsymbol{\theta}_j) \\ 0 & \sin(\boldsymbol{\alpha}_j) & \cos(\boldsymbol{\alpha}_j) & \mathbf{d}_j \\ 0 & 0 & 0 & 1 \end{array} \right] \end{aligned}$$

Example of Computing Jacobian Using DH frames

Let us reconsider the planar two-link manipulator studied in Chapter 1. Following the DH convention, the three frames can be assigned as shown below



We would like to compute the 4 entries of the Jacobian matrix in the next expression

$$\begin{bmatrix} \mathbf{v}_n^0(t) \\ \boldsymbol{\omega}_{0,n}^0(t) \end{bmatrix} = \begin{bmatrix} \mathbf{J}_{v_1}(q(t)) & \mathbf{J}_{v_2}(q(t)) \\ \mathbf{J}_{\omega_1}(q(t)) & \mathbf{J}_{\omega_2}(q(t)) \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}$$

Since both joints (so, $n = 2$) are revolute, we put $\rho_1 = \rho_2 = 1$ in the general formulae derived above to obtain the symbolic answer

$$\begin{bmatrix} \mathbf{v}_n^0(t) \\ \boldsymbol{\omega}_{0,n}^0(t) \end{bmatrix} = \begin{bmatrix} \mathbf{z}_0^0 \times (\mathbf{o}_2^0 - \mathbf{o}_0^0) & \mathbf{z}_1^0 \times (\mathbf{o}_2^0 - \mathbf{o}_1^0) \\ \mathbf{z}_0^0 & \mathbf{z}_1^0 \end{bmatrix} \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}$$

It is left to derive the expressions for the entries according to the particular choices of the origins (the three bold points in the picture above). This can be done either directly using simple geometry or following the forward kinematics HT computations with the DH convention routine, see Chapter 2. The result is

$$\mathbf{z}_0^0 = \mathbf{z}_1^0 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad \mathbf{o}_0^0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{o}_1^0 = \begin{bmatrix} a_1 c_1 \\ a_1 s_1 \\ 0 \end{bmatrix}, \quad \mathbf{o}_2^0 = \begin{bmatrix} a_1 c_1 + a_2 c_{12} \\ a_1 s_1 + a_2 s_{12} \\ 0 \end{bmatrix}$$

3.2.5 Analytic Jacobian

Sometimes, it is useful for a particular manipulation task to compute the rates of changes of the three angles (or other parameters of the rotation matrix) defining the orientation of the end-effector frame. As we have seen above, there are various way to parametrize a rotation matrix and one standard approach is to use the Euler angles. In fact, the time-derivatives of these angles are often more informative than the **3** components of the angular velocity of the frame defined by the matrix of spin.

In order to compute them, consider again the HT defining the solution of the forward kinematics problem

$$T_n^0(q) = \begin{bmatrix} R_n^0(q) & o_n^0(q) \\ 0 & 1 \end{bmatrix}, \quad q = [q_1, \dots, q_n]^T$$

There are several ways to represent (and to parametrize) the rotational matrix $R_n^0(q)$. Let us consider the following one of them

$$R_n^0 = R(\phi, \theta, \psi) = R_{z,\phi} R_{y,\theta} R_{z,\psi}$$

with (ϕ, θ, ψ) being the Euler angles (**ZYZ**-parametrization).

Let $\alpha(t) = [\phi(t), \theta(t), \psi(t)]^T$. Since

$$R_n^0(q) = R(\alpha)$$

and the components of the angular velocity $\omega_{0,n}^0(t)$ are defined by the **3** off-diagonal entries of the skew symmetric matrix

$$S(\omega_{0,n}^0(t)) = \frac{d}{dt} R_n^0(q(t)) (R_n^0(q(t)))^T = \frac{d}{dt} R(\alpha(t)) (R(\alpha(t)))^T$$

obviously, there exists $B(\alpha)$ such that

$$\omega_{0,n}^0(t) = B(\alpha(t)) \frac{d}{dt} \alpha(t)$$

In fact, it can be shown that

$$\omega_{0,n}^0(t) = \begin{bmatrix} c_\phi s_\theta \dot{\psi} - s_\phi \dot{\theta} \\ s_\phi s_\theta \dot{\psi} + c_\phi \dot{\theta} \\ c_\theta \dot{\psi} + \dot{\phi} \end{bmatrix} = \begin{bmatrix} 0 & -\sin(\phi) & \cos(\phi) \sin(\theta) \\ 0 & \cos(\phi) & \sin(\phi) \sin(\theta) \\ 1 & 0 & \cos(\theta) \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}$$

defining the components of $B(\alpha)$ with $\det\{B(\alpha)\} = -\sin(\theta)$.

Now, using the components of $\dot{\alpha} = [\dot{\phi}(t), \dot{\theta}(t), \dot{\psi}(t)]^T$ instead of $\omega_{0,n}^0(t)$

in the expression defining the normal Jacobian, we search for the expression

$$\begin{bmatrix} v_n^0(t) \\ \dot{\alpha}(t) \end{bmatrix} = \mathbf{J}_a(\mathbf{q}) \dot{\mathbf{q}}$$

that defines the so-called **analytic Jacobian**.

It is not hard to find the relation between the manipulator Jacobian and the analytic Jacobian:

$$\begin{aligned} \begin{bmatrix} v_n^0(t) \\ \omega_{0,n}^0(t) \end{bmatrix} &= \mathbf{J}(\mathbf{q}(t)) \dot{\mathbf{q}}(t) = \begin{bmatrix} v_n^0(t) \\ B(\alpha(t))\dot{\alpha}(t) \end{bmatrix} \\ &= \begin{bmatrix} I & 0 \\ 0 & B(\alpha(t)) \end{bmatrix} \begin{bmatrix} v_n^0(t) \\ \dot{\alpha}(t) \end{bmatrix} \\ &= \begin{bmatrix} I & 0 \\ 0 & B(\alpha(t)) \end{bmatrix} \mathbf{J}_a(\mathbf{q}(t)) \dot{\mathbf{q}}(t) \end{aligned}$$

and to compute it whenever necessary (assuming $B(\alpha)$ is invertible).

3.2.6 Singularities

Some properties of the Jacobian are important for understanding limitations and possibilities to perform various motions. In particular, analyzing the Jacobian one can compute so-called singular configuration.

The manipulator Jacobian $\mathbf{J}(\mathbf{q})$ is a $6 \times n$ -matrix mapping

$$\begin{aligned} \xi &= \begin{bmatrix} v_n^0 \\ \omega_{0,n}^0 \end{bmatrix} = \mathbf{J}(\mathbf{q}) \frac{d}{dt} \mathbf{q} = \begin{bmatrix} J_v(\mathbf{q}) \\ J_\omega(\mathbf{q}) \end{bmatrix} \frac{d}{dt} \mathbf{q} \\ &= [J_1(\mathbf{q}), J_2(\mathbf{q}), \dots, J_n(\mathbf{q})] \begin{bmatrix} \dot{q}_1 \\ \vdots \\ \dot{q}_n \end{bmatrix} \end{aligned}$$

Note that

$$\xi \in \mathbb{R}^6 \Rightarrow \text{rank} [J_1(\mathbf{q}), J_2(\mathbf{q}), \dots, J_n(\mathbf{q})] \leq 6, \forall \mathbf{q}$$

Now, all the configurations $\mathbf{q}^s = [q_1^s, \dots, q_n^s]$ for which

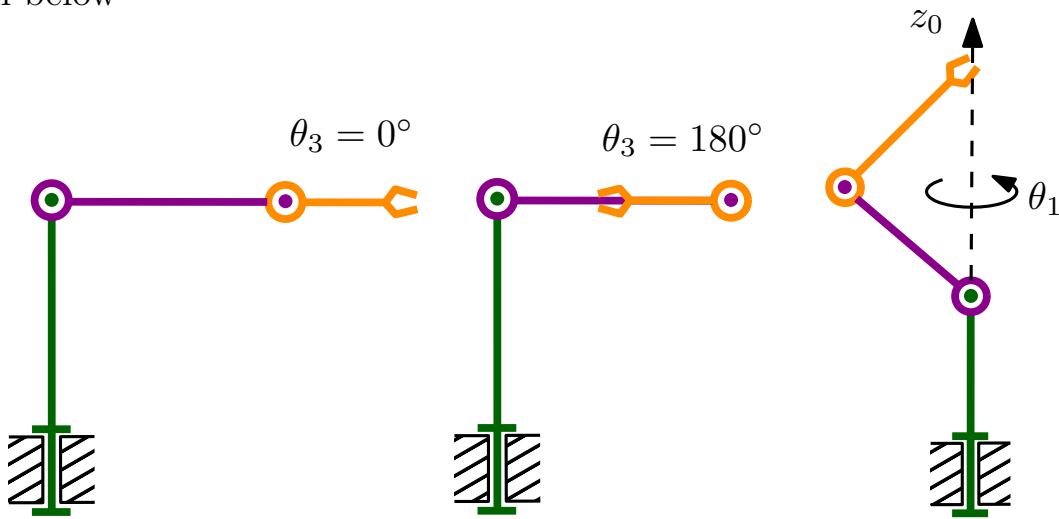
$$\text{rank} [J_1(\mathbf{q}_s), \dots, J_n(\mathbf{q}_s)] < \max_q \{ \text{rank} [J_1(\mathbf{q}), \dots, J_n(\mathbf{q})] \}$$

are called **singular**.

Identifying manipulator singularities are important due to the following

- Singularities represent configurations from which certain direction of motion of the end-effector may be unattainable;
- At singularity, bounded end-effector velocity ξ may correspond to unbounded joint velocities $\dot{q}$.
- Singularities often correspond to points on the boundary of the manipulator workspace.

Two examples of singular configurations for the elbow manipulator are shown below



3.3 Inverse Velocity Kinematics

As discussed above, the manipulator Jacobian defines the velocity of the end-effector frame and its angular velocity from derivatives of the joint variables. Sometimes, it is useful to solve the inverse problem. For example, if we know the desired velocity characteristics for the tool, it is useful to know the corresponding target values of the directly controlled joint velocities.

3.3.1 Assigning Joint Velocities

Suppose we are given the desired vector of end-effector velocities $\xi = \xi_\star$; how to find the corresponding vector $\dot{q}_\star$ of joint velocities?

The manipulator Jacobian $J(q)$ is a $6 \times n$ -matrix mapping

$$\xi = \begin{bmatrix} v_0^0 \\ \omega_{0,n}^0 \end{bmatrix} = J(q) \frac{d}{dt} q = [J_1(q), J_2(q), \dots, J_n(q)] \begin{bmatrix} \dot{q}_1 \\ \vdots \\ \dot{q}_n \end{bmatrix}$$

To solve the inverse velocity kinematics problem in the special case when the Jacobian is square and the configuration is not singular, we can invert it

$$\dot{\mathbf{q}}_{\star} = \mathbf{J}^{-1}(\mathbf{q}_{\star}) \dot{\boldsymbol{\xi}}_{\star}$$

When $n > 6$ there are many solutions, but all of them can be found using the pseudo-inverse matrices as follows

$$\dot{\mathbf{q}}_{\star} = \mathbf{J}^{+} \dot{\boldsymbol{\xi}}_{\star} + (\mathbf{I} - \mathbf{J}^{+} \mathbf{J}) \mathbf{b}, \quad \forall \mathbf{b} \in \mathbb{R}^n$$

where $\mathbf{J}^{+}(\mathbf{q}_{\star}) = \mathbf{J}^T (\mathbf{J} \mathbf{J}^T)^{-1}$ or, for example, as

$$\dot{\mathbf{q}}_{\star} = \mathbf{W}^{-1} \mathbf{J}_a^T [\mathbf{J}_a \mathbf{W}^{-1} \mathbf{J}_a^T]^{-1} \dot{\boldsymbol{\xi}}_{\star} + \left(\mathbf{I} - \mathbf{W}^{-1} \mathbf{J}_a^T [\mathbf{J}_a \mathbf{W}^{-1} \mathbf{J}_a^T]^{-1} \mathbf{J}_a \right) \mathbf{b},$$

where $\mathbf{J}_a$ is the algebraic Jacobian, $\mathbf{b} \in \mathbb{R}^n$ is taken arbitrary (or optimized to achieve an additional goal or a certain property) and $\mathbf{W}$ is a positive definite weighting matrix.

3.3.2 Inverse Kinematics based on Jacobian

Let us return to the case when the Jacobian is a square matrix, which is not singular in a vicinity of a planned motion.

With a given planned $\dot{\mathbf{X}}_{\star}(t)$ one can integrate the equation above to obtain a numerical solution for the inverse position/orientation kinematics problem. For example, with

$$\mathbf{q}_{\star}(t) = \int_0^t \mathbf{J}_a^{-1}(\mathbf{q}_{\star}(\tau)) \dot{\mathbf{X}}_{\star}(\tau) d\tau + \mathbf{q}_{\star}(0)$$

However, especially if $\mathbf{X}_{\star}(t) = \begin{bmatrix} o_{n\star}^0(t) \\ \alpha_{\star}(t) \end{bmatrix}$ is given and $\dot{\mathbf{X}}_{\star}(t)$ is computed via differentiation, the difference between $\mathbf{X}_{\star}(t)$ and $\mathbf{f}(\mathbf{q}_{\star}(t))$, where $\mathbf{f}(\cdot)$ denotes the solution of the forward kinematics problem, will grow:

$$\mathbf{q}_{\star}(t) = \int_0^t \left(\mathbf{J}_a^{-1}(\mathbf{q}_{\star}(\tau)) \dot{\mathbf{X}}_{\star}(\tau) + \text{off-set} \right) d\tau + \mathbf{q}_{\star}(0) = \dots + (\text{off-set}) t$$

To fix the off-set-related and accumulation of numerical integration errors, one can solve the differential equation

$$\dot{\mathbf{q}}_\star(t) = \mathbf{J}_a^{-1}(\mathbf{q}_\star) \left(\underbrace{\begin{bmatrix} \mathbf{v}_{n\star}^0(t) \\ \dot{\boldsymbol{\alpha}}_\star(t) \end{bmatrix}}_{\dot{\mathbf{X}}_\star(t)} + \mathbf{K} \underbrace{\left(\begin{bmatrix} \mathbf{o}_{n\star}^0(t) \\ \boldsymbol{\alpha}_\star(t) \end{bmatrix} - \mathbf{f}(\mathbf{q}_\star(t)) \right)}_{\mathbf{e}(t)} \right) \Rightarrow \dot{\mathbf{e}} + \mathbf{K}\mathbf{e} = \mathbf{0}$$

with $\mathbf{K} > \mathbf{0}$ that leads to exponentially vanishing $\mathbf{e}(t) = \mathbf{X}_\star(t) - \mathbf{f}(\mathbf{q}_\star(t))$, provided $\mathbf{f}(\mathbf{q}) \equiv \begin{bmatrix} \mathbf{o}_n^0(\mathbf{q}) \\ \boldsymbol{\alpha}(\mathbf{q}) \end{bmatrix}$ denotes the appropriate solution of the forward kinematics problem and $\mathbf{J}_a(\mathbf{q}) \equiv \frac{\partial \mathbf{f}(\mathbf{q})}{\partial \mathbf{q}}$ is the related algebraic Jacobian.

3.3.3 Manipulability

Suppose that we have restricted possible joint velocities of a n -DOF robot as follows

$$\|\dot{\mathbf{q}}\|^2 = \|\dot{q}_1\|^2 + \dots + \|\dot{q}_n\|^2 \leq 1$$

Then, clearly, we have restricted possible body velocities by

$$\boldsymbol{\xi}^T \mathbf{P}(\mathbf{q}_a) \boldsymbol{\xi} = \boldsymbol{\xi}^T (\mathbf{J}(\mathbf{q}_a) \mathbf{J}^T(\mathbf{q}_a))^{-1} \boldsymbol{\xi} = (\mathbf{J}^+(\mathbf{q}_a) \boldsymbol{\xi})^T \mathbf{J}^+(\mathbf{q}_a) \boldsymbol{\xi} = \|\dot{\mathbf{q}}\|^2 \leq 1$$

The set of values of $\boldsymbol{\xi}$ that satisfy the inequality above is called the **manipulability ellipsoid**. It illustrates how easy or difficult would be to move the end-effector in certain directions from the configuration $\mathbf{q}_a$.

Part II

Path Planning and Dynamics

Chapter 4

Path and Trajectory Planning

Here we would like to discuss path and trajectory planning. By “**path**” we mean a geometric curve to follow. It can be a curve for an end-effector to follow in the $\mathbf{3D}$ space or a curve defining the manipulator’s postures by a infinite sequence of values for the generalized coordinates. When we equip such a curve with a particular time-evolution characteristics, such as velocity at every moment, we obtain a “**trajectory**”.

For various tasks, trajectories must be understood either as a set of functions describing time-evolution of the generalized coordinates or as a set of functions describing time-evolution of the end-effector. Relations between the two descriptions of a path can be obtained from a solution of the kinematics problem, see Chapter 2. While to establish relations between the two descriptions of a trajectory, one needs to use a solution of the velocity kinematics problem, see Chapter 3.

Let us proceed with a useful terminological overview.

4.1 Space Description

For a topology-based approaches the next spaces are typically introduced.

4.1.1 Configuration Space

Given a robot with n -links, the following terminology is used.

- A complete specification of location of all the links of the robot, by e.g. particular values of the generalized coordinates $\mathbf{q}$, is called its **configuration**.
- The set of all possible configurations is known as the **configuration space** $\mathcal{Q} = \{\mathbf{q}\}$. It must be a subspace (or an embedded submanifold)

of the n -dimensional Euclidean space: $\mathcal{Q} \subset \mathbb{R}^n$.

For example, for 1-link revolute arm $\mathcal{Q}$ is the set of all possible orientations of the link, i.e., it is (topologically) a circle in plane:

$$\mathcal{Q} = \mathbb{S}^1 \quad \text{or} \quad \mathcal{Q} = SO(2)$$

As another example, consider a 2-link planar arm with revolute joints. The configuration space becomes a cross product of two circles, which is a two-dimensional torus:

$$\mathcal{Q} = \mathbb{S}^1 \times \mathbb{S}^1 = \mathbb{T}^2 \quad \leftarrow \quad \text{torus}$$

If one consider a rigid object moving on a plane, its location can be defined by coordinates of a single point and by an angle of its orientation. Hence, the configuration space is the cross product of a plane and a circle:

$$\mathcal{Q} = \{x, y, \theta\} = \mathbb{R}^2 \times \mathbb{S}^1$$

4.1.2 Workspace

Given a robot with n -links and its configuration space $\mathcal{Q} \subset \mathbb{R}^n$, the following is useful.

- Denote by $\mathcal{W} \subset \mathbb{R}^3$, the subset of the **3D** space, where the robot moves. It is called **workspace** of the robot.
- The workspace $\mathcal{W}$ might contain various obstacles that occupy certain spaces $\mathcal{O}_i \subset \mathbb{R}^3$, $i = 1, 2, \dots$, the number of obstacles.
- Denote by $\mathcal{A}(q)$ a subset of workspace $\mathcal{W}$, which is occupied by the robot, described by a particular values of the generalized coordinates $q \in \mathcal{Q}$.
- Introduce a subset of configuration space that is occupied by obstacles

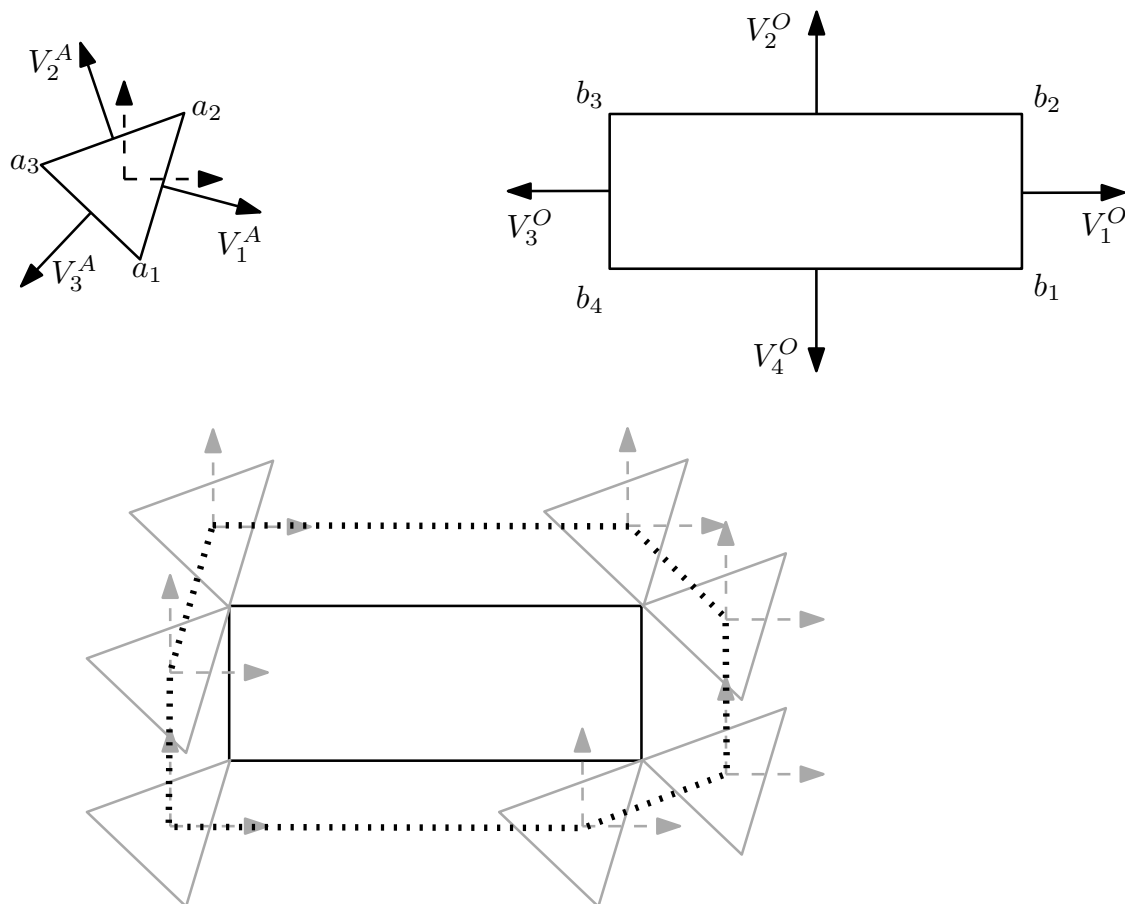
$$\mathcal{QO} := \{q \in \mathcal{Q} : \mathcal{A}(q) \cap \mathcal{O}_i \neq \emptyset, \forall i\}$$

in other words, $q \in \mathcal{QO}$ means that the robot is in collision with one of the obstacles.

- Then, the **collision-free configurations** are defined by

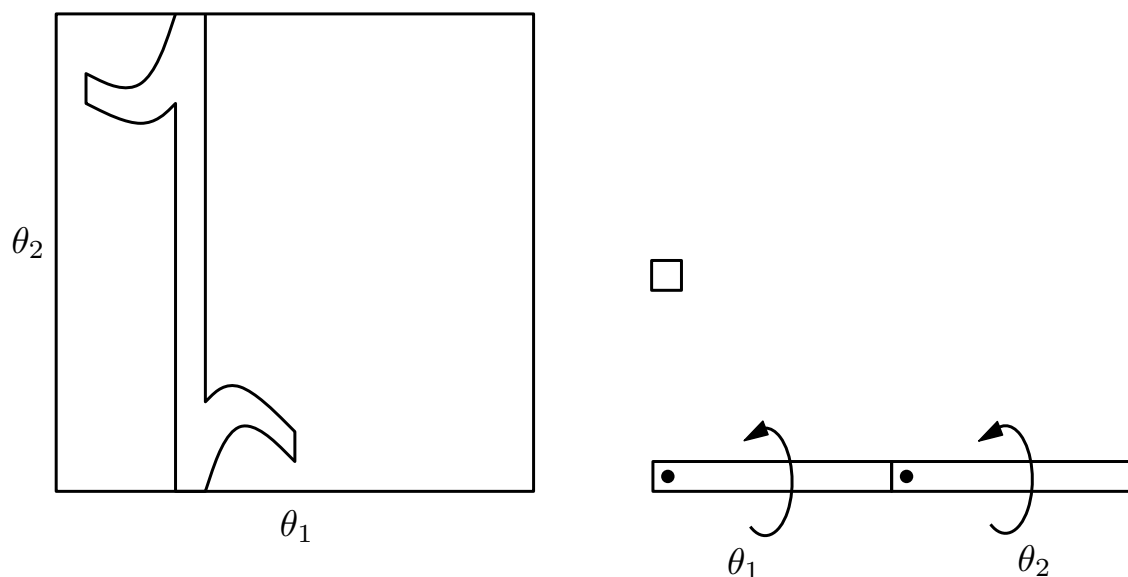
$$\mathcal{Q}_{free} := \mathcal{Q} \setminus \mathcal{QO}$$

For example, consider a robot, with an end-effector shaped in the form of a triangle and moving in plane with a fixed orientation. Suppose the workspace, which is a plane, contains a rectangular obstacle. Then, obviously, $\mathcal{QO}$ is the closed set with the dashed boundary given in the figure below.



However, typically, computing $\mathcal{QO}$ (and even $\mathcal{A}(q)$) is not a trivial task.

As another example, consider a two-links planar RR -type robot, the workspace of which contains a single square obstacle. The configuration space and the set $\mathcal{QO}$ occupied by the obstacle is shown below.



Note that from the figure above one can visualize the shaded region on the torus imagining that the edges of the square marked with θ_1 and θ_2 are glued together.

Whenever a collision free space $\mathcal{Q}_{free}$ is computed, one would need to find an appropriate for a given task path that belongs to it.

4.2 Path Planning

Problem of Path Planning is typical formulated as the task to find a path in the configuration space $\mathcal{Q}$ that

- connects an initial configuration q_s to a final configuration q_f ,
- does not collide any obstacle as the robot traverses the path.

Note that as soon as this is done, the pass of the end-effector is unambiguously defined by the solution of the forward kinematics problem.

So, formally, the task is to find a continuous function $\gamma(\cdot)$ such that

$$\gamma : [0, 1] \rightarrow \mathcal{Q}_{free} \quad \text{with } \gamma(0) = q_s, \text{ and } \gamma(1) = q_f.$$

However, typically, there are additional requirements such as

- Some intermediate (pass through) points q_i may be specified;
- The path should be sufficiently smooth;
- Some optimality is required (smallest path length, smallest curvature, etc.);
- Avoiding closeness to singular configurations.

There are several approaches to solve the path planning problem.

4.2.1 Path Planning: Potential Field Approach

The basic idea of this approach is the following.

- Treat a robot as a particle in $\mathbb{R}^n$ moving under the influence of an artificial potential field $U(\cdot)$ to be defined.
- Function $U(\cdot)$ should have
 - global minimum at $q_f \Rightarrow$ this point is attractive

- maximum or to be $+\infty$ in the points of $\mathcal{QO}$ $\Rightarrow$ these points repel the robot

It is a good idea to construct such function $U(\cdot)$ in a simple form, where one can easily add or remove an obstacle and change q_f . The commonly used form for $U(\cdot)$ is

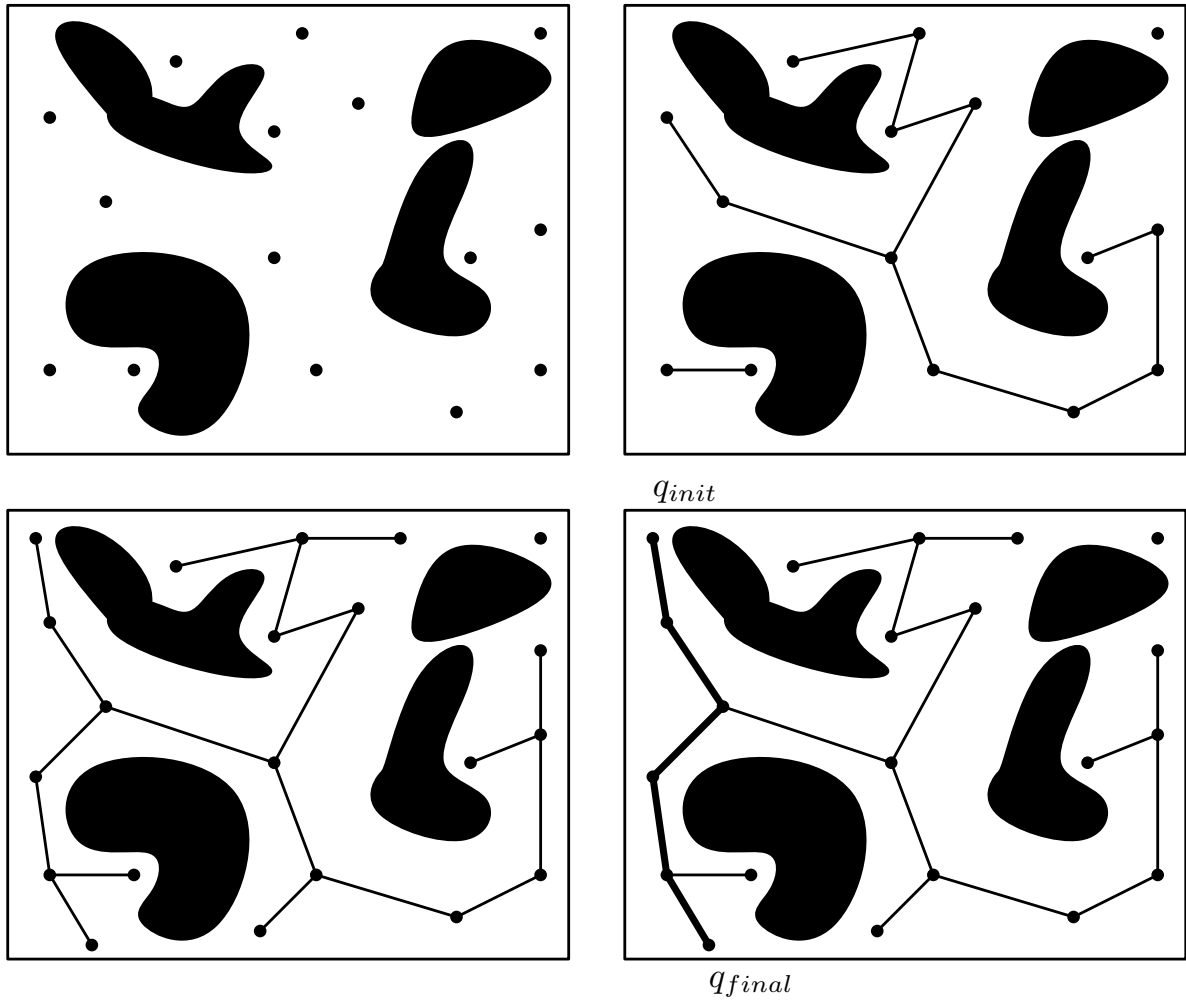
$$U(q) = U_{att}(q) + \left(U_{rep}^{(1)}(q) + U_{rep}^{(2)}(q) + \cdots + U_{rep}^{(N)}(q) \right)$$

4.2.2 Path Planning: Probabilistic Road Map Method

The basic idea of this approach is the following.

- Sample randomly the configuration space $\mathcal{Q}$, i.e. randomly choose a large set of points in $\mathcal{Q}$.
- Those samples that belong to $\mathcal{QO}$ are disregarded, while with the others one proceed as follows.
- Connect the points that are close pairwise in a particular way satisfying all the requirements and bridge all the isolated components (add additional points if necessary), e.g.
 - Choose the way to measure distance between points in $\mathcal{Q}$, i.e. define a continuous real-valued distance function $d(\cdot)$.
 - Choose $\varepsilon > 0$ and find k neighbors of distance no more than ε that can be connected to the current one without violating any requirements while following a simple pattern, such as connecting points along a straight line. This step may result in fragmentation of the workspace consisting of several disjoint components.
 - Make enhancement, that is, try to connect disjoint components, possibly introducing more samples.
- Try to compute a path by connecting the initial and final points to the closest samples and using a set of pairwise connection among the samples.

An example of following such a procedure in plane is shown below.



4.3 Trajectory Planning

4.3.1 From path to trajectory

In essence, trajectory is a path $\gamma : [0, 1] \rightarrow \mathcal{Q}_{free}$ enhanced by an explicit parametrization of time:

$$[T_s, T_f] \ni \textcolor{red}{t} \rightsquigarrow \textcolor{red}{\tau} \in [0, 1] : \textcolor{red}{q}(\textcolor{red}{t}) = \gamma(\textcolor{red}{\tau}) \in \mathcal{Q}_{free}$$

Specifying a trajectory means that, in addition to a geometrical description of a path, we make specifications on e.g.

- velocity $\frac{d}{dt}\textcolor{red}{q}(t)$ of a motion,
- acceleration $\frac{d^2}{dt^2}\textcolor{red}{q}(t)$ of a motion,
- jerk $\frac{d^3}{dt^3}\textcolor{red}{q}(t)$ of a motion,
- ...

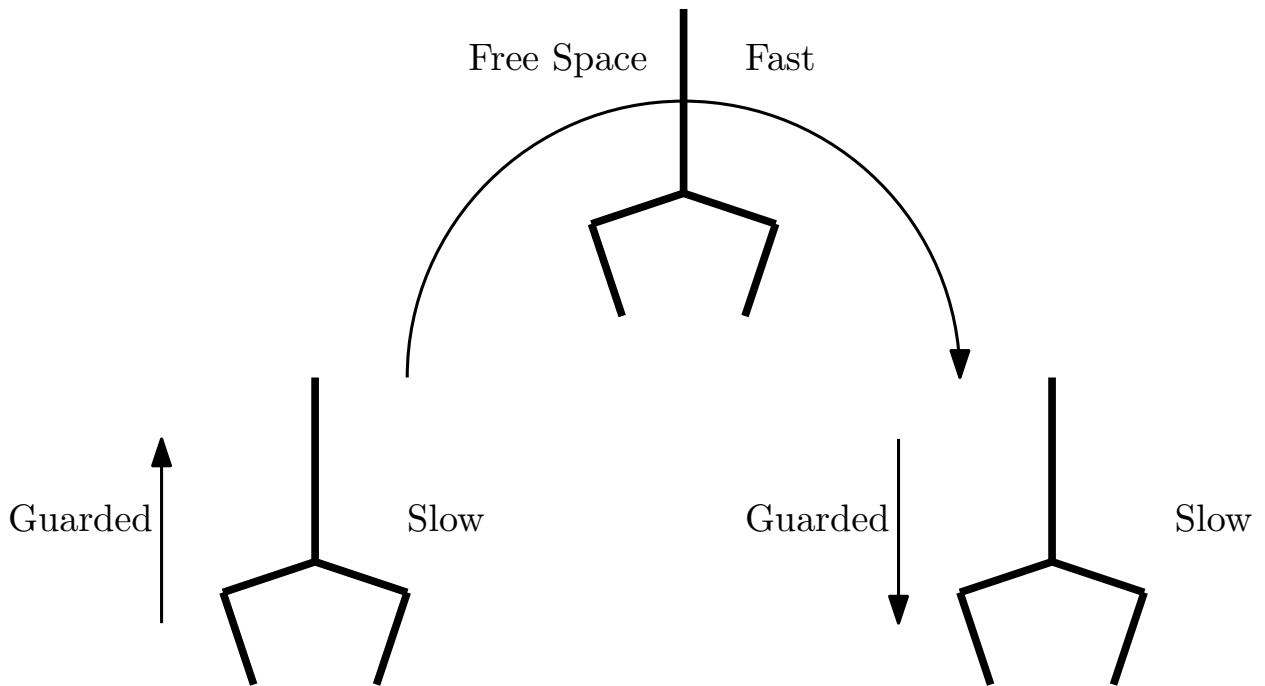
at each moment of time.

However, it is common to specify (or to redefine) a path in a form of a family of snap-shots

$$\mathbf{q}_s, \quad \mathbf{q}_1, \quad \mathbf{q}_2, \quad \mathbf{q}_3, \quad \dots, \quad \mathbf{q}_f$$

that can be obtained substituting various numbers from $[0, 1]$ into a computed path description $\gamma(\cdot)$.

Often, it is useful to group the points (make a segmentation) of snap-shots according to the task-defined accuracy. As an example, one can decompose a path into segments with fast and slow velocity profiles as illustrated below.



4.3.2 Trajectories for Point to Point Motions

Suppose now that a path is given in a form of a set of points with, possibly, additional specifications of velocities, accelerations, etc. at each of them, that are sufficiently distant from the obstacles. It is reasonable then to try finding a trajectory as a combination of pieces connecting pairs of close points.

Consider the i^{th} joint of a robot and suppose that the specification

$$\begin{aligned} \text{at time } t = t_0 \text{ is : } & \quad \mathbf{q}_i(t_0) = \mathbf{q}_0, \quad \frac{d}{dt}\mathbf{q}(t_0) = \mathbf{v}_0 \\ \text{at time } t = t_f \text{ is : } & \quad \mathbf{q}_i(t_f) = \mathbf{q}_f, \quad \frac{d}{dt}\mathbf{q}(t_f) = \mathbf{v}_f \end{aligned}$$

In addition, we might be given (or not) constraints on the accelerations

$$\frac{d^2}{dt^2}\mathbf{q}(t_0) = \alpha_0, \quad \frac{d^2}{dt^2}\mathbf{q}(t_f) = \alpha_f$$

Let us generate a polynomial of a sufficiently large degree m

$$q(t) = a_0 + a_1 t + a_2 t^2 + \dots + a_m t^m$$

that will allow us to satisfy the interpolation conditions defined by the specifications above.

Assuming the interpolation constraints

$$\begin{aligned} \text{at time } t = t_0 \text{ is : } & q_i(t_0) = \mathbf{q}_0, \quad \frac{d}{dt}q(t_0) = \mathbf{v}_0 \\ \text{at time } t = t_f \text{ is : } & q_i(t_f) = \mathbf{q}_f, \quad \frac{d}{dt}q(t_f) = \mathbf{v}_f \end{aligned}$$

for the 3^{rd} -order polynomial

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3, \quad \frac{d}{dt}q(t) = a_1 + 2a_2 t + 3a_3 t^2$$

one immediately obtains the following relations to satisfy

$$\begin{aligned} \mathbf{q}_0 &= a_0 + a_1 t_0 + a_2 t_0^2 + a_3 t_0^3 \\ \mathbf{v}_0 &= a_1 + 2a_2 t_0 + 3a_3 t_0^2 \\ \mathbf{q}_f &= a_0 + a_1 t_f + a_2 t_f^2 + a_3 t_f^3 \\ \mathbf{v}_f &= a_1 + 2a_2 t_f + 3a_3 t_f^2 \end{aligned}$$

They can be written in the following matrix form

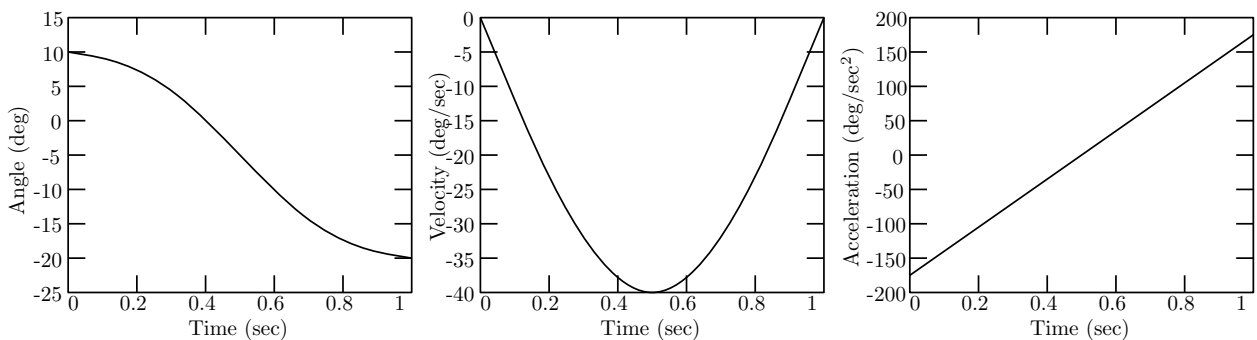
$$\begin{bmatrix} \mathbf{q}_0 \\ \mathbf{v}_0 \\ \mathbf{q}_f \\ \mathbf{v}_f \end{bmatrix} = \begin{bmatrix} 1 & t_0 & t_0^2 & t_0^3 \\ 0 & 1 & 2t_0 & 3t_0^2 \\ 1 & t_f & t_f^2 & t_f^3 \\ 0 & 1 & 2t_f & 3t_f^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

So, to be able to compute the required coefficients a_i , one needs to ensure that the determinant of the matrix above is not equal to zero.

Consider the following parameters for interpolation:

$$t = 0 \text{ and } t_f = 1, \quad q_0 = 10 \text{ and } q_f = -20, \quad v_0 = v_f = 0$$

the resulting trajectory is shown below



As can be seen the obtained trajectory is unacceptable due to very high accelerations. To avoid generating such trajectories one, obviously, needs to define additional interpolation conditions.

Consider the i^{th} joint of a robot and suppose that the specifications are

$$\begin{aligned} \text{at time } t = t_0 \text{ is : } & \quad q_i(t_0) = q_0, \quad \frac{d}{dt}q(t_0) = v_0 \\ \text{at time } t = t_f \text{ is : } & \quad q_i(t_f) = q_f, \quad \frac{d}{dt}q(t_f) = v_f \end{aligned}$$

and introduce additional constraints on the accelerations

$$\frac{d^2}{dt^2}q(t_0) = \alpha_0 \quad \frac{d^2}{dt^2}q(t_f) = \alpha_f$$

Clearly, to find an interpolating polynomial, one needs to choose a polynomial of order ≥ 5 such as

$$q(t) = a_0 + a_1t + a_2t^2 + a_3t^3 + a_4t^4 + a_5t^5$$

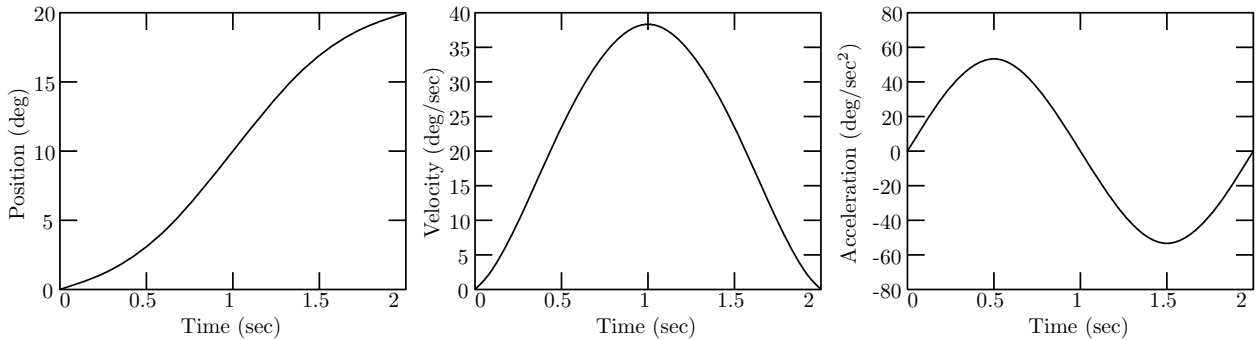
Consider, as above, the following parameters for interpolation:

$$t = 0 \text{ and } t_f = 1, \quad q_0 = 10 \text{ and } q_f = -20, \quad v_0 = v_f = 0$$

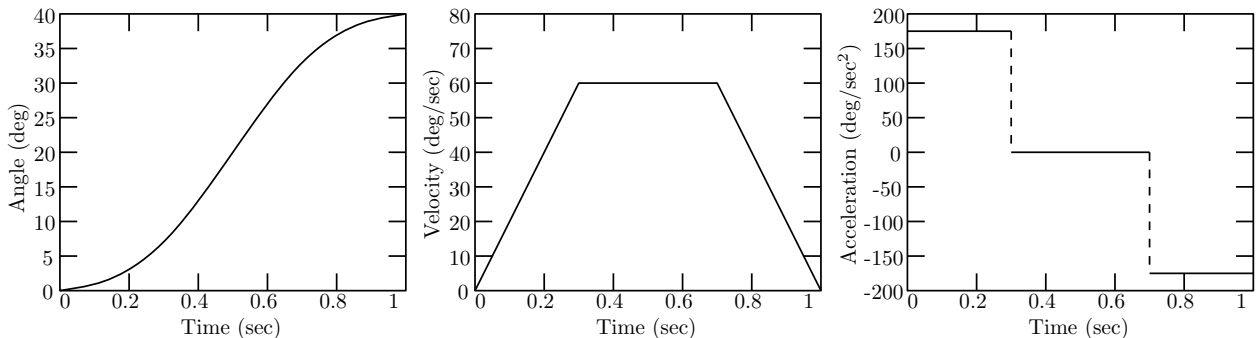
but this time let us require

$$\alpha_0 = 0 \text{ and } \alpha_f = 0.$$

The resulting trajectory is shown below



In the case when time optimality is desirable, trajectories are often generated using LSPB — Linear segments with parabolic blends, that are illustrated below.



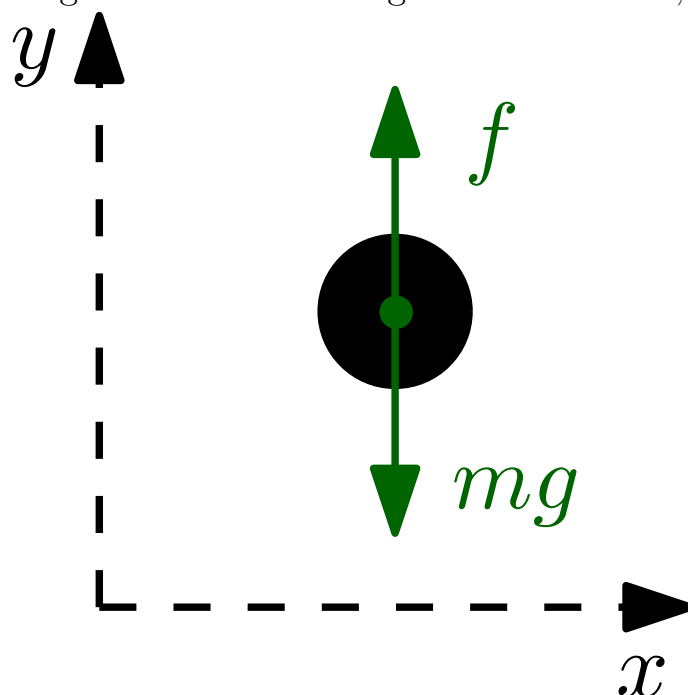
Chapter 5

Dynamics: Euler-Lagrange Equations

Let us now describe how to derive a mathematical equations describing dynamics. We will follow the algebraic approach, introduced by Lagrange. Although, it has several well-known shortcomings that include difficulty in introducing not-ideal physical interactions between various components of a robot, this seems to be the most straightforward way to derive equations of motion without using mechanical intuition.

5.1 Example: from Newton's Law to Euler-Lagrange Equations

Let us start with rewriting a classical Newton's equation of motion for a particle moving along a vertical line in a gravitational field, illustrated below.



The **2nd** Newton's law for the particle states that the particle's acceleration is proportional to the sum of all the forces acting on it, i.e.

$$m \frac{d^2}{dt^2} y = \sum F_i = f - mg$$

where

- m is the mass, which is the (constant) inertial characteristic of the particle,
- f is the cumulative external (vertical) force, and
- $-mg$ is the gravitational force acting on the particle, which is proportional to the acceleration of the gravity that is equal to $-g$.

The above equation of motion of the particle, can be rewritten in a quite different way!

The left-hand side of the equation of motion can be represented as

$$m \frac{d^2}{dt^2} y = \frac{d}{dt} \left(m \frac{d}{dt} y \right) = \frac{d}{dt} \left(m \frac{\partial}{\partial \dot{y}} \left[\frac{1}{2} \dot{y}^2 \right] \right) = \frac{d}{dt} \left(\frac{\partial}{\partial \dot{y}} \mathcal{K} \right)$$

where

$$\mathcal{K} = \frac{1}{2} m \dot{y}^2$$

is so-called **kinetic energy**.

The gravitational force, appearing in the equation of motion, can be represented as

$$mg = \frac{\partial}{\partial y} [mgy] = \frac{\partial}{\partial y} \mathcal{P}$$

where

$$\mathcal{P} = mgy$$

is so-called **potential energy** of the gravity.

Then, the second Newton's law can be rewritten as the so-called **Euler-Lagrange equation**

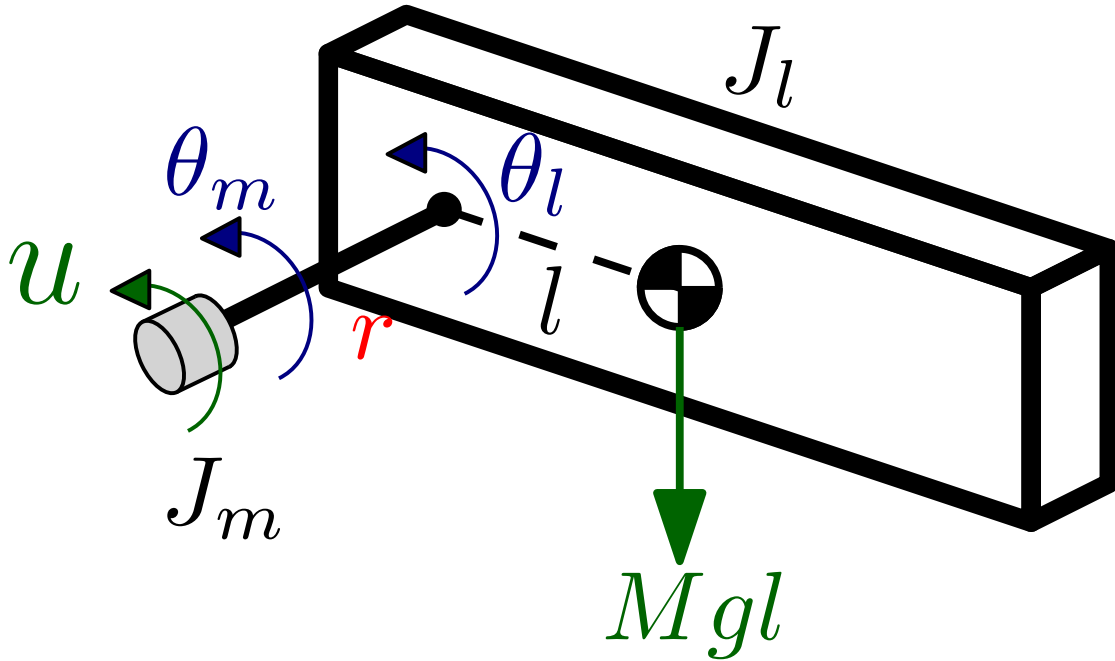
$$\frac{d}{dt} \left(\frac{\partial}{\partial \dot{y}} \mathcal{L} \right) - \frac{\partial}{\partial y} \mathcal{L} = f \quad \text{with} \quad \mathcal{L} = \mathcal{K} - \mathcal{P}$$

where the function $\mathcal{L}(y, \dot{y})$ is called the **Lagrangian** and is computed as the difference between the kinetic and the potential energies.

Equations of motion to describe dynamics of many mechanical systems can be derived using such formalism instead of direct application of the Newton's second law. Evidently, one just needs to know how to compute the energies.

5.2 Example: Dynamics of a Single Link Robot

We will discuss later how to compute the energies for a multi-link robot in a systematic way. Let us, however, illustrate derivation of the equation of motion for a one-link rotational robot schematically shown below.



Here, we have a rigid link of mass M , absolute location of which is defined by the angle θ_l , that is coupled through a gearbox with ration r , to a rotor of a DC motor, location of which is defined by the angle θ_m . The torque produced by the motor is denoted by u and it is amplified by the gearbox with the gain r . Note that $\theta_m = r\theta_l$ since (in the considered ideal case) the angle is reduced by the gear box with the gain $1/r$.

The kinetic energy of our robot is a sum of kinetic energies of the link and of the rotor. Each of them is proportional to the square of the corresponding angular velocity, i.e.

- Kinetic energy:

$$\mathcal{K} = \frac{1}{2}J_m\dot{\theta}_m^2 + \frac{1}{2}J_l\dot{\theta}_l^2 = \frac{1}{2}(r^2J_m + J_l)\dot{\theta}_l^2$$

where J_m and J_l are moments of inertia defining inertial characteristics of the two rigid bodies (the rotor of the motor and the link, respectively).

The potential energy (up to a constant initial value) is proportional to the height of the center of mass of the link with the coefficient of proportionality equal to Mg , i.e.

- Potential energy:

$$\mathcal{P} = Mgl(1 - \cos \theta_l)$$

Therefore,

- the Lagrangian is

$$\mathcal{L} = \mathcal{K} - \mathcal{P} = \frac{1}{2} (\mathbf{r}^2 J_m + J_l) \dot{\theta}_l^2 - M g l (1 - \cos \theta_l)$$

and one can easily compute the equation for the dynamics to be

$$\frac{d}{dt} \left(\frac{\partial}{\partial \dot{\theta}_l} \mathcal{L} \right) - \frac{\partial}{\partial \theta_l} \mathcal{L} = \tau_l = \mathbf{r} u$$

where $\mathbf{f} = \tau_l = \mathbf{r} u$ since the torque of the motor is amplified by the gearbox.

Finally, one obtains

$$(\mathbf{r}^2 J_m + J_l) \ddot{\theta}_l + M g l \sin \theta_l = \mathbf{r} u$$

The same equation can be obtained using the Lagrangian in terms of the motor variable

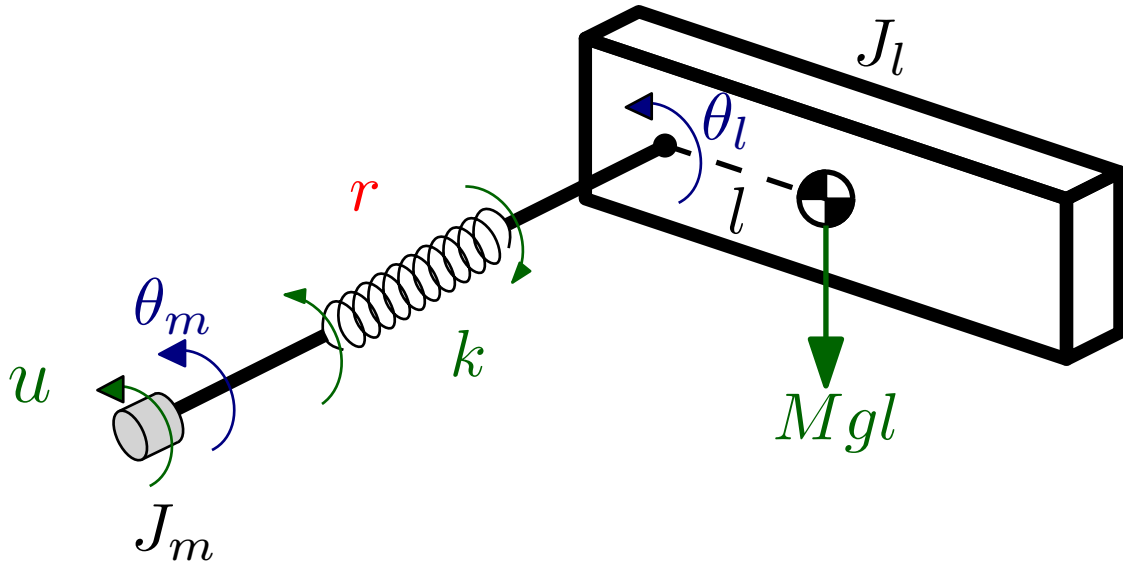
$$\mathcal{L} = \frac{1}{2} \left(J_m + \frac{J_l}{\mathbf{r}^2} \right) \dot{\theta}_m^2 - M g l \left(1 - \cos \frac{\theta_m}{\mathbf{r}} \right)$$

and the Euler-Lagrange equation

$$\frac{d}{dt} \left(\frac{\partial}{\partial \dot{\theta}_m} \mathcal{L} \right) - \frac{\partial}{\partial \theta_m} \mathcal{L} = \tau_m = u$$

Note that in the right-hand side here one must use the torque applied to the motor, τ_m , and not the torque applied to the link, τ_l , as in the previous case. This is related to the notion of the generalized force. We will see later that the choice of a generalized coordinate is not important (it could be, e.g., θ_l or θ_m) but one must be careful with obtaining a correct expression for the generalized force. In fact, $\theta_m = \mathbf{r} \theta_l$ implies $\tau_l = \mathbf{r} \tau_m$. This is related to the principle of virtual work, to be formulated later, that in the considered simple case can be written as $\tau_l \delta \theta_l = \tau_m \delta \theta_m$.

It is also useful to derive equations of motions for the same single link example but dropping the assumption of a perfect gearbox, that is with possibility to have $\theta_m \neq \mathbf{r} \theta_l$. To proceed, one needs to propose a mechanical model for the connection of the two rigid bodies: the rotor of the motor and the link. One possibility is to use an ideal spring model illustrated below.



In this case, the energies become functions of two independent generalized coordinates and their derivatives. Including into the expression of the potential energy a quadratic term in $(\theta_m - \mathbf{r}\theta_l)$ with a gain equal to the coefficient of stiffness of the spring, one obtains the Lagrangian

$$\mathcal{L} = \frac{1}{2}J_m\dot{\theta}_m^2 + \frac{1}{2}J_l\dot{\theta}_l^2 - Mgl(1 - \cos \theta_l) - \frac{1}{2}k\left(\frac{\theta_m}{\mathbf{r}} - \theta_l\right)^2$$

and use it to obtain two differential equations

$$\frac{d}{dt}\left(\frac{\partial}{\partial \dot{\theta}_l}\mathcal{L}\right) - \frac{\partial}{\partial \theta_l}\mathcal{L} = \tau_l, \quad \frac{d}{dt}\left(\frac{\partial}{\partial \dot{\theta}_m}\mathcal{L}\right) - \frac{\partial}{\partial \theta_m}\mathcal{L} = \tau_m$$

Realizing that the generalized forces are $\tau_l = 0$ and $\tau_m = u$, we have

$$J_l\ddot{\theta}_l + Mgl\sin\theta_l = k\left(\frac{\theta_m}{\mathbf{r}} - \theta_l\right), \quad J_m\ddot{\theta}_m + \frac{k}{\mathbf{r}}\left(\frac{\theta_m}{\mathbf{r}} - \theta_l\right) = u$$

It is useful to recover the previous equation of dynamics obtained under assumption of the identity $\theta_m - \mathbf{r}\theta_l \equiv 0$, which is called a holonomic constraint (to be introduced in the next section). This can be done if we also assume that the stiffness is infinite, i.e $k = \infty$ and rewrite the differential equations substituting the elastic force with an unknown reaction force keeping the perfect relation between the two angles

$$F = k\left(\frac{\theta_m}{\mathbf{r}} - \theta_l\right), \quad J_l\ddot{\theta}_l + Mgl\sin\theta_l = F, \quad J_m\ddot{\theta}_m + \frac{1}{\mathbf{r}}F = u$$

The last two differential equations together with the holonomic constraint (the identity imposed on the variables) are called Euler-Lagrange equations of the first kind or constrained Euler-Lagrange equations.

Now, a single differential equation describing dynamics can be obtained

substituting either $\theta_m = r\theta_l$ or $\theta_l = \theta_m/r$ and eliminating the unknown reaction force $\mathbf{F}$. However, in addition to $(r^2 J_m + J_l) \ddot{\theta}_l + Mgl \sin \theta_l = ru$ we also obtain the expression for the force $\mathbf{F} = Mgl \sin \theta_l + J_l \frac{ru - Mgl \sin \theta_l}{r^2 J_m + J_l}$ that can be used to verify systems' integrity.

We will derive below Euler-Lagrange equations for a robotic manipulator. After that, we will show how to compute potential and kinetic energies using the variables of the DH convention defined in Chapter 2. However, in order to proceed, we need to introduce a few useful notions such as holonomic constraints, degrees of freedom, virtual work, generalized coordinates, and generalized forces.

5.3 Holonomic Constraints and Virtual Work

The Euler-Lagrange formalism leads to a description of dynamics for a mechanical system in the form of a system of nonlinear interconnected differential equations. The number of these equations coincides with the so-called number of degrees of freedom (DOF), provided the mechanical parts of the system are subject to so-called holonomic constraints. Both notions are to be defined next.

5.3.1 Degrees of Freedom and Holonomic Constraints

The **number of degrees of freedom** (DOF) is the number of scalar variables that are necessary and sufficient to describe location of all the components of a mechanical system. For example, location of a particle (material point) in $\mathbf{3D}$ can be described by its $\mathbf{3}$ Cartesian coordinates and therefore, a single particle has $\mathbf{3}$ DOF.

If, by some reasons, we know that a particle is restricted to stay on a particular plane, then its location can be described by $\mathbf{2}$ Cartesian (or polar) coordinates introduced on the plane. Hence, it has $\mathbf{2}$ DOF. Here, it is useful to realize, that the restriction of the particle location can be described by a single (linear) scalar algebraic equation on its $\mathbf{3}$ coordinates (recall the equation of a plane in a Euclidean space). Such a linear or nonlinear identity satisfied by coordinates is called a holonomic constraint. Formally, a **holonomic constraint** is an algebraic relation among the scalar quantities specifying locations of various components of a mechanical system. One can formally

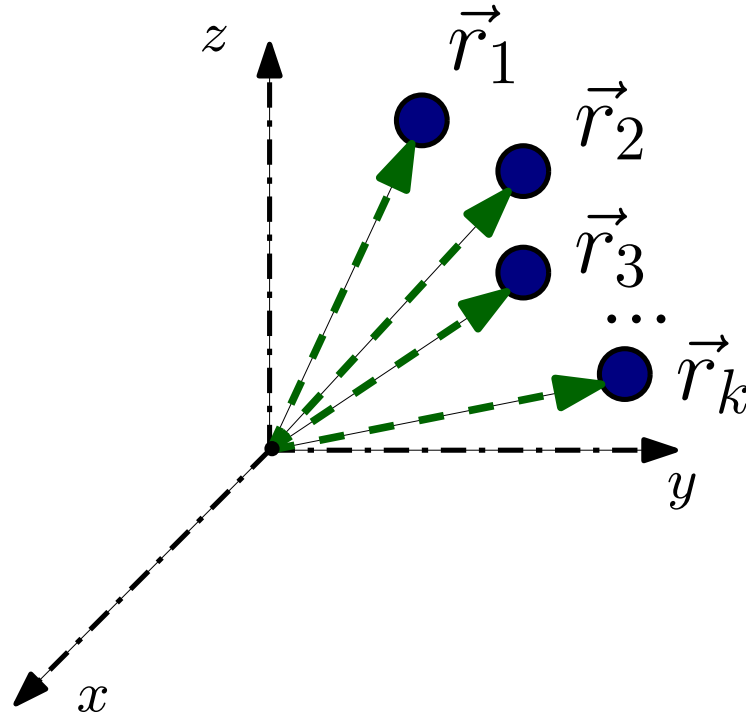
find the number of DOF of a system with constraints as a difference between the number of DOF for a system without constraints and the number of constraints,

provided the constraints are independent (no one of them is implied by others) and the number of constraints on each component is not bigger than the number of DOF of this component.

For an illustration, consider a system of **2** particles in a vertical plane connected by a weightless rod. To use the definition directly, one can compute the number of DOF as follows: To describe location of the system we need to specify **2** coordinates of one particle and **1** angle that the rod makes with a fixed line on the plane. Hence, the number of DOF for the system is $\mathbf{3} = \mathbf{2} + \mathbf{1}$. To compute this number using constraints, one say that there are **2** particles in plane with **2** DOFs each and **1** holonomic constraint (the distance between them), so that we have $\mathbf{3} = \mathbf{2} \cdot \mathbf{2} - \mathbf{1}$. Alternatively, one can see **2** particles in space, i.e. with **3** DOFs each, and **3** holonomic constraints (**1** fixed distance and **2** restrictions to the plane), so that $\mathbf{3} = \mathbf{3} \cdot \mathbf{2} - (\mathbf{1} + \mathbf{2})$.

A more interesting example is a free rigid body in space. We know from our discussion in Chapter 2 that, due to rigidity, locations of all points can be described by **6** variables: **3** describe location of a single point, called a pole, while **3** other describe the orientation of the body. From the other hand, note that, formally, a rigid body consists of an infinite number of particles, distances among which are pairwise constant. So, taking **3** particles belonging to the body that do not belong to a single line (to ensure that two constraints do not imply the other) we compute that the number of DOF is not less than $\mathbf{6} = \mathbf{3} \cdot \mathbf{3} - \mathbf{3}$, where we counted **3** particles with **3** DOFs each and **3** fixed distances. Now, one can notice that adding another particle would add **3** DOFs together with at least **3** fixed distances to the other particles. So, the number of DOF seems really to be **6**.

Let us now, for simplicity, forget the fact that we work with rigid bodies and formalize the notions just discussed for a mechanical system formed by a finite set of particles. To this end, consider an unconstrained system of $\mathbf{k}$ particles in $\mathbf{3D}$. The number of DOF is $\mathbf{3k}$ if they are not restricted and is less, if the particles are constrained.



Each of l (independent) constraints imposed on k particles with coordinates $\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_k \in \mathbb{R}^3$, written with respect to an inertial frame¹, is called **holonomic**, if it can be written as a scalar identity

$$g_i(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_k) = 0, \quad i = 1, 2, \dots, l$$

For example, for the system of 2 particles joined by a massless rigid wire of length L we have

$$\mathbf{r}_1, \mathbf{r}_2 \in \mathbb{R}^3 : g(\mathbf{r}_1, \mathbf{r}_2) = \|\mathbf{r}_1 - \mathbf{r}_2\|^2 - L^2 = (\mathbf{r}_1 - \mathbf{r}_2)^T (\mathbf{r}_1 - \mathbf{r}_2) - L^2 = 0$$

It is important to realize that

Presence of each holonomic constraint implies presence of (internal) forces (called **constraint forces**), that make the particles to obey the constraint.

Differentiating each constraint function g_i with respect to time, one obtains another constraint, which is another identity that must be satisfied by the state variables, i.e. by the positions and velocities

$$\frac{d}{dt}g_i(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_k) = \frac{\partial g_i}{\partial \mathbf{r}_1} \frac{d}{dt}\mathbf{r}_1 + \dots + \frac{\partial g_i}{\partial \mathbf{r}_k} \frac{d}{dt}\mathbf{r}_k = 0$$

or

$$\frac{\partial g_i}{\partial \mathbf{r}_1} d\mathbf{r}_1 + \dots + \frac{\partial g_i}{\partial \mathbf{r}_k} d\mathbf{r}_k = 0$$

¹Here and below we skip the superscript indicating this particular frame since it is the only frame.

that is a constraint in the form

$$\omega_1(r_1, \dots, r_k)dr_1 + \dots + \omega_k(r_1, \dots, r_k)dr_k = 0$$

If a constraint of this kind appears not as a consequence of a holonomic constraint, that is if it cannot be integrated to a form of a holonomic constraint, than it is called **non-holonomic**. Non-holonomic constraints can be often understood as a consequence of ideal sliding that restricts direction of possible velocities but not possible positions (e.g., a car cannot move in the direction orthogonal to the wheels but can certainly be driven in the corresponding parallel location).

5.3.2 Generalized Coordinates, Virtual Displacements

Let us consider the case when only holonomic constraints are present and, for the sake of simplicity, assume as above that we have just a finite number of particles, rigidly connected pairwise by weightless wires.

If a system of k particles, positions of which are specified as components of $3D$ vectors $\vec{r}_i$, $i = 1, \dots, k$, is subject to a set of $l \geq 1$ holonomic constraints, then

- it may be possible to express the coordinates of the particles (locally) as functions of n variables, $q_1, \dots, q_n$, with $n \leq 3k$

$$r_1 = r_1(q_1, \dots, q_n), r_2 = r_2(q_1, \dots, q_n), \dots, r_k = r_k(q_1, \dots, q_n)$$

Suppose this is done. Then,

- the smallest set of such variables $q_1, \dots, q_n$ is called the set of **generalized coordinates**, while
- the number of such variables, n , is called the **number of degrees of freedom**.

It should be noted that if the system consists of an **infinite** number of particles, then it might have **finite** number of DOF. For example, the number of DOF for an unconstrained rigid body, moving in $3D$ is **6**.

Given a system of k -particles under l holonomic constraint

$$g_i(r_1, r_2, \dots, r_k) = 0, \quad i = 1, 2, \dots, l$$

Suppose, r_i for $i = 1, \dots, k$ satisfy the constraints, by definition the set of infinitesimal displacements

$$\delta r_1, \delta r_2, \dots, \delta r_k$$

that are consistent with the constraints, i.e.

$$g_i(r_1 + \delta r_1, r_2 + \delta r_2, \dots, r_k + \delta r_k) = 0, \quad i = 1, 2, \dots, l$$

are called **virtual displacements**.

Expanding the left-hand side of the last expression into the Taylor series, using the previous expression and dropping higher-order terms, one immediately obtains

$$\frac{\partial g_i}{\partial r_1} \delta r_1 + \dots + \frac{\partial g_i}{\partial r_k} \delta r_k = 0, \quad i = 1, 2, \dots, l$$

This is a system of l linear equation with respect to k vector variables δr_i that must be satisfied. The fact that the constraints are independent mathematically means that the rectangular matrix constructed from the partial derivatives has the maximal rank. It follows that only

$$n = 3k - l \quad \text{in } 3D \quad \text{and} \quad n = 2k - l \quad \text{in } 2D$$

virtual displacement are independent. The respective n independent linear combinations of the virtual displacement can be taken arbitrary and define the freedom of motions that are consistent with all the holonomic constraints. This is why n is called the number of DOF.

As an example, let us compute virtual displacements of a rigid planar bar. Such infinitesimal motions do not destroy the virtual constraint on the end-points of the bar

$$g = (r_1 - r_2)^T (r_1 - r_2) - L^2 = 0$$

if r_1 and r_2 are perturbed

$$r_1 \rightarrow (r_1 + \delta r_1), \quad r_2 \rightarrow (r_2 + \delta r_2)$$

that is

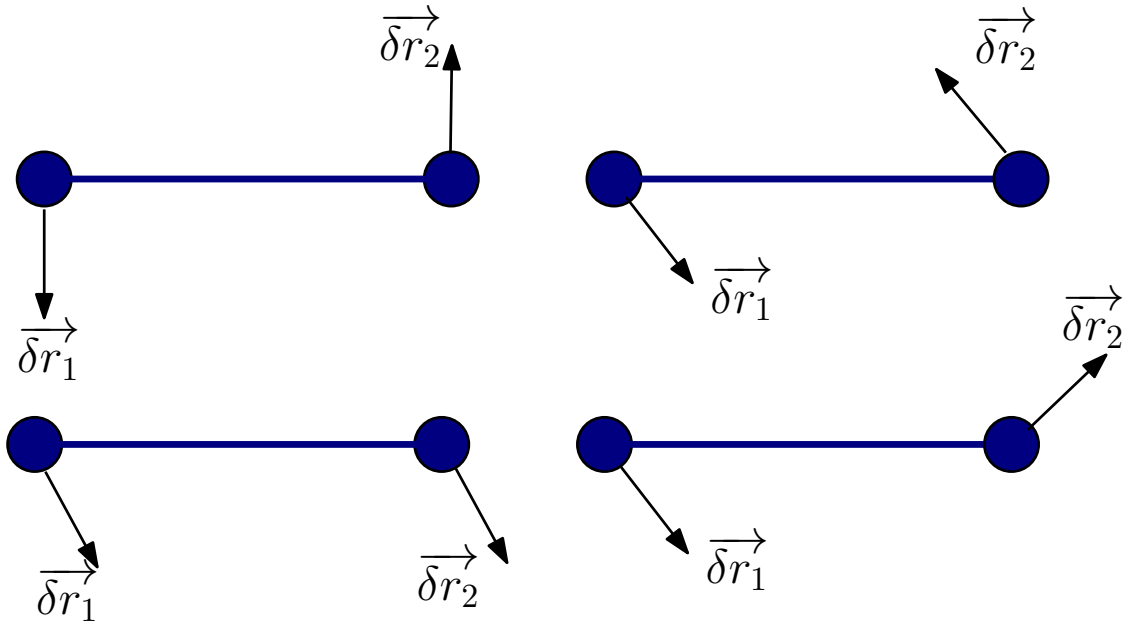
$$\left((r_1 + \delta r_1) - (r_2 + \delta r_2) \right)^T \left((r_1 + \delta r_1) - (r_2 + \delta r_2) \right) = L^2$$

It follows (dropping the second order terms) that

$$(r_1 - r_2)^T (\delta r_1 - \delta r_2) = 0$$

This means that the vector $\vec{r}_1 - \vec{r}_2$, defining relative displacement of the end-point of the bar, must be orthogonal to the vector $\delta \vec{r}_1 - \delta \vec{r}_2$, which is defined by the differences of the possible virtual displacement of the end

points. Some examples, including the pure translations of the bar and the pure rotation of the bar are shown in the figure below.



Note that all other possible rigid motions are clearly combinations of translations and rotations. Let us assume now that the bar is restricted to the plane. Instead of introducing two other holonomic constraints, let us assume that the vectors defining the end-point are in $2\mathbf{D}$, i.e.

$$\mathbf{r}_1 = \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}, \quad \mathbf{r}_2 = \begin{bmatrix} x_2 \\ y_2 \end{bmatrix}$$

Let us introduce $\mathbf{n} = \mathbf{3} = 4 - 1 = 2\mathbf{k} - \mathbf{l}$ generalized coordinates as the coordinates of the left end-points and the angle defining orientation of the bar on the plane

$$q_1 = x_1, \quad q_2 = y_1, \quad q_3 = \text{atan2}(y_2 - y_1, x_2 - x_1)$$

Expressing the vectors in terms of the generalized coordinates, one obtains

$$\mathbf{r}_1 = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}, \quad \mathbf{r}_2 = \begin{bmatrix} q_1 + L \cos(q_3) \\ q_2 + L \sin(q_3) \end{bmatrix}$$

while for the (not independent) virtual displacements we have

$$\delta \mathbf{r}_1 = \begin{bmatrix} \delta q_1 \\ \delta q_2 \end{bmatrix}, \quad \delta \mathbf{r}_2 = \begin{bmatrix} \delta q_1 - L \sin(q_3) \delta q_3 \\ \delta q_2 + L \cos(q_3) \delta q_3 \end{bmatrix}$$

Re-computing the restriction on the virtual displacements one obtains the

identity

$$(\mathbf{r}_1 - \mathbf{r}_2)^T (\delta \mathbf{r}_1 - \delta \mathbf{r}_2) = \begin{bmatrix} L \cos(q_3) \\ L \sin(q_3) \end{bmatrix}^T \begin{bmatrix} -L \sin(q_3) \delta q_3 \\ L \cos(q_3) \delta q_3 \end{bmatrix} \equiv 0$$

that tells us that all three generalized coordinates are indeed independent and not restricted by the holonomic constraint in any way. Note however, that it is typically easy to compute the number of DOF but may in general be difficult to introduce the generalized coordinates. Whenever they are introduced, the holonomic constraints must be automatically satisfied.

Using the concept of virtual displacements one can introduce a very important principle of mechanics.

5.3.3 Principle of Virtual Work and Constraint Forces

Consider a system of k -particles; suppose that

- The system has $l \geq 1$ holonomic constraints, that is each i -th particle is exposed to a (lumped) **constraint force** $\mathbf{f}_i^c$, $i = 1, \dots, k$;
- There are, possibly, (lumped) external forces $\mathbf{f}_i^e$, $i = 1, \dots, k$, applied to the particles as well;
- Each particle is in an equilibrium (i.e. at rest) $\Rightarrow \mathbf{f}_i^c + \mathbf{f}_i^e = \mathbf{0}$, $\forall i$.

As a result, trivially, the **virtual work**, which is defined as the product of the total force and the virtual displacement,

$$\delta W^i = (\mathbf{f}_i^c + \mathbf{f}_i^e)^T \delta \mathbf{r}_i,$$

done by all the forces acting on each i^{th} particle, is equal to zero.

Then, the total virtual work done by all the forces applied to the whole system of particle is zero as well:

$$\delta W = \sum_{i=1}^k \delta W^i = \sum_{i=1}^k (\mathbf{f}_i^c + \mathbf{f}_i^e)^T \delta \mathbf{r}_i = 0$$

Now, the constraint forces are defined as forces that keep the constraints satisfied, while the virtual displacements are assumed to be consistent with them. Hence, one can assume that the work done by the constraint forces along the virtual displacement must be equal to zero, i.e.

$$\sum_{i=1}^k (\mathbf{f}_i^c)^T \delta \mathbf{r}_i = 0$$

Whenever this key assumption is true, the holonomic constraints are called **ideal**; and we will assume that this is the case.

Combining the last two equations, one concludes that for a system of particles, which are at rest, the work done by all the external forces along the virtual displacement is equal to zero as well

$$\sum_{i=1}^k (f_i^e)^T \delta \mathbf{r}_i = \mathbf{0}$$

This equation is called the **principle of virtual work**.

Next, we will introduce a generalization of this principle to the case of a system of moving particles and use it to derive the equations of motions that do not contain the internal forces implied by the constraints.

As a result, we will be able to directly predict and compute the motions as solutions of a system of differential equations without computation of the constraint forces, provided we have all the external forces specified as functions of time or other internal variables.

5.4 D'Alembert Principle and Dynamics

Let us derive the Euler-Lagrange equations of motion for a finite family of particles using generalized coordinates and a fundamental general principle introduced next.

5.4.1 D'Alembert Principle

Consider a system of k -particles; suppose that

- The system has $l \geq 1$ **ideal holonomic constraints**, that is each i -th particle is exposed to a (lumped) **constraint force** $\mathbf{f}_i^c$, $i = 1, \dots, k$;
- There are, possibly, (lumped) external forces $\mathbf{f}_i^e$, $i = 1, \dots, k$, applied to the particles as well;
- **The system is moving.**

The D'Alembert Principle states that for a system of moving particles the work done by all the external forces along the virtual displacements is equal to zero provided we count the inertia forces, i.e.

$$\sum_{i=1}^k (\mathbf{f}_i^e - \frac{d}{dt} [\mathbf{m}_i \dot{\mathbf{r}}_i])^T \delta \mathbf{r}_i = 0$$

where $\mathbf{m}_i$ is the mass of the i^{th} particle.

Obviously, the expression above reduces to the principle of virtual work whenever the particles are not moving so that $\dot{\mathbf{r}}_i = \mathbf{0} \forall i$.

To derive equations of motions we will proceed as follows:

- Rewrite $\sum_{i=1}^k (\mathbf{f}_i^e)^T \delta \mathbf{r}_i$ as a function of generalized coordinates $\mathbf{q}_i$ and the (independent) **virtual generalized displacements** $\delta \mathbf{q}_i$, that are different from the virtual displacements $\delta \mathbf{r}_i$. This will lead to a definition of a generalized force.
- Rewrite $\sum_{i=1}^k \frac{d}{dt} [\mathbf{m}_i \dot{\mathbf{r}}_i]^T \delta \mathbf{r}_i$ as a function of generalized coordinates $\mathbf{q}_i$ and the virtual generalized displacements $\delta \mathbf{q}_i$ as well.
- Substitute everything into the equation of the D'Alembert principle and use the fact that $\delta \mathbf{q}_i$ are independent and arbitrary by combining the terms and equating to zeros the coefficients in front of each $\delta \mathbf{q}_i$.

5.4.2 Generalized Forces

Using the chain rule, the not-independent virtual displacements $\delta \mathbf{r}_i$, $i = 1, \dots, k$ in $\mathbf{3D}$, which are subject to l holonomic constraints, are computed in terms of the independent scalar generalized displacements δq_j , $j = 1, \dots, n$, as follows

$$\delta \mathbf{r}_i = \sum_{j=1}^n \delta q_j \frac{\partial \mathbf{r}_i}{\partial q_j}, \quad i = 1, \dots, k$$

where the partial derivatives are taken component-wise and, as before, $n = 3k - l$. Then,

$$\begin{aligned} \sum_{i=1}^k (\mathbf{f}_i^e)^T \delta \mathbf{r}_i &= \sum_{i=1}^k (\mathbf{f}_i^e)^T \left(\delta q_j \sum_{j=1}^n \frac{\partial \mathbf{r}_i}{\partial q_j} \right) \\ &= \sum_{j=1}^n \delta q_j \left(\sum_{i=1}^k (\mathbf{f}_i^e)^T \frac{\partial \mathbf{r}_i}{\partial q_j} \right) = \sum_{j=1}^n \delta q_j \psi_j \end{aligned}$$

The function ψ_j is called j^{th} **generalized force**:

$$\psi_j = \sum_{i=1}^k \left((\mathbf{f}_i^e)^T \frac{\partial \mathbf{r}_i}{\partial q_j} \right) = \sum_{i=1}^k \left((\mathbf{f}_i^e)^T \frac{\partial \dot{\mathbf{r}}_i}{\partial \dot{q}_j} \right)$$

where for the last equality we have used the identity $\boxed{\frac{\partial \mathbf{r}_i}{\partial q_j} = \frac{\partial \dot{\mathbf{r}}_i}{\partial \dot{q}_j}}$ that follows from the chain rule of differentiation

$$\dot{\mathbf{r}}_i = \frac{\partial \mathbf{r}_i}{\partial q_j} \dot{q}_j + \sum_{k \neq j} \frac{\partial \mathbf{r}_i}{\partial q_k} \dot{q}_k \quad \Rightarrow \quad \frac{\partial}{\partial \dot{q}_j} (\dot{\mathbf{r}}_i) = \frac{\partial}{\partial \dot{q}_j} \left(\frac{\partial \mathbf{r}_i}{\partial q_j} \dot{q}_j + \dots \right)$$

The next two cases are somewhat generic:

- When q_j is defined as a position variable, e.g. when it is one of the coordinates of one of the particles, ψ_j can be associated with a projection of a force acting along its direction of change.
- When q_j is defined as an angular variable, e.g. when it is an angle (partially or fully) defining orientation of a mass-less link connecting two particles, ψ_j can be associated with a moment acting along the corresponding rotation.

As an example, let us return to a planar bar but, for the sake of simplicity, assume that it is formed by two particles connected by a rigid mass-less rod.

We have

$$\dot{\mathbf{r}}_1 = \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}, \quad \dot{\mathbf{r}}_2 = \begin{bmatrix} \dot{q}_1 - L \sin(q_3) \dot{q}_3 \\ \dot{q}_2 + L \cos(q_3) \dot{q}_3 \end{bmatrix}$$

Suppose now that there are two (lumped) external forces acting on the particles

$$\mathbf{f}_1^e = \begin{bmatrix} F_{1x} \\ F_{1y} \end{bmatrix}, \quad \mathbf{f}_2^e = \begin{bmatrix} F_{2x} \\ F_{2y} \end{bmatrix}$$

Using the formula above one can easily compute the generalized forces

$$\psi_1 = F_{1x} + F_{2x}, \quad \psi_2 = F_{1y} + F_{2y}, \quad \psi_3 = F_{2x}(-L \sin q_3) + F_{2y}(L \cos q_3)$$

So, the first generalized force is the projection of the sum of forces applied to the system onto the $\mathbf{x}$ -axis, the second generalized force is the projection of the sum of forces applied to the system onto the $\mathbf{y}$ -axis, while the third generalized force is the moment of all the forces applied to the system computed with respect to the left end-point of the bar.

In a similar way, the notion of a **generalized force**, which is typically a force or a moment, can be introduced in the case when the mechanical system consists of several **rigid bodies**. The difference is that the fact that there are infinitely many points give rise to the notion of a total (external) torque applied to a rigid body in parallel with a total (external) force. As a result, the generalized force ψ_j acting on the virtual displacement $\delta \mathbf{q}_j$ are formed not only by forces $\mathbf{f}_i^e$ but also by torques τ_i^e applied to the system of bodies (they can be understood as action of two forces with identical magnitudes, opposite directions that are applied at different points). The generalized forces can be computed using the next formula

$$\psi_j = \sum_{i=1}^k \left((\mathbf{f}_i^e)^T \frac{\partial \mathbf{v}_i^e}{\partial \dot{\mathbf{q}}_j} + (\tau_i^e)^T \frac{\partial \boldsymbol{\omega}_i^e}{\partial \dot{\mathbf{q}}_j} \right)$$

where $\mathbf{v}_i^e = \dot{\mathbf{r}}_i^e = -\dot{\mathbf{r}}_i$ represents coordinates of the vector opposite to the linear velocity of the point of application of the force and $\boldsymbol{\omega}_i^e = -\boldsymbol{\omega}_i$ represents the coordinates of the vector opposite to the angular velocity of the i -th rigid body respectively. Obviously, this formula can be re-written using appropriate Jacobian matrices as

$$\psi_j = \sum_{i=1}^k \left(\left(\frac{\partial \mathbf{v}_i^e}{\partial \dot{\mathbf{q}}_j} \right)^T \mathbf{f}_i^e + \left(\frac{\partial \boldsymbol{\omega}_i^e}{\partial \dot{\mathbf{q}}_j} \right)^T \tau_i^e \right) = \sum_{i=1}^k \left(\mathbf{J}_{\mathbf{v}_i^e}^T \mathbf{f}_i^e + \mathbf{J}_{\boldsymbol{\omega}_i^e}^T \tau_i^e \right)$$

Let us return to the case of a finite number of particles and proceed according to the stated above plan.

5.4.3 Virtual Work of the Inertial Forces

The expression for the virtual work of the inertial forces can be rewritten as

$$\sum_{i=1}^k \frac{d}{dt} [m_i \dot{\mathbf{r}}_i]^T \delta \mathbf{r}_i = \sum_{i=1}^k m_i \ddot{\mathbf{r}}_i^T \left(\delta \mathbf{q}_j \sum_{j=1}^n \frac{\partial \mathbf{r}_i}{\partial q_j} \right) = \sum_{i=1}^k \sum_{j=1}^n m_i \delta q_j \ddot{\mathbf{r}}_i^T \frac{\partial \mathbf{r}_i}{\partial q_j}$$

Let us now use the following identity, obtained using the product rule for differentiation

$$\frac{d}{dt} \left[\dot{\mathbf{r}}_i^T \frac{\partial \mathbf{r}_i}{\partial q_j} \right] = \ddot{\mathbf{r}}_i^T \frac{\partial \mathbf{r}_i}{\partial q_j} + \dot{\mathbf{r}}_i^T \frac{d}{dt} \left[\frac{\partial \mathbf{r}_i}{\partial q_j} \right]$$

It follows that

$$\sum_{j=1}^n \sum_{i=1}^k m_i \delta q_j \ddot{\mathbf{r}}_i^T \frac{\partial \mathbf{r}_i}{\partial q_j} = \sum_{j=1}^n \delta q_j \left[\sum_{i=1}^k m_i \left(\frac{d}{dt} \left[\dot{\mathbf{r}}_i^T \frac{\partial \mathbf{r}_i}{\partial q_j} \right] - \dot{\mathbf{r}}_i^T \frac{d}{dt} \left[\frac{\partial \mathbf{r}_i}{\partial q_j} \right] \right) \right]$$

Now, we will use the following identities

$$\mathbf{v}_i = \dot{\mathbf{r}}_i = \sum_{j=1}^n \frac{\partial \mathbf{r}_i}{\partial q_j} \dot{q}_j = \sum_{l=1}^n \frac{\partial \mathbf{r}_i}{\partial q_l} \dot{q}_l \Rightarrow \frac{\partial \mathbf{v}_i}{\partial \dot{q}_j} = \frac{\partial \mathbf{r}_i}{\partial q_j}$$

and

$$\frac{d}{dt} \left[\frac{\partial \mathbf{r}_i}{\partial q_j} \right] = \sum_{l=1}^n \frac{\partial^2 \mathbf{r}_i}{\partial q_j \partial q_l} \dot{q}_l = \frac{\partial}{\partial q_j} \left[\sum_{l=1}^n \frac{\partial \mathbf{r}_i}{\partial q_l} \dot{q}_l \right] = \frac{\partial \mathbf{v}_i}{\partial q_j}$$

Substituting them into the last expression for the virtual work of the inertial forces, we obtain

$$\begin{aligned} \sum_{i=1}^k \frac{d}{dt} [m_i \dot{\mathbf{r}}_i]^T \delta \mathbf{r}_i &= \sum_{j=1}^n \delta q_j \left[\sum_{i=1}^k \frac{d}{dt} \left[m_i \mathbf{v}_i^T \frac{\partial \mathbf{v}_i}{\partial q_j} \right] - m_i \mathbf{v}_i^T \frac{\partial \mathbf{v}_i}{\partial q_j} \right] \\ &= \sum_{j=1}^n \delta q_j \left[\frac{d}{dt} \frac{\partial \mathcal{K}}{\partial \dot{q}_j} - \frac{\partial \mathcal{K}}{\partial q_j} \right] \end{aligned}$$

where

$$\mathcal{K} = \mathcal{K}(\mathbf{q}, \dot{\mathbf{q}}) = \frac{1}{2} \sum_{i=1}^k m_i \mathbf{v}_i^T \mathbf{v}_i = \frac{1}{2} \sum_{i=1}^k m_i \|\mathbf{v}_i\|^2 = \frac{1}{2} \sum_{i=1}^k m_i \|\dot{\mathbf{r}}_i\|^2$$

is called the **kinetic energy** of the system of particles. It can be treated as a function of generalized coordinates and generalized velocities and can be computed from the amplitudes of the velocities (speeds) of all the particles.

5.4.4 Equations of Motion for Systems of Particles

To summarize, the expression for the d'Alembert principle

$$\sum_{i=1}^k \left(\mathbf{f}_i^e - \frac{d}{dt} [m_i \dot{\mathbf{r}}_i] \right)^T \delta \mathbf{r}_i = 0$$

can be rewritten using the following two expressions derived above

$$\sum_{i=1}^k \frac{d}{dt} [m_i \dot{\mathbf{r}}_i]^T \delta \mathbf{r}_i = \sum_{j=1}^n \left[\frac{d}{dt} \frac{\partial \mathcal{K}}{\partial \dot{q}_j} - \frac{\partial \mathcal{K}}{\partial q_j} \right] \delta q_j, \quad \sum_{i=1}^k (\mathbf{f}_i^e)^T \delta \mathbf{r}_i = \sum_{j=1}^n \psi_j \delta q_j$$

as follows

$$\sum_{j=1}^n \left\{ \frac{d}{dt} \frac{\partial \mathcal{K}}{\partial \dot{q}_j} - \frac{\partial \mathcal{K}}{\partial q_j} - \psi_j \right\} \delta q_j = 0$$

Using the fact that the generalized virtual displacement δq_j are independent and arbitrary, one concludes that the coefficients in front of δq_j must be equal to zeros and therefore

$$\frac{d}{dt} \frac{\partial \mathcal{K}}{\partial \dot{q}_j} - \frac{\partial \mathcal{K}}{\partial q_j} = \psi_j, \quad j = 1, \dots, n$$

It is customary to separate from the generalized forces ψ_j the ones that can be computed as partial derivatives of a scalar function $\mathcal{P}(\mathbf{q})$ that is called **potential energy** and often is caused by gravity. With such a separation, the equation above can be rewritten as

$$\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{q}_j} - \frac{\partial \mathcal{L}}{\partial q_j} = \tau_j, \quad \psi_j = -\frac{\partial \mathcal{P}}{\partial q_j} + \tau_j, \quad \mathcal{L} = \mathcal{K}(\mathbf{q}, \dot{\mathbf{q}}) - \mathcal{P}(\mathbf{q}),$$

where $\mathcal{L}(\mathbf{q}, \dot{\mathbf{q}})$ is called the **Lagrangian** and is computed as the difference between the kinetic energy and the potential energy of the system.

Let us continue consideration of our planar bar example. Suppose a system of two particles connected by a mass-less link is moving in a vertical plane as above but now it is under influence of not only external forces but also gravitational ones.

We already know that the planar two-component forces applied to the articles result in

$$\psi_1 = F_{1x} + F_{2x}, \quad \psi_2 = F_{1y} + F_{2y}, \quad \psi_3 = -F_{2x}L \sin(q_3) + F_{2y}L \cos(q_3)$$

Differentiating the expressions for the coordinates of the position vectors

$\vec{r}_1$ and $\vec{r}_2$ with respect to time we obtain as before

$$\mathbf{v}_1 = \dot{\mathbf{r}}_1 = \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}, \quad \mathbf{v}_2 = \dot{\mathbf{r}}_2 = \begin{bmatrix} \dot{q}_1 - L \sin(q_3) \dot{q}_3 \\ \dot{q}_2 + L \cos(q_3) \dot{q}_3 \end{bmatrix}$$

Hence, the kinetic energy is

$$\begin{aligned} \mathcal{K} = \frac{1}{2} (m_1 \|\mathbf{v}_1\|^2 + m_2 \|\mathbf{v}_2\|^2) &= (m_1 + m_2) (\dot{q}_1^2 + \dot{q}_2^2) + \frac{m_2 L^2}{2} \dot{q}_3^2 \\ &\quad + m_2 L (\dot{q}_2 \cos(q_3) - \dot{q}_1 \sin(q_3)) \dot{q}_3 \end{aligned}$$

To compute the potential energy, we just add potential energies of the particles that (up to a non-important constant offset) are proportional to the y -coordinates of the articles. So,

$$\mathcal{P} = (m_1 r_1 + m_2 r_2) \begin{bmatrix} 0 \\ g \end{bmatrix} = g \left((m_1 + m_2) q_2 + m_2 L \sin(q_3) \right)$$

Now, $\mathcal{L} = \mathcal{K} - \mathcal{P}$, the partial derivatives are

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial q_1} &= 0, \quad \frac{\partial \mathcal{L}}{\partial q_2} = -(m_1 + m_2)g, \quad \frac{\partial \mathcal{L}}{\partial q_3} = -m_2 L \left((g + \dot{q}_3 \dot{q}_1) \cos(q_3) + \dot{q}_3 \dot{q}_2 \sin(q_3) \right) \\ \frac{\partial \mathcal{L}}{\partial \dot{q}_1} &= 2(m_1 + m_2) \dot{q}_1 - m_2 L \sin(q_3) \dot{q}_3, \quad \frac{\partial \mathcal{L}}{\partial \dot{q}_2} = 2(m_1 + m_2) \dot{q}_2 + m_2 L \cos(q_3) \dot{q}_3, \\ \frac{\partial \mathcal{L}}{\partial \dot{q}_3} &= m_2 L^2 \dot{q}_3 + m_2 L (\dot{q}_2 \cos(q_3) - \dot{q}_1 \sin(q_3)) \end{aligned}$$

and substituting them into the equations of motion we obtain the final answer

$$\begin{aligned} 2(m_1 + m_2) \ddot{q}_1 - m_2 L \sin(q_3) \ddot{q}_3 - m_2 L \cos(q_3) \dot{q}_3^2 &= F_{1x} + F_{2x}, \\ 2(m_1 + m_2) \ddot{q}_2 + m_2 L \cos(q_3) \ddot{q}_3 - m_2 L \sin(q_3) \dot{q}_3^2 &= (F_{1y} - g m_1) + (F_{2y} - g m_2), \\ -m_2 \sin(q_3) \ddot{q}_1 + m_2 \cos(q_3) \ddot{q}_2 + m_2 L \ddot{q}_3 &= (F_{2y} - g m_2) \cos(q_3) - F_{2x} \sin(q_3) \end{aligned}$$

Equations of motion for other systems of particles with holonomic constraints can be derived in a similar straightforward way.

Moreover, the same expression,

$$\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{q}_j} - \frac{\partial \mathcal{L}}{\partial q_j} = \tau_j, \quad j = 1, \dots, n, \quad \mathcal{L} = \mathcal{K}(q, \dot{q}) - \mathcal{P}(q),$$

known as **Euler-Lagrange equations**, is valid for a system of rigid bodies with n DOF as well. However, the expressions for the energies are more involved than in the case of a finite number of particles. Let us derive them.

5.5 Lagrangian for an n -Link Robot

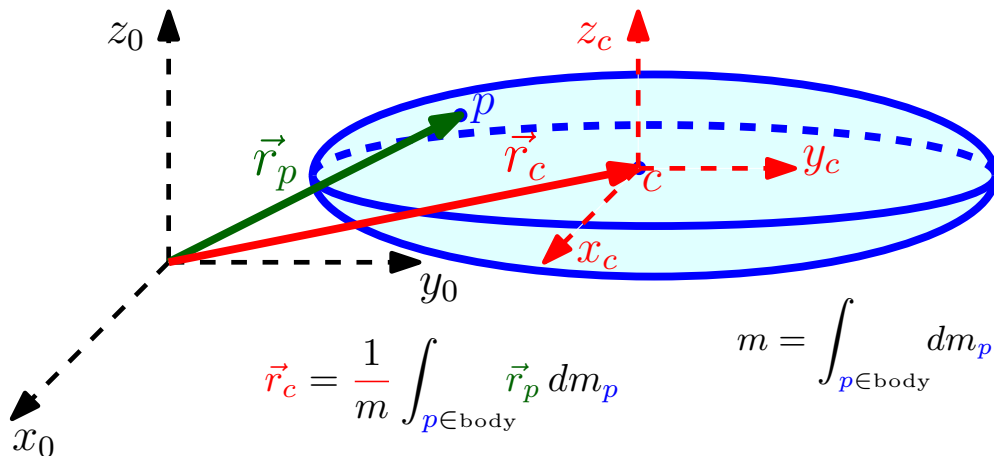
An n -link robot is a mechanical system with n DOF, where the variables of the kinematic description with the DH convention of Chapter 2 can be taken as the generalized coordinates.

The **Lagrangian** for a robot can be computed adding the kinetic energies of the links and subtracting their potential energies.

5.5.1 Kinetic Energy for a Rigid Body

We start with a rigid body, or a single link that is not restricted by any holonomic constraints.

A kinetic energy of a rigid body is a quadratic form of linear velocities of its single chosen point, called the pole, and of either components of the angular velocity or other quantities specifying the rate of rotation (such as, e.g., derivatives of the Euler angles).



In the particular case when the pole is taken at the center of mass, the kinetic energy can be computed as a sum of two terms responsible for the energy of rotation and the energy of translation, more precisely,

$$\begin{aligned}
 \mathcal{K} &= \frac{1}{2} \int_{p \in \text{body}} \underbrace{\left| \vec{v} + \vec{\omega} \times (\vec{r}_p - \vec{r}) \right|}_{v_p^T v_p} dm_p \\
 &= \frac{1}{2} \int_{p \in \text{body}} \left(\vec{v} + \vec{\omega} \times (\vec{r}_p - \vec{r}) \right)^T \left(\vec{v} + \vec{\omega} \times (\vec{r}_p - \vec{r}) \right) dm_p \\
 &= \frac{1}{2} \int_{p \in \text{body}} \left(\vec{v} - S(\vec{r}_p - \vec{r}) \vec{\omega} \right)^T \left(\vec{v} - S(\vec{r}_p - \vec{r}) \vec{\omega} \right) dm_p \\
 &= \frac{1}{2} \int_{p \in \text{body}} \vec{v}^T \vec{v} dm_p + \frac{1}{2} \int_{p \in \text{body}} \vec{\omega}^T S^T(\vec{r}_p - \vec{r}) S(\vec{r}_p - \vec{r}) \vec{\omega} dm_p \\
 &= \frac{1}{2} m \vec{v}^T \vec{v} + \frac{1}{2} \vec{\omega}^T \mathcal{I} \vec{\omega}
 \end{aligned}$$

where $\mathbf{v} = \dot{\mathbf{r}}$ represents the velocity in $\mathbf{3D}$ of the center of mass, defined by $\mathbf{r} = \frac{1}{m} \int_{p \in \text{body}} \mathbf{r}_p d\mathbf{m}_p$, and $\boldsymbol{\omega}$ represents the body's angular velocity, $\mathbf{m} = \int_{p \in \text{body}} d\mathbf{m}_p$ is the total mass, while the $\mathbf{3} \times \mathbf{3}$ symmetric positive definite matrix $\mathcal{I}$ is so-called matrix of inertia.

An unrestricted rigid body has **6** DOF. So, to find a useful expression for $\mathcal{K}$ one needs:

- To choose **6** generalized coordinates $\mathbf{q}_1, \dots, \mathbf{q}_6$,
- To compute the coordinates of the vectors specifying the velocity of the center of mass $\mathbf{v} = \dot{\mathbf{r}}$ and of the angular velocity $\boldsymbol{\omega}$ as functions of $\mathbf{q}_1, \dots, \mathbf{q}_6$ and their derivatives,
- To compute from CAD-drawings or to identify otherwise the mass $\mathbf{m}$ and the **6** independent components of $\mathcal{I}$.

One should compute vector $\mathbf{r}$ and find its time derivative. To compute the angular velocity, see Chapter 3, one needs to find the matrix of rotation $\mathbf{R}(t)$, see Chapter 2, compute the matrix of spin

$$\mathbf{S}(t) = \frac{d}{dt} \mathbf{R}(t) \dot{\mathbf{R}}^T(t)$$

and find the components of $\boldsymbol{\omega}$ from the components of $\mathbf{S}(t) = \boldsymbol{\omega}(t) \times \mathbf{I}$.

The time varying components of the matrix of inertia $\mathcal{I}(t)$ in the inertial frame can be computed from components of the constant matrix of inertia in the body frame $\mathcal{I}_0$:

$$\mathcal{I}(t) = \int_{p \in \text{body}} \mathbf{S}^T(\vec{\mathbf{r}}_p - \vec{\mathbf{r}}) \mathbf{S}(\vec{\mathbf{r}}_p - \vec{\mathbf{r}}) d\mathbf{m}_p = \mathbf{R}(t) \mathcal{I}_0 \mathbf{R}^T(t)$$

due to the change of coordinates formula for representations of the tensor of inertia, i.e. $\mathbf{A}^1 = \mathbf{R}_1^0 \mathbf{A}^0 (\mathbf{R}_1^0)^T$, see Chapter 2.

The $\mathbf{3} \times \mathbf{3}$ matrix of inertia

$$\mathcal{I}_0 = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix}, \quad \mathcal{I}_0 = \mathcal{I}_0^T$$

is constant in the body frame and since

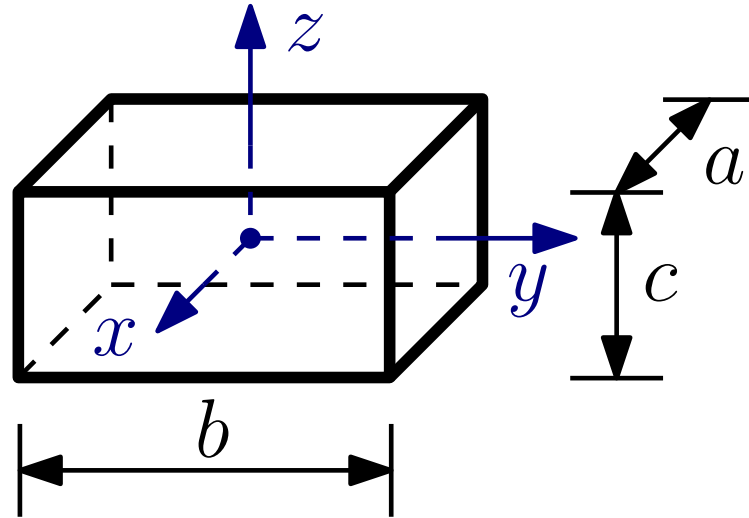
$$\mathcal{I}_0 = \int_{p \in \text{body}} \underbrace{\begin{bmatrix} 0 & -z & y \\ z & 0 & -x \\ -y & x & 0 \end{bmatrix}}_{\mathbf{S}^T([x,y,z]^T)} \underbrace{\begin{bmatrix} 0 & z & -y \\ -z & 0 & x \\ y & -x & 0 \end{bmatrix}}_{\mathbf{S}([x,y,z]^T)} d\mathbf{m}_p$$

its components can be computed as follows

$$\begin{aligned}
 I_{xx} &= \int \int \int (y^2 + z^2) \rho(x, y, z) dx dy dz \\
 I_{yy} &= \int \int \int (x^2 + z^2) \rho(x, y, z) dx dy dz \\
 I_{zz} &= \int \int \int (y^2 + x^2) \rho(x, y, z) dx dy dz \\
 I_{xy} = I_{yx} &= - \int \int \int xy \rho(x, y, z) dx dy dz \\
 I_{xz} = I_{zx} &= - \int \int \int xz \rho(x, y, z) dx dy dz \\
 I_{yz} = I_{zy} &= - \int \int \int yz \rho(x, y, z) dx dy dz
 \end{aligned}$$

where $\rho(x, y, z)$ represents the mass density function, while the integration is done over the whole volume occupied by the body assuming that the origin coincides with the center of mass. Note that $m = \int \int \int \rho(x, y, z) dx dy dz$.

Let us consider a simple example of a rectangular brick with uniform mass density $\rho(x, y, z) = \rho = \text{const}$ and dimensions $a \times b \times c$ shown below



One can verify that the center of mass coincides with the origin put in the geometric center by checking that

$$\begin{aligned}
 x_c &= \frac{1}{m} \int_{-c/2}^{c/2} \int_{-b/2}^{b/2} \int_{-a/2}^{a/2} \rho x dx dy dz = 0, & y_c &= \frac{1}{m} \int_{-c/2}^{c/2} \int_{-b/2}^{b/2} \int_{-a/2}^{a/2} \rho y dx dy dz = 0, \\
 z_c &= \frac{1}{m} \int_{-c/2}^{c/2} \int_{-b/2}^{b/2} \int_{-a/2}^{a/2} \rho z dx dy dz = 0
 \end{aligned}$$

Clearly,

$$\begin{aligned} m &= \int_{-c/2}^{c/2} \int_{-b/2}^{b/2} \int_{-a/2}^{a/2} \rho \, dx dy dz = \rho abc, \\ I_{xx} &= \int_{-c/2}^{c/2} \int_{-b/2}^{b/2} \int_{-a/2}^{a/2} (y^2 + z^2) \rho \, dx dy dz \\ &= \rho \frac{abc}{12} (b^2 + c^2) = \frac{m}{12} (b^2 + c^2) \end{aligned}$$

and in a similar fashion

$$I_{yy} = \frac{m}{12} (a^2 + c^2), \quad I_{zz} = \frac{m}{12} (a^2 + b^2), \quad I_{xy} = I_{xz} = I_{yz} = 0.$$

Let us move now to a more useful case for us and consider an (open) kinematic chain of rigid bodies.

5.5.2 Kinetic Energy for an n -Link Robot

We will use the formula

$$\mathcal{K} = \frac{1}{2} \sum_{i=1}^n (m_i v_i^T v_i + \omega_i^T \mathcal{I}_i \omega_i).$$

where $\mathcal{I}_i = R(t) \mathcal{I}_{0i} R^T(t)$ and m_i are matrices of inertia and masses of the links.

So, we need to express

- $v_i = \dot{r}_i$ as a function of generalized coordinates q and velocities $\dot{q}$ and
- ω_i as function of generalized coordinates q and velocities $\dot{q}$.

for $i = 1, \dots, n$.

The necessary relations are given by the Jacobian matrices

$$v_i = J_{v_i}(q) \dot{q}, \quad \omega_i = J_{\omega_i}(q) \dot{q}$$

that can be computed as shown in Chapter 3.

Therefore, the final form of kinetic energy is

$$\mathcal{K} = \frac{1}{2} \dot{q}^T \left[\sum_{i=1}^n (m_i J_{v_i}^T(q) J_{v_i}(q) + J_{\omega_i}^T(q) R_i^0(q) \mathcal{I}_{0i} (R_i^0(q))^T J_{\omega_i}(q)) \right] \dot{q}$$

5.5.3 Potential Energy for an n -Link Robot

The potential energy is a sum of potential energies of all the links.

$$\mathcal{P} = \sum_{i=1}^n \mathcal{P}_i$$

Potential energy of i^{th} -link is

$$\mathcal{P}_i = -m_i \mathbf{g}^T \mathbf{r}_{ci}^0$$

where $\mathbf{r}_{ci}^0$ represents the position of the link's center of mass and $\mathbf{g} \equiv \mathbf{g}^0$ denotes the direction of the gravity in $\mathbf{0}$ -frame multiplied by the gravity constant, which is around **9.8** meters per second.

So, the total potential energy of the robot

$$\mathcal{P} = \sum_{i=1}^n \mathcal{P}_i = - \sum_{i=1}^n m_i \mathbf{g}^T \mathbf{r}_{ci}^0(\mathbf{q})$$

where $\mathbf{g}$ represents the vector acceleration of gravity in the $\mathbf{0}$ frame and $\mathbf{r}_{ci}^0(\mathbf{q})$ can be computed in the similar way as the solution of the forward kinematics problem given in Chapter 2 from the constant locations of the centers of mass $\mathbf{r}_{ci}^i$ of the links with respect to the attached frame: $\mathbf{r}_{ci}^0(\mathbf{q}) = \mathbf{o}_i^0 + \mathbf{R}_i^0(\mathbf{q}) \mathbf{r}_{ci}^i$.

5.6 Equations for Dynamics an n -Link Robot

We are ready to discuss the equations of motion written in terms of the generalized coordinates that can be taken as the variables of the DH-convention introduced in Chapter 2.

5.6.1 General form of equations of motion

As has been discussed above, under certain assumptions, the equations of motions of a mechanical system (of particles or rigid bodies) with purely holonomic constraints and external forces or/and torques can be derived as follows.

Procedure 1 (Computational Procedure) *One can proceed according to the following steps.*

1. Identify the smallest required number of independent variables, known as the number of degrees of freedom (DOF) $\mathbf{n}$;

2. Introduce a set of such variables, called generalized coordinates $(\mathbf{q}_1, \dots, \mathbf{q}_n)$;
3. Compute kinetic energy $\mathcal{K}(\mathbf{q}, \dot{\mathbf{q}})$ and potential energy $\mathcal{P}(\mathbf{q})$ as function of these variables and their derivatives, known as generalized velocities;
4. Compute the Lagrangian $\mathcal{L} = \mathcal{K} - \mathcal{P}$ and derive the system of n second order differential equations, known as Euler-Lagrange equations:

$$\frac{d}{dt} \left[\frac{\mathcal{L}(\mathbf{q}, \dot{\mathbf{q}})}{\partial \dot{\mathbf{q}}_j} \right] - \frac{\mathcal{L}(\mathbf{q}, \dot{\mathbf{q}})}{\partial \mathbf{q}_j} = \tau_j, \quad j = 1, \dots, n$$

where generalized forces τ_j are found using the principle of virtual work

$$\begin{aligned} \tau_j &= \sum_{i=1}^{k_f} (\mathbf{f}_i^e)^T \frac{\partial \mathbf{v}_i^e}{\partial \dot{\mathbf{q}}_j} + \sum_{i=1}^{k_m} (\boldsymbol{\tau}_i^e)^T \frac{\partial \boldsymbol{\omega}_i^e}{\partial \dot{\mathbf{q}}_j} \\ &= \sum_{i=1}^{k_f} \left(\mathbf{J}_{\mathbf{v}_i^e}^j(\mathbf{q}) \right)^T \mathbf{f}_i^e + \sum_{i=1}^{k_m} \left(\mathbf{J}_{\boldsymbol{\omega}_i^e}^j(\mathbf{q}) \right)^T \boldsymbol{\tau}_i^e \end{aligned}$$

from external forces $\left\{ \vec{\mathbf{f}}_i^e \right\}_{i=1}^{k_f}$ that produce work along directions defined by $\left\{ \vec{\mathbf{v}}_i^e \right\}_{i=1}^{k_f}$ or/and torques $\left\{ \vec{\boldsymbol{\tau}}_i^e \right\}_{i=1}^{k_m}$ that produce work rotating rigid bodies along directions defined by angular velocities $\left\{ \vec{\boldsymbol{\omega}}_i^e \right\}_{i=1}^{k_m}$, which are not accounted in the potential energy.

In the case when, except for the control torques $\mathbf{u}$ and gravity, the only other external force $\vec{\mathbf{f}}^e$ is applied to the tool at the origin of the last frame and the only external torque $\vec{\boldsymbol{\tau}}^e$ is applied to the tool, the matrices in the expressions for the generalized forces become related to the manipulator Jacobian, introduced in Chapter 3 resulting in

$$\boldsymbol{\tau} = -(\mathbf{J}_v(\mathbf{q}))^T \mathbf{f}^e - (\mathbf{J}_\omega(\mathbf{q}))^T \boldsymbol{\tau}^e + \frac{\partial \mathbf{q}^u}{\partial \mathbf{q}} \mathbf{u} = -(\mathbf{J}(\mathbf{q}))^T \begin{bmatrix} \mathbf{f}^e \\ \boldsymbol{\tau}^e \end{bmatrix} + \mathbf{B}(\mathbf{q}) \mathbf{u}$$

where $\mathbf{q}^u$ define the variables along which the control forces work and correspondingly $\mathbf{B}(\mathbf{q})$ is the actuation-dependent matrix of partial derivatives, which is the identity matrix in the most common case when the control torques work directly on the introduced generalized coordinates: i.e. when $\mathbf{q}^u = \mathbf{q}$. Note that “ $-$ ” appears above due to “the change of the view” from forces and torques produced by a manipulator at a possible contact to those applied to the tool from outside.

The procedure described above should be combined with computation of the energies described above as well as with appropriate expressions for

linear and angular velocities derived in Chapter 3 using DH-convention from Chapter 2.

Let us summarize what we have shown above in relation to computations of energies.

- In general, the kinetic energy is

$$\mathcal{K} = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{D}(\mathbf{q}) \dot{\mathbf{q}} = \sum_{i=1}^n \sum_{j=1}^n d_{ij}(\mathbf{q}) \dot{q}_i \dot{q}_j$$

where

$$\mathbf{D}(\mathbf{q}) = \sum_{i=1}^n [m_i \mathbf{J}_{v_i}^T(\mathbf{q}) \mathbf{J}_{v_i}(\mathbf{q}) + \mathbf{J}_{\omega_i}^T(\mathbf{q}) \mathbf{R}_i^0(\mathbf{q}) \mathcal{I}_{0i}(\mathbf{R}_i^0(\mathbf{q}))^T \mathbf{J}_{\omega_i}(\mathbf{q})]$$

- The potential forces can be separated from the other generalized forces:

$$\psi_j = -\frac{\partial \mathcal{P}(\mathbf{q})}{\partial q_j} + \tau_j \quad \text{and the potential energy is}$$

$$\mathcal{P}(\mathbf{q}) = -\sum_{i=1}^n m_i \mathbf{g}^T \mathbf{r}_{ci}^0(\mathbf{q}),$$

where the centers of masses $\mathbf{r}_{ci}^0(\mathbf{q})$ and the vector of gravitational acceleration $\mathbf{g}$ are expressed in the inertial frame.

- We can introduce a scalar function, called Lagrangian, $\mathcal{L} = \mathcal{K}(\mathbf{q}, \dot{\mathbf{q}}) - \mathcal{P}(\mathbf{q})$ and write the equation of motion in the compact form

$$\frac{d}{dt} \frac{\partial \mathcal{L}}{\partial \dot{q}_j} - \frac{\partial \mathcal{L}}{\partial q_j} = \tau_j, \quad j = 1, \dots, n$$

5.6.2 Structure of the Equations of Motion

The equations of motion have a particular structure implied by

$$\frac{d}{dt} \frac{\partial \mathcal{K}}{\partial \dot{q}_k} - \frac{\partial (\mathcal{K} - \mathcal{P})}{\partial q_k} = \tau_k, \quad k = 1, \dots, n$$

and the form of the Lagrangian.

Indeed, the first term can be expressed as

$$\frac{\partial \mathcal{K}}{\partial \dot{q}_k} = \frac{\partial}{\partial \dot{q}_k} \left[\frac{1}{2} \dot{\mathbf{q}}^T \mathbf{D}(\mathbf{q}) \dot{\mathbf{q}} \right] = \sum_{j=1}^n d_{kj} \dot{q}_j \Rightarrow \frac{d}{dt} \frac{\partial \mathcal{K}}{\partial \dot{q}_k} = \frac{d}{dt} \left[\sum_{j=1}^n d_{kj}(\mathbf{q}) \dot{q}_j \right]$$

and rewritten as follows

$$\begin{aligned}
 \frac{d}{dt} \left[\sum_{j=1}^n d_{kj} \dot{q}_j \right] &= \sum_{j=1}^n d_{kj}(q) \ddot{q}_j + \sum_{j=1}^n \frac{d}{dt} [d_{kj}(q)] \dot{q}_j \\
 &= \sum_{j=1}^n d_{kj}(q) \ddot{q}_j + \sum_{j=1}^n \left(\sum_{i=1}^n \frac{\partial d_{kj}(q)}{\partial q_i} \dot{q}_i \right) \dot{q}_j \\
 &= \sum_{j=1}^n d_{kj}(q) \ddot{q}_j + \sum_{j=1}^n \sum_{i=1}^n \left(\frac{\partial d_{kj}(q)}{\partial q_i} \right) \dot{q}_i \dot{q}_j \\
 &= \sum_{j=1}^n d_{kj}(q) \ddot{q}_j + \frac{1}{2} \sum_{j=1}^n \sum_{i=1}^n \left(\frac{\partial d_{kj}(q)}{\partial q_i} + \frac{\partial d_{ki}(q)}{\partial q_j} \right) \dot{q}_i \dot{q}_j
 \end{aligned}$$

The second term of the equations of motion is equal to

$$\begin{aligned}
 \frac{\partial(\mathcal{K} - \mathcal{P})}{\partial q_k} &= \frac{\partial}{\partial q_k} \left[\frac{1}{2} \dot{q}^T D(q) \dot{q} - \mathcal{P} \right] = \frac{1}{2} \dot{q}^T \left[\frac{\partial}{\partial q_k} D(q) \right] \dot{q} - \frac{\partial}{\partial q_k} \mathcal{P} \\
 &= \frac{1}{2} \sum_{j=1}^n \sum_{i=1}^n \frac{\partial d_{ij}(q)}{\partial q_k} \dot{q}_i \dot{q}_j - \frac{\partial}{\partial q_j} \mathcal{P}(q)
 \end{aligned}$$

To sum up, the equations of motion can be rewritten as follows

$$\sum_{j=1}^n d_{kj} \ddot{q}_j + \frac{1}{2} \sum_{j=1}^n \sum_{i=1}^n \left(\frac{\partial d_{kj}}{\partial q_i} + \frac{\partial d_{ki}}{\partial q_j} \right) \dot{q}_i \dot{q}_j - \frac{1}{2} \sum_{j=1}^n \sum_{i=1}^n \frac{\partial d_{ij}}{\partial q_k} \dot{q}_i \dot{q}_j + \frac{\partial \mathcal{P}}{\partial q_k} = \tau_k$$

and, combining the terms in the middle, as

$$\sum_{j=1}^n d_{kj}(q) \ddot{q}_j + \sum_{j=1}^n \sum_{i=1}^n \mathbf{c}_{ijk}(q) \dot{q}_i \dot{q}_j + \mathbf{g}_k(q) = \tau_k$$

with

$$\mathbf{c}_{ijk}(q) = \frac{1}{2} \left(\frac{\partial d_{kj}(q)}{\partial q_i} + \frac{\partial d_{ki}(q)}{\partial q_j} - \frac{\partial d_{ij}(q)}{\partial q_k} \right), \quad \mathbf{g}_k(q) = \frac{\partial}{\partial q_k} \mathcal{P}(q)$$

Finally, introducing the so-called matrix of centrifugal and Coriolis forces $C(q, \dot{q})$ defined by its components

$$C_{kj}(q, \dot{q}) = \sum_{i=1}^n \mathbf{c}_{ijk}(q) \dot{q}_i = \frac{1}{2} \sum_{i=1}^n \left(\frac{\partial d_{kj}(q)}{\partial q_i} + \frac{\partial d_{ki}(q)}{\partial q_j} - \frac{\partial d_{ij}(q)}{\partial q_k} \right) \dot{q}_i$$

one can obtain the equations in the following compact matrix form

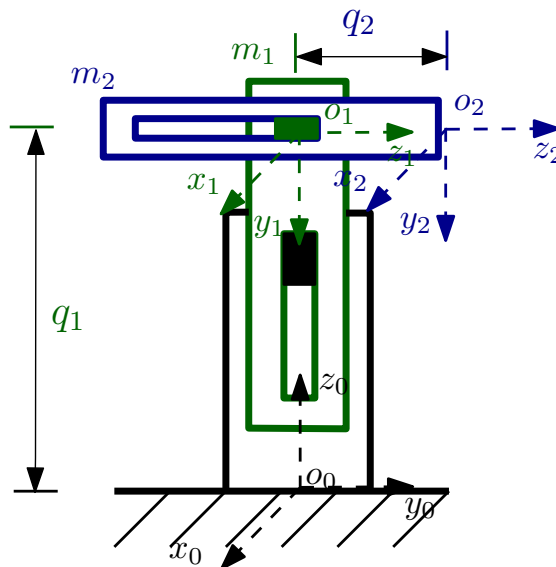
$$D(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau$$

Let us see examples of deriving the equations of motions for particular examples of robotic manipulators.

5.7 Examples for Equations of Motion

5.7.1 Two-Link Cartesian Manipulator

Consider the following 2-link robot of PP -type



For this system we need

- to solve forward kinematics problem;
- to compute manipulator Jacobian;
- to compute kinetic and potential energies and the Euler-Lagrange equations.

Forward Kinematics and Jacobian

DH parameters for computing homogeneous transformations

$$T(q_i) = \text{Rot}_{z,\theta} \cdot \text{Trans}_{z,d} \cdot \text{Trans}_{x,a} \cdot \text{Rot}_{x,\alpha}$$

are

$$\begin{aligned} T_1^0 : \quad \theta &= 0, \quad d = q_1, \quad a = 0, \quad \alpha = -\frac{\pi}{2} \\ T_2^1 : \quad \theta &= 0, \quad d = q_2, \quad a = 0, \quad \alpha = 0 \end{aligned}$$

The kinetic energy of the system is

$$\mathcal{K} = \frac{1}{2} [m_1 v_{c1}^T v_{c1} + \omega_1^T \mathcal{I}_1 \omega_1] + \frac{1}{2} [m_2 v_{c2}^T v_{c2} + \omega_2^T \mathcal{I}_2 \omega_2]$$

where the velocities of the centers of masses coincide with velocities of all the other points of the respective links

$$\begin{aligned} v_{c1} &= J_{v1} \dot{q} = \begin{bmatrix} J_{v1}^{(1)} & J_{v1}^{(2)} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} = J_{v1}^{(1)} \dot{q}_1 + J_{v1}^{(2)} \dot{q}_2 \\ v_{c2} &= J_{v2} \dot{q} = \begin{bmatrix} J_{v2}^{(1)} & J_{v2}^{(2)} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} = J_{v2}^{(1)} \dot{q}_1 + J_{v2}^{(2)} \dot{q}_2 \end{aligned}$$

and the angular velocities of the links

$$\begin{aligned} \omega_1 &= J_{\omega1} \dot{q} = \begin{bmatrix} J_{\omega1}^{(1)} & J_{\omega1}^{(2)} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} = J_{\omega1}^{(1)} \dot{q}_1 + J_{\omega1}^{(2)} \dot{q}_2 \\ \omega_2 &= J_{\omega2} \dot{q} = \begin{bmatrix} J_{\omega2}^{(1)} & J_{\omega2}^{(2)} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} = J_{\omega2}^{(1)} \dot{q}_1 + J_{\omega2}^{(2)} \dot{q}_2 \end{aligned}$$

can be computed using the velocity Jacobian.

To compute the Jacobian we can use the expressions derived above for the case of DH-frames, i.e, non-zero vectors are obtained for $i \leq k$ according to the following expressions

$$\begin{aligned} J_{vk}^{(i)} &= \begin{cases} z_{i-1}^0, & \text{for prismatic joint} \\ z_{i-1}^0 \times [o_{ck}^0 - o_{i-1}^0], & \text{for revolute joint} \end{cases} \\ J_{\omega k}^{(i)} &= \begin{cases} 0, & \text{for prismatic joint} \\ z_{i-1}^0, & \text{for revolute joint} \end{cases} \end{aligned}$$

So, we have

$$\begin{aligned} J_{v1} = [z_0^0 \quad 0_{3 \times 1}] &= \begin{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} & \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \end{bmatrix}, \quad J_{v2} = [z_0^0 \quad z_1^0] = \begin{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} & \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \end{bmatrix} \\ J_{\omega1} &= 0_{3 \times 1}, \quad J_{\omega2} = 0_{3 \times 1} \end{aligned}$$

To sum up:

- Angular velocities ω_1 and ω_2 of both links are **zeros**.

- Linear velocities of centers of masses are

$$\begin{aligned} \mathbf{v}_{c1} &= \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \dot{q}_1 + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \dot{q}_2 = \begin{bmatrix} 0 \\ 0 \\ \dot{q}_1 \end{bmatrix} \\ \mathbf{v}_{c2} &= \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \dot{q}_1 + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \dot{q}_2 = \begin{bmatrix} 0 \\ \dot{q}_2 \\ \dot{q}_1 \end{bmatrix} \end{aligned}$$

- The kinetic energy is

$$\mathcal{K} = \frac{1}{2} \mathbf{m}_1 \mathbf{v}_{c1}^T \mathbf{v}_{c1} + \frac{1}{2} \mathbf{m}_2 \mathbf{v}_{c2}^T \mathbf{v}_{c2} = \frac{1}{2} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}^T \begin{bmatrix} m_1 + m_2 & 0 \\ 0 & m_2 \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}$$

Potential Energy (PE) for Two-Link Cartesian Manipulator

Here are immediate observations:

- PE is independent of the second link position,
- It depends on the height of center of mass of the whole robot.

It follows that, up to an additive constant off-set, $\mathcal{P} = g (m_1 + m_2) q_1$.

Euler-Lagrange Equations for 2-Link Cartesian Manipulator

Given the kinetic $\mathcal{K}$ and potential $\mathcal{P}$ energies, the dynamics are

$$\frac{d}{dt} \left[\frac{\partial \mathcal{L}}{\partial \dot{q}} \right] - \frac{\partial \mathcal{L}}{\partial q} = \tau$$

where the difference between the kinetic and potential energies is

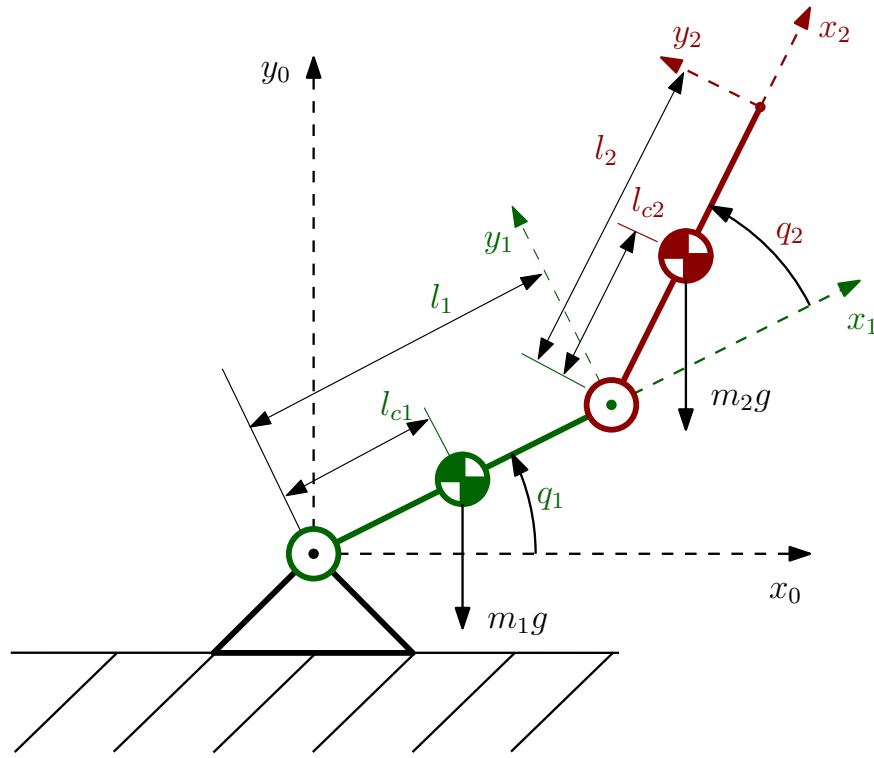
$$\mathcal{L} = \frac{1}{2} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}^T \begin{bmatrix} m_1 + m_2 & 0 \\ 0 & m_2 \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} - g (m_1 + m_2) q_1$$

Differentiating we obtain the Euler-Lagrange equations

$$\begin{aligned} (m_1 + m_2) \ddot{q}_1 + g(m_1 + m_2) &= \tau_1 \\ m_2 \ddot{q}_2 &= \tau_2 \end{aligned}$$

5.7.2 Planar Elbow Manipulator

Consider the planar **2**-link manipulator of **RR** type (two-link planar articulated robot) shown below together with an assignment of DH frames.



This manipulator has been already studied in Chapter 1, where we have stated the equations of motion without derivations.

For this system we would like

- to compute forward kinematics and manipulator Jacobian,
- to compute the energies and the Euler-Lagrange equations.

Forward Kinematics and Jacobian

DH parameters for computing homogeneous transformations

$$T(q_i) = \text{Rot}_{z,\theta} \cdot \text{Trans}_{z,d} \cdot \text{Trans}_{x,a} \cdot \text{Rot}_{x,\alpha}$$

are

$$\begin{aligned} T_1^0 : \quad & \theta = q_1, \quad d = 0, \quad a = l_1, \quad \alpha = 0 \\ T_2^1 : \quad & \theta = q_2, \quad d = 0, \quad a = l_2, \quad \alpha = 0 \end{aligned}$$

The kinetic energy of the system is

$$\mathcal{K} = \frac{1}{2} [m_1 v_{c1}^T v_{c1} + \omega_1^T \mathcal{I}_1 \omega_1] + \frac{1}{2} [m_2 v_{c2}^T v_{c2} + \omega_2^T \mathcal{I}_2 \omega_2]$$

where the velocities of the centers of mass can be computed in a similar

way as the velocities of the end-points of the links

$$\begin{aligned} \mathbf{v}_{c1} &= \mathbf{J}_{v1} \dot{\mathbf{q}} = \begin{bmatrix} \mathbf{J}_{v1}^{(1)} & \mathbf{J}_{v1}^{(2)} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} = \mathbf{J}_{v1}^{(1)} \dot{q}_1 + \mathbf{J}_{v1}^{(2)} \dot{q}_2 \\ \mathbf{v}_{c2} &= \mathbf{J}_{v2} \dot{\mathbf{q}} = \begin{bmatrix} \mathbf{J}_{v2}^{(1)} & \mathbf{J}_{v2}^{(2)} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} = \mathbf{J}_{v2}^{(1)} \dot{q}_1 + \mathbf{J}_{v2}^{(2)} \dot{q}_2 \end{aligned}$$

and the angular velocities of the links

$$\begin{aligned} \boldsymbol{\omega}_1 &= \mathbf{J}_{\omega 1} \dot{\mathbf{q}} = \begin{bmatrix} \mathbf{J}_{\omega 1}^{(1)} & \mathbf{J}_{\omega 1}^{(2)} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} = \mathbf{J}_{\omega 1}^{(1)} \dot{q}_1 + \mathbf{J}_{\omega 1}^{(2)} \dot{q}_2 \\ \boldsymbol{\omega}_2 &= \mathbf{J}_{\omega 2} \dot{\mathbf{q}} = \begin{bmatrix} \mathbf{J}_{\omega 2}^{(1)} & \mathbf{J}_{\omega 2}^{(2)} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} = \mathbf{J}_{\omega 2}^{(1)} \dot{q}_1 + \mathbf{J}_{\omega 2}^{(2)} \dot{q}_2 \end{aligned}$$

can be computed using the velocity Jacobian.

To compute the Jacobian we can use the expressions derived above for the case of DH-frames, i.e, non-zero vectors are obtained for $i \leq k$ according to the following expressions

$$\begin{aligned} \mathbf{J}_{vk}^{(i)} &= \begin{cases} \mathbf{z}_{i-1}^0, & \text{for prismatic joint} \\ \mathbf{z}_{i-1}^0 \times [\mathbf{o}_{ck}^0 - \mathbf{o}_{i-1}^0], & \text{for revolute joint} \end{cases} \\ \mathbf{J}_{\omega k}^{(i)} &= \begin{cases} \mathbf{0}, & \text{for prismatic joint} \\ \mathbf{z}_{i-1}^0, & \text{for revolute joint} \end{cases} \end{aligned}$$

So, we have for the linear velocity Jacobian

$$\begin{aligned} \mathbf{J}_{v1}^{(1)} &= \mathbf{z}_0^0 \times (\mathbf{o}_{c1} - \mathbf{o}_0) = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ 1 \end{bmatrix} \times \begin{bmatrix} l_{c1} \cos q_1 \\ l_{c1} \sin q_1 \\ 0 \end{bmatrix} = \begin{bmatrix} -l_{c1} \sin q_1 \\ l_{c1} \cos q_1 \\ 0 \end{bmatrix}, \\ \mathbf{J}_{v2}^{(1)} &= \mathbf{z}_0^0 \times (\mathbf{o}_{c2} - \mathbf{o}_0) \\ &= \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ 1 \end{bmatrix} \times \left(\begin{bmatrix} l_1 \cos q_1 \\ l_1 \sin q_1 \\ 0 \end{bmatrix} + \begin{bmatrix} l_{c2} \cos(q_1 + q_2) \\ l_{c2} \sin(q_1 + q_2) \\ 0 \end{bmatrix} \right) = \begin{bmatrix} -l_1 \sin q_1 - l_{c2} \sin(q_1 + q_2) \\ l_1 \cos q_1 + l_{c2} \cos(q_1 + q_2) \\ 0 \end{bmatrix} \\ \mathbf{J}_{v1}^{(2)} &= \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ 0 \end{bmatrix}, \\ \mathbf{J}_{v2}^{(2)} &= \mathbf{z}_1^0 \times (\mathbf{o}_{c2} - \mathbf{o}_1) = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ 1 \end{bmatrix} \times \begin{bmatrix} l_{c2} \cos(q_1 + q_2) \\ l_{c2} \sin(q_1 + q_2) \\ 0 \end{bmatrix} = \begin{bmatrix} -l_{c2} \sin(q_1 + q_2) \\ l_{c2} \cos(q_1 + q_2) \\ 0 \end{bmatrix} \end{aligned}$$

For the angular velocity Jacobian we have

$$\mathbf{J}_{\omega 1}^{(1)} = \mathbf{z}_0^0 = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ 1 \end{bmatrix}, \quad \mathbf{J}_{\omega 2}^{(1)} = \mathbf{z}_0^0 = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ 1 \end{bmatrix}, \quad \mathbf{J}_{\omega 1}^{(2)} = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ 0 \end{bmatrix}, \quad \mathbf{J}_{\omega 2}^{(2)} = \mathbf{z}_1^0 = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ 1 \end{bmatrix}$$

Finally, the kinetic energy $\mathcal{K}$ is

$$\begin{aligned}
 \mathcal{K} &= \frac{1}{2} [m_1 v_{c1}^T v_{c1} + \omega_1^T \mathcal{I}_1 \omega_1] + \frac{1}{2} [m_2 v_{c2}^T v_{c2} + \omega_2^T \mathcal{I}_2 \omega_2] \\
 &= \frac{1}{2} \left[m_1 \left(J_{v_1}^{(1)} \dot{q}_1 \right)^2 + I_1 \left(J_{\omega_1}^{(1)} \dot{q}_1 \right)^2 \right] \\
 &\quad + \frac{1}{2} \left[m_2 \left(J_{v_2}^{(1)} \dot{q}_1 + J_{v_2}^{(2)} \dot{q}_2 \right)^2 + I_2 \left(J_{\omega_2}^{(1)} \dot{q}_1 + J_{\omega_2}^{(2)} \dot{q}_2 \right)^2 \right] \\
 &= \frac{1}{2} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}^T \begin{bmatrix} d_{11} & d_{12} \\ d_{12} & d_{22} \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}
 \end{aligned}$$

where

$$\begin{aligned}
 d_{11} &= m_1 l_{c1}^2 + m_2 (l_1^2 + l_{c2}^2 + 2l_1 l_{c2} \cos q_2) + I_1 + I_2 \\
 d_{12} &= m_2 (l_{c2}^2 + l_1 l_{c2} \cos q_2) + I_2 \\
 d_{22} &= m_2 l_{c2}^2 + I_2
 \end{aligned}$$

Potential Energy (PE) for Two-Link Elbow Manipulator

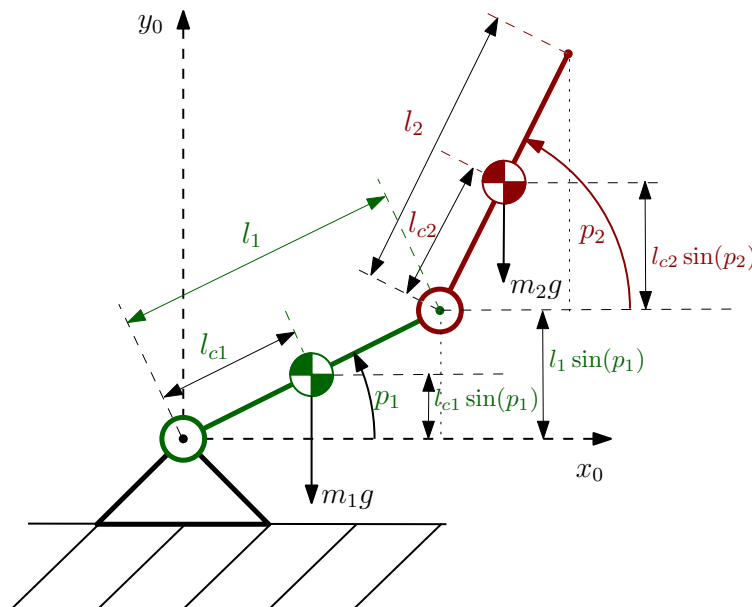
We proceed as follows

- PE of the 1st link is $\mathcal{P}_1 = m_1 g y_{c1} = m_1 g l_{c1} \sin q_1$;
- PE of the 2nd link is $\mathcal{P}_2 = m_2 g y_{c2} = m_2 g (l_1 \sin q_1 + l_{c2} \sin(q_1 + q_2))$;
- So, total PE is $\mathcal{P}_1 + \mathcal{P}_2$.

The result of applying the Euler-Lagrange formulae to $\mathcal{L} = \mathcal{K} - \mathcal{P}$ has been already presented in Chapter 1 above.

5.7.3 Double Link Pendulum with Absolute Angles

Let us now show how to use Euler-Lagrange equations without the machinery of using DH parameters. Consider the same planar **RR**-type manipulator but suppose we would like to use the absolute angles p_1 and p_2 , see the figure below, as the generalized coordinates.



If we would like to deriving the equations of motion directly in these coordinates, we should proceed as follows

- Compute the kinetic energy $\mathcal{K}(p_1, p_2, \dot{p}_1, \dot{p}_2)$;
- Compute the potential energy $\mathcal{P}(p_1, p_2)$;
- Use them to compute the Lagrangian and to obtain the Euler-Lagrange equations.

Note that a possible reason for such a choice of the generalized coordinates would be the fact that the generalized forces work exactly on δp_1 and δp_2 respectively. This would be the case if both motors driving the links are fixed to the base so that, in particular, the second link is actuated through a power-train such as a belt. In this case, in the right-hand sides of the Euler-Lagrange equations we should have the torques τ_1 and τ_2 produced by the motors (possibly magnified if gearboxes are present).

Note that an alternative way would be to use the DH-parameters q_1 and q_2 but to recompute the generalized forces for the Euler-Lagrange equations, written in terms of q_1 and q_2 using the principle of virtual work

$$\tau_1^{(p)} = \tau_1^{(q)} \frac{\partial \omega_1}{\partial \dot{q}_1} + \tau_2^{(q)} \frac{\partial \omega_1}{\partial \dot{q}_2}, \quad \tau_2^{(p)} = \tau_1^{(q)} \frac{\partial \omega_2}{\partial \dot{q}_1} + \tau_2^{(q)} \frac{\partial \omega_2}{\partial \dot{q}_2}$$

where, as computed above, $\omega_1 = \dot{q}_1$ and $\omega_2 = \dot{q}_1 + \dot{q}_2$ so that the right-hand sides of the equations, written in terms of q_1 and q_2 should be $\tau_1^{(q)} = \tau_1^{(p)}$ and $\tau_2^{(q)} = \tau_2^{(p)} - \tau_1^{(p)}$.

Let us, however, avoid using these arguments and write the equations in terms of p_1 and p_2 directly with, correspondingly, the right-hand sides taken directly as $\tau_1 = \tau_1^{(p)}$ and $\tau_2 = \tau_2^{(p)}$.

Let us start with kinetic energy

$$\mathcal{K} = \frac{1}{2} [\mathbf{m}_1 \mathbf{v}_{c_1}^T \mathbf{v}_{c_1} + \boldsymbol{\omega}_1^T \mathcal{I}_1 \boldsymbol{\omega}_1] + \frac{1}{2} [\mathbf{m}_2 \mathbf{v}_{c_2}^T \mathbf{v}_{c_2} + \boldsymbol{\omega}_2^T \mathcal{I}_2 \boldsymbol{\omega}_2]$$

In order to compute $\mathbf{v}_{c_1}$ and $\mathbf{v}_{c_2}$, we use simple geometry to get the coordinates of the center of mass of the first link.

The coordinates of the center of mass of the first link are

$$x_{c_1} = l_{c_1} \cos(p_1), \quad y_{c_1} = l_{c_1} \sin(p_1)$$

Therefore,

$$\dot{x}_{c_1} = -l_{c_1} \sin(p_1) \dot{p}_1, \quad \dot{y}_{c_1} = l_{c_1} \cos(p_1) \dot{p}_1$$

while it is obvious that the angular velocity of the first link is

$$\boldsymbol{\omega}_1 = \dot{p}_1 \mathbf{z}_0^0,$$

since we have the simple case of rotation in plane.

The $\mathbf{x}$ -coordinate of the center of mass of the second link is

$$x_{c_2} = l_1 \cos(p_1) + l_{c_2} \cos(p_2)$$

and its derivative is

$$\dot{x}_{c_2} = -l_1 \sin(p_1) \dot{p}_1 - l_{c_2} \sin(p_2) \dot{p}_2$$

The $\mathbf{y}$ -coordinate of the center of mass of the second link is

$$y_{c_2} = l_1 \sin(p_1) + l_{c_2} \sin(p_2)$$

and its derivative is

$$\dot{y}_{c_2} = l_1 \cos(p_1) \dot{p}_1 + l_{c_2} \cos(p_2) \dot{p}_2$$

while it is again obvious that the angular velocity of the second link is

$$\boldsymbol{\omega}_2 = \dot{p}_2 \mathbf{z}_1^0.$$

It follows that

$$\begin{aligned}
 \mathcal{K} &= \frac{1}{2} \left[m_1 v_{c_1}^T v_{c_1} + \omega_1^T \mathcal{I}_1 \omega_1 \right] + \frac{1}{2} \left[m_2 v_{c_2}^T v_{c_2} + \omega_2^T \mathcal{I}_2 \omega_2 \right] \\
 &= \frac{1}{2} \begin{bmatrix} \dot{p}_1 \\ \dot{p}_2 \end{bmatrix}^T D(p_1, p_2) \begin{bmatrix} \dot{p}_1 \\ \dot{p}_2 \end{bmatrix} \\
 &= \frac{1}{2} \begin{bmatrix} \dot{p}_1 \\ \dot{p}_2 \end{bmatrix}^T \begin{bmatrix} m_1 l_{c_1}^2 + m_2 l_1^2 + I_1 & m_2 l_1 l_{c_2} \cos(p_2 - p_1) \\ m_2 l_1 l_{c_2} \cos(p_2 - p_1) & m_2 l_{c_2}^2 + I_2 \end{bmatrix} \begin{bmatrix} \dot{p}_1 \\ \dot{p}_2 \end{bmatrix}
 \end{aligned}$$

Now, the potential energy can be easily computed

$$\begin{aligned}
 \mathcal{P} &= m_1 g y_{c_1} + m_2 g y_{c_2} \\
 &= m_1 g l_{c_1} \sin(p_1) + m_2 g [l_1 \sin(p_1) + l_{c_2} \sin(p_2)]
 \end{aligned}$$

Finally, the equations of motion can be computed from

$$\frac{d}{dt} \left[\frac{\partial \mathcal{K}}{\partial \dot{p}_1} \right] - \frac{\partial (\mathcal{K} - \mathcal{P})}{\partial p_1} = \tau_1, \quad \frac{d}{dt} \left[\frac{\partial \mathcal{K}}{\partial \dot{p}_2} \right] - \frac{\partial (\mathcal{K} - \mathcal{P})}{\partial p_2} = \tau_2$$

Here and above we have considered the case of so-called open kinematic chains. Following DH-convention routine typically allows saving much time for developing equation for kinematics and dynamics. However, this is not possible if the number of links is smaller than the number of DOF due to presence of a kinematic loop.

5.7.4 Euler-Lagrange equations in excessive coordinates

Sometimes, it is to compute Lagrangian as a function of more variables than the number of degrees of freedom. Suppose we have $k \geq 2$ variables

$$\theta_1, \quad \theta_2, \quad \dots, \quad \theta_k, \quad \rightarrow \quad \theta = [\theta_1 \quad \theta_2 \quad \dots \quad \theta_k]^T$$

and $1 \leq l < k$ relations, called holonomic constraints, among them

$$g_i(\theta_1, \theta_2, \dots, \theta_k) = 0, \quad i = 1, 2, \dots, l \quad \rightarrow \quad g(\theta) = 0_{l \times 1}$$

and these relations are not easy to solve for $n = k - l$ independent variables, called generalized coordinates. It is possible to modify the equations describing dynamics, derived above for a system of particles, to include the additional l constraint forces. First, one computes the energies as functions of all the not independent variables. After that, applying the principle of virtual work, and introducing l constraint forces:

$$f_1^c, \quad f_2^c, \quad \dots, \quad f_l^c, \quad \rightarrow \quad F^c = [f_1^c \quad f_2^c \quad \dots \quad f_l^c]^T$$

one obtains k differential equations of second order

$$\frac{d}{dt} \left[\frac{\partial \mathcal{K}}{\partial \dot{\theta}_i} \right] - \frac{\partial(\mathcal{K} - \mathcal{P})}{\partial \theta_i} = \tau_i - (J^c(\theta))^T F^c, \quad i = 1, \dots, k$$

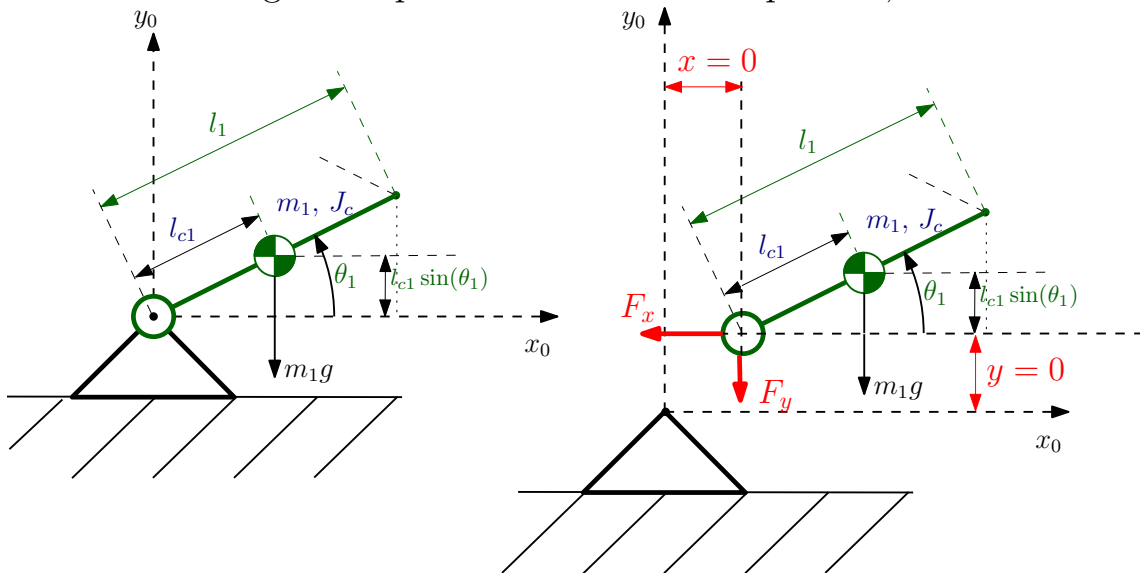
where τ_i denote generalized external forces acting on θ_i and the Jacobian of constraints is obtained by differentiations of the constraint identities:

$$J^c(\theta) = \frac{\partial g(\theta)}{\partial \theta} \quad \Leftarrow \quad \frac{d}{dt} g(\theta) = J^c(\theta) \dot{\theta}$$

The equations above are sometimes called Euler-Lagrange equations of first kind or Euler-Lagrange equations with constraint forces in excessive coordinates.

Often the constraint forces can be eliminated from the equations as in the case illustrated in the next simple example.

Consider a single-link planar rotational manipulator, illustrated below.



The dynamics can be described either using the single generalized coordinate θ_1 or using the three variable: x , y , and θ_1 , which are subject to two holonomic constraints:

$$g(\theta) = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Computing coordinates of the center of mass and their derivatives

$$\begin{bmatrix} x_{c1} \\ y_{c1} \end{bmatrix} = \begin{bmatrix} x + l_{c1} \cos \theta_1 \\ y + l_{c1} \sin \theta_1 \end{bmatrix}, \quad \begin{bmatrix} \dot{x}_{c1} \\ \dot{y}_{c1} \end{bmatrix} = \begin{bmatrix} \dot{x} - l_{c1} \dot{\theta}_1 \sin \theta_1 \\ \dot{y} + l_{c1} \dot{\theta}_1 \cos \theta_1 \end{bmatrix}$$

one can obtain the expressions for the energies

$$\mathcal{K} = \frac{1}{2} m_1 (\dot{x}_{c1}^2 + \dot{y}_{c1}^2) + \frac{1}{2} J_c \dot{\theta}_1^2, \quad \mathcal{P} = m_1 g l_{c1} \sin \theta_1$$

and of the Euler-Lagrange equations with constraint forces:

$$\begin{aligned}\frac{d}{dt} \left[\frac{\partial(\mathcal{K} - \mathcal{P})}{\partial \dot{\theta}_1} \right] - \frac{\partial(\mathcal{K} - \mathcal{P})}{\partial \theta_1} &= \tau_1 = \tau_1 + \frac{\partial [x \ y]}{\partial \theta} \begin{bmatrix} \mathbf{F}_x \\ \mathbf{F}_y \end{bmatrix} \\ \frac{d}{dt} \left[\frac{\partial(\mathcal{K} - \mathcal{P})}{\partial \dot{x}} \right] - \frac{\partial(\mathcal{K} - \mathcal{P})}{\partial x} &= \mathbf{F}_x = \frac{\partial [x \ y]}{\partial x} \begin{bmatrix} \mathbf{F}_x \\ \mathbf{F}_y \end{bmatrix} \\ \frac{d}{dt} \left[\frac{\partial(\mathcal{K} - \mathcal{P})}{\partial \dot{y}} \right] - \frac{\partial(\mathcal{K} - \mathcal{P})}{\partial y} &= \mathbf{F}_y = \frac{\partial [x \ y]}{\partial y} \begin{bmatrix} \mathbf{F}_x \\ \mathbf{F}_y \end{bmatrix}\end{aligned}$$

which can be rewritten as

$$\begin{aligned}m_1 l_{c1}^2 \ddot{\theta}_1 + m_1 l_{c1} (\ddot{y} \cos \theta_1 - \ddot{x} \sin \theta_1) + m_1 g l_{c1} \cos \theta_1 &= \tau_1 \\ m_1 \left(\ddot{x} - l_{c1} \ddot{\theta}_1 \sin \theta_1 - l_{c1} \dot{\theta}_1^2 \cos \theta_1 \right) &= \mathbf{F}_x \\ m_1 \left(\ddot{y} + l_{c1} \ddot{\theta}_1 \cos \theta_1 - l_{c1} \dot{\theta}_1^2 \sin \theta_1 \right) &= \mathbf{F}_y\end{aligned}$$

Now, substituting the relations that follow from the constraints $\mathbf{x} \equiv \mathbf{0}$ and $\mathbf{y} \equiv \mathbf{0}$, i.e. $\ddot{x} = \mathbf{0}$ and $\ddot{y} = \mathbf{0}$, we recover not only the equation that can be derived using the generalized coordinate θ_1 without constraints and additional variables:

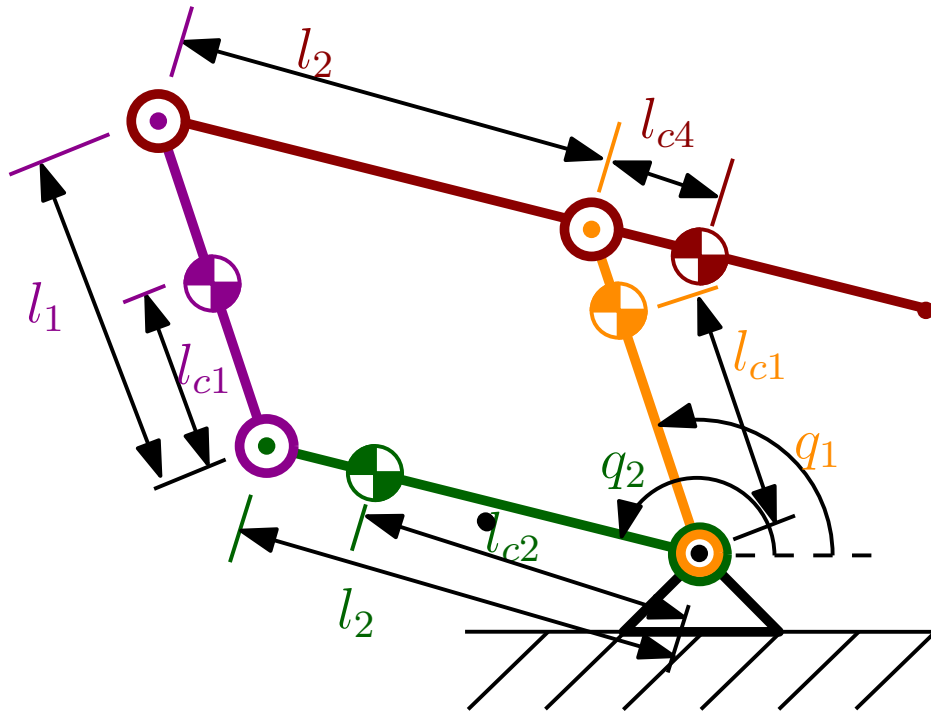
$$m_1 l_{c1}^2 \ddot{\theta}_1 + m_1 g l_{c1} \cos \theta_1 = \tau_1$$

but also the expressions for the reaction (constraint) forces:

$$\begin{aligned}\mathbf{F}_x &= \left(\frac{\tau_1}{l_{c1}} - m_1 g \cos \theta_1 \right) \sin \theta_1 - m_1 l_{c1} \dot{\theta}_1^2 \cos \theta_1 \\ \mathbf{F}_y &= \left(\frac{\tau_1}{l_{c1}} - m_1 g \cos \theta_1 \right) \cos \theta_1 - m_1 l_{c1} \dot{\theta}_1^2 \sin \theta_1\end{aligned}$$

5.7.5 Mechanism of a Closed Kinematic Chain

Let us consider now a more challenging example of a mechanism shown below.



Since we have a closed kinematic chain, using DH-convention is not possible. Hence, we need to introduce generalized coordinates and proceed with all the derivations without the DH-machinery.

We need, besides choosing the coordinates,

- to compute the kinetic energy $\mathcal{K}$;
- to compute the potential energy $\mathcal{P}$;
- to substitute them into the Euler-Lagrange equations.

First of all, it is obvious that two angles completely define location of all the components. Hence, the number of DOF is **2**. A possible choice of the two generalized coordinates is taking the two absolute angles q_1 and q_2 defining orientations of the first two links, see the figure above.

Let us start with the kinetic energy

$$\mathcal{K} = \frac{1}{2} [m_1 v_{c_1}^T v_{c_1} + \omega_1^T \mathcal{I}_1 \omega_1] + \frac{1}{2} [m_2 v_{c_2}^T v_{c_2} + \omega_2^T \mathcal{I}_2 \omega_2] \\ + \frac{1}{2} [m_3 v_{c_3}^T v_{c_3} + \omega_3^T \mathcal{I}_3 \omega_3] + \frac{1}{2} [m_4 v_{c_4}^T v_{c_4} + \omega_4^T \mathcal{I}_4 \omega_4]$$

Begin with the coordinates of the centers of mass of the two links, location of which are directly specified by q_1 and q_2

$$\begin{aligned} x_{c_1} &= l_{c_1} \cos q_1, & y_{c_1} &= l_{c_1} \sin q_1 \\ x_{c_2} &= l_{c_2} \cos q_2, & y_{c_2} &= l_{c_2} \sin q_2 \end{aligned}$$

It can be seen that for the other two links, essentially using the symmetry,

we have

$$\begin{aligned} x_{c_3} &= l_2 \cos q_2 + l_{c_3} \cos q_1 \\ y_{c_3} &= l_2 \sin q_2 + l_{c_3} \sin q_1 \end{aligned}$$

and

$$\begin{aligned} x_{c_4} &= l_1 \cos q_1 - l_{c_4} \cos q_2 \\ y_{c_4} &= l_1 \sin q_1 - l_{c_4} \sin q_2 \end{aligned}$$

Now, as above and using the symmetry

$$\begin{aligned} \vec{\omega}_1 &= \vec{\omega}_3 = \dot{q}_1 \vec{k} \\ \vec{\omega}_2 &= \vec{\omega}_4 = \dot{q}_2 \vec{k} \end{aligned}$$

where $\vec{k}$ is orthogonal to the plane and directed towards us.

It follows that

$$\begin{aligned} \mathcal{K} &= \frac{1}{2} [m_1 v_{c_1}^T v_{c_1} + \omega_1^T \mathcal{I}_1 \omega_1] + \frac{1}{2} [m_2 v_{c_2}^T v_{c_2} + \omega_2^T \mathcal{I}_2 \omega_2] \\ &\quad + \frac{1}{2} [m_3 v_{c_3}^T v_{c_3} + \omega_3^T \mathcal{I}_3 \omega_3] + \frac{1}{2} [m_4 v_{c_4}^T v_{c_4} + \omega_4^T \mathcal{I}_4 \omega_4] \\ &= \frac{1}{2} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix}^T \begin{bmatrix} d_{11}(q_1, q_2) & d_{12}(q_1, q_2) \\ d_{12}(q_1, q_2) & d_{22}(q_1, q_2) \end{bmatrix} \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} \end{aligned}$$

where

$$\begin{aligned} d_{11} &= m_1 l_{c_1}^2 + m_3 l_{c_3}^2 + m_4 l_4^2 + I_1 + I_3 \\ d_{12} &= [m_3 l_2 l_{c_3} - m_4 l_1 l_{c_4}] \cos(q_2 - q_1) \\ d_{22} &= m_2 l_{c_2}^2 + m_3 l_2^2 + m_4 l_{c_4}^2 + I_2 + I_4 \end{aligned}$$

Computing the potential energy is not hard

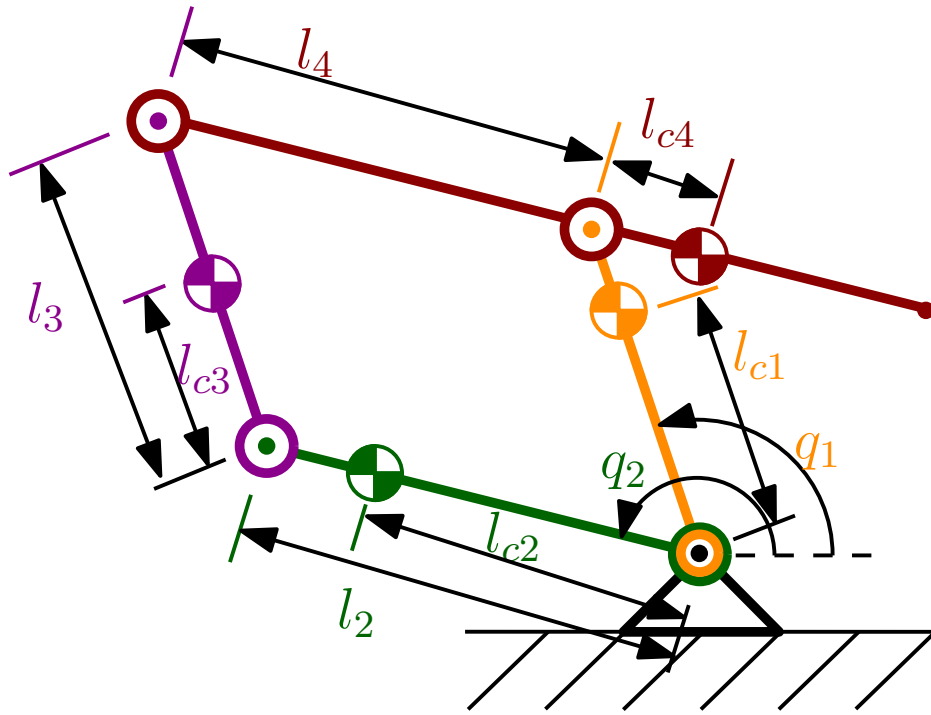
$$\begin{aligned} \mathcal{P} &= \sum_{i=1}^4 m_i g y_{c_i} \\ &= [m_1 l_{c_1} + m_3 l_{c_3} + m_4 l_1] g \sin q_1 \\ &\quad + [m_2 l_{c_2} + m_3 l_2 - m_4 l_{c_4}] g \sin q_2 \end{aligned}$$

Finally, Euler-Lagrange formalism can be used to derive the equations of motion. Note that in the special case when $m_3 l_2 l_{c_3} - m_4 l_1 l_{c_4} = 0$, the matrix of inertia is diagonal and constant. As a result, the obtained equations

$$d_{11} \ddot{q}_1 + \frac{\partial}{\partial q_1} \mathcal{P} = \tau_1 \quad d_{22} \ddot{q}_2 + \frac{\partial}{\partial q_2} \mathcal{P} = \tau_2$$

are decoupled and simple.

It is of interest to consider the case without symmetry, i.e. assuming $l_3 \neq l_1$ and/or $l_4 \neq l_2$, illustrated below

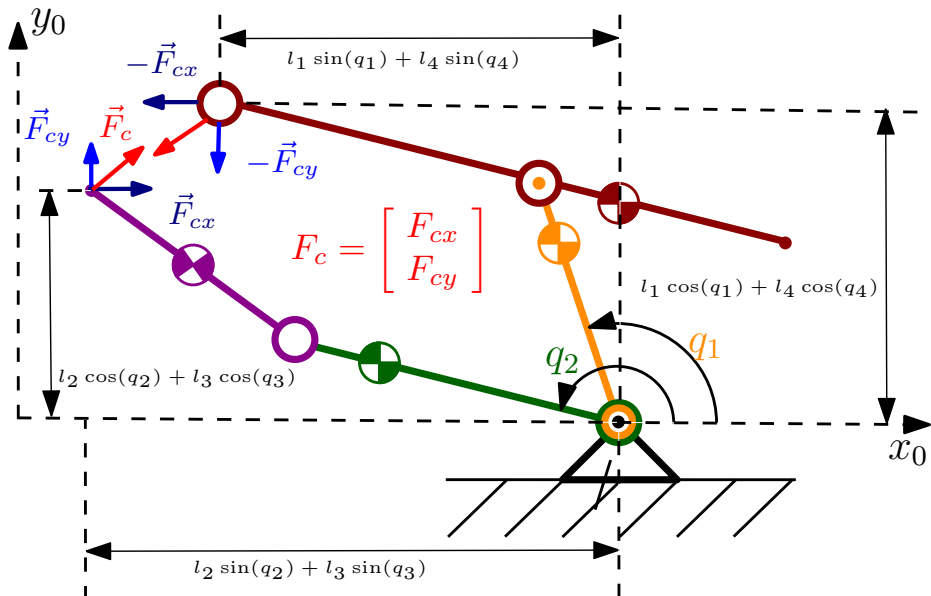


Now, we have to modify the expressions for the centers of mass for the third and fourth links since we will have almost always $q_3 \neq q_1$ and $q_4 \neq q_2$:

$$\begin{bmatrix} x_{c3} \\ y_{c3} \end{bmatrix} = \begin{bmatrix} l_2 \cos q_2 + l_{c3} \cos q_3 \\ l_2 \sin q_2 + l_{c3} \sin q_3 \end{bmatrix}, \quad \begin{bmatrix} x_{c4} \\ y_{c4} \end{bmatrix} = \begin{bmatrix} l_1 \cos q_1 - l_{c4} \cos q_4 \\ l_1 \sin q_1 - l_{c4} \sin q_4 \end{bmatrix}$$

and can compute the Lagrangian in a similar way as above but as a function of all eight variables: $q_1, q_2, q_3, q_4, \dot{q}_1, \dot{q}_2, \dot{q}_3$, and $\dot{q}_4$, which are not independent.

There exist two independent holonomic constraints $N_c(q) \in \mathbb{R}^2$, related to presence of the closed kinematic loop. They can be found by computing x and y coordinates of a point following two branches of the loop. For example, “braking” the loop as illustrated below



one obtains

$$\begin{aligned} N_c(q) &= \begin{bmatrix} l_1 \cos q_1 + l_4 \cos q_4 - l_2 \cos q_2 - l_3 \cos q_3 \\ l_1 \sin q_1 + l_4 \sin q_4 - l_2 \sin q_2 - l_3 \sin q_3 \end{bmatrix} = 0 \\ \frac{d}{dt} N_c(q) &= J_{c,1}(q) \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} + J_{c,2}(q) \begin{bmatrix} \dot{q}_3 \\ \dot{q}_4 \end{bmatrix} = 0 \\ \frac{d^2}{dt^2} N_c(q) &= [\dot{J}_{c,1}(q, \dot{q}) \quad \dot{J}_{c,2}(q, \dot{q})] \dot{q} + [J_{c,1}(q) \quad J_{c,2}(q)] \ddot{q} = 0 \end{aligned}$$

The right-hand side of the system of four second-order Euler-Lagrange equations must be modified by adding constraint forces, which appear pre-multiplied by transposed Jacobian of the constraint, according to the virtual work principle:

$$\frac{d}{dt} \left[\frac{\partial(\mathcal{K} - \mathcal{P})}{\partial \dot{q}} \right] - \frac{\partial(\mathcal{K} - \mathcal{P})}{\partial q} = \tau + J_c^T F_c$$

One can compute constraint forces F_c and reduce the dynamics to just $4 - 2 = 2$ second-order differential equations for two generalized coordinates, e.g. $\{q_1, q_2\} \in \mathbb{R}^2$.

Grouping the variables $q_r = [q_1, q_2]^T$ and $q_c = [q_3, q_4]^T$, one can observe that the dynamics is structured as

$$\begin{bmatrix} D_1(q) \\ D_2(q) \end{bmatrix} \begin{bmatrix} \ddot{q}_r \\ \ddot{q}_c \end{bmatrix} + \begin{bmatrix} C_1(q, \dot{q}) \\ C_2(q, \dot{q}) \end{bmatrix} \begin{bmatrix} \dot{q}_r \\ \dot{q}_c \end{bmatrix} + \begin{bmatrix} G_1 \\ G_2 \end{bmatrix} = \begin{bmatrix} \tau_r \\ 0 \end{bmatrix} + \begin{bmatrix} J_{c,1}^T F_c \\ J_{c,2}^T F_c \end{bmatrix}$$

From the holonomic constraint we can substitute

$$\begin{aligned} q_c &= f(q_r), \quad f : N_c(q_r, q_c) = 0 \rightarrow q_c \quad (\text{nontrivial when no symmetry!}) \\ \dot{q}_c &= -J_{c,2}^{-1}(q) J_{c,1}(q) \dot{q}_r \\ \ddot{q}_c &= -J_{c,2}^{-1}(q) \left[\dot{J}_{c,1}(q, \dot{q}_r) \dot{q}_r + J_{c,1}(q) \ddot{q}_r + \dot{J}_{c,2}(q, \dot{q}_r) \dot{q}_c \right] \end{aligned}$$

The constraint force is computed from the second equation

$$F_c = \begin{bmatrix} F_{cx} \\ F_{cy} \end{bmatrix} = \left(J_{c,2}^T \right)^{-1} \left(D_2(q) \begin{bmatrix} \ddot{q}_r \\ \ddot{q}_c \end{bmatrix} + C_2(q, \dot{q}_r) \begin{bmatrix} \dot{q}_r \\ \dot{q}_c \end{bmatrix} + G_2 \right)$$

and we can substitute it into the first equation. After collecting similar terms the reduced dynamics takes the form

$$D_r(q) \begin{bmatrix} \ddot{q}_1 \\ \ddot{q}_2 \end{bmatrix} + C_r(q, \dot{q}) \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} + G_r = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix}$$

In the symmetric case ($l_1 = l_3$ and $l_2 = l_4$), one can proceed in the same way and would obtain equations identical to the ones derived above.

However, a bonus for these more complicated derivation is the possibility to compute the internal constraint forces that in practice should be kept within certain limits in order to avoid braking the system.

Let us discuss now some structural properties of the equations of motion that can be used to check results of numerical simulations and to derive dynamic-model-based feedback control laws.

5.8 Properties of Equations of Motion

Some properties of the system can be derived directly from the structure of the Euler-Lagrange equations. Let us present some of such properties that will be used later for analysis and for design of feedback control laws.

5.8.1 Passivity Relation

Given a mechanical system

$$\frac{d}{dt} \left[\frac{\partial \mathcal{L}}{\partial \dot{q}} \right] - \frac{\partial \mathcal{L}}{\partial q} = \tau \Leftrightarrow D(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau$$

with

$$\mathcal{L} = \frac{1}{2} \dot{q}^T D(q) \dot{q} - P(q)$$

the total mechanical energy is given by

$$\mathcal{H} = \frac{1}{2} \dot{q}^T D(q) \dot{q} + P(q).$$

Let us compute the time-derivative of the energy along the solutions under an arbitrary vector of generalized forces τ using the resolved Euler-Lagrange equation

$$D(q)\ddot{q} = \tau - C(q, \dot{q})\dot{q} - g(q)$$

Differentiating $\mathcal{H}$ along a solution of the system, we have

$$\begin{aligned} \frac{d}{dt} \mathcal{H} &= \frac{d}{dt} \left[\frac{1}{2} \dot{q}^T D(q) \dot{q} + P(q) \right] \\ &= \frac{1}{2} \ddot{q}^T D(q) \dot{q} + \frac{1}{2} \dot{q}^T D(q) \ddot{q} + \frac{1}{2} \dot{q}^T \frac{d}{dt} [D(q)] \dot{q} + \dot{q}^T \frac{\partial P}{\partial q} \\ &= \dot{q}^T D(q) \ddot{q} + \frac{1}{2} \dot{q}^T \frac{d}{dt} [D(q)] \dot{q} + \dot{q}^T \frac{\partial P}{\partial q} \\ &= \dot{q}^T [\tau - C(q, \dot{q})\dot{q} - g(q)] + \frac{1}{2} \dot{q}^T \frac{d}{dt} [D(q)] \dot{q} + \dot{q}^T \frac{\partial P}{\partial q} \end{aligned}$$

$$\begin{aligned}
&= \dot{\mathbf{q}}^T \boldsymbol{\tau} + \dot{\mathbf{q}}^T \left(\frac{1}{2} \frac{d}{dt} [D(\mathbf{q})] - C(\mathbf{q}, \dot{\mathbf{q}}) \right) \dot{\mathbf{q}} + \underbrace{\dot{\mathbf{q}}^T \left(\frac{\partial \mathcal{P}}{\partial \mathbf{q}} - \mathbf{g}(\mathbf{q}) \right)}_{=0} \\
&= \dot{\mathbf{q}}^T \boldsymbol{\tau} + \underbrace{\dot{\mathbf{q}}^T \left(\frac{1}{2} \frac{d}{dt} [D(\mathbf{q})] - C(\mathbf{q}, \dot{\mathbf{q}}) \right) \dot{\mathbf{q}}}_{=0} \\
&= \dot{\mathbf{q}}^T \boldsymbol{\tau}
\end{aligned}$$

In the derivations above we have used two identities.

The first one, $\frac{\partial \mathcal{P}}{\partial \mathbf{q}} = \mathbf{g}(\mathbf{q})$, follows from the definition, while the second one, $\dot{\mathbf{q}}^T \left(\frac{1}{2} \frac{d}{dt} [D(\mathbf{q})] - C(\mathbf{q}, \dot{\mathbf{q}}) \right) \dot{\mathbf{q}} = 0$, is to be derived below.

Meanwhile, the obtained differential relation

$$\frac{d}{dt} \mathcal{H} = \dot{\mathbf{q}}^T \boldsymbol{\tau},$$

which in the case of $\boldsymbol{\tau} = \mathbf{0}$ corresponds to the energy conservation law, can be integrated

$$\int_0^T \frac{d}{dt} \mathcal{H}(\mathbf{q}, \dot{\mathbf{q}}) dt = \mathcal{H}(\mathbf{q}(T), \dot{\mathbf{q}}(T)) - \mathcal{H}(\mathbf{q}(0), \dot{\mathbf{q}}(0)) = \int_0^T \dot{\mathbf{q}}(t)^T \boldsymbol{\tau}(t) dt$$

and, in the case when the total energy is non-negative, we conclude

$$\int_0^T \dot{\mathbf{q}}(t)^T \boldsymbol{\tau}(t) dt \geq -\mathcal{H}(\mathbf{q}(0), \dot{\mathbf{q}}(0))$$

This is called passivity (dissipativity) relation in integral form.

Note that the total energy can be assumed positive inside every compact set of the state space since the potential energy is defined up to an arbitrary constant off-set.

Let us show now that $\dot{\mathbf{q}}^T \left(\frac{1}{2} \frac{d}{dt} [D(\mathbf{q})] - C(\mathbf{q}, \dot{\mathbf{q}}) \right) \dot{\mathbf{q}} = 0$. In fact, we shall show a stronger property

$$\mathbf{v}^T \left(\frac{1}{2} \frac{d}{dt} [D(\mathbf{q})] - C(\mathbf{q}, \dot{\mathbf{q}}) \right) \mathbf{v} = 0 \quad \forall \mathbf{v} \in \mathbb{R}^n$$

that is equivalent to the skew-symmetry of the matrix $\frac{d}{dt} [D(\mathbf{q})] - 2C(\mathbf{q}, \dot{\mathbf{q}})$.

Skew Symmetry of $\dot{D}(\mathbf{q}) - 2C(\mathbf{q}, \dot{\mathbf{q}})$

To check that

$$\mathbf{N} = \frac{d}{dt} [D(\mathbf{q})] - 2C(\mathbf{q}, \dot{\mathbf{q}}) \quad \Rightarrow \quad \mathbf{N}^T = -\mathbf{N}$$

we compute the $(\mathbf{k}, j)^{th}$ element of this matrix

$$\begin{aligned} n_{kj} &= \frac{d}{dt} d_{kj} - 2c_{kj} = \sum_{i=1}^n \frac{\partial d_{kj}}{\partial q_i} \dot{q}_i - \sum_{i=1}^n \left[\frac{\partial d_{kj}}{\partial q_i} + \frac{\partial d_{ki}}{\partial q_j} - \frac{\partial d_{ij}}{\partial q_k} \right] \dot{q}_i \\ &= \sum_{i=1}^n \left\{ \frac{\partial d_{kj}}{\partial q_i} - \left[\frac{\partial d_{kj}}{\partial q_i} + \frac{\partial d_{ki}}{\partial q_j} - \frac{\partial d_{ij}}{\partial q_k} \right] \right\} \dot{q}_i \\ &= \sum_{i=1}^n \left\{ \frac{\partial d_{ij}}{\partial q_k} - \frac{\partial d_{ki}}{\partial q_j} \right\} \dot{q}_i = \sum_{i=1}^n \left\{ \frac{\partial d_{ji}}{\partial q_k} - \frac{\partial d_{ki}}{\partial q_j} \right\} \dot{q}_i \end{aligned}$$

where for the last equality we have used the symmetry of $\mathbf{D}$: $d_{ij} = d_{ji}$ $\forall i, j$ follows from $\mathbf{D} = \mathbf{D}^T$.

It is now obvious that

$$n_{kj} = -n_{jk}$$

and so the skew-symmetry property is established.

5.8.2 Bounds on Matrix of Inertia

The kinetic energy of any mechanical system

$$\mathcal{K}(\mathbf{q}, \dot{\mathbf{q}}) = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{D}(\mathbf{q}) \dot{\mathbf{q}}$$

is non-negative, since it is derived as a sum of positive terms, each of which is computed as a product of a positive constant and the square of the magnitude of a velocity vector.

It is clear that $\mathbf{D}(\mathbf{q})$ is symmetric and non-negative matrix at an arbitrary configuration defined by values of the generalized coordinates. Moreover, there exist two non-negative scalar functions $m(\mathbf{q})$ and $M(\mathbf{q})$ such that

$$0 \leq m(\mathbf{q}) \mathbf{I} \leq \mathbf{D}(\mathbf{q}) \leq M(\mathbf{q}) \mathbf{I}$$

for every $\mathbf{q}$.

However, in general, it is not true that $\mathbf{D}(\mathbf{q})$ is positive definite, i.e. it can happen that there exists $\mathbf{q}^*$ such that $m(\mathbf{q}^*) = 0$ and $\det \mathbf{D}(\mathbf{q}^*) = 0$.

As an example, consider the point-mass spherical pendulum of length l , orientation of which is defined by the two angles θ and ϕ of the spherical coordinate system. The kinetic energy can be computed as the product of the mass m and the magnitude of the velocity expressed in the generalized

coordinates. The result is

$$\boldsymbol{\kappa} = \frac{1}{2}ml^2 \left[\sin^2(\theta)\dot{\phi}^2 + \dot{\theta}^2 \right] = \frac{1}{2} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \end{bmatrix}^T \begin{bmatrix} ml^2 \sin^2 \theta & 0 \\ 0 & ml^2 \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \end{bmatrix}$$

and we have a singularity at $\theta = 0$.

5.8.3 Linearity of Dynamics in Parameters

Consider the equations of motion of a mechanical system derived above

$$D(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$$

and suppose that we have measured (or estimated somehow)

- Control torque $\tau = \tau(t)$, $t \in [0, T]$
- Positions, velocities and accelerations

$$q = q(t), \quad \dot{q} = \dot{q}(t), \quad \ddot{q} = \ddot{q}(t), \quad t \in [0, T]$$

on some nontrivial interval of time, i.e. with $T > 0$.

An important question is the following:

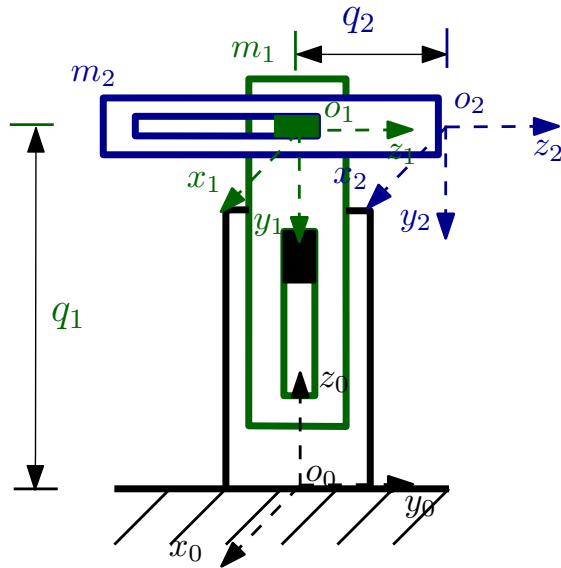
Can we reconstruct the physical parameters of the system (masses, coefficients of inertia matrices, dimensions) ?

and if the answer is yes, how this can be done?

Let us attempt to answer these questions for an example.

Example: Two Link Cartesian Manipulator

Let us reconsider the two-link planar manipulator of **PP**-type shown below



The equations of motion are

$$(m_1 + m_2)\ddot{q}_1 + g(m_1 + m_2) = \tau_1, \quad m_2\ddot{q}_2 = \tau_2$$

and involve only **2** parameters.

They can be rewritten as follows

$$\begin{bmatrix} \ddot{q}_1(t) + g & 0 \\ 0 & \ddot{q}_2(t) \end{bmatrix} \begin{bmatrix} m_1 + m_2 \\ m_2 \end{bmatrix} = \begin{bmatrix} \tau_1(t) \\ \tau_2(t) \end{bmatrix}$$

The equation for parameters

$$\underbrace{Y(\ddot{q}_1, \ddot{q}_2)}_{\text{REGRESSOR}} \underbrace{\begin{bmatrix} p_1 \\ p_2 \end{bmatrix}}_{\text{UNKNOWN CONSTANTS}} = \begin{bmatrix} \tau_1(t) \\ \tau_2(t) \end{bmatrix}$$

with $Y(\ddot{q}_1, \ddot{q}_2) = \begin{bmatrix} \ddot{q}_1(t) + g & 0 \\ 0 & \ddot{q}_2(t) \end{bmatrix}$, $p_1 = \mathbf{m}_1 + \mathbf{m}_2$, and $p_2 = \mathbf{m}_2$, can be solved for every moment of time when $\ddot{q}_2 \neq 0$ and $\ddot{q}_1 \neq -g$.

It is clear that

$$m_2 = \tau_2 / \ddot{q}_2, \quad m_1 = \tau_1 / (\ddot{q}_1 - g) - \tau_2 / \ddot{q}_2$$

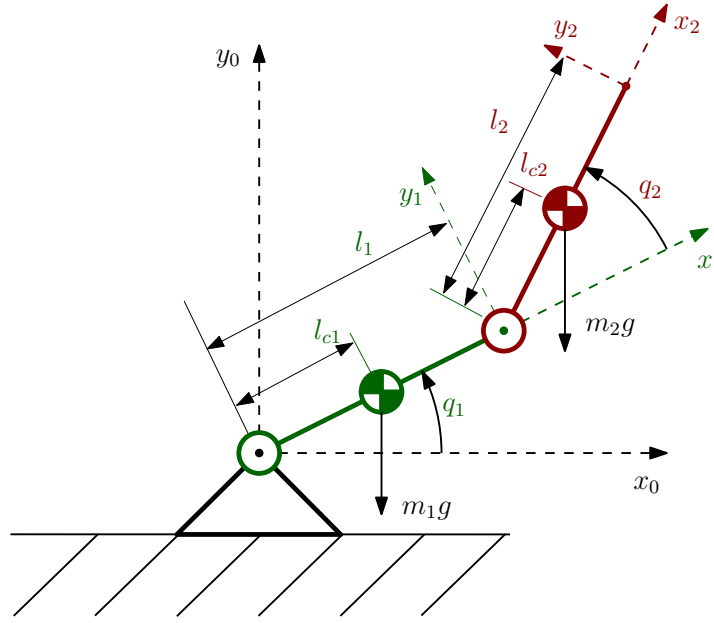
In general, it is possible to write the dynamics in the following form

$$D(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) \equiv Y(q, \dot{q}, \ddot{q})p = \tau$$

where the so-called regressor $Y(q, \dot{q}, \ddot{q})$ is formed by known functions and p contains various combinations of physical parameters (masses, moments of inertia, and various lengths). Note, however, that finding the appropriate vector of parameters p with the lowest dimension and the corresponding

regressor can be a nontrivial computational task for a many-DOF system.

For example, for a planar elbow manipulator



we have

$$\begin{aligned}
 D(q) &= \begin{bmatrix} p_1 + p_2 + 2p_3 \cos q_2 & p_2 + p_3 \cos q_2 \\ p_2 + p_3 \cos q_2 & p_3 \end{bmatrix} \\
 C(q, \dot{q}) &= \begin{bmatrix} -p_3 \sin(q_2) \dot{q}_2 & -p_3 \sin(q_2) (\dot{q}_1 + \dot{q}_2) \\ p_3 \sin(q_2) \dot{q}_1 & 0 \end{bmatrix} \\
 G(q) &= \begin{bmatrix} p_4 \cos q_1 + p_5 \cos(q_1 + q_2) \\ p_5 \cos(q_1 + q_2) \end{bmatrix}
 \end{aligned}$$

where a minimal set of $l = 5$ parameters can be taken as

$$p = \begin{bmatrix} p_1 \\ p_2 \\ p_3 \\ p_4 \\ p_5 \end{bmatrix} = \begin{bmatrix} m_1 l_{c1}^2 + m_2 l_1^2 + I_1 \\ m_2 l_{c2}^2 + I_2 \\ m_2 l_1 l_{c2} \\ (m_1 l_{c1} + m_2 l_1)g \\ m_2 l_{c2} g \end{bmatrix}$$

leading to the regressor

$$\begin{aligned}
 Y(q, \dot{q}, \ddot{q}) &\equiv D(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \\
 &\begin{bmatrix} \ddot{q}_1 & \ddot{q}_1 + \ddot{q}_2 & (2\ddot{q}_1 + \ddot{q}_2)\cos q_2 - (2\dot{q}_1\dot{q}_2 + \dot{q}_2^2)\sin q_2 & \cos q_1 & \cos(q_1 + q_2) \\ 0 & \ddot{q}_1 & \ddot{q}_1 \cos q_2 + \ddot{q}_2 + \dot{q}_1^2 \sin q_2 & 0 & \cos(q_1 + q_2) \end{bmatrix}
 \end{aligned}$$

It is also often a nontrivial or impossible (as in this case) task to obtain the estimates for the values of the real physical parameters from the values of the components of p .

Part III

Robotics Control

Chapter 6

Multivariable Control of Manipulators

Here we discuss a few mathematics-based approaches for derivation of feedback control laws. Let us start with a small improvement of our dynamic model. We will assume that our actuators are DC-motors that produce generalized forces acting through gear-boxes directly on each of the generalized coordinates. Let us merge a simplified model of the actuators with the mechanical model described above.

6.1 Dynamics with Geared DC-Motors

Given a mechanical system with n DOF, dynamics of which is described by the following Euler-Lagrange equations

$$D(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau, \quad q = [q_1, \dots, q_n]^T, \quad \tau = [\tau_1, \dots, \tau_n]^T$$

and suppose that each DOF is controlled by a geared DC-motor, dynamics of which can be described as a separated 1-DOF mechanical system with viscous friction and a generalized force, which is proportional to a control signal V_k ,

$$J_{m_k}\ddot{\theta}_k + B_k\dot{\theta}_k = K_{m_k}V_k + F_k, \quad F_k = -\frac{1}{r_k}\tau_k, \quad k = 1, \dots, n$$

where $F_k = -\frac{1}{r_k}\tau_k$ represents the constraint force corresponding to the reaction force that is applied to the mechanical part τ_k , which is reduced in amplitude due to the gear and has an opposite sign, while

- θ_k is the angle of the rotor of the k^{th} motor;
- r_k is the gear ration;

- J_{m_k} , B_k , K_{m_k} are parameters of the k^{th} DC-motor:
 - J_{m_k} is the inertia of the rotor,
 - B_k is the combined coefficient of the viscous friction and of the induced back electromotive force, and
 - K_{m_k} is the so-called motor constant, which is the gain between the applied voltage and the produced torque.

Moreover, suppose that the generalized coordinates for the n -DOF mechanical system and for the motors are related by the gear ratios

$$\theta_{m_k} = r_k q_k, \quad k = 1, \dots, n$$

Then, the actuator equations can be rewritten as follows

$$r_k^2 J_{m_k} \ddot{q}_k + r_k^2 B_k \dot{q}_k = r_k K_{m_k} V_k - \tau_k, \quad k = 1, \dots, n$$

We will give a few more details on derivation of these equation in Chapter 7.

The equations of the combined dynamical system

$$D(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau, \quad q = [q_1, \dots, q_n]^T, \quad \tau = [\tau_1, \dots, \tau_n]^T$$

$$r_k^2 J_{m_k} \ddot{q}_k + r_k^2 B_k \dot{q}_k = r_k K_{m_k} V_k - \tau_k, \quad k = 1, \dots, n$$

can be rewritten as one, if we exclude τ , which is the vector of internal forces, keeping valid our n holonomic constraints $\theta_{m_k} = r_k q_k$ for $k = 1, \dots, n$!

Finally, we obtain

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + B\dot{q} + g(q) = u$$

where

- $M(q) = D(q) + J$ with $J = \text{diag}\{r_1^2 J_{m_1}, \dots, r_n^2 J_{m_n}\}$, so $M(q)$ is positive definite;
- $B = [r_1^2 B_1, r_2^2 B_2, \dots, r_n^2 B_n]^T$
- $u = [u_1, u_2, \dots, u_n]^T$ with $u_k = r_k K_{m_k} V_k$, $k = 1, \dots, n$

Let us consider possible strategies for designing stabilizing feedback control laws.

6.2 PD-Controller Design

Given a mechanical system

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + B\dot{q} + g(q) = \mathbf{u}$$

we are interested

- to design a controller to stabilize a particular configuration of the robot:
 $q = q_d = \text{const}$,
- to analyze the closed-loop system stability.

6.2.1 The Case of Absent Gravity

Suppose first that $B = 0$ and $g(p) = 0$. We will show later how to remove this assumption.

Let the control law be taken as the classical proportional-differential (PD) feedback

$$\mathbf{u} = -K_p (q - q_d) - K_d \dot{q}$$

with K_p and K_d taken as either diagonal matrices with positive elements or arbitrary symmetric positive definite matrices.

To analyze the behavior of the closed-loop system

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} = \mathbf{u} = -K_p (q - q_d) - K_d \dot{q}$$

consider the scalar function

$$V(q, \dot{q}) = \frac{1}{2} \dot{q}^T M(q) \dot{q} + \frac{1}{2} (q - q_d)^T K_p (q - q_d)$$

Its time-derivative along solutions of the closed-loop system is

$$\begin{aligned} \frac{d}{dt} V &= \dot{q}^T M(q) \ddot{q} + \dot{q}^T \frac{d}{dt} \frac{1}{2} [M(q)] \dot{q} + \dot{q}^T K_p (q - q_d) \\ &= \dot{q}^T [\mathbf{u} - C(q, \dot{q}) \dot{q}] + \dot{q}^T \frac{d}{dt} \frac{1}{2} [M(q)] \dot{q} + \dot{q}^T K_p (q - q_d) \\ &= \dot{q}^T [\mathbf{u} + K_p (q - q_d)] + \underbrace{\dot{q}^T \left\{ \frac{d}{dt} \frac{1}{2} [D(q) + J] - C(q, \dot{q}) \right\} \dot{q}}_{= 0} \\ &= \dot{q}^T [\mathbf{u} + K_p (q - q_d)] \\ &= \dot{q}^T [-K_p (q - q_d) - K_d \dot{q} + K_p (q - q_d)] = -\dot{q}^T K_d \dot{q} \end{aligned}$$

So, finally, since $K_d > 0$ or $K_d = \text{diag} \{K_{d1}, K_{d2}, \dots, K_{dn}\} > 0$, we conclude

$$\frac{d}{dt} V = -\dot{q}^T K_d \dot{q} \leq 0.$$

It follows that

- $V(q(t), \dot{q}(t))$ is monotonically not increasing!

and it is not hard to see that

- V is positive definite (hence, bounded from below by 0), i.e. $V \geq 0$ and $V(q, \dot{q}) = 0 \Leftrightarrow \{q = q_d, \dot{q} = 0\}$

Therefore, according to elementary results from real analysis,

$$\exists \lim_{t \rightarrow +\infty} V(q(t), \dot{q}(t)) = V_\infty \quad \text{and} \quad \exists \lim_{t \rightarrow +\infty} \dot{q}(t) = \dot{q}_\infty(t) = 0$$

If we substitute this limit trajectory into the dynamics, we obtain

$$M(q_\infty) \underbrace{\ddot{q}_\infty}_{=0} + C(q_\infty, \dot{q}_\infty) \underbrace{\dot{q}_\infty}_{=0} = -K_p (q_\infty - q_d) - K_d \underbrace{\dot{q}_\infty}_{=0}$$

that is

$$0 = -K_p (q_\infty - q_d)$$

Now,

$$K_p > 0 \quad \text{or} \quad K_p = \text{diag} \{K_{p1}, K_{p2}, \dots, K_{pn}\} > 0 \Rightarrow q_\infty = q_d$$

Therefore, according to either the Barbashin-Krasovskiĭ theorem or the invariance principle $\exists \lim_{t \rightarrow +\infty} q(t) = q_d$, and, moreover, $(q, \dot{q}) = (q_d, 0)$ is a globally asymptotically stable (GAS) equilibrium of the closed-loop system.

6.2.2 PD-Controller with Gravity Compensation

Given a mechanical system

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + B\dot{q} + g(q) = u,$$

how to modify the previously PD controller

$$u = -K_p (q - q_d) - K_d \dot{q}$$

to work in the case of $B \neq 0$ and $g(p) \neq 0$?

Obviously, the controller

$$u = -K_p (q - q_d) - K_d \dot{q} + g(q)$$

is stabilizing, if K_p and K_d are diagonal matrices with positive elements or just positive definite matrices such that

$$K_p > 0, \quad K_d + B > 0.$$

We will leave to the reader investigation of the so-called $PD+$ controller

$$\mathbf{u} = -\mathbf{K}_p (\mathbf{q} - \mathbf{q}_d) - \mathbf{K}_d \dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}_d)$$

that can be shown to ensure GAS as well with the help of the following auxiliary function

$$V(\mathbf{e}, \dot{\mathbf{q}}) = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}} + \frac{1}{2} \mathbf{e}^T \mathbf{K}_p \mathbf{e} + P(\mathbf{e} + \mathbf{q}_d) - P(\mathbf{q}_d) + \mathbf{e}^T \mathbf{g}(\mathbf{q}_d),$$

where $\mathbf{e} = \mathbf{q} - \mathbf{q}_d$, provided the gravitational force is globally Lipschitz, i.e. $\exists \alpha > 0$ s.t.

$$\left(\mathbf{g}(\mathbf{q}) - \mathbf{g}(\mathbf{q}_d) \right)^T \left(\mathbf{g}(\mathbf{q}) - \mathbf{g}(\mathbf{q}_d) \right) \leq \alpha (\mathbf{q} - \mathbf{q}_d)^T (\mathbf{q} - \mathbf{q}_d), \quad \forall \mathbf{q}$$

and

$$\mathbf{K}_p - \alpha \mathbf{I} > 0, \quad \mathbf{K}_d + \mathbf{B} > 0.$$

where $\mathbf{I}$ is the identity matrix.

6.3 Feedback Linearization

Let us consider now one possible control design strategy that can be applied in the case when $\mathbf{q}_d(t) \neq \text{const.}$ This is known as **tracking** problem as opposed to the **point-to-point** or **regulation** problem solved above.

6.3.1 Feedback Linearization Under No Uncertainty

Given a mechanical system

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{B}\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{u},$$

how to define a control transformation

$$\mathbf{u} = \alpha(\mathbf{q}, \dot{\mathbf{q}}) + \beta(\mathbf{q}, \dot{\mathbf{q}})\mathbf{v}$$

such that the closed-loop system is **linear and stabilizable**? In other words, such that, possibly after a transformation of state variables

$$[\mathbf{q}, \dot{\mathbf{q}}] \longleftrightarrow \mathbf{x},$$

the original dynamics is equivalent to

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{v}$$

where the pair (A, B) is stabilizable?

In order to cancel many nonlinear terms we can simply start with

$$\mathbf{u} = M(\mathbf{q})\mathbf{v} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + B\dot{\mathbf{q}} + g(\mathbf{q})$$

Then, obviously, we obtain

$$M(\mathbf{q})\ddot{\mathbf{q}} = M(\mathbf{q})\mathbf{v} \iff \ddot{\mathbf{q}} = \mathbf{v}$$

using that $M(\mathbf{q}) > 0$.

Now,

$$\ddot{\mathbf{q}} = \mathbf{v} \iff \dot{\mathbf{x}} = \underbrace{\begin{bmatrix} \mathbf{0}_n & \mathbf{I}_n \\ \mathbf{0}_n & \mathbf{0}_n \end{bmatrix}}_{=A} \mathbf{x} + \underbrace{\begin{bmatrix} \mathbf{0}_n \\ \mathbf{I}_n \end{bmatrix}}_{=B} \mathbf{v}, \quad \mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$$

A stabilizing feedback control law for the obtained linear system can be designed easier. A possible choice is again a **PD** feedback but this time it is applied after a nonlinear feedback transformation.

So, for a given mechanical system

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + B\dot{\mathbf{q}} + g(\mathbf{q}) = \mathbf{u}$$

and a desired twice continuously differentiable trajectory $\mathbf{q}_d = \mathbf{q}_d(t)$, let us introduce the controller

$$\mathbf{u} = M(\mathbf{q})\mathbf{v} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + B\dot{\mathbf{q}} + g(\mathbf{q})$$

where

$$\mathbf{v} = \ddot{\mathbf{q}}_d(t) - K_p(\mathbf{q} - \mathbf{q}_d(t)) - K_d(\dot{\mathbf{q}} - \dot{\mathbf{q}}_d(t))$$

with $K_p > 0$ and $K_d > 0$.

Then, the closed-loop system is

$$\ddot{\mathbf{q}} = \mathbf{v} = \ddot{\mathbf{q}}_d(t) - K_p(\mathbf{q} - \mathbf{q}_d(t)) - K_d(\dot{\mathbf{q}} - \dot{\mathbf{q}}_d(t))$$

and it can be rewritten in the error-variables as

$$\ddot{\mathbf{e}} + K_d\dot{\mathbf{e}} + K_p\mathbf{e} = 0, \quad \mathbf{e} = \mathbf{q} - \mathbf{q}_d(t)$$

It is obvious that $\mathbf{e} \rightarrow 0$ as $t \rightarrow \infty$, while this is usually not true in the case when the simple **PD**-feedback, suggested above for the special case of constant reference trajectory, is applied directly for the tracking problem.

6.3.2 Robust and Adaptive Motion Control

We would like to propose now some modifications of the feedback linearization controller presented above. To this end, we notice that, in reality, for a given mechanical system

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \mathbf{u}$$

with a desired smooth trajectory $q_d = q_d(t)$, the controller

$$\mathbf{u} = M(q)\mathbf{v} + C(q, \dot{q})\dot{q} + g(q)$$

$$\mathbf{v} = \ddot{q}_d(t) - K_p(q - q_d(t)) - K_d(\dot{q} - \dot{q}_d(t))$$

cannot be implemented since the values of the parameters, which appear in the expressions for the elements of $M(q)$, $C(q, \dot{q})$, and $g(q)$, are never known precisely!

Of course, approximations might be used

$$\mathbf{u} = [M + \Delta M]\mathbf{v} + [C + \Delta C]\dot{q} + [g + \Delta g]$$

$$\mathbf{v} = \ddot{q}_d(t) - K_p(q - q_d(t)) - K_d(\dot{q} - \dot{q}_d(t))$$

If performance of the approximate realization of the feedback linearization-based design is not acceptable, one can use one the following two standard approaches to improve it.

- **Robust Control:**

Design K_p , K_d , and, if necessary, an additional input $\mathbf{w}$ in the controller

$$\mathbf{u} = [M + \Delta M]\mathbf{v} + [C + \Delta C]\dot{q} + [g + \Delta g]$$

$$\mathbf{v} = \ddot{q}_d(t) - K_p(q - q_d(t)) - K_d(\dot{q} - \dot{q}_d(t)) + \mathbf{w}$$

such that the error signal

$$e(t) = q(t) - q_d(t) \approx 0 \quad \forall \{\Delta M, \Delta C, \Delta g\} \in \mathcal{W}$$

- **Adaptive Control:** Improve the estimates for $M(q)$, $C(q, \dot{q})$, and $g(q)$ on-line until the error signal vanishes. Here we might aim at finding the law of change for the estimates achieving $\Delta M, \Delta C, \Delta g \rightarrow 0$.

6.3.3 Robust Control Based on Feedback Linearization

Given a trajectory $\mathbf{q} = \mathbf{q}_d(t)$, consider our system

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{u}$$

with approximate feedback linearization control law based on available approximations (nominal values) of the system parameters

$$\mathbf{u} = [\mathbf{M} + \Delta\mathbf{M}] \mathbf{v} + [\mathbf{C} + \Delta\mathbf{C}] \dot{\mathbf{q}} + [\mathbf{g} + \Delta\mathbf{g}]$$

Substituting, we obtain

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = [\mathbf{M} + \Delta\mathbf{M}] \mathbf{v} + [\mathbf{C} + \Delta\mathbf{C}] \dot{\mathbf{q}} + [\mathbf{g} + \Delta\mathbf{g}]$$

$$\Leftrightarrow \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} = [\mathbf{M} + \Delta\mathbf{M}] \mathbf{v} + \Delta\mathbf{C}\dot{\mathbf{q}} + \Delta\mathbf{g}$$

$$\Leftrightarrow \ddot{\mathbf{q}} = \mathbf{M}^{-1} [\mathbf{M} + \Delta\mathbf{M}] \mathbf{v} + \mathbf{M}^{-1} [\Delta\mathbf{C}\dot{\mathbf{q}} + \Delta\mathbf{g}] \pm \mathbf{M}^{-1}\mathbf{M}\mathbf{v}$$

Finally, the closed-loop system can be rewritten as

$$\ddot{\mathbf{q}} = \mathbf{v} + \boldsymbol{\eta}(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{v})$$

where

$$\boldsymbol{\eta}(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{v}) = \mathbf{M}^{-1}(\mathbf{q}) [\Delta\mathbf{M}(\mathbf{q})] \mathbf{v} + \mathbf{M}^{-1}(\mathbf{q}) [\Delta\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \Delta\mathbf{g}(\mathbf{q})]$$

Since

$$\Delta\mathbf{M} = \mathbf{0}, \quad \Delta\mathbf{C} = \mathbf{0}, \quad \Delta\mathbf{g} = \mathbf{0} \quad \Rightarrow \quad \boldsymbol{\eta} = \mathbf{0}$$

it makes sense to modify our previous design

$$\mathbf{v} = \ddot{\mathbf{q}}_d(t) - \mathbf{K}_p(\mathbf{q} - \mathbf{q}_d(t)) - \mathbf{K}_d(\dot{\mathbf{q}} - \dot{\mathbf{q}}_d(t)) + \mathbf{w}$$

to keep the same performance in the unrealistic case of perfect knowledge of the parameters with $\mathbf{w} = \mathbf{0}$.

With this choice, our closed-loop system

$$\ddot{\mathbf{q}} = \mathbf{v} + \boldsymbol{\eta}(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{v})$$

becomes

$$(\ddot{\mathbf{q}} - \ddot{\mathbf{q}}_d) + \mathbf{K}_d(\dot{\mathbf{q}} - \dot{\mathbf{q}}_d) + \mathbf{K}_p(\mathbf{q} - \mathbf{q}_d) = \mathbf{w} + \boldsymbol{\eta}(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{v})$$

and can be rewritten as

$$\frac{d}{dt}e = \underbrace{\begin{bmatrix} \mathbf{0}_n & \mathbf{I}_n \\ -\mathbf{K}_p & -\mathbf{K}_d \end{bmatrix}}_A e + \underbrace{\begin{bmatrix} \mathbf{0}_n \\ \mathbf{I}_n \end{bmatrix}}_B (\mathbf{w} + \eta), \quad e = \begin{bmatrix} \mathbf{q} - \mathbf{q}_d \\ \dot{\mathbf{q}} - \dot{\mathbf{q}}_d \end{bmatrix}$$

or

$$\frac{d}{dt}e = A e + B (\mathbf{w} + \eta(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{v})), \quad e = \begin{bmatrix} e_1 \\ e_2 \end{bmatrix} \equiv \begin{bmatrix} \mathbf{q} - \mathbf{q}_d(t) \\ \dot{\mathbf{q}} - \dot{\mathbf{q}}_d(t) \end{bmatrix}$$

We should now design $\mathbf{w}$ for the system

$$\frac{d}{dt}e = A e + B [\mathbf{w} + \bar{\eta}(t, e, \mathbf{w})]$$

where $\eta(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{v})$ is rewritten as

$$\bar{\eta}(t, e, \mathbf{w}) = \eta\left(\underbrace{e_1 + \mathbf{q}_d(t)}_q, \underbrace{e_2 + \dot{\mathbf{q}}_d(t)}_{\dot{q}}, \underbrace{\ddot{\mathbf{q}}_d(t) - [\mathbf{K}_p \quad \mathbf{K}_d] e + \mathbf{w}}_{\mathbf{v}}\right)$$

Under certain assumptions on the mechanical system, it can be verified that there are positive constants $\alpha, \gamma_{1,2,3}$ such that the following estimate on $\bar{\eta}(t, e, \mathbf{w})$ is true

$$\|\bar{\eta}(t, e, \mathbf{w})\| \leq \alpha \|\mathbf{w}\| + \gamma_1 \|e\| + \gamma_2 \|e\|^2 + \gamma_3, \quad \alpha < 1$$

Now, the matrix A is stable; hence $\forall Q > 0, \exists P = P^T > 0$ such that

$$A^T P + P A = -Q$$

With such P , consider a Lyapunov function candidate

$$V(e) = e^T P e$$

and compute its time-derivative along the solutions

$$\begin{aligned} \frac{d}{dt}V &= \frac{d}{dt}e^T P e + e^T P \frac{d}{dt}e \\ &= e^T (A^T P + P A) e + 2e^T P B [\mathbf{w} + \bar{\eta}(t, e, \mathbf{w})] \\ &= -e^T Q e + 2e^T P B [\mathbf{w} + \bar{\eta}] \\ &\leq 0 \quad \leftarrow \text{How to achieve this by choosing } \mathbf{w}? \end{aligned}$$

Let us look at the second term in

$$\frac{d}{dt}V = -e^T Q e + 2e^T P B [\mathbf{w} + \bar{\eta}]$$

when $\mathbf{w}$ has the form

$$\mathbf{w} = -\rho(\cdot) \frac{\mathbf{z}}{\sqrt{\mathbf{z}^T \mathbf{z}}}, \quad \mathbf{z} = \mathbf{B}^T \mathbf{P} \mathbf{e}, \quad \rho \text{ is a function to choose}$$

We have

$$\mathbf{e}^T \mathbf{P} \mathbf{B} [\mathbf{w} + \bar{\boldsymbol{\eta}}] = \mathbf{z}^T \left(-\rho \frac{\mathbf{z}}{\sqrt{\mathbf{z}^T \mathbf{z}}} + \bar{\boldsymbol{\eta}} \right) \|\mathbf{z}\| (-\rho + \|\bar{\boldsymbol{\eta}}\|)$$

To sum up, we search for a scalar function $\rho(\cdot)$ such that

$$(-\rho + \|\bar{\boldsymbol{\eta}}\|) \leq 0 \Leftrightarrow \|\bar{\boldsymbol{\eta}}\| \leq \rho$$

and we have

$$\|\bar{\boldsymbol{\eta}}(t, \mathbf{e}, \mathbf{w})\| \leq \alpha \|\mathbf{w}\| + \gamma_1 \|\mathbf{e}\| + \gamma_2 \|\mathbf{e}\|^2 + \gamma_3, \quad \alpha < 1$$

with

$$\mathbf{w} = -\rho(\cdot) \frac{\mathbf{z}}{\sqrt{\mathbf{z}^T \mathbf{z}}}$$

Therefore, we must have

$$\begin{aligned} \alpha \left\| \rho(\cdot) \frac{\mathbf{z}}{\sqrt{\mathbf{z}^T \mathbf{z}}} \right\| + \gamma_1 \|\mathbf{e}\| + \gamma_2 \|\mathbf{e}\|^2 + \gamma_3 &\leq \rho(\cdot) \\ \Rightarrow \alpha \rho(\cdot) + \gamma_1 \|\mathbf{e}\| + \gamma_2 \|\mathbf{e}\|^2 + \gamma_3 &\leq \rho(\cdot) \\ \Rightarrow \rho(\mathbf{e}) &\geq \frac{1}{1 - \alpha} [\gamma_1 \|\mathbf{e}\| + \gamma_2 \|\mathbf{e}\|^2 + \gamma_3] \end{aligned}$$

Finally, for a given trajectory $\mathbf{q} = \mathbf{q}_d(t)$, we design stabilizing feedback controller for

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{u}$$

in the following form

$$\begin{aligned} \mathbf{u} &= [\mathbf{M}(\mathbf{q}) + \Delta \mathbf{M}(\mathbf{q})] \mathbf{v} + [\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \Delta \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})] \dot{\mathbf{q}} + [\mathbf{g}(\mathbf{q}) + \Delta \mathbf{g}(\mathbf{q})] \\ \mathbf{v} &= \ddot{\mathbf{q}}_d(t) - \mathbf{K}_p(\mathbf{q} - \mathbf{q}_d(t)) - \mathbf{K}_d(\dot{\mathbf{q}} - \dot{\mathbf{q}}_d(t)) + \dot{\mathbf{q}}_d(t) + \mathbf{w} \\ \mathbf{w} &= \begin{cases} -\rho(\mathbf{e}) \frac{\mathbf{z}}{\sqrt{\mathbf{z}^T \mathbf{z}}}, & \text{if } \mathbf{z} = \mathbf{B}^T \mathbf{P} \mathbf{e} \neq 0 \\ 0, & \text{if } \mathbf{z} = \mathbf{B}^T \mathbf{P} \mathbf{e} = 0 \end{cases} \end{aligned}$$

where $\rho(\cdot)$ is any function that satisfies the inequality

$$\rho \geq \frac{1}{1 - \alpha} [\gamma_1 \|\mathbf{e}\| + \gamma_2 \|\mathbf{e}\|^2 + \gamma_3]$$

where constants α , γ_1 - γ_2 are from the inequality

$$\|\eta\| \leq \alpha \|\mathbf{w}\| + \gamma_1 \|e\| + \gamma_2 \|e\|^2 + \gamma_3, \quad \alpha < 1$$

and

$$\eta = M^{-1}(q) [\Delta M(q)\mathbf{v} + \Delta C(q, \dot{q})\dot{q} + \Delta g(q)], \quad e = \begin{bmatrix} q - q_d \\ \dot{q} - \dot{q}_d \end{bmatrix}$$

Implemented controller above, however, often leads to very fast switching of the control signal known as **chattering** phenomenon. Let us discuss what it means and how to avoid it. If the control variable is scalar then the expression

$$\mathbf{w} = \begin{cases} -\rho(e) \frac{z}{\sqrt{z^T z}}, & \text{if } z = B^T P e \neq 0 \\ 0, & \text{if } z = B^T P e = 0 \end{cases}$$

becomes

$$\mathbf{w} = \begin{cases} -\rho(e) \text{sign}(z), & \text{if } z = B^T P e \neq 0 \\ 0, & \text{if } z = B^T P e = 0 \end{cases}$$

This means that whenever $z(t)$ is changing its sign, the control signal has to be changed instantaneously with a non-zero increment! Since such change may be impossible to implement by an actuator, and due to various delays and other implementation imperfections, high frequency oscillations in the control signal with large amplitude, known as chattering, appear.

One of possible choices to avoid the discontinuity in the control signal is to use the following continuous control law

$$\mathbf{w} = \begin{cases} -\rho(e) \text{sign}(B^T P e), & \text{if } |B^T P e| > \delta \\ -\frac{1}{\delta} \rho(e) B^T P e, & \text{if } |B^T P e| \leq \delta \end{cases}$$

Let us analyze the implications of such a modification.

The closed-loop system can be rewritten as

$$\frac{d}{dt}e = A e + B [\mathbf{w}(e) + \eta], \quad \mathbf{w} \in \mathbb{R}^1, \quad \eta \in \mathbb{R}^1$$

where we have assumed that

$$|\eta| \leq \rho(e)$$

with some known in advance function $\rho(e)$.

As above, we have

$$A = \begin{bmatrix} 0_n & I_n \\ -K_p & -K_d \end{bmatrix}, \quad B = \begin{bmatrix} 0_n \\ I_n \end{bmatrix}, \quad e = \begin{bmatrix} q - q_d \\ \dot{q} - \dot{q}_d \end{bmatrix}$$

Since matrix $\mathbf{A}$ is stable (Hurwitz), $\forall \mathbf{Q} > \mathbf{0}$, $\exists \mathbf{P} = \mathbf{P}^T > \mathbf{0}$ such that

$$\mathbf{A}^T \mathbf{P} + \mathbf{P} \mathbf{A} = -\mathbf{Q}$$

As before, we consider the quadratic Lyapunov function candidate

$$V(\mathbf{e}) = \mathbf{e}^T \mathbf{P} \mathbf{e}$$

and compute its time derivative along the systems' trajectories

$$\begin{aligned} \frac{d}{dt} V &= \frac{d}{dt} \mathbf{e}^T \mathbf{P} \mathbf{e} + \mathbf{e}^T \mathbf{P} \frac{d}{dt} \mathbf{e} \\ &= \mathbf{e}^T (\mathbf{A}^T \mathbf{P} + \mathbf{P} \mathbf{A}) \mathbf{e} + 2 \mathbf{e}^T \mathbf{P} \mathbf{B} [\mathbf{w} + \boldsymbol{\eta}] \\ &= -\mathbf{e}^T \mathbf{Q} \mathbf{e} + 2 \mathbf{e}^T \mathbf{P} \mathbf{B} [\mathbf{w} + \boldsymbol{\eta}] \end{aligned}$$

With the modified control law, for those $\mathbf{e}$ that satisfy $|\mathbf{B}^T \mathbf{P} \mathbf{e}| > \delta$ we have as before

$$\begin{aligned} \frac{d}{dt} V &= -\mathbf{e}^T \mathbf{Q} \mathbf{e} + 2 \mathbf{e}^T \mathbf{P} \mathbf{B} [\mathbf{w} + \boldsymbol{\eta}] \\ &= -\mathbf{e}^T \mathbf{Q} \mathbf{e} + 2z [\mathbf{w} + \boldsymbol{\eta}] \\ &< 2z [-\rho(\mathbf{e}) \text{sign}(z) + \boldsymbol{\eta}] \\ &= -2|z| [\rho(\mathbf{e}) - \boldsymbol{\eta} \text{sign}(z)] \\ &\leq -2|z| [\rho(\mathbf{e}) - |\boldsymbol{\eta}|] \leq 0 \end{aligned}$$

while for those $\mathbf{e}$ that satisfy $|z| = |\mathbf{B}^T \mathbf{P} \mathbf{e}| \leq \delta$ we have

$$\begin{aligned} \frac{d}{dt} V &= -\mathbf{e}^T \mathbf{Q} \mathbf{e} + 2 \mathbf{e}^T \mathbf{P} \mathbf{B} [\mathbf{w} + \boldsymbol{\eta}] \\ &= -\mathbf{e}^T \mathbf{Q} \mathbf{e} + 2z \left[-\frac{1}{\delta} \rho(\mathbf{e}) z + \boldsymbol{\eta} \right] \\ &= -\mathbf{e}^T \mathbf{Q} \mathbf{e} + 2 \left[-\frac{1}{\delta} \rho(\mathbf{e}) |z|^2 + z \boldsymbol{\eta} \right] \\ &\leq -\mathbf{e}^T \mathbf{Q} \mathbf{e} + 2 \left[-\frac{1}{\delta} \rho(\mathbf{e}) |z|^2 + |z| |\boldsymbol{\eta}| \right] \\ &\leq -\mathbf{e}^T \mathbf{Q} \mathbf{e} + 2 \left[-\frac{1}{\delta} \rho(\mathbf{e}) |z|^2 + \rho(\mathbf{e}) |z| \right] \\ &\leq -\mathbf{e}^T \mathbf{Q} \mathbf{e} + 2 \rho(\mathbf{e}) \underbrace{\left[-\frac{1}{\delta} |z|^2 + |z| \right]} \end{aligned}$$

What is the maximum of this expression?

$$\leq -\mathbf{e}^T \mathbf{Q} \mathbf{e} + 2 \rho(\mathbf{e}) \underbrace{\left[-\frac{1}{\delta} \left| \frac{\delta}{2} \right|^2 + \left| \frac{\delta}{2} \right| \right]}$$

the maximum is at $z = \frac{\delta}{2}$

$$= -e^T Q e + 2\rho(e) \frac{\delta}{4} = -e^T Q e + \rho(e) \frac{\delta}{2}$$

Hence, for those e that satisfies $|z| = |B^T P e| \leq \delta$ we have

$$\frac{d}{dt} V \leq -e^T Q e + \rho(e) \frac{\delta}{2} \leq 0$$

provided

$$\left[\|Q\| \right] \|e\|^2 \geq e^T Q e > \frac{\delta}{2} \rho(e)$$

with

$$\frac{\delta}{2} \rho(e) \geq \frac{\delta/2}{1-\alpha} \left[\gamma_1 \|e\| + \gamma_2 \|e\|^2 + \gamma_3 \right]$$

Let us summarize the results:

- To avoid chattering, the next modification is suggested

$$w = \begin{cases} -\rho(e) \operatorname{sign}(B^T P e), & \text{if } |B^T P e| > \delta \\ -\frac{1}{\delta} \rho(e) B^T P e, & \text{if } |B^T P e| \leq \delta \end{cases}$$

- With such modification, we cannot prove asymptotic stability of the closed-loop system :(
- But we can show that the value of the Lyapunov function is decreasing everywhere, except some vicinity of the origin of the error dynamics.
- The size of this set can be modified, and depends on δ . Moreover, it can be made arbitrary small by reducing δ .
- Such property is called **ultimate boundedness**.

Example:

Consider a smooth target reference signal $q_d(t)$ and the system

$$M \ddot{q} = u, \quad M = 1 + \varepsilon$$

Suppose our nominal (estimated) value for M is 1 and so it has the error

$$\Delta M = -\varepsilon$$

where the value of ε is not known but it is reasonable to assume that we know an interval of values to which it must belong.

The system is linear, so that the transforms $\mathbf{u} \rightarrow \mathbf{v} \rightarrow \mathbf{w}$ are

$$\begin{aligned}\mathbf{u} &= (\mathbf{M} + \Delta\mathbf{M})\mathbf{v} = \mathbf{v} \\ \mathbf{v} &= \ddot{\mathbf{q}}_d(t) - \mathbf{K}_p(\mathbf{q} - \mathbf{q}_d(t)) - \mathbf{K}_d(\dot{\mathbf{q}} - \dot{\mathbf{q}}_d(t)) + \mathbf{w}\end{aligned}$$

If for instance, $\mathbf{K}_p = \mathbf{K}_d = \mathbf{1}$, then the transformed system is

$$(1 + \varepsilon)\ddot{\mathbf{q}} = \ddot{\mathbf{q}}_d(t) - (\mathbf{q} - \mathbf{q}_d(t)) - (\dot{\mathbf{q}} - \dot{\mathbf{q}}_d(t)) + \mathbf{w}$$

or

$$(\ddot{\mathbf{q}} - \ddot{\mathbf{q}}_d(t)) + (\dot{\mathbf{q}} - \dot{\mathbf{q}}_d(t)) + (\mathbf{q} - \mathbf{q}_d(t)) = \mathbf{w} + \boldsymbol{\eta}(\cdot)$$

with

$$\boldsymbol{\eta} = \frac{\varepsilon}{1 + \varepsilon} [\ddot{\mathbf{q}}_d(t) - (\mathbf{q} - \mathbf{q}_d(t)) - (\dot{\mathbf{q}} - \dot{\mathbf{q}}_d(t)) + \mathbf{w}]$$

To proceed with robust design we need

- To rewrite the system

$$(\ddot{\mathbf{q}} - \ddot{\mathbf{q}}_d(t)) + (\dot{\mathbf{q}} - \dot{\mathbf{q}}_d(t)) + (\mathbf{q} - \mathbf{q}_d(t)) = \mathbf{w} + \boldsymbol{\eta}(\cdot)$$

in a state-space form

$$\frac{d}{dt}\mathbf{e} = \mathbf{A}\mathbf{e} + \mathbf{B}[\mathbf{w} + \boldsymbol{\eta}(\cdot)], \quad \mathbf{e} = \begin{bmatrix} \mathbf{q} - \mathbf{q}_d \\ \dot{\mathbf{q}} - \dot{\mathbf{q}}_d \end{bmatrix}$$

- To solve the Lyapunov equation

$$\mathbf{A}^T \mathbf{P} + \mathbf{P} \mathbf{A} = -\mathbf{Q}, \quad \text{with, e.g.,} \quad \mathbf{Q} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- To find a range of ε such that the function

$$\boldsymbol{\eta} = \frac{\varepsilon}{1 + \varepsilon} [\ddot{\mathbf{q}}_d(t) - (\mathbf{q} - \mathbf{q}_d(t)) - (\dot{\mathbf{q}} - \dot{\mathbf{q}}_d(t)) + \mathbf{w}]$$

satisfies the bound

$$\|\boldsymbol{\eta}\| \leq \alpha \|\mathbf{w}\| + \gamma_1 \|\mathbf{e}\| + \gamma_2 \|\mathbf{e}\|^2 + \gamma_3, \quad \alpha < 1$$

Clearly, we can take

$$\gamma_1 = \mathbf{K}_p + \mathbf{K}_d = 2, \quad \gamma_2 = 0, \quad \alpha = \sup_{\varepsilon} \frac{\varepsilon}{1 + \varepsilon}, \quad \gamma_3 = \alpha \sup_{t \geq 0} |\ddot{\mathbf{q}}_d(t)|$$

6.3.4 Adaptive Control Based on Feedback Linearization

Let us consider now an alternative approach to the robust design, known as adaptive design. Here, instead of introducing another control input, we derive the so-called adaptation law that defines how to change on-line our guesses for the systems' parameters.

Given a mechanical system

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \mathbf{u}$$

and the desired trajectory $q = q_d(t)$, introduce the controller

$$\begin{aligned} \mathbf{u} &= \hat{M}(q)\mathbf{v} + \hat{C}(q, \dot{q})\dot{q} + \hat{g}(q) \\ \mathbf{v} &= \ddot{q}_d(t) - K_p(q - q_d(t)) - K_d(\dot{q} - \dot{q}_d(t)) \end{aligned}$$

where in opposite to the robust design:

- The elements of $\hat{M}(\cdot)$, $\hat{C}(\cdot)$, $\hat{g}(\cdot)$ are not fixed but changing with time in an appropriate way and they are variables to tune, while
- There is no additional control input, i.e. $\mathbf{w} = \mathbf{0}$!

Let us find the dynamical equations for updating values of $\hat{M}(\cdot)$, $\hat{C}(\cdot)$, $\hat{g}(\cdot)$ provided that we measure $q(t)$, $\dot{q}(t)$, and $\ddot{q}(t)$.

Let us rewrite the system

$$\begin{aligned} M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) &= \mathbf{u}, \quad \mathbf{u} = \hat{M}(q)\mathbf{v} + \hat{C}(q, \dot{q})\dot{q} + \hat{g}(q) \\ \mathbf{v} &= \ddot{q}_d(t) - K_p(q - q_d(t)) - K_d(\dot{q} - \dot{q}_d(t)) \end{aligned}$$

as follows

$$\begin{aligned} \hat{M}(q)\ddot{q} + \left[M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) - \hat{M}(q)\ddot{q} \right] &= \mathbf{u} \\ \mathbf{u} &= \hat{M}(q)\mathbf{v} + \hat{C}(q, \dot{q})\dot{q} + \hat{g}(q) \end{aligned}$$

or

$$\begin{aligned} \hat{M}(q)\ddot{q} &= \hat{M}(q)\mathbf{v} + \left[\left(\hat{M}(q) - M(q) \right) \ddot{q} + \left(\hat{C}(q, \dot{q}) - C(q, \dot{q}) \right) \dot{q} \right. \\ &\quad \left. + \hat{g}(q) - g(q) \right] \\ &= \hat{M}(q)\mathbf{v} + Y(q, \dot{q}, \ddot{q}) \left[\hat{\theta} - \theta \right] \end{aligned}$$

where

- θ is the vector of true (unknown) parameters of the model,

- $\hat{\theta}$ is the vector of current estimates of the parameters,
- $Y(\cdot)$ is the vector function, values of which we can compute measuring $q(t)$, $\dot{q}(t)$, $\ddot{q}(t)$, $q_d(t)$ and its derivatives, i.e.

$$\begin{aligned} Y(q, \dot{q}, \ddot{q}) \theta &= M(q) \ddot{q} + C(q, \dot{q}) \dot{q} + g(q) \\ Y(q, \dot{q}, \ddot{q}) \hat{\theta} &= \hat{M}(q) \ddot{q} + \hat{C}(q, \dot{q}) \dot{q} + \hat{g}(q) \end{aligned}$$

Our closed-loop system is

$$\ddot{q} = \mathbf{v} + \hat{M}(q)^{-1} Y(q, \dot{q}, \ddot{q}) [\hat{\theta} - \theta]$$

and with the PD part of the feedback linearization design we have

$$(\ddot{q} - \ddot{q}_d) + K_d(\dot{q} - \dot{q}_d) + K_p(q - q_d) = \hat{M}^{-1}(q) Y(q, \dot{q}, \ddot{q}) [\hat{\theta} - \theta]$$

where there are **variables** that we can change! How to do this update?

To answer this question, let us rewrite the system in the state-space form

$$\frac{d}{dt}e = \underbrace{\begin{bmatrix} 0 & I \\ -K_p & -K_d \end{bmatrix}}_{=A} e + \underbrace{\begin{bmatrix} 0 \\ I \end{bmatrix}}_{=B} \Phi [\hat{\theta} - \theta], \quad e = \begin{bmatrix} q - q_d(t) \\ \dot{q} - \dot{q}_d(t) \end{bmatrix}$$

where

$$\Phi = \Phi(\hat{\theta}, q, \dot{q}, \ddot{q}) = \hat{M}^{-1}(q) Y(q, \dot{q}, \ddot{q})$$

Now we need to choose the update law for $\hat{\theta}$ in

$$\frac{d}{dt}e = Ae + B\Phi [\hat{\theta} - \theta]$$

to ensure that $e \rightarrow 0$. Note that since θ is unknown, we can not just take $\hat{\theta} = \theta$ but since it is constant, we might be able to introduce an integral action to attenuate its impact.

Let us take the solution of the Lyapunov equation

$$A^T P + P A = -Q < 0, \quad P > 0$$

and consider the Lyapunov function candidate

$$V = e^T P e + \frac{1}{2} [\hat{\theta} - \theta]^T \Gamma [\hat{\theta} - \theta], \quad \Gamma > 0$$

Then

$$\begin{aligned}
\frac{d}{dt}V &= \frac{d}{dt}\{e^T P e\} + \frac{d}{dt}\left\{\frac{1}{2}[\hat{\theta} - \theta]^T \Gamma [\hat{\theta} - \theta]\right\} \\
&= \left[Ae + B\Phi[\hat{\theta} - \theta]\right]^T P e + e^T P \left[Ae + B\Phi[\hat{\theta} - \theta]\right] \\
&\quad + \frac{1}{2}\left[\frac{d}{dt}\hat{\theta} - \mathbf{0}\right]^T \Gamma [\hat{\theta} - \theta] + \frac{1}{2}[\hat{\theta} - \theta]^T \Gamma \left[\frac{d}{dt}\hat{\theta} - \mathbf{0}\right] \\
&= e^T [A^T P + P A] e + [\hat{\theta} - \theta]^T \left\{\Phi^T B^T P e + \Gamma \frac{d}{dt}\hat{\theta}\right\} \\
&= -e^T Q e + [\hat{\theta} - \theta]^T \underbrace{\left\{\Phi^T B^T P e + \Gamma \frac{d}{dt}\hat{\theta}\right\}}_{=0} \leq 0
\end{aligned}$$

With the proposed update law for $\hat{\theta}$ the closed-loop system is

$$\frac{d}{dt}e = Ae + B\Phi[\theta - \hat{\theta}], \quad \frac{d}{dt}\hat{\theta} = -\Gamma^{-1}\Phi^T B^T P e$$

The inequality

$$\frac{d}{dt}V = \frac{d}{dt}\left[e^T P e + \frac{1}{2}[\hat{\theta} - \theta]^T \Gamma [\hat{\theta} - \theta]\right] = -e^T Q e$$

implies that (thanks to Barbalat lemma)

$$e(t) \rightarrow 0 \text{ as } t \rightarrow +\infty \quad \text{and} \quad (\hat{\theta}(t) - \theta) \text{ is bounded}$$

However, some difficulty may occur trying to implement the adaptive control law above. Note that it is necessary

- To measure (or accurately estimate) $\ddot{\mathbf{q}}$ for computing the regressor;
- To ensure that the matrix $\hat{M}(\mathbf{q})$ at each time moment is invertible.

6.4 Passivity Based Motion Control

Let us discuss now how to redesign the feedback controller above avoiding the need of computing $\ddot{\mathbf{q}}$. We will use the passivity property.

6.4.1 Passivity Based Control Under No Uncertainty

Given a trajectory $\mathbf{q} = \mathbf{q}_d(t)$, consider the system

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{u}$$

with the controller

$$\mathbf{u} = M(\mathbf{q})\mathbf{a} + C(\mathbf{q}, \dot{\mathbf{q}})\mathbf{b} + g(\mathbf{q}) - K\mathbf{r}, \quad K = \text{diag}\{k_1, \dots, k_n\}$$

where the variables $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{r}$ are to be chosen.

Note that for feedback linearization we have used the controller above with

$$\mathbf{a} = \ddot{\mathbf{q}}_d - K_d(\dot{\mathbf{q}} - \dot{\mathbf{q}}_d) - K_p(\mathbf{q} - \mathbf{q}_d), \quad \mathbf{b} = \dot{\mathbf{q}}, \quad \mathbf{r} = \mathbf{0}$$

to obtain a linear closed-loop system. Here, we will suggest another choice of $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{r}$ that will lead to a nonlinear closed-loop system of special structure, which would allow us to suggest an energy-like Lyapunov function candidate for analysis.

If we substitute the controller, we obtain

$$M(\mathbf{q})[\ddot{\mathbf{q}} - \mathbf{a}] + C(\mathbf{q}, \dot{\mathbf{q}})[\dot{\mathbf{q}} - \mathbf{b}] + K\mathbf{r} = \mathbf{0}$$

Let us impose the following relations among $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{r}$

$$\frac{d}{dt}\mathbf{r} = \ddot{\mathbf{q}} - \mathbf{a}, \quad \mathbf{r} = \dot{\mathbf{q}} - \mathbf{b} \quad \Rightarrow \quad \frac{d}{dt}\mathbf{b} = \mathbf{a}$$

and take

$$\mathbf{b} = \dot{\mathbf{q}}_d(t) - \Lambda(\mathbf{q} - \mathbf{q}_d(t))$$

with $\Lambda = \text{diag}\{\lambda_1, \dots, \lambda_n\} > \mathbf{0}$, so that

$$\mathbf{r} = \dot{\mathbf{q}} - \dot{\mathbf{q}}_d(t) + \Lambda(\mathbf{q} - \mathbf{q}_d(t)),$$

$$\mathbf{a} = \ddot{\mathbf{q}}_d(t) - \Lambda(\dot{\mathbf{q}} - \dot{\mathbf{q}}_d(t)).$$

Clearly, $\mathbf{q} = \mathbf{q}_d(t) \Rightarrow \mathbf{b} = \dot{\mathbf{q}}_d(t)$ and $\mathbf{a} = \ddot{\mathbf{q}}_d(t)$
while $\mathbf{r} \equiv \mathbf{0} \Rightarrow \mathbf{q} - \mathbf{q}_d(t) \rightarrow \mathbf{0}$ (exponentially) as $t \rightarrow \infty$.

Then, the closed-loop system can be written as

$$M(\mathbf{q})\frac{d}{dt}\mathbf{r} + C(\mathbf{q}, \dot{\mathbf{q}})\mathbf{r} + K\mathbf{r} = \mathbf{0}$$

and we would like to show that $\mathbf{r} \rightarrow \mathbf{0}$.

Consider the Lyapunov function candidate

$$V(\tilde{\mathbf{q}}, \mathbf{r}) = \frac{1}{2}\mathbf{r}^T M(\tilde{\mathbf{q}} + \mathbf{q}_d)\mathbf{r} + \tilde{\mathbf{q}}^T K \Lambda \tilde{\mathbf{q}}$$

where $\tilde{\mathbf{q}} = \mathbf{q} - \mathbf{q}_d$.

Computing its time derivative along the solutions we obtain

$$\begin{aligned}
\frac{d}{dt}V &= \frac{d}{dt} \left[\frac{1}{2} \mathbf{r}^T M(\mathbf{q}) \mathbf{r} \right] + \frac{d}{dt} [\tilde{\mathbf{q}}^T K \Lambda \tilde{\mathbf{q}}] \\
&= \mathbf{r}^T M(\mathbf{q}) \frac{d}{dt} [\mathbf{r}] + \frac{1}{2} \mathbf{r}^T \frac{d}{dt} [M(\mathbf{q})] \mathbf{r} + 2\tilde{\mathbf{q}}^T K \Lambda \frac{d}{dt} \tilde{\mathbf{q}} \\
&= \mathbf{r}^T [-C(\mathbf{q}, \dot{\mathbf{q}}) \mathbf{r} - K \mathbf{r}] + \frac{1}{2} \mathbf{r}^T \frac{d}{dt} [M(\mathbf{q})] \mathbf{r} + 2\tilde{\mathbf{q}}^T K \Lambda \frac{d}{dt} \tilde{\mathbf{q}} \\
&= -\mathbf{r}^T K \mathbf{r} + \underbrace{\mathbf{r}^T \left[\frac{1}{2} \frac{d}{dt} M(\mathbf{q}) - C(\mathbf{q}, \dot{\mathbf{q}}) \right] \mathbf{r}}_{=0} + 2\tilde{\mathbf{q}}^T K \Lambda \frac{d}{dt} \tilde{\mathbf{q}} \\
&= -\mathbf{r}^T K \mathbf{r} + 2\tilde{\mathbf{q}}^T K \Lambda \tilde{\mathbf{q}}
\end{aligned}$$

where we can substitute

$$\mathbf{r} = \dot{\mathbf{q}} - \dot{\mathbf{q}}_d + \Lambda (\mathbf{q} - \mathbf{q}_d) = \frac{d}{dt} \tilde{\mathbf{q}} + \Lambda \tilde{\mathbf{q}}$$

with $\tilde{\mathbf{q}} = \mathbf{q} - \mathbf{q}_d$ to obtain

$$\begin{aligned}
\frac{d}{dt}V &= - \left(\frac{d}{dt} \tilde{\mathbf{q}} + \Lambda \tilde{\mathbf{q}} \right)^T K \left(\frac{d}{dt} \tilde{\mathbf{q}} + \Lambda \tilde{\mathbf{q}} \right) + 2\tilde{\mathbf{q}}^T K \Lambda \frac{d}{dt} \tilde{\mathbf{q}} \\
&= - \left[\frac{d}{dt} \tilde{\mathbf{q}} \right]^T K \frac{d}{dt} \tilde{\mathbf{q}} - \tilde{\mathbf{q}}^T \Lambda^T K \Lambda \tilde{\mathbf{q}} < 0
\end{aligned}$$

Finally, for the time-dependent closed-loop system

$$M(\tilde{\mathbf{q}} + \mathbf{q}_d(t)) \frac{d}{dt} \mathbf{r} + C(\tilde{\mathbf{q}} + \mathbf{q}_d(t), \dot{\tilde{\mathbf{q}}} + \dot{\mathbf{q}}_d(t)) \mathbf{r} + K \mathbf{r} = 0$$

with state variables $\tilde{\mathbf{q}}$ and $\mathbf{r}$ we have positive definite

$$V = \frac{1}{2} \mathbf{r}^T M(\mathbf{q}) \mathbf{r} + \tilde{\mathbf{q}}^T K \Lambda \tilde{\mathbf{q}} > 0$$

and negative definite

$$\frac{d}{dt}V = -(\mathbf{r} - \Lambda \tilde{\mathbf{q}})^T K (\mathbf{r} - \Lambda \tilde{\mathbf{q}}) - \tilde{\mathbf{q}}^T \Lambda^T K \Lambda \tilde{\mathbf{q}} < 0$$

Global asymptotic stability follows as well as $\tilde{\mathbf{q}} \rightarrow 0$ and $\mathbf{r} \rightarrow 0$.

6.4.2 Adaptive Passivity Based Motion Control

Let us transform a feedback controller above into an adaptive one that can be used whenever the parameters are unknown.

Given a trajectory $\mathbf{q} = \mathbf{q}_d(t)$, consider the system

$$M(\mathbf{q}) \ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + g(\mathbf{q}) = \mathbf{u}$$

with the controller

$$\mathbf{u} = \hat{M}(\mathbf{q}) \mathbf{a} + \hat{C}(\mathbf{q}, \dot{\mathbf{q}}) \mathbf{b} + \hat{g}(\mathbf{q}) - K \mathbf{r}, \quad K = \text{diag} \{k_1, \dots, k_n\}$$

with the variables $\mathbf{a}$, $\mathbf{b}$, $\mathbf{r}$ defined as above by

$$\mathbf{a} = \ddot{\mathbf{q}}_d - \Lambda (\dot{\mathbf{q}} - \dot{\mathbf{q}}_d), \quad \mathbf{b} = \dot{\mathbf{q}}_d - \Lambda (\mathbf{q} - \mathbf{q}_d), \quad \mathbf{r} = \dot{\mathbf{q}} - \dot{\mathbf{q}}_d + \Lambda (\mathbf{q} - \mathbf{q}_d)$$

The closed-loop system can be rewritten as

$$\begin{aligned} M(\mathbf{q}) [\ddot{\mathbf{q}} - \mathbf{a}] + C(\mathbf{q}, \dot{\mathbf{q}}) [\dot{\mathbf{q}} - \mathbf{b}] + \mathbf{K}\mathbf{r} \\ = [\hat{M}(\mathbf{q}) - M(\mathbf{q})] \mathbf{a} + [\hat{C}(\mathbf{q}, \dot{\mathbf{q}}) - C(\mathbf{q}, \dot{\mathbf{q}})] \mathbf{b} + [\hat{g}(\mathbf{p}) - g(\mathbf{p})] \\ = Y(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{a}, \mathbf{b}) [\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}] \end{aligned}$$

which leads to

$$M(\mathbf{q}) \frac{d}{dt} \mathbf{r} + C(\mathbf{q}, \dot{\mathbf{q}}) \mathbf{r} + \mathbf{K}\mathbf{r} = Y(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{a}, \mathbf{b}) [\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}]$$

To find the update law for $\hat{\boldsymbol{\theta}}$ -variable we will use the Lyapunov function candidate

$$V = \frac{1}{2} \mathbf{r}^T M(\mathbf{q}) \mathbf{r} + (\mathbf{q} - \mathbf{q}_d(t))^T \mathbf{K} \Lambda (\mathbf{q} - \mathbf{q}_d(t)) + \frac{1}{2} [\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}]^T \Gamma [\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}]$$

Its time-derivative along a solution of the system is

$$\frac{d}{dt} V = -\dot{\mathbf{q}}^T \mathbf{K} \dot{\mathbf{q}} - \tilde{\mathbf{q}}^T \Lambda^T \mathbf{K} \Lambda \tilde{\mathbf{q}} + [\hat{\boldsymbol{\theta}} - \boldsymbol{\theta}]^T \underbrace{\left\{ \Gamma \frac{d}{dt} \hat{\boldsymbol{\theta}} + Y(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{a}, \mathbf{b})^T \mathbf{r} \right\}}_{=0} \leq 0$$

where we have used the following adaptation law

$$\frac{d}{dt} \hat{\boldsymbol{\theta}} = -\Gamma^{-1} Y^T(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{a}, \mathbf{b}) \mathbf{r}$$

Similar conclusions about the global stability and convergence to zero of the tracking error can be drawn as in the case of feedback-linearization-based adaptive design. However, here we have no need to accurately estimate $\ddot{\mathbf{q}}$.

Note that in a similar way as with feedback linearization, a robust controller can be designed based on passivity.

To conclude, we remark that the control laws derived in this chapter are based on the nonlinear dynamical model developed earlier in the form of Euler-Lagrange equations. To the best of our knowledge they are still considered to be too advanced and are not implemented in the currently available on the market industrial robotic manipulators.

Let us now discuss a few approaches that are less complex for implementation. They are to be derived with less rigor and are to be based on various simplifications of the model that might be unacceptable for a particular task to be performed by the robotic manipulator.

Chapter 7

Independent Joint Control

There are several methods that can be used to design feedback control laws making the robots follow a desired working scenario. Let us discuss first the steps that are necessary before using any of the advanced control designs such as the ones described in Chapter 6. After that, we will discuss a few related issues and briefly mention a less mathematically justified approach to feedback control design.

7.1 Development of Control Systems for Robots

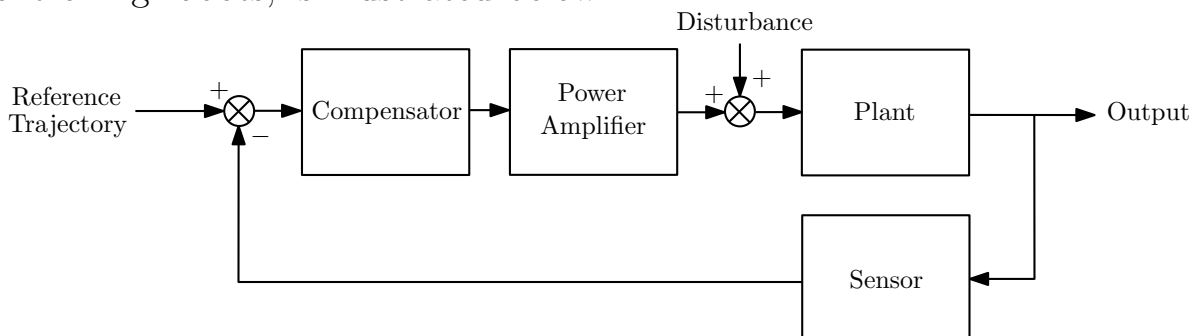
In order to apply any of the advanced methods of Chapter 6, one can proceed as follows.

- Compute appropriate mechanical models for describing possible behavior of the robot, in particular, describe
 - Kinematics (see, e.g., Chapter 2, 3) and use it to specify a scenario in terms of desired time-evolutions of the generalized coordinates such as DH-variables (see, e.g., Chapter 4);
 - Dynamics (see, e.g., Chapter 5);
- Estimate or/and compute from CAD drawings parameters of the kinematic and dynamic models. Here one can figure out necessary specifications of the system based on the dynamical description such as necessary for implementation ranges for joint velocities and accelerations as well as required magnitudes of the generalized forces produced by the actuators.
- Either choose (if possible) actuators and gears according to the specifications above or modify the trajectories designed based on kinematics to make them feasible according to the dynamical model.

- Either choose (if possible) sensors and their allocation or make appropriate models for sensors and actuators according to available specifications to improve the dynamical model for verification of achievable performance via simulation.
- Choose a control architecture, taking into account
 - Possibility to use linear vs. nonlinear methods;
 - Implementation of compensation for friction, delays, lack of velocity measurements, and other features that are not adequately described by the available model;
 - Anti-Windup Methods if integrators are to be used in the control law;
 - Adaptive Mechanisms for On-line Parameters Estimation;
 - Robustness ...
- Tune parameters of a controller with a chosen architecture via simulation and experimental study. Here, additional parameter identification is typically required.

7.2 Conceptual Control Loop for a Single Joint

Basic structure of a feedback control system, which is often appropriate for controlling robots, is illustrated below.

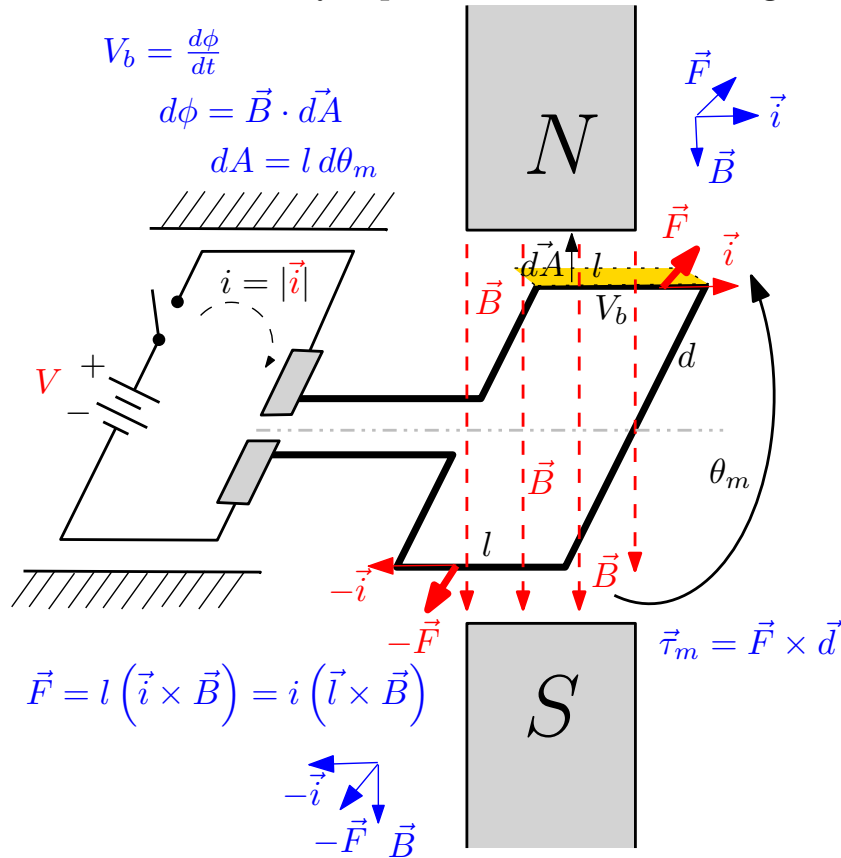


Here the chosen control law is denoted by the “Compensator” block, which, clearly, is driven by the tracking error signal of the single joint variable and produces an output driving the amplifier for the corresponding actuator. The influence of dynamics describing the other DOF is ignored. Such structure often fails in the case when precise or comparably fast motions are targeted. However, sometimes it is sufficient.

7.3 Actuator Dynamics

Let us now briefly discuss slightly more accurate way to improve our model taking into account dynamics of actuators than the modification suggested in the beginning of Chapter 6. More precisely, let us remind the basic principle of operation of a permanent magnet DC motor, assuming that our actuators are of this type.

The motor is schematically represented in the next figure.



Our motor creates a magnetic field $\vec{B}$ characterized by a magnetic flux $\vec{\phi}$ and a current $\vec{i}$ in a conductor. A straight current-carrying conductor of length l in the magnetic field experience a Lorentz force

$$\vec{F} = l (\vec{i} \times \vec{B})$$

and, due to an appropriate bearing, starts rotating producing a torque $\vec{\tau}_m$ applied to the mechanical frame.

The magnitude of the the torque is proportional to the magnitude of the created force. In particular, for a rectangle $l \times d$

$$\vec{\tau}_m = \vec{F} \times \vec{d} \quad \Rightarrow \quad \tau_m = |\vec{\tau}_m| = (l d |\vec{B}|) |\vec{i}| = K_m i$$

At the same time, whenever a conductor moves in a magnetic field $\vec{B}$, the

voltage $\mathbf{V}_b$ is induced according to the Faraday's law of induction:

$$\mathbf{V}_b = \frac{d\phi}{dt} = \vec{\mathbf{B}} \cdot \frac{d\vec{\mathbf{A}}}{dt} = \vec{\mathbf{B}} \cdot \left(\frac{d\vec{\theta}_m}{dt} \times \vec{l} \right) = K_b \frac{d\theta_m}{dt}$$

With $\vec{\mathbf{B}} = \text{const}$, a sufficiently large number of coil windings N etc., we can relate angular velocity of the rotor with induced voltage by a back emf constant $K_b = N l |\vec{\mathbf{B}}|$, which is inverse of the so-called speed constant.

Alternatively, the same relation can be derived from consideration of transformation of electrical power into the mechanical one: The external force causes motion so that mechanical work is done

$$W = \int_{\theta_m(t_1)}^{\theta_m(t_2)} \tau_m d\theta_m = \int_{t_1}^{t_2} \tau_m \frac{d\theta_m}{dt} dt = \sum_{\text{number of coils}} \int_{t_1}^{t_2} \mathbf{V}_b(t) i(t) dt = E$$

which must be equal to a loss of electrical energy in the circuit explaining the induced voltage $\mathbf{V}_b$ (back emf) opposite to the current flow, and, therefore $\tau_m \frac{d\theta_m}{dt} = (K_m i) \frac{d\theta_m}{dt} = (K_m \frac{d\theta_m}{dt}) i = N \mathbf{V}_b i$ leading to a similar relation. Of course, for other structures, the constants should be computed according to the particular geometry of the coils.

Now, assuming certain electrical inductance L and resistance R in the armature, we have the following relation between the armature current $i = |\vec{i}|$ and the input voltage V

$$L \left(\frac{di}{dt} \right) + R i = V - V_b$$

where, under assumption of constant flux $|\vec{\phi}| = \text{const}$,

$$V_b = K_b \frac{d\theta_m}{dt}$$

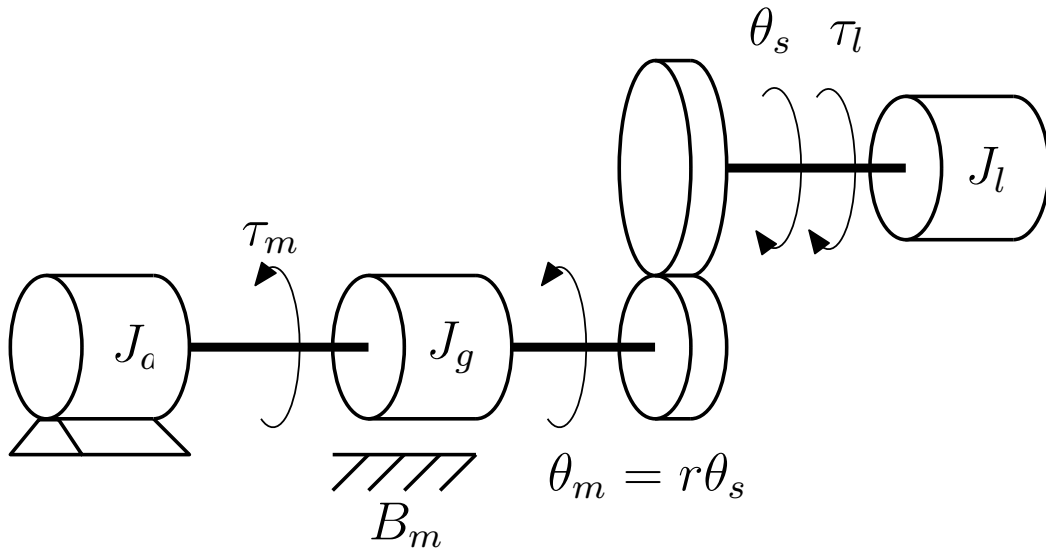
and the produced torque to the mechanical part is

$$\tau_m = K_m i$$

Let us use this expression to combine the obtained dynamical equation with a mechanical model, that is taken for simplicity as a single DOF system.

7.4 Dynamical Model for a Single Joint

Consider the next model of a single link with actuator/gear transmission.



In terms of the motor angle θ_m the equation of motion is

$$J_m \ddot{\theta}_m = \tau_m + F_m - B_m \dot{\theta}_m, \quad \tau_m = K_m i, \quad F_m = -\frac{1}{r} \tau_l,$$

where $J_m = J_a + J_g$ is the combined moment of inertia for the motor and the gear, B_m is the viscous friction gain, τ_l and F_m are the reaction forces applied to the link and to the rotor of the motor respectively (note that they have opposite signs and the ratio of their magnitudes is the gear ratio r).

Now, one should augment the mechanical model with the electrical one derived above

$$L \left(\frac{d}{dt} i \right) + R i = V - K_b \left(\frac{d}{dt} \theta_m \right),$$

where V is the control input to be assigned.

At the same time, we have

$$J_l \ddot{\theta}_s + B_s \dot{\theta}_s = \tau_l + F_s,$$

where F_s is typically highly nonlinear and can represent the sum of various forces acting on the link, such as gravity or interaction forces with other links of the manipulator. This force can, in general, depend not only on θ_s but also on the other generalized coordinates as well as on their first and second derivatives.

To proceed with feedback control design one can:

- Assume that the gear box is flexible and so $\theta_m \neq r \theta_s$. One of the commonly used assumption is that τ_l is proportional to $(\theta_m - r \theta_s)$. We will consider this situation at the end of this chapter as well as in Chapter 8, where we will derive the so-called dynamical model with flexible joints and use the ideas of geometric nonlinear feedback control theory for design.

- Assume that $L \ll R$ so that $L/R \approx 0$, so that

$$0 \left(\frac{d}{dt} i \right) + i = \frac{1}{R} \mathbf{V} - \frac{1}{R} K_b \left(\frac{d}{dt} \theta_m \right) \quad \Rightarrow \quad \tau_m = \mathbf{u} - B_e \left(\frac{d}{dt} \theta_m \right),$$

where $\mathbf{u} = \frac{K_m}{R} \mathbf{V}$ and $B_e = \frac{K_m}{R} K_b$. This is, in essence, the assumption made during the derivation of the dynamic model in Chapter 6.

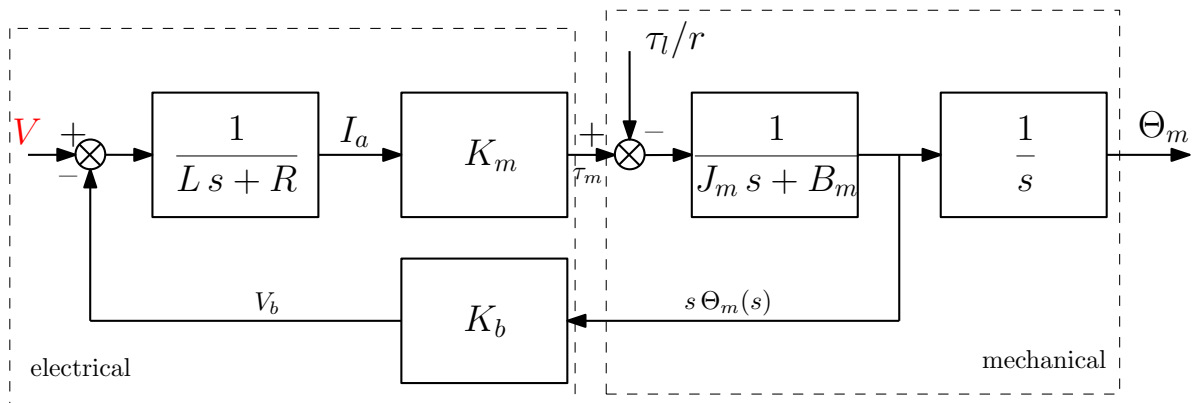
- Assume that the transmission through the gear box is perfectly rigid and there are no additional DOFs due to mechanical part of the motor-gear-transmission system, so that $\theta_m \equiv r \theta_s$. Furthermore, one can
 - Assume that the motions are relatively slow and the gear ratio r is relatively high, and $\frac{1}{r} \mathbf{F}_s$ is relatively small with respect to the other terms and can be treated as a small bounded disturbance. The equation should be modified excluding τ_l and θ_s and arriving at a similar model with $\mathbf{F}_s$ in place of τ_l .
 - Assume that the motions are relatively slow and the gear ratio r is relatively high, and $\frac{1}{r} \tau_l$ is relatively small with respect to the other terms and can be treated as a small bounded disturbance. We will proceed in this way.

So, for now, suppose that a reasonable combined models is the following

$$\begin{aligned} L \left(\frac{d}{dt} i \right) + R i &= \mathbf{V} - K_b \left(\frac{d}{dt} \theta_m \right) \\ J_m \left(\frac{d^2}{dt^2} \theta_m \right) + B_m \left(\frac{d}{dt} \theta_m \right) &= K_m i - \frac{1}{r} \tau_l \end{aligned}$$

with τ_l/r being a relatively small signal that can be treated as a small bounded disturbance.

The corresponding system is illustrated in the next figure.



Note that the system is linear! So, we can use classical linear feedback control design methods for it!

The transfer function from the input $\mathbf{V}(s)$ to the output $\Theta_m(s)$ is

$$G_{\{v \rightarrow \theta\}}(s) = \frac{\Theta_m(s)}{\mathbf{V}(s)} = \frac{K_m}{[(Ls + R)(J_ms + B_m) + K_m K_b] s}$$

while the transfer function from the disturbance $\tau_l(s)$ to the output $\Theta_m(s)$ is

$$G_{\{\tau_l \rightarrow \theta\}}(s) = \frac{\Theta_m(s)}{\tau_l(s)} = \frac{1}{r} \frac{-(Ls + R)}{[(Ls + R)(J_ms + B_m) + K_m K_b] s}$$

These transfer functions should be used to achieve certain performance specifications in the closed-loop system as well as a certain level of disturbance attenuation.

Note that in the case when $L \ll R$ so that $L/R \approx 0$ one often can base the design on the reduced transfer functions

$$G_{\{v \rightarrow \theta\}}(s) \approx \frac{K_m}{[R(J_ms + B_m) + K_m K_b] s}$$

and

$$G_{\{\tau_l \rightarrow \theta\}}(s) \approx -\frac{1}{r [(J_ms + B_m) + K_m K_b] s}$$

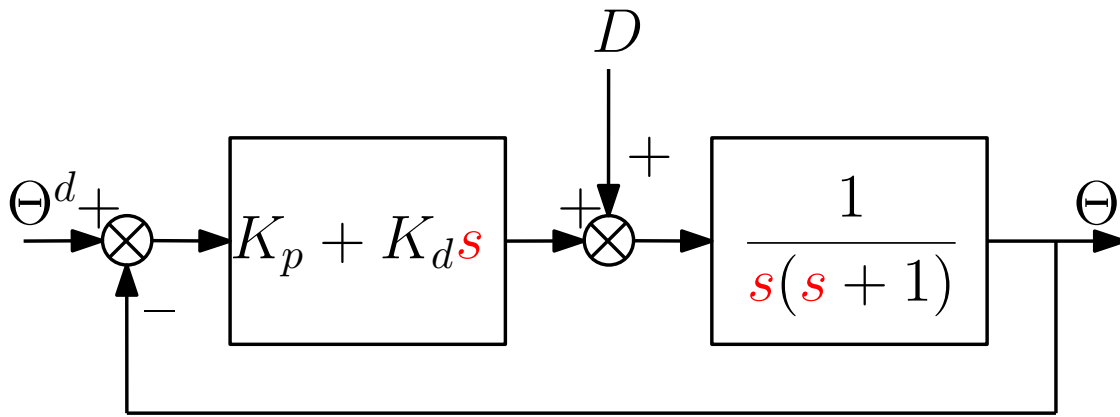
However, the correctness of reduction of the model (i.e. how small L/R should be) should be justified via e.g. **balanced model reduction** method.

7.5 PD and PID Tuning Based on a Simplified Linear Model

Suppose the gear ratio r is so large that $G_{\{\tau_l \rightarrow \theta\}} \approx 0$, $L \ll R$, and the physical parameters are such that

$$G_{\{v \rightarrow \theta\}}(s) \approx \frac{K_m}{[R(J_m \mathbf{s} + B_m) + K_m K_b] \mathbf{s}} = \frac{1}{(\mathbf{s} + 1) \mathbf{s}}$$

Let us design a PD-controller, illustrated in the next diagram



such that the closed-loop poles are located at -3 and -1 .

The transfer functions for the plant and for the controller are

$$G_{\{v \rightarrow \theta\}}(s) = \frac{B(s)}{A(s)} = \frac{1}{(s+1)s}, \quad H(s) = \frac{P(s)}{L(s)} = \frac{K_p + K_d s}{1}$$

and so the pole placement is equivalent to solving the so-called Diophantine equation

$$\begin{aligned} A(s) \cdot L(s) + B(s) \cdot P(s) &= A_{clp}(s) \\ (s+1)s \cdot 1 + 1 \cdot (K_p + K_d s) &= (s+3)(s+1) \\ s^2 + (K_d + 1)s + K_p &= s^2 + 4s + 3 \end{aligned}$$

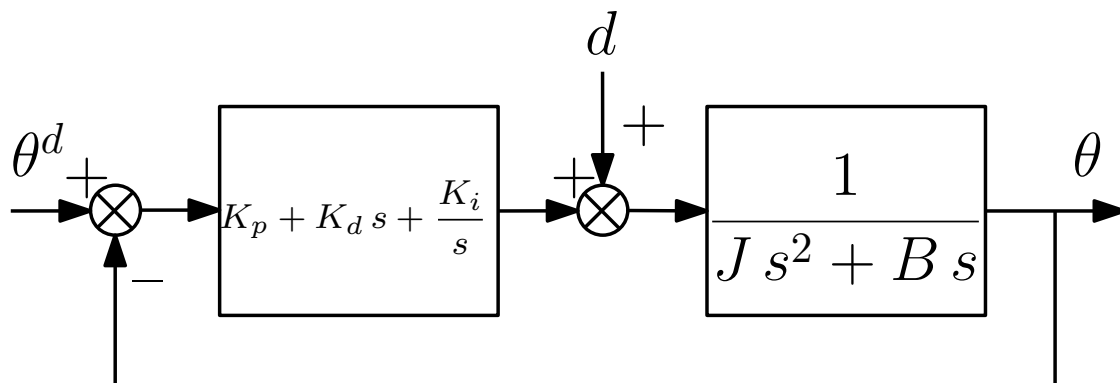
Hence, we should take

$$K_d = 3, \quad K_p = 3$$

The reader should be able to easily solve the following exercise. Suppose

$$G_{\{V \rightarrow \theta\}}(s) \approx \frac{K_m}{[R(J_m s + B_m) + K_m K_b] s} = \frac{1}{J s^2 + B s},$$

find a PID-controller gains such that that closed-loop system illustrated below



is stable.

However, in practice, stability is not sufficient and performance of the

closed-loop system should be reasonable in certain sense. There are several practical techniques for tuning the gains of a PID controller, such as the classical Ziegler-Nichols method.

Often, the gains of a PID-controller are renamed as follows

$$u = P + I + D, \quad P = K e(t), \quad I = \frac{K}{T_i} \int_0^\infty e(\tau) d\tau, \quad D \approx K T_d \dot{e}(t),$$

where $e = \theta_d - \theta$ is the tracking error, K is the proportional gain, $T_i > 0$ and $T_d \geq 0$ are the time constants characterizing the integral and differential parts, while D can be computed using an approximation for a differentiating filter or a velocity observer.

The most common sampled discrete-time realization of such a regulator assuming a zero-order-hold (ZOH) model of the digital-to-analog signal converter with the step $h > 0$ is

$$u(t) = u[k], \quad kh \leq t < (k+1)h, \quad k = 0, 1, 2, \dots$$

where the digital feedback signal on the sampled measurement

$$e[k] = \theta_d(kh) - \theta(kh)$$

is computed as

$$u[k] = P[k] + I[k] + D[k] \quad \text{with} \quad P[k] = K e[k]$$

and using approximations by the forward Euler difference for the integral feedback

$$I[k+1] = I[k] + \frac{K h}{T_i} e[k], \quad I[0] = 0, \quad k = 1, 2, \dots$$

and by the backward Euler difference for the derivative feedback based on the first order differential filter $\frac{T_d}{N} \dot{D} + D = -K T_d \dot{e}$ with $N \gg 1$ (usually, $10 \leq N \leq 100$) realized by

$$D[k] = \frac{T_d}{T_d + N h} \left(D[k-1] - K N (e[k] - e[k-1]) \right), \quad D[0] = 0$$

for $k = 1, 2, \dots$

Tuning of the gains can be done using Ziegler-Nichols method based either on recorded step response or on periodic response to critical pure proportional feedback.

In the first approach, one record the output in response to $u(t) = 0$ for $t < 0$ and $u(t) = U = \text{const}$ for $t \geq 0$, where $U > 0$ should be chosen

to obtain a monotonous transition from the initial value to another constant steady-state. After that, one finds on the graph of the response the point with the steepest slope and computes this slope R as well as the value L of the interception of the corresponding tangent line with the time axis.

After that one takes either

$$K = \frac{6}{5 R L}, \quad T_i = 2 L, \quad T_d = \frac{L}{2}$$

or

$$K = \frac{9}{10 R L}, \quad T_i = 3 L, \quad T_d = 0.$$

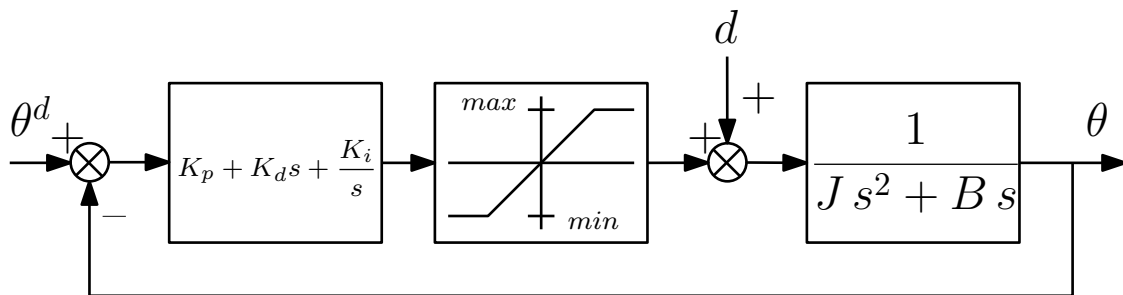
For the other approach one needs to perform a series of experiment with proportional feedback $u(t) = K_{test} (\theta_d(t) - \theta(t))$ under slowly changing reference signal $\theta_d(t)$. The coefficient K_{test} should be gradually increased until further increase would lead to instability. The corresponding extremal (critical) value of the proportional gain K_u should be recorded together with an estimate of the period of achieved oscillations T_u right before instability. Based on these values one takes either

$$K = \frac{3 K_u}{5}, \quad T_i = \frac{T_u}{2}, \quad T_d = \frac{T_u}{8}$$

or

$$K = \frac{9 K_u}{20}, \quad T_i = \frac{5 T_u}{6}, \quad T_d = 0.$$

It should be noticed that, often, one have also to address the problem of input saturation. The closed-loop system can be drastically different if we take into account limits on the signals in the loop:

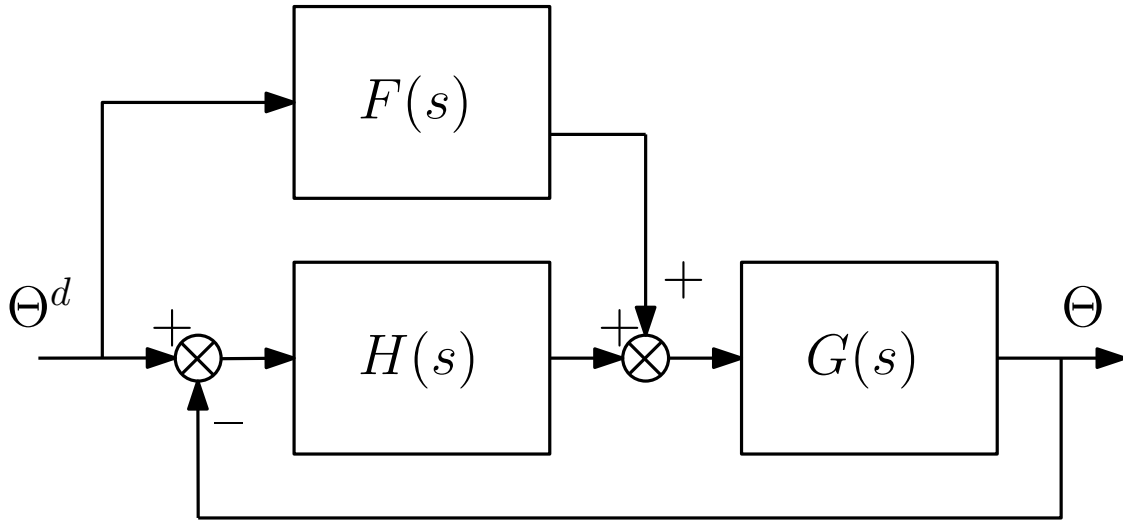


Here, various anti-windup scheme may be useful but in any case, performance of the closed-loop system should be verified taking such limitations into account.

Let us now briefly remind another useful technique from the standard linear control theory.

7.6 Simplified Feedforward Control Design

The following structure of the controller is often useful



Here,

- $\mathbf{G}(s)$ represents a simplified model of the control system for a particular link,
- $\mathbf{H}(s)$ could be a PID-like controller that is tuned as above,
- while the transfer function $\mathbf{F}(s)$ is so-called feedforward term such that
 - $\mathbf{F}(s)$ is stable;
 - $\mathbf{F}(s)$ is proper;

A reasonable choice is $\mathbf{F}(s) \approx 1/\mathbf{G}(s)$, for a particular range of frequencies, provided the plant model is minimum-phase, i.e. there are no unstable zeros as in the case of the transfer functions considered above.

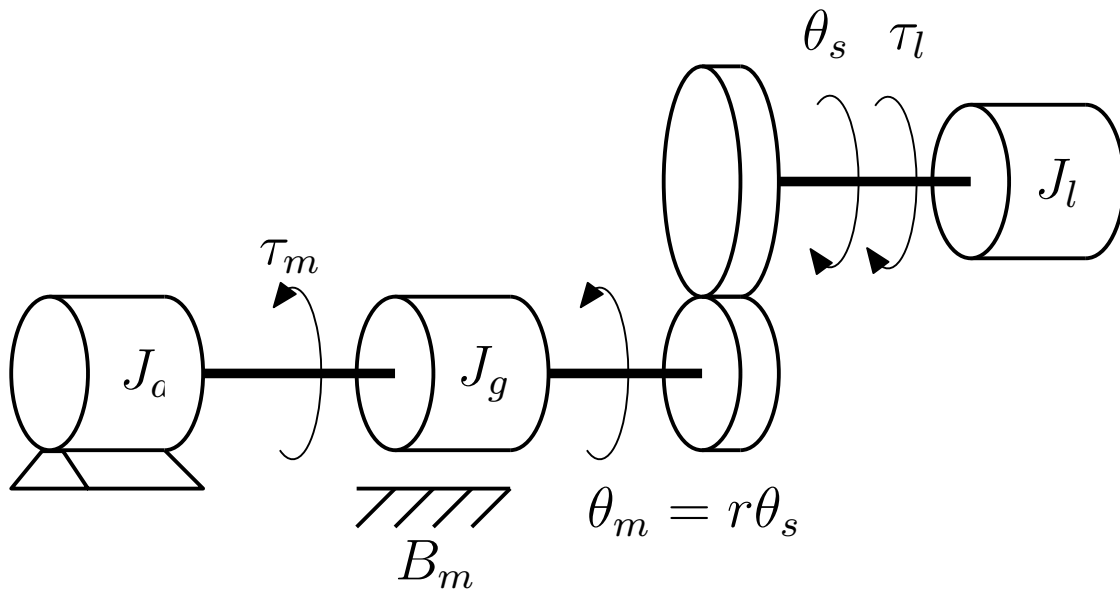
In the considered example with $\mathbf{G}(s) = \frac{1}{Js^2 + Bs}$, one can implement the feedforward term $\mathbf{F}(s) \Theta^d(s)$ as $\mathbf{J} \ddot{\Theta}_d(t) + \mathbf{B} \dot{\Theta}_d(t)$ avoiding approximate differentiation but using generation of not only the reference signal but its two required derivatives.

It is well-known that introduction of a feedforward term can significantly improve performance.

Let us now discuss another issue that may be important to take into account at the control design stage.

7.7 Dealing with Flexible Transmission

Let us re-consider our motor-transmission-gearbox system



but drop the often unrealistic rigidity assumption, so that $\theta_m \neq r\theta_s$. Note that

- Gear-box always introduces additional friction and delay;
- Gear-box can complicate the dynamics and limit the performance.

Suppose now that we can ignore the electrical dynamics, i.e. that $\mathbf{L} \ll \mathbf{R}$ so that $\mathbf{L}/\mathbf{R} \approx \mathbf{0}$ and

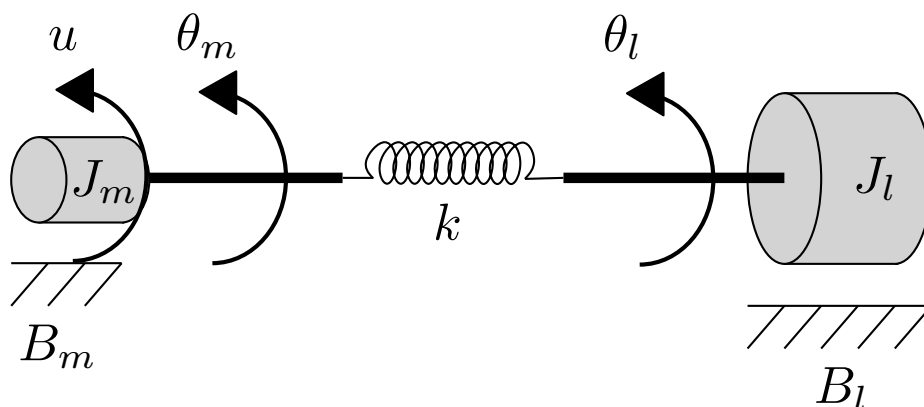
$$\tau_m = \mathbf{u} - B_e \left(\frac{d}{dt} \theta_m \right),$$

as above.

Let us also assume that the transmission, such as a harmonic drive, can be modeled as a torsional spring, so that

$$\tau_l = k \left(\frac{\theta_m}{r} - \theta_s \right)$$

with some coefficient $k > 0$, as illustrated on the next figure.



The dynamics of the system takes the following form

$$\begin{aligned} J_l \ddot{\theta}_s + B_l \dot{\theta}_s + k \left(\theta_s - \frac{\theta_m}{r} \right) &= F_s \\ J_m \ddot{\theta}_m + B_m \dot{\theta}_m - \frac{k}{r} \left(\theta_s - \frac{\theta_m}{r} \right) &= \mathbf{u} \end{aligned}$$

where $\mathbf{u}$ is a control torque.

Now, suppose $F_s \approx 0$ and rename the variables as

$$\theta_l \leftarrow \theta_s, \quad \theta_m \leftarrow \theta_m/r, \quad \mathbf{u} \leftarrow r^2 \mathbf{u}$$

and the constants as

$$J_m \leftarrow r^2 J_m, \quad B_m \leftarrow r^2 B_m,$$

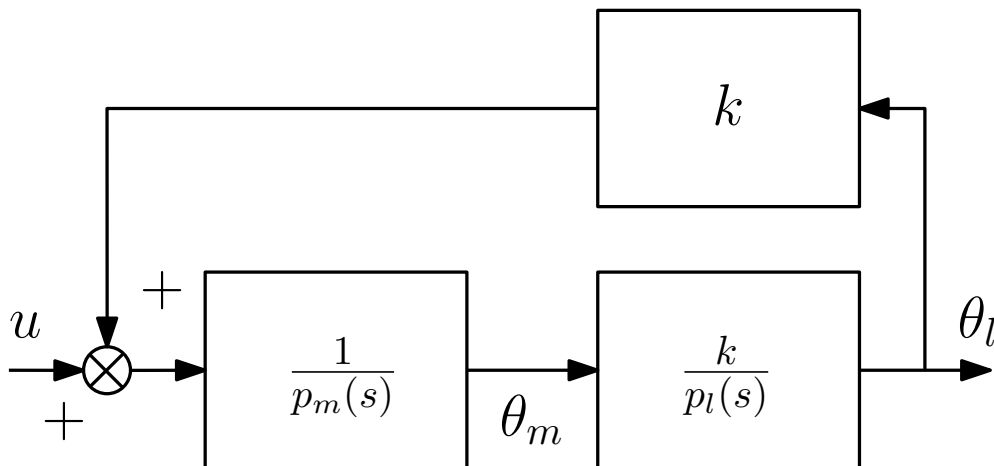
so that the motor related quantities are reflected through the reduction ratios. Then, we can rewrite dynamics as follows

$$\begin{aligned} J_l \ddot{\theta}_l + B_l \dot{\theta}_l + k\theta_l &= k\theta_m \\ J_m \ddot{\theta}_m + B_m \dot{\theta}_m + k\theta_m &= k\theta_l + \mathbf{u} \end{aligned}$$

and compute the transfer functions relating the two variables and the control input:

$$\begin{aligned} \Theta_l(s) &= \frac{k}{p_l(s)} \Theta_m(s) = \frac{k}{J_l s^2 + B_l s + k} \Theta_m(s) \\ \Theta_m(s) &= \frac{1}{p_m(s)} (k\Theta_l(s) + \mathbf{U}(s)) = \frac{k\Theta_l(s) + \mathbf{U}(s)}{J_m s^2 + B_m s + k} \end{aligned}$$

The result is illustrated on the diagram below



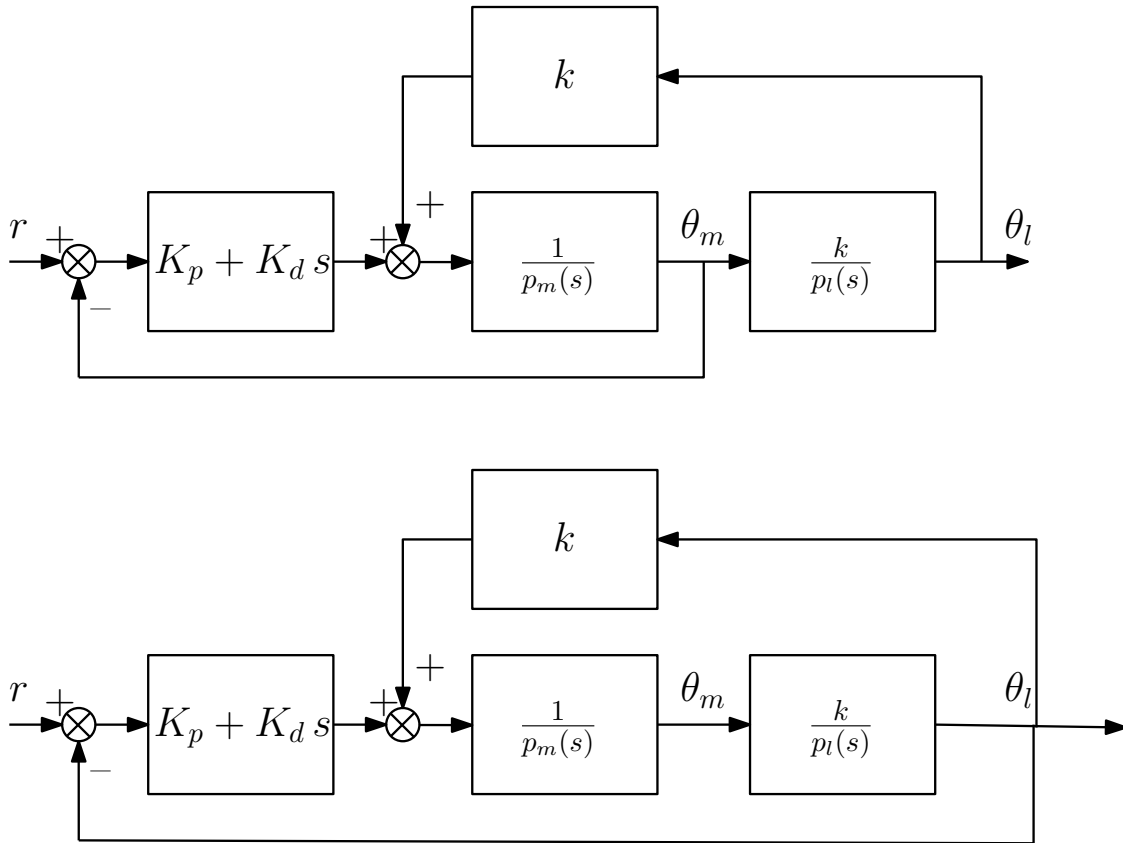
Combining the two expressions we obtain the transfer function from the

input to the link position

$$\begin{aligned}
 \Theta_l(s) &= \frac{k}{p_l(s)p_m(s) - k^2} U(s) \\
 &= \frac{k}{J_m J_l s^4 + (J_l B_m + J_m B_l) s^3 + (k(J_m + J_l) + B_m B_l) s^2 + k(B_m + B_l) s} U(s) \\
 &\approx \frac{k}{J_m J_l s^4 + k(J_m + J_l) s^2} U(s)
 \end{aligned}$$

where the last approximation is valid when the friction coefficients are small.

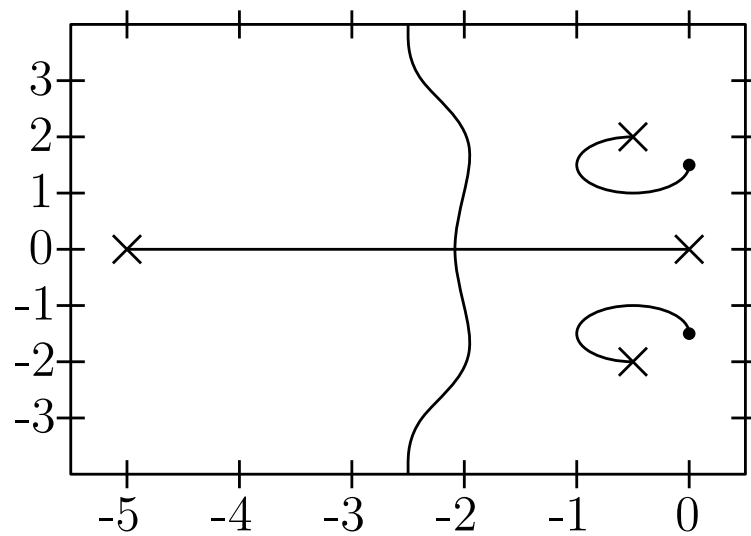
Obviously, the designer might be able to implement PD controller with respect to either θ_m or θ_l . The two possibilities are illustrated below



Which design is better? One of the differences can be observed from root-locus analysis. The root-locus of the closed-loop system with θ_m -feedback

$$U = -(K_P + K_D s) \Theta_m = K(a + s) \Theta_m$$

is shown below

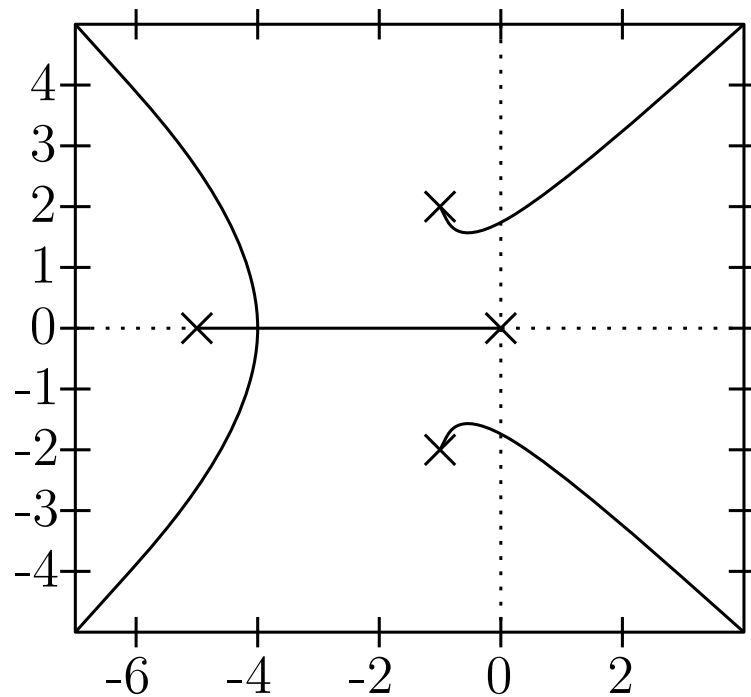


Note that stability is guaranteed for all (positive) values of the gains!

The root-locus of the closed-loop system with θ_l -feedback

$$U = -(K_P + K_D s) \Theta_l = K(a + s) \Theta_l$$

is shown below



Note that stability is lost when the gains are large. In fact, it can be shown that the system becomes unstable when the proportional gain reaches the value of the coefficient of elasticity.

Chapter 8

Geometric Nonlinear Control

Here we would like to review a particular geometry-based technique for designing feedback control laws for highly nonlinear systems that cannot be linearized by trivial direct cancellation of nonlinearities.

This technique is useful for model-based robot control in case when the rigid model, derived in Chapter 5, does not describe dynamics adequately in relation to a particular class of target motions. In particular, this is the case when the number of DOF that must be taken into account is less than the number of independent control inputs. Such systems are called **underactuated** and cover the two examples that we have briefly mentioned above: the flexible joint case and the case when dynamics of the actuators can not be approximated by static relations.

8.1 Introduction to Differential Geometry

We start with an introduction of a few useful mathematical concepts.

8.1.1 Smooth Embedded Manifold in $\mathbb{R}^n$

From an intuitive point of view, a manifold is something that is locally equivalent to a usual finite-dimensional Euclidean space but may have a more complex global structure. There exists a rigorous mathematical definition involving such notions as topological distances, maps, coordinates, atlases, etc., which are inspired by topography. In what follows, we will give a somewhat simplified definition of what is typically called a smooth manifold embedded into an Euclidean space. This will be sufficient for our goals, while an interested reader may consult a standard textbook on differential geometry for a rigorous overview of the subject.

A smooth (nontrivial) manifold $\mathcal{M}$ of dimension m , with $m \geq 1$,

is a subset of points in $\mathbb{R}^n$, with $n \geq 2$, defined as the zero set of a family of $p = n - m \geq 1$ smooth scalar functions

$$\begin{aligned} h_1(x_1, x_2, \dots, x_n) &= 0 \\ &\vdots \\ h_p(x_1, x_2, \dots, x_n) &= 0 \end{aligned}$$

such that the differentials

$$dh_1(\cdot), \quad dh_2(\cdot), \quad \dots, \quad dh_p(\cdot)$$

are linearly independent for any point of $\mathcal{M}$.

At each point $x \in \mathcal{M}$ we can attach a **tangent space $T_x\mathcal{M}$** , which is an m -dimensional space that specifies the set of possible directions of velocities at this point implying not leaving $\mathcal{M}$. It can be associated with $\mathbb{R}^m$.

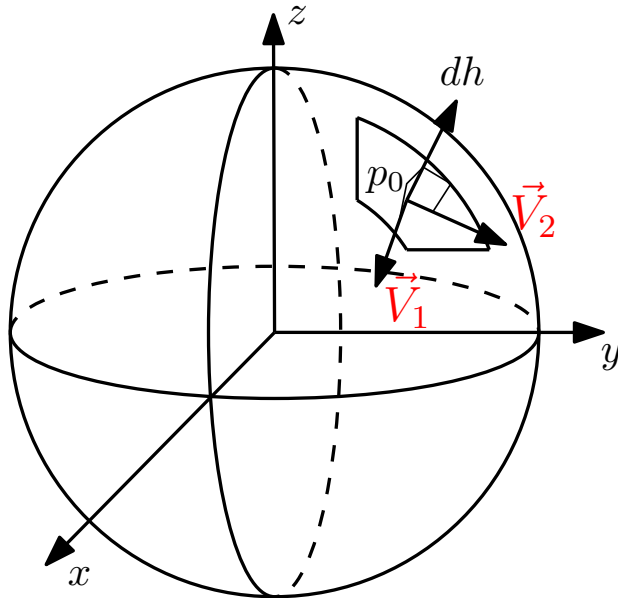
Let us illustrate these notions with an example.

Example: a sphere as a manifold

Consider a unit sphere $\mathcal{S}^2$ in $\mathbb{R}^3$ defined by a single relation

$$h(x, y, z) = x^2 + y^2 + z^2 - 1 = 0$$

on the coordinates in a certain fixed frame and represented on the next figure together with a tangent space at one point.



Since $n = 3$ and there is a single relation, i.e. $p = 1$, the dimension of this manifold is $m = 2 = 3 - 1$.

The differential $dh(\cdot)$ of the function

$$h(x, y, z) = x^2 + y^2 + z^2 - 1 = 0$$

at a point $p_0 = (x_0, y_0, z_0) \in S^2$ is the row vector

$$\begin{aligned} dh(x_0, y_0, z_0) &= \left[\frac{\partial}{\partial x} h, \frac{\partial}{\partial y} h, \frac{\partial}{\partial z} h \right] \Big|_{x=x_0, y=y_0, z=z_0} \\ &= [2x_0, 2y_0, 2z_0], \end{aligned}$$

where, obviously, $x_0^2 + y_0^2 + z_0^2 = 1$, defines the direction orthogonal to the surface of the sphere at the particular point.

How to compute a tangent space $T_{p_0}S^2$

to the sphere S^2 at $p_0 = (x_0, y_0, z_0) \in S^2$?

By definition

$$\begin{aligned} T_{p_0}S^2 &= \{V = [v_1, v_2, v_3]^T : dh(p_0) \perp V\} \\ &= \left\{ V = [v_1, v_2, v_3]^T : 2x_0 v_1 + 2y_0 v_2 + 2z_0 v_3 = 0 \right\} \end{aligned}$$

Suppose for the given $p_0 \in S^2$ we have $z_0 \neq 0$, than we can suggest the following two representative solutions for $V \in T_{p_0}S^2$:

$$\mathbf{V}_1 = [1, 0, -x_0/z_0], \quad \mathbf{V}_2 = [0, 1, -y_0/z_0]$$

that form a basis, i.e.

$$T_{p_0}S^2 = \text{span}\{\mathbf{V}_1, \mathbf{V}_2\} = \{V = \alpha \mathbf{V}_1 + \beta \mathbf{V}_2 : \alpha, \beta \in \mathbb{R}\},$$

where here and below $\text{span}\{\cdot\}$ is defined as the set of all possible linear combinations.

8.1.2 Smooth Vector and Co-vector Fields

The following two notions are useful as well.

- **A smooth vector field** on a manifold $\mathcal{M}$ is a smooth function

$$x \in \mathcal{M} \rightarrow f(x) = \begin{bmatrix} f_1(x) \\ \vdots \\ f_m(x) \end{bmatrix} \in T_x \mathcal{M}$$

- **A smooth co-vector field** on a manifold $\mathcal{M}$ is a smooth function

$$x \in \mathcal{M} \rightarrow w(x) = [w_1(x), \dots, w_m(x)] \in T_x^* \mathcal{M}$$

Here $T_x^* \mathcal{M}$ is co-tangent space to the manifold $\mathcal{M}$ at its point x .

Every fixed co-vector field $w(x)$ can be associated with a linear operator (functional) that associates with every arbitrary vector field $f(x)$ a real number:

$$\mathcal{A}_{w(x)} : T_x \mathcal{M} \ni f(x) \mapsto \mathcal{A}_{w(x)} f(x) \in \mathbb{R}$$

for every point in $\mathcal{M}$, computed as the usual product of the row, representing $w(x)$, and the column, representing $f(x)$:

$$\begin{aligned} \mathcal{A}_{w(x)} f(x) &= [w_1(x), \dots, w_m(x)] \begin{bmatrix} f_1(x) \\ \vdots \\ f_m(x) \end{bmatrix} \\ &= w_1(x) f_1(x) + \dots + w_m(x) f_m(x) \end{aligned}$$

8.1.3 Distributions and Co-distributions

Vector fields and co-vector fields can be grouped.

- Let $X_1(x), \dots, X_k(x)$ be vector fields on $\mathcal{M}$ that are linearly independent for each $x \in \mathcal{M}$. Then, a **distribution** $\Delta(\cdot)$ is a family of linear subspaces of $T\mathcal{M}$ defined for each point x as

$$\Delta(x) = \text{span}\{X_1(x), \dots, X_k(x)\}$$

- Let $W_1(x), \dots, W_k(x)$ be co-vector fields on $\mathcal{M}$ linearly independent for each $x \in \mathcal{M}$. Then a **co-distribution** $\Omega(\cdot)$ is a family of linear subspaces of $T^*\mathcal{M}$ defined for each point x as

$$\Omega(x) = \text{span}\{W_1(x), \dots, W_k(x)\}$$

A vector field $f(x)$ can be associated with a differential equation

$$\frac{d}{dt}x = f(x)$$

since by the very definition, whenever $x \in \mathcal{M}$, we have $\frac{d}{dt}x \in T_x \mathcal{M}$.

More importantly, a vector field $f(x)$, called the drift term, together with a distribution $\Omega(x) = \text{span}\{g_1(x), \dots, g_m(x)\}$ can be associated with a control system defined by an infinite dimensional family of differential equations

$$\frac{d}{dt}x = f(x) + g_1(x) u_1 + \dots + g_m(x) u_m,$$

where $u_1, \dots, u_m$ are control inputs.

We are going to see that some properties of solutions for a control system, defined in this way, can be deduced from analysis of certain properties of the corresponding distribution $\Omega(\mathbf{x})$ and the vector field $\mathbf{f}(\mathbf{x})$, which can be verified by some computation procedures related to the formal operations introduced next.

8.1.4 Lie Bracket of Vector Fields

Given two vector fields $\mathbf{f}(\cdot)$ and $\mathbf{g}(\cdot)$ on $\mathcal{M} = \mathbb{R}^n$, the **Lie bracket** of $\mathbf{f}$ and $\mathbf{g}$ is the vector field in $\mathbb{R}^n$ denoted by $[\mathbf{f}, \mathbf{g}]$ and defined for each $\mathbf{x} \in \mathcal{M} = \mathbb{R}^n$ by

$$[\mathbf{f}, \mathbf{g}](\mathbf{x}) := \left[\frac{\partial}{\partial \mathbf{x}} \mathbf{g}(\mathbf{x}) \right] \mathbf{f}(\mathbf{x}) - \left[\frac{\partial}{\partial \mathbf{x}} \mathbf{f}(\mathbf{x}) \right] \mathbf{g}(\mathbf{x})$$

For example, suppose $\mathbf{f} = \mathbf{A}\mathbf{x}$ and $\mathbf{g} = \mathbf{B}\mathbf{x}$, then

$$\begin{aligned} [\mathbf{f}, \mathbf{g}](\mathbf{x}) &= \left[\frac{\partial}{\partial \mathbf{x}} \mathbf{g}(\mathbf{x}) \right] \mathbf{f}(\mathbf{x}) - \left[\frac{\partial}{\partial \mathbf{x}} \mathbf{f}(\mathbf{x}) \right] \mathbf{g}(\mathbf{x}) \\ &= \mathbf{B} \mathbf{A} \mathbf{x} - \mathbf{A} \mathbf{B} \mathbf{x} = \underbrace{(\mathbf{B} \mathbf{A} - \mathbf{A} \mathbf{B})}_{=\mathbf{C}} \mathbf{x} \end{aligned}$$

We will also use **repeated Lie brackets** defined recursively as follows

$$\begin{aligned} ad_f^0(\mathbf{g}) &= \mathbf{g} \\ ad_f^1(\mathbf{g}) &= [\mathbf{f}, \mathbf{g}] \\ ad_f^2(\mathbf{g}) &= [\mathbf{f}, [\mathbf{f}, \mathbf{g}]] \\ &\vdots \\ ad_f^k(\mathbf{g}) &= [\mathbf{f}, ad_f^{(k-1)}(\mathbf{g})] \end{aligned}$$

For example, suppose for given $\mathbf{A} \in \mathbb{R}^{n \times n}$ and $\mathbf{b} \in \mathbb{R}^n$ we define $\mathbf{f} = \mathbf{A}\mathbf{x}$, $\mathbf{g} = \mathbf{b}$, and a control system $\frac{d}{dt}\mathbf{x} = \mathbf{A}\mathbf{x} + \mathbf{b}u$. Then, $ad_f^0(\mathbf{g}) = \mathbf{b}$, $ad_f^1(\mathbf{g}) = -\mathbf{A}\mathbf{b}$, $ad_f^2(\mathbf{g}) = \mathbf{A}^2\mathbf{b}, \dots, ad_f^n(\mathbf{g}) = (-1)^n \mathbf{A}^n \mathbf{b}$, and the rank of the matrix formed by them is the controllability degree.

8.1.5 Lie Derivative

Given

- a scalar function $h(\cdot)$ on $\mathbb{R}^n$, i.e. $h : \mathbb{R}^n \rightarrow \mathbb{R}$ and
- a vector field $\mathbf{f}(\cdot)$ on $\mathbb{R}^n$, i.e. $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^n$,

the **Lie derivative** of $h(\cdot)$ with respect to $f(\cdot)$ is the scalar function denoted as $\mathcal{L}_f h$ and defined by

$$\mathcal{L}_f h = \left[\frac{\partial}{\partial x} h(x) \right] f(x) = \left[\frac{\partial}{\partial x_1} h(x) \right] f_1(x) + \cdots + \left[\frac{\partial}{\partial x_n} h(x) \right] f_n(x)$$

Lie derivative $\mathcal{L}_f h$, called also directional derivative, defines the rate of change of a scalar function h of a vector argument x along solutions of the differential equation associated with the vector field $f(x)$. This is exactly derivative that we compute differentiating a Lyapunov function.

We will also use **repeated Lie derivatives** defined recursively as follows

$$\mathcal{L}_f^k h = \mathcal{L}_f \left[\mathcal{L}_f^{(k-1)} h \right], \quad k = 1, 2, \dots, \quad \mathcal{L}_f^0 h = h$$

The following technical result can be proved via direct computation.

Lemma: Let $h : \mathbb{R}^n \rightarrow \mathbb{R}$ be a scalar function and f, g be vector fields on $\mathbb{R}^n$. Then the following identity holds

$$\mathcal{L}_{[f,g]} h \equiv \mathcal{L}_f [\mathcal{L}_g h] - \mathcal{L}_g [\mathcal{L}_f h]$$

This will be used later.

Equipped with some basic notions from differential geometry, we can approach some problems of finding solutions of differential equations. This will be used later to define a systematic procedure of feedback control design.

8.2 Frobenius Theorem

Here we will introduce a very useful result known in differential geometry as Frobenius Theorem.

8.2.1 Manifolds as Solutions of PDEs

Consider the following system of two partial differential equations (PDEs)

$$\frac{\partial z}{\partial x} = f(x, y, z), \quad \frac{\partial z}{\partial y} = g(x, y, z)$$

Suppose, we are looking for a solution of this system of equations, i.e. we would like to express z in the form

$$z = \phi(x, y)$$

such that

$$\frac{\partial}{\partial x} \phi(x, y) = f(x, y, \phi(x, y)), \quad \frac{\partial}{\partial y} \phi(x, y) = g(x, y, \phi(x, y))$$

If we find a solution, then it obviously corresponds to a surface in $\mathbb{R}^3$

$$\mathcal{M} = \{(x, y, z) \in \mathbb{R}^3 : z - \phi(x, y) = 0\}$$

in other words, to a 2-dimensional manifold.

How to compute a tangent plane to this surface (manifold)?

8.2.2 Tangent Space to a Solution of a PDE

The differential $dh(\cdot)$ of the function

$$h(x, y, z) = z - \phi(x, y) = 0$$

at the point $p_0 = (x_0, y_0, z_0) \in \mathbb{R}^3$ is the row vector

$$\begin{aligned} dh(p_0) &= \left[\frac{\partial}{\partial x} h, \frac{\partial}{\partial y} h, \frac{\partial}{\partial z} h \right] \Big|_{x=x_0, y=y_0, z=z_0} \\ &= \left[-\frac{\partial}{\partial x} \phi(x, y), -\frac{\partial}{\partial y} \phi(x, y), 1 \right] \Big|_{x=x_0, y=y_0, z=z_0} \\ &= [-f(x_0, y_0, \phi(x_0, y_0)), -g(x_0, y_0, \phi(x_0, y_0)), 1] \end{aligned}$$

Consequently, the tangent space to the manifold $\mathcal{M}$ of the solution of the system of PDEs defined above can be computed as follows

$$\begin{aligned} T_{p_0} \mathcal{M} &= \{V = [v_1, v_2, v_3]^T : dh(p_0) \perp V\} \\ &= \{V : -f(\cdot)v_1 - g(\cdot)v_2 + 1 \cdot v_3 = 0\} \end{aligned}$$

A basis for $T_{p_0} \mathcal{M}$ can be taken as

$$V_1 = [1, 0, f(\cdot)]^T \quad \text{and} \quad V_2 = [0, 1, g(\cdot)]^T$$

so that

$$T_{p_0} \mathcal{M} = \text{span}\{V_1, V_2\}$$

We see now that the functions $f(\cdot)$ and $g(\cdot)$ appearing in the right-hand sides of the system of PDEs unambiguously and directly define the tangent space to the surface representing a solution. In other words, they define the solution geometrically.

8.2.3 Integral Manifolds

The following conclusion can be drawn:

A search for a

solution $z = \phi(x, y)$ of a system of two partial differential equations

$$\frac{\partial}{\partial x} z = f(x, y, z), \quad \frac{\partial}{\partial y} z = g(x, y, z)$$

is equivalent to a search for a

surface $\mathcal{M}$ in $\mathbb{R}^3$ whose tangent space $T_p \mathcal{M}$ is

spanned by $V_1 = [1, 0, f(x, y, z)]^T$ and $V_2 = [0, 1, g(x, y, z)]^T$

If such surface $\mathcal{M}$ can be found then it is called an **integral manifold** of the system while the equations are called **integrable**.

8.2.4 Lie Derivative Representation for PDEs

The problem of finding an integral manifold for a system of 2 PDEs can be rewritten as a problem of finding a function, two Lie derivatives of which vanish simultaneously. These derivatives should be taken along the two vectors computed above.

In other words, the problem of finding a solution $z = \phi(x, y)$ of equations

$$\frac{\partial}{\partial x} z = f(x, y, z), \quad \frac{\partial}{\partial y} z = g(x, y, z)$$

can be rewritten as the problem of finding a solution $h = h(x, y, z)$, in the particular form $h(x, y, z) = z - \phi(x, y)$, of the following two equations

$$\mathcal{L}_{V_1} h = 0, \quad \mathcal{L}_{V_2} h = 0$$

where

$$V_1 = [1, 0, f(x, y, z)]^T \quad \text{and} \quad V_2 = [0, 1, g(x, y, z)]^T$$

Indeed, by definition, we have

$$\begin{aligned} \mathcal{L}_{V_1} h &= \left[\frac{\partial}{\partial x} h(x, y, z) \right] \cdot 1 + \left[\frac{\partial}{\partial y} h(x, y, z) \right] \cdot 0 + \left[\frac{\partial}{\partial z} h(x, y, z) \right] f(x, y, z) \\ &= \frac{\partial}{\partial x} h(x, y, z) + \frac{\partial}{\partial z} h(x, y, z) f(x, y, z) \\ &= \frac{\partial}{\partial x} (z - \phi(x, y)) + \frac{\partial}{\partial z} (z - \phi(x, y)) f(x, y, z) \\ &= -\frac{\partial}{\partial x} \phi(x, y) + f(x, y, z) \end{aligned}$$

and

$$\begin{aligned}
 \mathcal{L}_{V_2} h &= \left[\frac{\partial}{\partial x} h(x, y, z) \right] \cdot 0 + \left[\frac{\partial}{\partial y} h(x, y, z) \right] \cdot 1 + \left[\frac{\partial}{\partial z} h(x, y, z) \right] g(x, y, z) \\
 &= \frac{\partial}{\partial y} h(x, y, z) + \frac{\partial}{\partial z} h(x, y, z) g(x, y, z) \\
 &= \frac{\partial}{\partial y} (z - \phi(x, y)) + \frac{\partial}{\partial z} (z - \phi(x, y)) g(x, y, z) \\
 &= -\frac{\partial}{\partial y} \phi(x, y) + g(x, y, z)
 \end{aligned}$$

so that equivalence of the two systems is obvious.

It is now time to notice that the fact that

$$\mathcal{L}_{V_1} h = 0 \quad \text{and} \quad \mathcal{L}_{V_2} h = 0$$

immediately implies that

$$\mathcal{L}_V h = 0 \quad \forall V \in \Delta = \text{span}\{V_1, V_2\}$$

i.e. h does not change along any direction from a distribution spanned by V_1 and V_2 .

We are going to claim next that existence of an integral manifold can be verified algebraically by checking a particular property of this distribution through computation of some repeated Lie brackets.

8.2.5 Integrability and Involutivity for Distributions

Let us introduce two more notions.

- The distribution

$$\Delta = \text{span}\{X_1(x), X_2(x), \dots, X_m(x)\}, \quad x \in \mathbb{R}^n$$

is said to be **completely integrable**, if there are functions $h_1(x), \dots, h_{n-m}(x)$ such that

- they are linearly independent for any $x \in \mathbb{R}^n$;
- they satisfy the system of partial differential equations

$$\mathcal{L}_{X_i} h_j = 0, \quad \forall i \in \{1, \dots, m\}, \forall j \in \{1, \dots, (n - m)\}$$

- The distribution Δ is said to be **involutive**, if there are scalar functions $\alpha_{ijk} : \mathbb{R}^n \rightarrow \mathbb{R}$ such that

$$[X_i, X_j] = \sum_{k=1}^n \alpha_{ijk} X_k \quad \forall i, j$$

8.2.6 Frobenius Theorem: integrability $\Leftrightarrow$ involutivity

We are ready to formulate the main result that we need from differential geometry.

A distribution Δ on $\mathbb{R}^n$ is integrable



A distribution Δ on $\mathbb{R}^n$ is involutive

This statements means that in order to verify existence of a solution for a system of partial differential equations, one can simply compute some Lie brackets and check that they can be expressed as linear combinations of certain vectors.

Note however, that a procedure for finding a solution is not clear from this theorem. Let us show next how this theorem can be used for feedback control design.

8.3 Feedback Linearization

For a given nonlinear control system, it is often not clear whether nonlinearities can be canceled via a feedback transformation. However, possibility for such cancellation is a useful structural property that may allow bringing powerful tools of linear control theory for nonlinear control design. Let us formulate the problem of finding an appropriate transformation that makes a nonlinear control system equivalent to a linear one.

8.3.1 Concept of Feedback Linearization

The control system

$$\frac{d}{dt}x = f(x) + g(x)u, \quad x \in \mathbb{R}^n, \quad u \in \mathbb{R}^1, \quad f(0) = 0$$

is said to be **feedback linearizable** if there are

- an invertible change of coordinates **$y = T(x)$** and
- a feedback transform **$u = \alpha(x) + \beta(x) v$**

such that the system dynamics written in new coordinates is

$$\frac{d}{dt}\mathbf{y} = \underbrace{\begin{bmatrix} 0 & 1 & 0 & & 0 \\ 0 & 0 & 1 & & 0 \\ \cdot & \cdot & \cdot & \ddots & \cdot \\ 0 & 0 & 0 & & 1 \\ 0 & 0 & 0 & \dots & 0 \end{bmatrix}}_{=A} \mathbf{y} + \underbrace{\begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix}}_{=B} v$$

Let us try to exploit this definition to obtain some conditions on the involved transformations.

8.3.2 How to find $T(\cdot)$, $\alpha(\cdot)$ and $\beta(\cdot)$?

If the system

$$\frac{d}{dt}\mathbf{x} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}, \quad \mathbf{x} \in \mathbb{R}^n, \quad \mathbf{u} \in \mathbb{R}^1, \quad \mathbf{f}(\mathbf{0}) = \mathbf{0}$$

is feedback linearizable, then there are new coordinates $\mathbf{y}$ defined by

$$\mathbf{y} = T(\mathbf{x}) \quad \Rightarrow \quad \frac{d}{dt}\mathbf{y} = \frac{d}{dt}[T(\mathbf{x})] = \frac{\partial}{\partial \mathbf{x}}[T(\mathbf{x})] \frac{d}{dt}\mathbf{x}$$

The last relation can be rewritten as an equation for the unknown $T_i(\mathbf{x})$

$$A\mathbf{y} + Bv = \frac{\partial}{\partial \mathbf{x}}[T(\mathbf{x})] (\mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}) = \begin{bmatrix} \frac{\partial}{\partial \mathbf{x}}T_1(\mathbf{x}) \\ \vdots \\ \frac{\partial}{\partial \mathbf{x}}T_n(\mathbf{x}) \end{bmatrix} (\mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u})$$

$$A \begin{bmatrix} T_1(\mathbf{x}) \\ \vdots \\ T_n(\mathbf{x}) \end{bmatrix} + Bv = \begin{bmatrix} \frac{\partial}{\partial \mathbf{x}}T_1(\mathbf{x}) \\ \vdots \\ \frac{\partial}{\partial \mathbf{x}}T_n(\mathbf{x}) \end{bmatrix} (\mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u})$$

and substituting A and B :

$$\begin{bmatrix} 0 & 1 & 0 & & 0 \\ 0 & 0 & 1 & & 0 \\ \cdot & \cdot & \cdot & \ddots & \cdot \\ 0 & 0 & 0 & & 1 \\ 0 & 0 & 0 & \dots & 0 \end{bmatrix} \begin{bmatrix} T_1(\mathbf{x}) \\ \vdots \\ T_n(\mathbf{x}) \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} v = \begin{bmatrix} \frac{\partial}{\partial \mathbf{x}}T_1(\mathbf{x}) \\ \vdots \\ \frac{\partial}{\partial \mathbf{x}}T_n(\mathbf{x}) \end{bmatrix} (\mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u})$$

$$\begin{bmatrix} T_2(x) \\ \vdots \\ T_n(x) \\ v \end{bmatrix} = \begin{bmatrix} \mathcal{L}_f T_1 + [\mathcal{L}_g T_1] u \\ \vdots \\ \mathcal{L}_f T_{n-1} + [\mathcal{L}_g T_{n-1}] u \\ \mathcal{L}_f T_n + [\mathcal{L}_g T_n] u \end{bmatrix}$$

In other words, we obtain the following system of equations

$$\begin{aligned} \mathcal{L}_f T_1 + [\mathcal{L}_g T_1] u &= T_2, \\ \mathcal{L}_f T_2 + [\mathcal{L}_g T_2] u &= T_3, \\ &\vdots \\ \mathcal{L}_f T_n + [\mathcal{L}_g T_n] u &= v \end{aligned}$$

that must be satisfied after substituting $u = \alpha(x) + \beta(x) v$ identically for all values of x and v .

Since we are looking at $T_1(x), \dots, T_n(x)$ that are independent on u , we must require

$$\mathcal{L}_g T_1 = \mathcal{L}_g T_2 = \dots = \mathcal{L}_g T_{n-1} = 0, \quad \mathcal{L}_g T_n \neq 0$$

Therefore, the equations to solve become

$$\mathcal{L}_f T_i = T_{i+1}, \quad i = 1, \dots, n-1, \quad \mathcal{L}_f T_n + (\mathcal{L}_g T_n) \alpha = 0, \quad (\mathcal{L}_g T_n) \beta = 1$$

Hence, to obtain a solution we must solve the system of PDEs

$$\mathcal{L}_g T_1 = 0, \quad \mathcal{L}_g T_2 = 0, \quad \dots, \quad \mathcal{L}_g T_{n-1} = 0$$

for $T_1, \dots, T_{n-1}$, which are defined by

$$T_2 = \mathcal{L}_f T_1, \quad T_3 = \mathcal{L}_f T_2, \quad \dots, \quad T_n = \mathcal{L}_f T_{n-1}$$

In order to use Frobenius Theorem, let us rewrite it as a system with respect to a single scalar function $T_1(x)$ by recursively substituting the expressions for $T_i(x)$.

It is easy to see that we obtain

$$\mathcal{L}_g T_1 = 0, \quad \mathcal{L}_g \mathcal{L}_f T_1 = 0, \quad \mathcal{L}_g \mathcal{L}_f^2 T_1 = 0, \quad \dots, \quad \mathcal{L}_g \mathcal{L}_f^{n-1} T_1 = 0$$

It is left to notice that the second equation can be substituted with the following one

$$\mathcal{L}_{[f,g]} T_1 = \mathcal{L}_f [\mathcal{L}_g T_1] - \mathcal{L}_g [\mathcal{L}_f T_1] = 0 - \mathcal{L}_g T_2 = 0$$

and in a similar way all of them can be substituted by repeated Lie derivatives

so that the system to solve becomes

$$\mathcal{L}_g T_1 = 0, \quad \mathcal{L}_{[f,g]} T_1 = 0, \quad \mathcal{L}_{[f,[f,g]]} T_1 = 0, \quad \dots, \quad \mathcal{L}_{ad_f^{n-1}g} T_1 \neq 0$$

Finally, we have a system of partial differential equations

$$\mathcal{L}_{X_i} T_1 = 0, \quad i = \{1, \dots, n-1\}, \quad \mathcal{L}_{X_n} T_1 \neq 0$$

with

$$X_1 = ad_f^0 g, \quad X_2 = ad_f^1 g, \quad \dots, \quad X_{n-1} = ad_f^{(n-2)} g, \quad X_n = ad_f^{(n-1)} g$$

and to ensure existence of a solution, according to **Frobenius Theorem**, the distribution $\Delta = \text{span}\{X_1, X_2, \dots, X_n\}$ should be

- of constant rank, i.e. X_i are to be linearly independent,
- involutive.

If both conditions are satisfied, then the solution $T_1(x)$ exists!

As soon as a solution is found, we take

$$T_2 = \mathcal{L}_f T_1, \quad T_3 = \mathcal{L}_f T_2, \quad \dots, \quad T_n = \mathcal{L}_f T_{n-1}$$

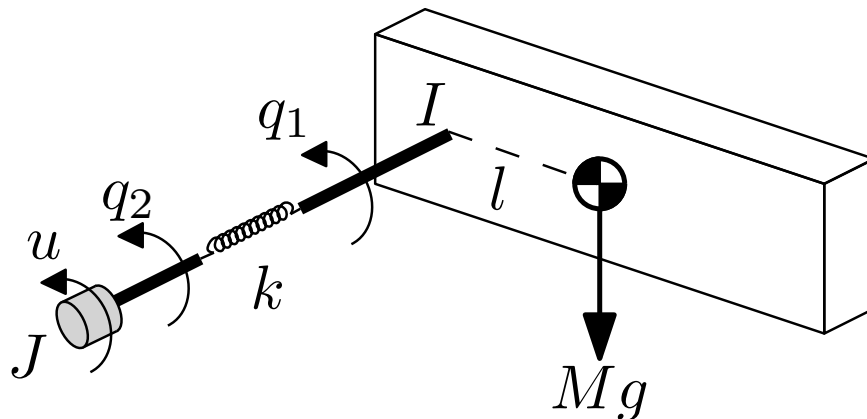
to define the coordinate transformation $y = T(x)$ and solve

$$(\mathcal{L}_g T_n) \alpha = -(\mathcal{L}_f T_n), \quad (\mathcal{L}_g T_n) \beta = 1$$

to obtain the expression for the control transformation $u = \beta(x) + \alpha(x) v$.

8.3.3 Feedback Linearization for Single Link Flexible Joint Robot

Consider the following simple diagram describing a one-revolute-link robot with a gear box modeled as a torsional spring.



The equations of motion are

$$I\ddot{q}_1 + Mgl \sin(q_1) + k(q_1 - q_2) = 0$$

$$J\ddot{q}_2 + k(q_2 - q_1) = u$$

where M is the mass of the link, l is the distance from the pivot point to the center of mass of the link, g is the gravitational constant, k is the spring constant, I and J are the moments of inertia, q_1 and q_2 are the angles of the motor rotor and the link respectively, and u is the applied control torque produced by the motor.

In order to show that the system is feedback linearizable if $k \neq 0$, $I \neq 0$, $J \neq 0$ we need to compute f , g , a few repeated Lie brackets, and verify involutivity! Let us do this and suggest a solution for the corresponding PDE.

Step 1: Writing Dynamics in a State-Space Form

Two second order equations

$$I\ddot{q}_1 + Mgl \sin(q_1) + k(q_1 - q_2) = 0$$

$$J\ddot{q}_2 + k(q_2 - q_1) = u$$

should be rewritten a single vector first order equation

$$\frac{d}{dt}x = f(x) + g(x)u$$

How to choose the vector x , and how to compute $f(x)$ and $g(x)$?

The most natural choice (among others) for $x = [x_1, x_2, x_3, x_4]^T$ is

$$x_1 = q_1, \quad x_2 = \frac{d}{dt}q_1, \quad x_3 = q_2, \quad x_4 = \frac{d}{dt}q_2$$

Let us compute derivatives of the chosen state variables

$$\frac{d}{dt}x_1 = x_2$$

$$\frac{d}{dt}x_2 = -Mgl \sin(x_1)/I - k(x_1 - x_3)/I$$

$$\frac{d}{dt}x_3 = x_4$$

$$\frac{d}{dt}x_4 = k(x_1 - x_3)/J + u/J$$

This leads to a definition for the vector fields:

$$\frac{d}{dt} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \underbrace{\begin{bmatrix} x_2 \\ -Mgl \sin(x_1)/I - k(x_1 - x_3)/I \\ x_4 \\ k(x_1 - x_3)/J \end{bmatrix}}_{f(x)} + \underbrace{\begin{bmatrix} 0 \\ 0 \\ 0 \\ 1/J \end{bmatrix}}_{g(x)} \mathbf{u}$$

Step 2: Checking the Conditions of Frobenius Theorem

For our system the vector fields $f(x)$ and $g(x)$ are

$$\begin{aligned} f(x) &= \left[x_2, -\left(\frac{Mgl}{I} \sin(x_1) + \frac{k}{I}(x_1 - x_3) \right), x_4, \frac{k}{J}(x_1 - x_3) \right]^T \\ g(x) &= \left[0, 0, 0, \frac{1}{J} \right]^T \end{aligned}$$

We need to compute the following repeated Lie brackets

$$X_1 = ad_f^0 g, \quad X_2 = ad_f^1 g, \quad X_3 = ad_f^2 g, \quad X_4 = ad_f^3 g$$

This can be done recursively as follows

$$X_1 = g, \quad X_2 = [f, X_1], \quad X_3 = [f, X_2], \quad X_4 = [f, X_3]$$

So, the second one is

$$\begin{aligned} X_2 &= [f, X_1] = \left[\frac{\partial}{\partial x} X_1(x) \right] f(x) - \left[\frac{\partial}{\partial x} f(x) \right] X_1(x) \\ &= \underbrace{\left[\frac{\partial}{\partial x} g(x) \right]}_{=0} f(x) - \left[\frac{\partial}{\partial x} f(x) \right] g(x) \\ &= - \left[\frac{\partial}{\partial x} f(x) \right] g(x) = -\frac{1}{J} \left[\frac{\partial}{\partial x_4} f(x) \right]^T \\ &= -\frac{1}{J} [0, 0, 1, 0]^T \end{aligned}$$

The third one is

$$\begin{aligned} X_3 &= [f, X_2] = \left[\frac{\partial}{\partial x} X_2(x) \right] f(x) - \left[\frac{\partial}{\partial x} f(x) \right] X_2(x) \\ &= - \left[\frac{\partial}{\partial x} f(x) \right] X_2(x) = \frac{1}{J} \left[\frac{\partial}{\partial x_3} f(x) \right]^T \\ &= \frac{1}{J} [0, \frac{k}{I}, 0, -\frac{k}{J}]^T \end{aligned}$$

The fourth one is

$$\begin{aligned}
\mathbf{X}_4 &= [\mathbf{f}, \mathbf{X}_3] = \left[\frac{\partial}{\partial x} \mathbf{X}_3(x) \right] \mathbf{f}(x) - \left[\frac{\partial}{\partial x} \mathbf{f}(x) \right] \mathbf{X}_3(x) \\
&= - \left[\frac{\partial}{\partial x_2} \mathbf{f}(x) \right] \frac{k}{JI} + \left[\frac{\partial}{\partial x_4} \mathbf{f}(x) \right] \frac{k}{J^2} \\
&= \left[-\frac{k}{JI}, 0, \frac{k}{J^2}, 0 \right]^T
\end{aligned}$$

Finally,

$$\mathbf{X}_1 = \left[0, 0, 0, \frac{1}{J} \right]^T, \quad \mathbf{X}_2 = -\frac{1}{J} \left[0, 0, 1, 0 \right]^T,$$

$$\mathbf{X}_3 = \frac{1}{J} \left[0, \frac{k}{I}, 0, -\frac{k}{J} \right]^T, \quad \mathbf{X}_4 = \left[-\frac{k}{JI}, 0, \frac{k}{J^2}, 0 \right]^T$$

Now, it is clear that the distribution Δ spanned by $\mathbf{X}_1, \dots, \mathbf{X}_4$ is of constant dimension since the vector fields are constant

$$[\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3, \mathbf{X}_4] = \begin{bmatrix} 0 & 0 & 0 & -\frac{k}{JI} \\ 0 & 0 & \frac{k}{JI} & 0 \\ 0 & -\frac{1}{J} & 0 & \frac{k}{J^2} \\ \frac{1}{J} & 0 & -\frac{k}{J^2} & 0 \end{bmatrix}$$

We need to verify whether it is involutive or not!

By definition, distribution Δ is said to be **involutive**, if there are scalar functions $\alpha_{ijk} : \mathbb{R}^n \rightarrow \mathbb{R}$ such that

$$[\mathbf{X}_i, \mathbf{X}_j] = \sum_{k=1}^n \alpha_{ijk} \mathbf{X}_k \quad \forall i, j$$

Obviously, for constant vector fields $\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3, \mathbf{X}_4$

$$[\mathbf{X}_i, \mathbf{X}_j] = 0, \quad 1 \leq i, j \leq 4$$

and hence we can take $\alpha_{ijk} = 0$.

To conclude

- Both conditions of Frobenius Theorem are valid!
- The dynamics of the system is **feedback linearizable**!

Hence, there is a change of coordinates

$$\mathbf{y}_1 = \mathbf{T}_1(\mathbf{x}), \mathbf{y}_2 = \mathbf{T}_2(\mathbf{x}), \mathbf{y}_3 = \mathbf{T}_3(\mathbf{x}), \mathbf{y}_4 = \mathbf{T}_4(\mathbf{x})$$

and there is a feedback transform

$$\mathbf{u} = \boldsymbol{\alpha}(\mathbf{x}) + \boldsymbol{\beta}(\mathbf{x})\mathbf{v}$$

such that the dynamics written in $(\mathbf{y}, \mathbf{v})$ is linear.

- The equations with respect to a scalar function $T_1(\mathbf{x})$

$$\mathcal{L}_g \mathbf{T}_1(\mathbf{x}) = 0, \mathcal{L}_{ad_f g} \mathbf{T}_1(\mathbf{x}) = 0, \mathcal{L}_{ad_f^2 g} \mathbf{T}_1(\mathbf{x}) = 0, \mathcal{L}_{ad_f^3 g} \mathbf{T}_1(\mathbf{x}) \neq 0$$

has a solution! Functions $\mathbf{T}_2(\cdot)$, $\mathbf{T}_3(\cdot)$, $\mathbf{T}_4(\cdot)$ can be found!

Step 3: Find $\mathbf{T}_1(\mathbf{x})$ as a solution for PDEs

The first PDE

$$\mathcal{L}_g \mathbf{T}_1(\mathbf{x}) = 0$$

becomes

$$\left[\frac{\partial}{\partial x_1} T_1, \frac{\partial}{\partial x_2} T_1, \frac{\partial}{\partial x_3} T_1, \frac{\partial}{\partial x_4} T_1 \right] \left[0, 0, 0, \frac{1}{J} \right]^T = 0 \Rightarrow \frac{\partial}{\partial x_4} T_1 = 0$$

The second PDE

$$\mathcal{L}_{ad_f g} \mathbf{T}_1(\mathbf{x}) = 0$$

becomes

$$\left[\frac{\partial}{\partial x_1} T_1, \frac{\partial}{\partial x_2} T_1, \frac{\partial}{\partial x_3} T_1, \frac{\partial}{\partial x_4} T_1 \right] \left[0, 0, -\frac{1}{J}, 0 \right]^T = 0 \Rightarrow \frac{\partial}{\partial x_3} T_1 = 0$$

The third PDE

$$\mathcal{L}_{ad_f^2 g} \mathbf{T}_1(\mathbf{x}) = 0$$

becomes

$$\left[\frac{\partial}{\partial x_1} T_1, \frac{\partial}{\partial x_2} T_1, \frac{\partial}{\partial x_3} T_1, \frac{\partial}{\partial x_4} T_1 \right] \left[0, \frac{k}{JI}, 0, -\frac{k}{J^2} \right]^T = 0 \Rightarrow \frac{\partial}{\partial x_2} T_1 = 0$$

The last inequality

$$\mathcal{L}_{ad_f^3 g} \mathbf{T}_1(\mathbf{x}) \neq 0$$

becomes

$$\left[\frac{\partial}{\partial x_1} T_1, \frac{\partial}{\partial x_2} T_1, \frac{\partial}{\partial x_3} T_1, \frac{\partial}{\partial x_4} T_1 \right] \left[\frac{-k}{JI}, 0, -\frac{k}{J^2}, 0 \right]^T \neq 0 \Rightarrow \frac{\partial}{\partial x_1} T_1 \neq 0$$

Finally, all the computed requirements for the function $T_1 = T_1(\mathbf{x})$

$$\frac{\partial}{\partial x_1} T_1(\mathbf{x}) \neq 0, \quad \frac{\partial}{\partial x_2} T_1(\mathbf{x}) = 0, \quad \frac{\partial}{\partial x_3} T_1(\mathbf{x}) = 0, \quad \frac{\partial}{\partial x_4} T_1(\mathbf{x}) = 0$$

imply that it should depend only on $\mathbf{x}_1$ variable in a nontrivial way!

There are many possible choices. Let us take

$$T_1(\mathbf{x}) = \mathbf{x}_1$$

Step 4: Compute $T_2(\mathbf{x})$, $T_3(\mathbf{x})$, $T_4(\mathbf{x})$

The functions $T_2(\cdot)$, $T_3(\cdot)$, $T_4(\cdot)$, $\alpha(\cdot)$, and $\beta(\cdot)$ can be easily found from the relations

$$\mathcal{L}_f T_1 = T_2, \quad \mathcal{L}_f T_2 = T_3, \quad \mathcal{L}_f T_3 = T_4,$$

and

$$(\mathcal{L}_g T_n) \alpha = -(\mathcal{L}_f T_n), \quad (\mathcal{L}_g T_n) \beta = 1$$

With $T_1(\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4) = \mathbf{x}_1$, we have

$$\begin{aligned} T_2(\mathbf{x}) &= \mathcal{L}_f T_1 = \left[\frac{\partial}{\partial x_1} T_1(\mathbf{x}) \right] \cdot f_1(\mathbf{x}) + \cdots + \left[\frac{\partial}{\partial x_4} T_1(\mathbf{x}) \right] \cdot f_4(\mathbf{x}) \\ &= \left[\frac{\partial}{\partial x_1} T_1(\mathbf{x}) \right] \cdot f_1(\mathbf{x}) + 0 \cdot f_2(\mathbf{x}) + 0 \cdot f_3(\mathbf{x}) + 0 \cdot f_4(\mathbf{x}) \\ &= f_1(\mathbf{x}) = x_2 \end{aligned}$$

since

$$f(\mathbf{x}) = \left[x_2, -\left(\frac{Mgl}{I} \sin(x_1) + \frac{k}{I} (x_1 - x_3) \right), x_4, \frac{k}{J} (x_1 - x_3) \right]^T$$

Proceeding in the same way

$$\begin{aligned} T_3(\mathbf{x}) &= \mathcal{L}_f T_2 = \left[\frac{\partial}{\partial x_1} T_2(\mathbf{x}) \right] \cdot f_1(\mathbf{x}) + \cdots + \left[\frac{\partial}{\partial x_4} T_2(\mathbf{x}) \right] \cdot f_4(\mathbf{x}) \\ &= 0 \cdot f_1(\mathbf{x}) + \left[\frac{\partial}{\partial x_2} T_2(\mathbf{x}) \right] \cdot f_2(\mathbf{x}) + 0 \cdot f_3(\mathbf{x}) + 0 \cdot f_4(\mathbf{x}) \\ &= f_2(\mathbf{x}) = -\left(\frac{Mgl}{I} \sin(x_1) + \frac{k}{I} (x_1 - x_3) \right) \end{aligned}$$

and

$$\begin{aligned} T_4(\mathbf{x}) &= \mathcal{L}_f T_3 = \left[\frac{\partial}{\partial x_1} T_3(\mathbf{x}) \right] \cdot f_1(\mathbf{x}) + \cdots + \left[\frac{\partial}{\partial x_4} T_3(\mathbf{x}) \right] \cdot f_4(\mathbf{x}) \\ &= -\left(\frac{Mgl}{I} \cos(x_1) + \frac{k}{I} \right) \cdot x_2 + \frac{k}{I} \cdot x_4 \end{aligned}$$

Step 5: Compute $\alpha(\cdot)$ and $\beta(\cdot)$

The equation for the feedback transform

$$\mathcal{L}_f T_4(\mathbf{x}) + \mathcal{L}_g T_4(\mathbf{x}) u = v$$

with

$$\begin{aligned} f(x) &= \left[x_2, -\left(\frac{Mgl}{I} \sin(x_1) + \frac{k}{I}(x_1 - x_3)\right), x_4, \frac{k}{J}(x_1 - x_3) \right]^T \\ g(x) &= \left[0, 0, 0, \frac{1}{J} \right]^T \\ T_4(x) &= -\left(\frac{Mgl}{I} \cos(x_1) + \frac{k}{I}\right) \cdot x_2 + \frac{k}{I} \cdot x_4 \end{aligned}$$

becomes

$$u = \alpha(x) + \beta(x)v$$

if

$$\begin{aligned} \beta(x) &= JI/k \\ \alpha(x) &= -\frac{JI}{k} \left[\frac{Mgl}{I} \cos(x_1) \cdot x_2^2 + \frac{k}{I} \frac{k}{J} (x_1 - x_3) \right. \\ &\quad \left. + \left(\frac{Mgl}{I} \cos(x_1) + \frac{k}{I} \right) \left(\frac{Mgl}{I} \sin(x_1) + \frac{k}{I} (x_1 - x_3) \right) \right] \end{aligned}$$

Finally, under the transformation, the equations of motion

$$\begin{aligned} I\ddot{q}_1 + Mgl \sin(q_1) + k(q_1 - q_2) &= 0 \\ J\ddot{q}_2 + k(q_2 - q_1) &= u \end{aligned}$$

become linear

$$\dot{y}_1 = y_2, \quad \dot{y}_2 = y_3, \quad \dot{y}_3 = y_4, \quad \dot{y}_4 = v$$

in the new coordinates: $y_1 = q_1, \quad y_2 = \dot{q}_1$

$$\begin{aligned} y_3 &= -\left(\frac{Mgl}{I} \sin(q_1) + \frac{k}{I}(q_1 - q_2)\right) \\ y_4 &= -\left(\frac{Mgl}{I} \cos(q_1) + \frac{k}{I}\right) \dot{q}_1 + \frac{k}{I} \dot{q}_2 \end{aligned}$$

and with the new control input v that defines the real control as

$$u = \alpha(q_1, \dot{q}_1, q_2, \dot{q}_2) + \beta(q_1, \dot{q}_1, q_2, \dot{q}_2) v$$

Now, v can be designed using the tools of linear control theory.

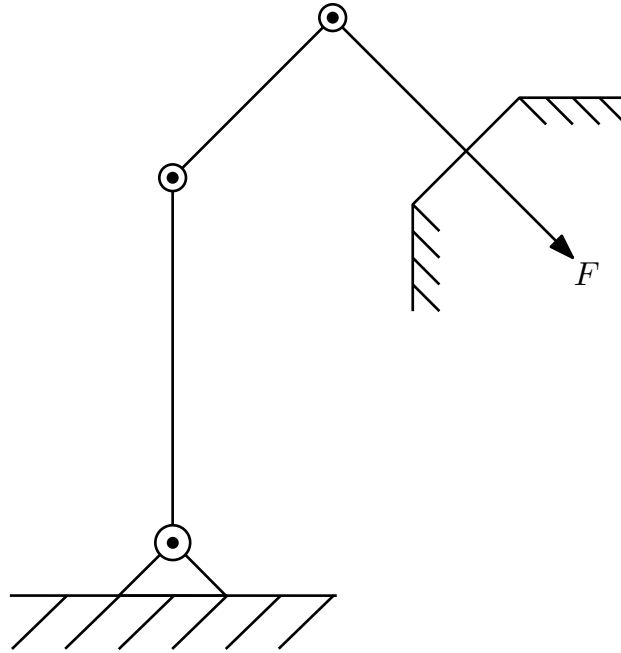
Supplementary Material: Elements of Force and Vision-Based Control

Appendix A

Elements of Force Control

We describe here some basic ideas that are currently used in robotics in various scenarios where a manipulator interacts with the environment or with the object of manipulation. In many cases the interaction forces not only influence the dynamics of the manipulator itself but also must be kept within certain limits in order to avoid distraction or sometimes to ensure completion of a task such as polishing a surface.

Whenever manipulator's tip touches an object, interaction forces appear. A force (or, sometimes, also a torque) will be applied to an object, as illustrated in the following figure.



Simultaneously, an opposite in direction and same in magnitude force (or, sometimes, also a torque) will be applied to the manipulator.

Whenever robot end-effector come into contact with a surface, the model derived in Chapter 5 is not valid from the moment of impact and until the contact is broken; it should be modified. Such a modification can be done in two ways:

1. The environment can be modeled as a separate mechanical system while

contact can be considered (after a brief period of impact) as a holonomic (or of a more general type) constraint that generates forces. Sometimes, it is also necessary to separately model the impact at the moment when impact is initiated. This approach may lead to a quite complicated models of hybrid nature (mixture of continuous and discrete dynamics) with not constant numbers of DOFs.

2. The dynamics of the environment may be completely or partially ignored while the control system relies on special sensors measuring the interaction forces/torques in addition to the usual sensors estimating the kinematic configuration. The interaction forces are added to the dynamical model derived for the interaction-free case: They appear in the dynamical equations through the manipulator Jacobian, which is, obviously, coincides with the Jacobian computed from a description of the contact as a constraint in terms of the manipulator's generalized coordinates.

To the best of our knowledge, all the commercially available systems today are based on the second approach. The main disadvantage of this is that feasible trajectories and interaction forces cannot be neither planned in advance nor, correspondingly, enforced simultaneously even if the advanced reliable sensors are installed. This issue is the subject of active current research.

A.1 Coordinate Frames and Constraints

To describe the interaction, the base frame coordinates of the vectors of linear and angular velocities and of the vectors of force and torque acting on the end-effector

$$\boldsymbol{\xi} = [\boldsymbol{v}^T, \boldsymbol{\omega}^T]^T, \quad \boldsymbol{F} = [\boldsymbol{f}^T, \boldsymbol{n}^T]^T$$

are typically used.

Let $\mathcal{M}$ be a set of coordinates of vectors of linear and angular velocities of the end-effector in the $\mathbf{0}$ frame

$$\boldsymbol{\xi} = [\boldsymbol{v}^T, \boldsymbol{\omega}^T]^T \in \mathcal{M}$$

known also as **twists**. They can be computed as in Chapter 3.

Let $\mathcal{F}$ be a similar set for vectors of forces and torques applied to the end-effector

$$\boldsymbol{F} = [\boldsymbol{f}^T, \boldsymbol{n}^T]^T \in \mathcal{F}$$

known also as **wrenches**. They are often assumed to be measured by a special sensor. Sometimes, they can be approximately computed under certain

assumptions about the environment and of the contact.

It is a common practice to describe the desired interaction task of a manipulator with the environment in terms of specifications on the twists and wrenches. When such specifications are described, it is often the case that the following property is valid. A twist $\xi \in \mathcal{M}$ and a wrench $F \in \mathcal{F}$ are called **reciprocal** if

$$\xi^T F = v^T f + \omega^T n = 0$$

This can be interpreted as zero elementary work during the target motion when performing the task and, in particular, assumes that the friction at the contact can be ignored.

A.2 Natural and Artificial Constraints

It is usually more natural to describe a task involving interaction in a particular frame that is attached to the surface of interaction or to the object of interaction. The frame, in which task is readily defined, is called a **compliance frame** or a constraint frame.

With respect to this frame, the task is specified in terms of constraints that can be grouped as

- the **natural** ones, which are implied by mechanics and cannot be changed by the behavior of the robot,
- the **artificial** ones, which are implied by our description of the task and can be assigned differently.

This is illustrated on an example next.

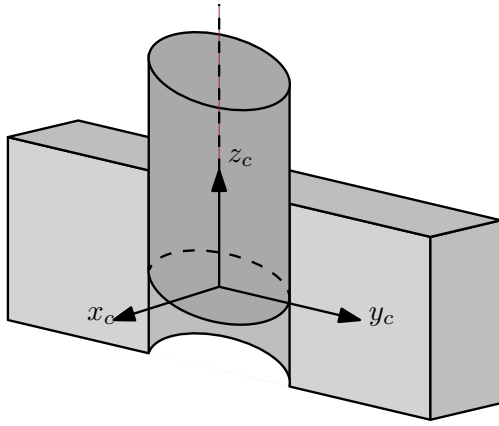
A.2.1 Inserting Task

Consider the task of inserting a cylindrical peg into a matching cylindrical hole in a static fixed rigid object. Let us assume that the walls of the hole and the peg itself are perfectly rigid and there is no friction between them.

When such a task is performed by a manipulator, it is clear that during the insertion the end-effector is prevented to have linear velocities in the directions orthogonal to the walls as well as angular velocities along such directions. At the same time, the object can produce neither a resiting force nor a resiting torque along the direction of insertion. We have just described the set of the **natural constraints**, which appear due to the interaction.

Now, in order not to damage both the object and the manipulator, the robot should not produce any forces or torques along the directions orthogonal to the walls of the hole. At the same time, insertion should be performed with a certain linear speed and a certain angular speed, so the target velocity along the hole should be a specified constant while the target angular velocity should be zero since this is a peg and not a screw. We have just described the set of the **artificial constraints**, which appear due to the task description.

The two types of constraints are summarized on the next figure.



Natural Constraints	Artificial Constraints
$v_x = 0$	$f_x = 0$
$v_y = 0$	$f_y = 0$
$f_z = 0$	$v_z = v_d$
$\omega_x = 0$	$n_x = 0$
$\omega_y = 0$	$n_y = 0$
$n_z = 0$	$\omega_z = 0$

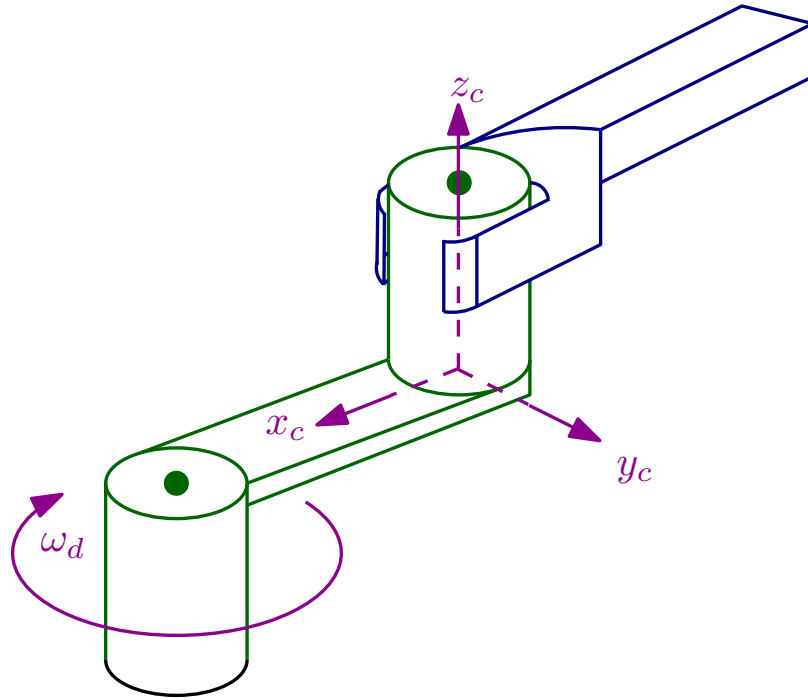
Note that for this example,

$$\xi^T F = v_x f_x + v_y f_y + v_z f_z + \omega_x n_x + \omega_y n_y + \omega_z n_z = \textcolor{red}{0}$$

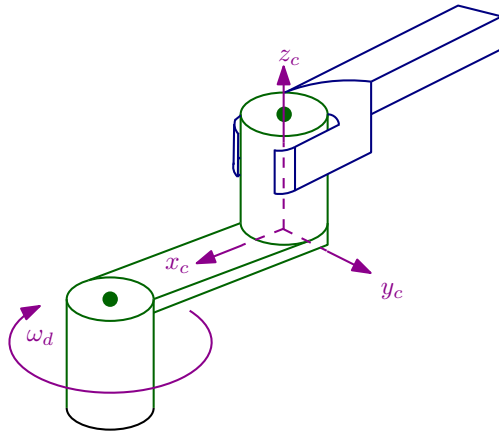
that is the constraints are reciprocal. Note that this is partially due to our assumption of frictionless contact.

A.2.2 Turning a Crank Task

Consider now the task illustrated below.



Which constraints are natural and which ones are artificial in this case? The answer is given next.



Natural Constraints	Artificial Constraints
$v_x = 0$	$f_x = 0$
$f_y = 0$	$v_y = 0$
$v_z = 0$	$f_z = 0$
$\omega_x = 0$	$n_x = 0$
$\omega_y = 0$	$n_y = 0$
$n_z = 0$	$\omega_z = \omega_d$

Note that the total elementary work is again zero and so the constraints are reciprocal.

A.3 Network Models and Impedance

In the situation when total elementary work along a target task is zero, that is the task describing constraints are reciprocal, it is clear how to formulate a control design task: In certain directions the position and the orientation of the end-effector should be achieved, while in the complimentary directions forces and torques should be produced.

However, in practice, such separation in the task description is often not possible without introducing models of the environment and of the interaction. To deal with the situation, it has been suggested to introduce partial

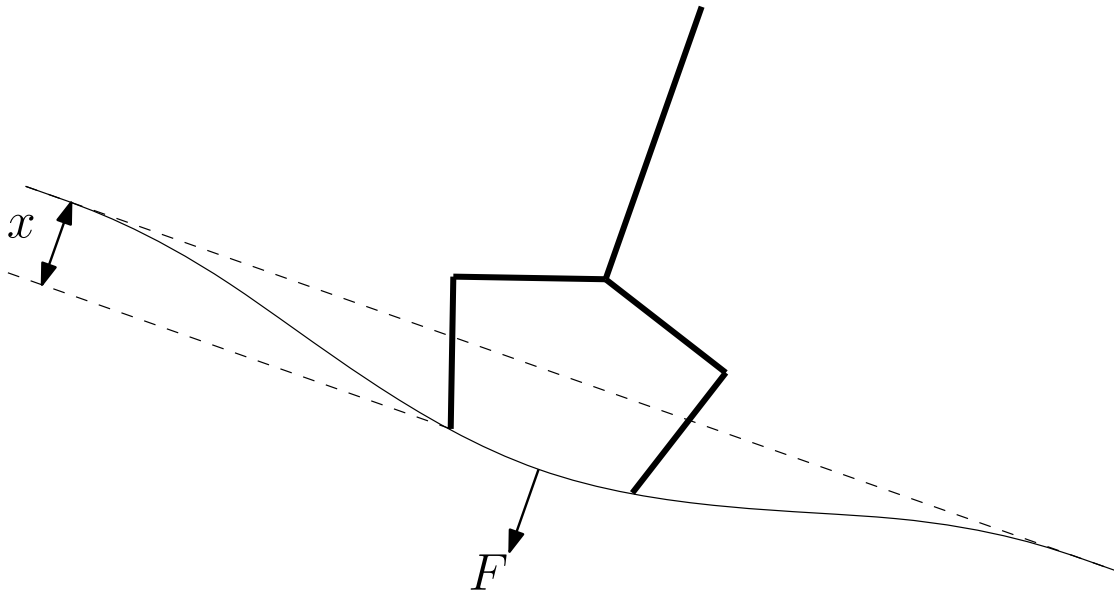
intuition-based models of the environment and of the corresponding interaction forces. These models are to allow generation of target trajectories (or just target velocities) and target interaction forces/torques that are not contradictory to each other.

A.3.1 Network Models

Let us introduce a useful concept of mechanical impedance.

Flexible surface

Suppose the end-effector comes into contact with a surface that is not rigid. Instead of attempting to generate a mechanical model of the object, surface of which is touched, let us introduce assumptions about the induced reaction force. Suppose first that the surface acts as a perfect resisting spring, i.e. whenever deformation of the surface appears as illustrated in the figure below



a reaction force proportional to the deformation in the direction normal to the surface appears as well. Introduce the stiffness coefficient of the surface k and suppose

$$\mathbf{F} = [f_x, f_y, f_z]^T = [k x, 0, 0]^T, \quad \mathbf{n} = 0$$

then, clearly,

$$\xi^T \mathbf{F}(t) = v_x f_x + v_y f_y + v_z f_z + \omega_x n_x + \omega_y n_y + \omega_z n_z \neq 0$$

and non-zero elementary work must be done to introduce deformation of the surface.

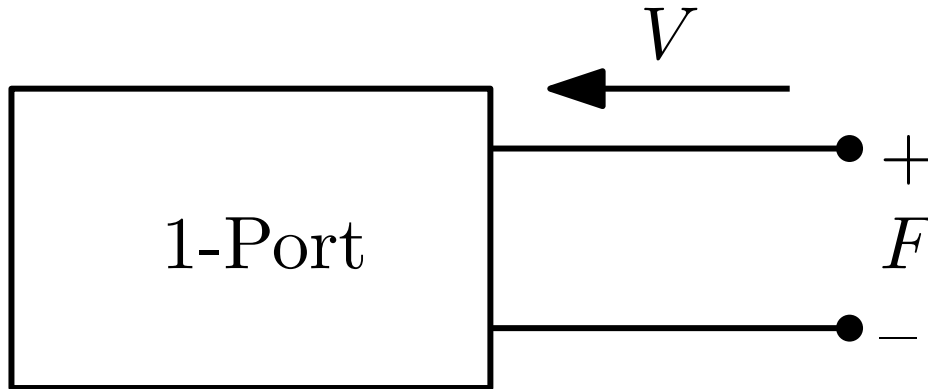
The work done along the motion of certain duration $\mathbf{T}$ can be computed as follows

$$\begin{aligned} \int_0^T \boldsymbol{\xi}(t)^T \mathbf{F}(t) dt &= \int_0^T [\mathbf{v}_x] \mathbf{f}_x dt = \int_0^T [\mathbf{v}_x] kx(t) dt \\ &= \int_0^T \frac{d}{dt}[x(t)] kx(t) dt = k \int_0^T \frac{d}{dt} \left[\frac{1}{2} x^2(t) \right] dt \\ &= \frac{1}{2} k [x^2(T) - x^2(0)] \end{aligned}$$

Note that this work is defined by the initial and final configurations.

One-Port Network

The above description of flexible surface of contact can be generalized as a network with velocity $\mathbf{V}$ (or twists) treated as an input and the law of change of the resistance force $\mathbf{F}$ (or wrenches) characterized by dissipated power during the interaction. The conceptual diagram is given below.



The variables $\mathbf{V}$ and $\mathbf{F}$ are such that

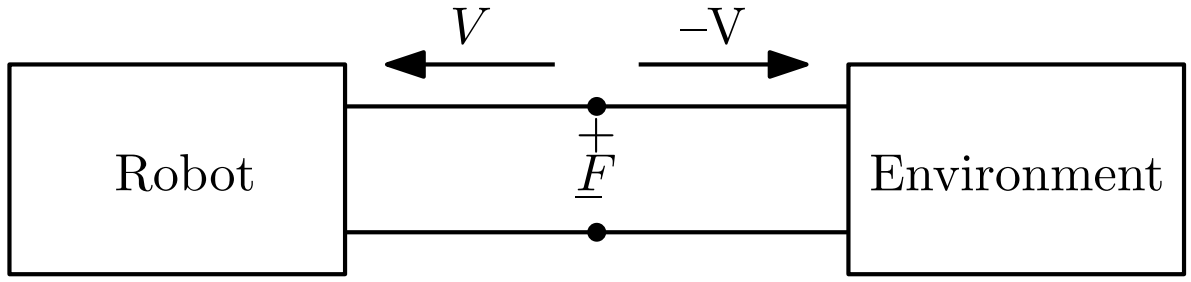
- $\mathbf{V}^T(t)\mathbf{F}(t)$ represents an instantaneous (elementary) power,
- $\int_0^T \mathbf{V}(t)^T \mathbf{F}(t) dt$ is the energy dissipated by the network on the time interval $t \in [0, T]$.

These variables have special names:

- $\mathbf{V}$ is called the **flow** variable,
- $\mathbf{F}$ is called the **effort** variable.

Interconnection of One-Port Networks

It is possible to describe interaction between the robot and the environment as a interconnection of two one-port networks as illustrated below



Note that in order to describe the energy exchange between the robot and the environment, it is convenient to consider connection of the two subsystems with opposite in sign flow variables and equal effort variables.

A.3.2 Mechanical Impedance

A network model of interaction allows to introduce the concept of (mechanical) impedance.

Impedance Operator for One-Port Network

If the system is linear, then the (steady-state) relation between the flow variable ($\mathbf{V}(t)$) and the effort variable ($\mathbf{F}(t)$) for a one-port network can be described by a transfer function, which is called the **impedance operator**

$$Z(s) = \frac{\tilde{\mathbf{F}}(s)}{\tilde{\mathbf{V}}(s)}$$

For example, in the case of flexible surface, we have

$$\mathbf{F}(t) = k\mathbf{x}(t) \Rightarrow \frac{d}{dt}\mathbf{F} = k\mathbf{V}(t) \Rightarrow s\tilde{\mathbf{F}}(s) = k\tilde{\mathbf{V}}(s) \Rightarrow Z(s) = \frac{k}{s}$$

where with some abuse of notation we have dropped the zero components redefining $\mathbf{V} \leftarrow \mathbf{v}_x$ and $\mathbf{F} \leftarrow \mathbf{f}_x$.

Let us consider now another example. Suppose dynamics of a one-port network can be described as a 1-DOF linear time-invariant mechanical system

$$M\ddot{\mathbf{x}} + B\dot{\mathbf{x}} + K\mathbf{x} = \mathbf{f}$$

Then, the impedance operator from $\mathbf{V} = \dot{\mathbf{x}}$ to $\mathbf{F} = \mathbf{f}$ is

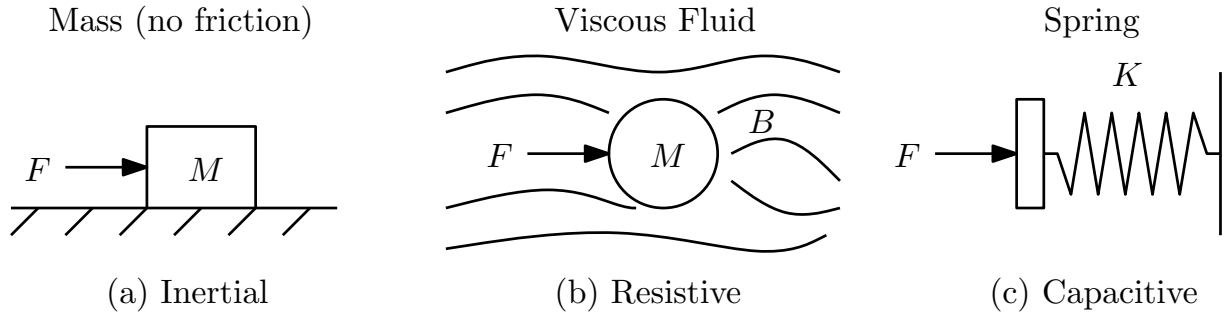
$$Z(s) = Ms + B + K \frac{1}{s}$$

Classification of Impedance Operators

An impedance operator $Z(s)$ is called

- **Inertial**, if $|Z(0)| = 0$
- **Resistive**, if $|Z(0)| = B$ with $0 < B < \infty$
- **Capacitive**, if $|Z(0)| = +\infty$

This classification is illustrated in the following diagram



The type of the impedance operator that is appropriate for description of the environment within a particular task have some consequences on how to approach design of a control law.

A.4 Task Space Dynamics and Impedance Control

Impedance description may be helpful in designing a control law.

A.4.1 Dynamics of a Rigid Manipulator in Contact with an Environment

Dynamics of the system can be obtained by modifying the model derived in Chapter 5. We just need to add the external forces due to interaction with the environment to obtain

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) + J^T(q)F_e = u$$

where the generalized force $J^T(q)F_e$ is computed as product of the transposed manipulator Jacobian (see Chapter 3) and the external generalized reaction force.

Now, in order to design a feedback control law to achieve a particular task involving interaction with the environment one have to know how the interaction force is formed. Avoiding modeling the environment as an interacting with the robot mechanical system, one may attempt to proceed in the following way:

- Assume that there is a description of the target trajectory $\mathbf{q} = \mathbf{q}_d(t)$ and the target interaction force $\mathbf{F} = \mathbf{F}_d(t)$ that are **consistent**, i.e. simultaneously possible without loosing the contact between the robot and the environment or the object of manipulation.
- Introduce feedback not only on the state variables but also on the generalized interaction force measurable by an appropriate sensor.

To understand the consequences and importance of the consistency assumption one needs to realize that the internal (reaction) forces between various pairs of links of a mechanical system, such as a manipulator, are completely defined by the values of the generalized coordinates, generalized velocities and generalized accelerations. The forces and the values of the coordinates can be computed along any target motion (e.g. by introducing appropriate excessive coordinates and writing dynamics in the form of a system of algebraic-differential equations with Lagrange multipliers representing magnitudes of the internal forces) but *can not be chosen independently!*

Nevertheless, let us assume that a particular task is described appropriately. For example, it seems obvious that zero interaction force can be achieved provided the target trajectory avoids collisions with the environment. Similarly, it seems to be reasonable to assume possibility to achieve constant interaction force together with a constant deformation of the environment accurately approximated by a flexible surface model.

As a possible control design strategy, one may modify the feedback-linearization-based control transformation described in Chapter 6 as follows

$$\mathbf{u} = \mathbf{M}(\mathbf{q})\mathbf{a}_q + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) + \mathbf{J}(\mathbf{q})^T \mathbf{a}_f$$

Then, we obtain the transformed dynamics

$$\mathbf{M}(\mathbf{q}) [\ddot{\mathbf{q}} - \mathbf{a}_q] + \mathbf{J}^T(\mathbf{q}) [\mathbf{F}_e - \mathbf{a}_f] = \mathbf{0}$$

and may try to design the two new control inputs:

We can design feedback implying certain relation between $\mathbf{a}_q$ and $\mathbf{a}_f$ to reach a particular interplay (or accuracy trade-off):

tracking *vs* force assignment

This approach seems to be particularly attractive in the case when the constraints describing the task are reciprocal, i.e. when the directions important for accurate tracking and for accurate interaction force assignment are naturally distinguished. However, in this case we need to overcome the difficulty related to the fact that the task is described in the compliance frame and not in the base frame of the manipulator.

A.4.2 Task Space Dynamics and Impedance Control

Consider the vector of linear and angular velocities of the end-effector

$$\xi = \begin{bmatrix} v_e \\ \omega_e \end{bmatrix} = \begin{bmatrix} \dot{x} \\ \omega \end{bmatrix} = \begin{bmatrix} J_v(q) \\ J_\omega(q) \end{bmatrix} \dot{q} = J(q)\dot{q},$$

related to the generalized velocities via the manipulator Jacobian.

Compute its time derivative

$$\frac{d}{dt}\xi = J(q)\ddot{q} + \frac{d}{dt}[J(q)]\dot{q}$$

and assume, for simplicity, that orientation dynamics is of no interest and can be dropped

$$\frac{d^2}{dt^2}x = J_v(q)\ddot{q} + \frac{d}{dt}[J_v(q)]\dot{q}$$

Introduce the following control transformation

$$\mathbf{a}_x = J(q)\mathbf{a}_q + \frac{d}{dt}[J(q)]\dot{q}$$

Combining this new transformation with

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) + \mathbf{J}^T(q)\mathbf{F}_e = \mathbf{u},$$

$$\mathbf{u} = M(q)\mathbf{a}_q + C(q, \dot{q})\dot{q} + g(q) + \mathbf{J}^T(q)\mathbf{a}_f$$

we obtain

$$\frac{d^2}{dt^2}x = \mathbf{a}_x + \mathbf{W}(q)[\mathbf{F}_e - \mathbf{a}_f]$$

where

$$\mathbf{W}(q) = J(q)M^{-1}(q)J^T(q)$$

is called the mobility tensor.

Assuming that our task can be described as achieving certain impedance in the closed-loop system, a possible choice of the new control inputs is

$$\mathbf{a}_f = \mathbf{F}_e$$

to cancel the current interaction force, and

$$\mathbf{a}_x = \ddot{x}_d - M_d^{-1}(B_d(\dot{x} - \dot{x}_d(t)) + K_d(x - x_d(t)) + F_e))$$

to achieve certain impedance of the end-effector dynamics in the closed-loop system

$$M_d\ddot{e} + B_d\dot{e} + K_de = -F_e$$

where $e = x - x_d(t)$ is the task-space tracking error.

Note that using this approach zero tracking error can not be achieved simultaneously with a nonzero interaction force.

Under assumption of accuracy of impedance-based description of interaction with the environment, it is possible to use a more advanced design in order to achieve certain specifications of the closed-loop system impedance together with partial assignment of interaction forces simultaneously with accurate partial tracking under some additional assumptions. However, we will not go deeper into this subject.

Appendix B

Elements of Computer Vision

We present here a brief introduction into computer vision, i.e. into various issues that should be address to make a robotic system detect and identify various objects in the workspace using video cameras.

B.1 Digital Image

Information provided by a standard video camera is organized in the form of digital images.

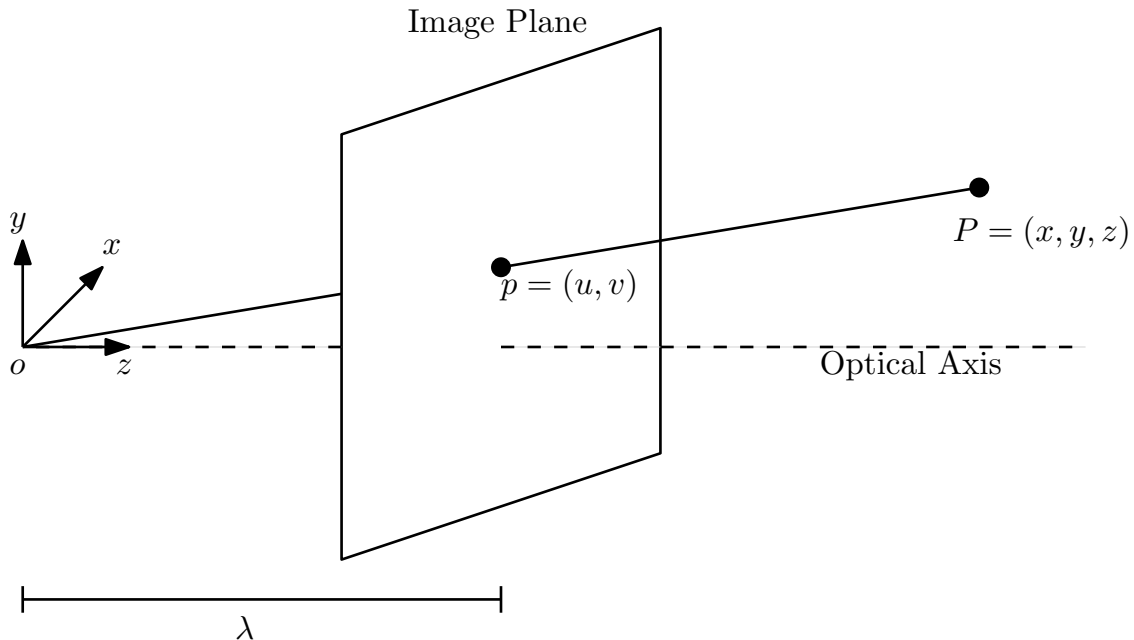
Digital image is a **two dimensional array**: $N_{rows} \times N_{cols}$ and the following remarks are in order.

- Elements of an image are called **pixels** (comes from “picture elements”).
- The image is formed by focusing light on a two-dimensional array of **sensing elements**.
- Each pixel has a numerical value that corresponds to the **intensity of light** incident on a particular sensing element.
- A lens is used to focus the light onto the sensing array, which is usually composed of CCD (charge-couple device) sensors.
- Frame grabber transfers the data from matrix of sensing elements to the pixel array (digital image).

In order to understand how the obtained information on the environment can be used, one needs to understand the basic principle of image formation.

B.1.1 Geometry of Image Formation

An image is always formed from a camera view relative to the coordinate frame attached to the camera. It is done via a projection onto a plane, called the **image plane**. The schematic is given below.



The following terminology is used.

- The origin of the coordinate frame is called the **center of projection**.
- The point, at which the optical axis intersects the image plane, is called the **principal point**.
- The distance from the center of projection to the image plane, λ , is called the **focal length**.
- Any point on the image plane has coordinates: (u, v, λ) in the camera frame and represents a point (x, y, z) by the light intensity.

Let us start our analysis deriving the relation between the two sets of coordinates (u, v, λ) and (x, y, z) .

B.1.2 Perspective Projection

Consider a point $P = (x, y, z)$ in the World (represented by coordinates in an inertial frame) and an induced by it point $p = (u, v, \lambda)$ in the image plane. Obviously, it is produced by a projection.

We see from the picture that the vectors $\vec{oP}$ and $\vec{op}$ from the origin o to the point P and to the point p , respectively, have the same directions.

Therefore, there exists $\mathbf{k}$ such that

$$\mathbf{k} \overrightarrow{oP} = \overrightarrow{op}$$

or, in coordinates,

$$\mathbf{k}x = u, \quad \mathbf{k}y = v, \quad \mathbf{k}z = \lambda$$

Eliminating the distance value (the depth of the point), one obtains the following

$$u = \frac{\lambda}{z} x, \quad \text{and} \quad v = \frac{\lambda}{z} y$$

Now, we know the relation between coordinates on the image plane (u, v) for a point with the coordinates (x, y, z) and would like to relate these values to the corresponding pixel of the digital image.

B.1.3 Coordinates on Image Plane vs Sensor Array

A digital image is a **two dimensional array** of pixels with an integer number of rows, N_{rows} , and an integer number of columns, N_{cols} .

Each pixel is defined by coordinates (r, c) with $r \in [1, N_{rows}]$ and $c \in [1, N_{cols}]$.

It is important to know the coordinates of the principal point (r_o, c_o) as well as the horizontal and vertical dimensions of a pixel, denoted below by s_x and s_y .

With such information at hand, the relation between the coordinates of the point $p = (u, v, \lambda)$ on the image plane and the indexes of the associated pixel in the array (r, c) can be represented as follows

$$-\frac{u}{s_x} \approx r - r_o, \quad -\frac{v}{s_y} \approx c - c_o$$

Here, the “ $-$ ” sign corresponds to the most frequently used direction of counting pixels.

The exact relations hold if we round the left-hand sides so that

$$r = r_o + \text{round} \left[-\frac{u}{s_x} \right], \quad c = c_o + \text{round} \left[-\frac{v}{s_y} \right]$$

Note that the frame grabber transfers the data from matrix of sensing elements to the pixel array in according to the formulae above.

However, one should realize by now that several parameters must be known in order to correctly interpret the information presented in a digital

image. Procedure of obtaining such information is called **camera celebration**; and we address it next.

B.2 Camera Calibration

Our ultimate goal is to relate the information recorded in a pixel $(\mathbf{r}, \mathbf{c})$ with the information observed at a point $(\mathbf{x}, \mathbf{y}, \mathbf{z})$. To make such relation unambiguous, one needs to calibrate the camera, i.e. find various parameters of a particular camera. The parameters are typically split into two groups. Let us describe them.

B.2.1 Extrinsic Camera Parameters

Let us remind our goal:

Given coordinates $(\mathbf{x}^w, \mathbf{y}^w, \mathbf{z}^w)$ of a point in the world frame, we need an algorithm for computing the coordinates $(\mathbf{r}, \mathbf{c})$ of the corresponding point on the digital image.

First of all, note that the camera frame and the world frame are typically different. In fact, the camera may even be attached to a moving link of the robot so that the camera frame orientation may continuously change with time. Independently on whether both frames are stationary or not, at each moment of time there is a homogeneous transform that relates them (see Chapter 2)

$$\mathbf{q}^w = \begin{bmatrix} x^w \\ y^w \\ z^w \end{bmatrix} = \mathbf{R}_c^w \begin{bmatrix} x^c \\ y^c \\ z^c \end{bmatrix} + \mathbf{c}_o^w = \mathbf{R}_c^w \mathbf{q}^c + \mathbf{c}_o^w$$

Here, coordinates of the point $\mathbf{q}$ are denoted by $\mathbf{q}^c$ in the camera frame and by $\mathbf{q}^w$ in the world frame.

Hence,

$$\mathbf{q}^c = [\mathbf{R}_c^w]^T \mathbf{q}^w - [\mathbf{R}_c^w]^T \mathbf{c}_o^w = \mathbf{R} \mathbf{q}^w + \mathbf{T}$$

In the most common situations, a camera is mounted on a tripod so that its orientation is conveniently defined by two parameters:

$$\mathbf{R} = \mathbf{R}_{z,\theta} \mathbf{R}_{x,\alpha}, \quad \text{where } \theta \text{ is the pan angle, } \alpha \text{ is the tilt angle.}$$

The components of the vector $\mathbf{T}$ together with the angles θ and α are called **extrinsic parameters** that must be estimated for a particular set-up.

B.2.2 Intrinsic Camera Parameters

With the values of the **extrinsic parameters** at hands, one can relate the coordinates (x^w, y^w, z^w) of a point in the world frame with the coordinates (x^c, y^c, z^c) of the point in the camera frame.

$$\begin{bmatrix} x^c \\ y^c \\ z^c \end{bmatrix} = R \begin{bmatrix} x^w \\ y^w \\ z^w \end{bmatrix} + T$$

We need now to find relation between (x^c, y^c, z^c) and (r, c) of the corresponding point of the **digital image**.

Earlier we have found the following relations

$$\begin{aligned} \text{round} \left[-\frac{u}{s_x} \right] &= r - o_x, & u &= \frac{\lambda}{z^c} x^c, \\ \text{round} \left[-\frac{v}{s_y} \right] &= c - o_y, & v &= \frac{\lambda}{z^c} y^c \end{aligned}$$

Combining them, we have

$$r \approx -\frac{\lambda}{s_x} \frac{x^c}{z^c} + r_o$$

and

$$c \approx -\frac{\lambda}{s_y} \frac{y^c}{z^c} + c_o$$

In other words,

$$r \approx -f_x \frac{x^c}{z^c} + r_o, \quad c \approx -f_y \frac{y^c}{z^c} + c_o$$

The values f_x and f_y are called the **intrinsic parameters**.

B.2.3 Steps in Camera Calibration

Finally, we know that we need to identify all the (extrinsic and intrinsic) parameters:

$$R, \quad T, \quad f_x, \quad r_o, \quad f_y, \quad c_o$$

This can be done as follows:

- Determine the position (r_o, c_o) of the principle point.
- Collect a data set $\{r_i, c_i, x_i^w, y_i^w, z_i^w\}_{i=1}^n$ from various measurements of test points with known locations in the world frame.

- Use the equations with respect to the unknown entries of $\mathbf{R}$ and $\mathbf{T}$

$$\begin{aligned}x^c &= r_{11}x^w + r_{12}y^w + r_{13}z^w + T_x \\y^c &= r_{21}x^w + r_{22}y^w + r_{23}z^w + T_y \\z^c &= r_{31}x^w + r_{32}y^w + r_{33}z^w + T_z\end{aligned}$$

to rewrite the equations

$$r - r_o = -f_x \frac{x^c}{z^c}, \quad c - c_o = -f_y \frac{y^c}{z^c}$$

as

$$z^c = -\frac{f_x}{r - r_o} x^c, \quad z^c = -\frac{f_y}{c - c_o} y^c$$

to obtain

$$[c - c_o] f_x x^c = [r - r_o] f_y y^c$$

that can be rewritten as

$$\begin{aligned}&[c - c_o] f_x (r_{11}x^w + r_{12}y^w + r_{13}z^w + T_x) \\&= [r - r_o] f_y (r_{21}x^w + r_{22}y^w + r_{23}z^w + T_y)\end{aligned}$$

- Use the relations

$$\begin{aligned}&[c - c_o] f_x (r_{11}x^w + r_{12}y^w + r_{13}z^w + T_x) \\&- [r - r_o] f_y (r_{21}x^w + r_{22}y^w + r_{23}z^w + T_y) = 0\end{aligned}$$

to find the parameters

$$r_{21}, \quad r_{22}, \quad r_{23}, \quad T_y, \quad \dots$$

searching for the best fit for the collected data set, i.e. make the following substitutions

$$r = r_i, \quad c = c_i, \quad x^w = x_i^w, \quad y^w = y_i^w, \quad z^w = z_i^w \quad i = 1, \dots, n$$

to obtain a system of equations for the parameters.

B.3 Image Processing: Segmentation

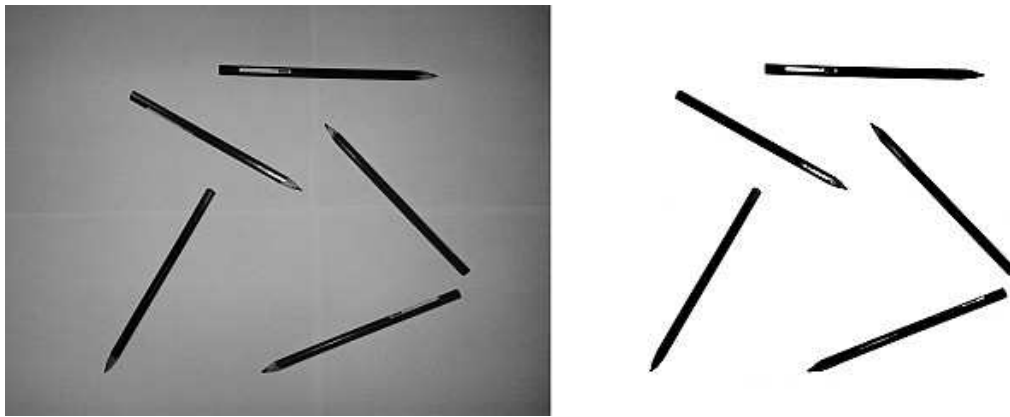
Each digital image from a calibrated camera should be processed to extract the information about various objects.

The first typical task is to identify groups of pixels that correspond to

the same object using the intensity information. This procedure is called segmentation.

B.3.1 Segmentation of Image

The basic idea is to separate objects from the background by choosing a trash-hold value and finding connected groups of pixels with intensity below it. A typical outcome of segmentation is identification of different parts of the image with different objects as illustrated below



B.3.2 Tools for Image Processing

Typically, for a given digital $N_{rows} \times N_{cols}$ image, each pixel is one of the following integers

$$z \in \{0, 1, 2, 3, \dots, (N - 1) \quad (= 2^8 - 1 = 255)\}$$

The image can be processed as follows.

- Compute the frequency of each value on the image

$$P(z) = \frac{H(z)}{N_{rows} \times N_{cols}}, \quad H(z) \text{ is the number of times } z \text{ occurs.}$$

- Compute the **mean value** for the whole image

$$\mu = \sum_{z=0}^{N-1} z P(z)$$

- Compute the **variance** for the whole image

$$\sigma^2 = \sum_{z=0}^{N-1} (z - \mu)^2 P(z)$$

- Find possible **segmentation of the image**.

To this end, note that *mean values of pixel values for the areas occupied by an object is different from the mean values of pixel values for the background*. So, we can do the following.

- Compute the frequency of each value not for the whole image, but for various sub-areas

$$P_i(z) = \frac{H_i(z)}{\sum_{z=0}^{N-1} H_i(z)}, \quad H_i(z) \text{ is the number of times } z \text{ occurs}$$

here $i = 0$ means pixels of (possible) background and $i = 1$ means pixels of a (possible) object.

- Compute **mean values** and **variances** for various areas

$$\mu_i = \sum_{z=0}^{N-1} z P_i(z) \quad \sigma_i^2 = \sum_{z=0}^{N-1} (z - \mu_i)^2 P_i(z)$$

and distinguish the areas on the image, e.g.

$$\mu_1 \neq \mu_0 \text{ (an object) or } \sigma_i \approx 0 \text{ (background)}.$$

Note that if we do not know segmentation of image into object and background areas, we can search for an **optimal** threshold value z_t

$$\mu_1 > z_t > \mu_0 \quad \text{or} \quad \mu_1 < z_t < \mu_0$$

such that the **variance** for each area

$$\sigma_i^2 = \sum_{z=0}^{N-1} (z - \mu_i)^2 P_i(z), \quad i = 0, 1$$

is as small as possible!

Alternatively, one can search for an optimal threshold z_t by minimizing a weighted sum of conditional variances

$$\Phi(z_t) = q_0(z_t)\sigma_0^2(z_t) + q_1(z_t)\sigma_1^2(z_t)$$

where weights are

$$q_0(\mathbf{z}_t) = \frac{\sum_{z=0}^{\mathbf{z}_t} H(z)}{N_{rows} \times N_{cols}} \quad q_1(\mathbf{z}_t) = \frac{\sum_{z=\mathbf{z}_t+1}^{N-1} H(z)}{N_{rows} \times N_{cols}}$$

The function $\Phi(\cdot)$ is known as the **within-group variance**.

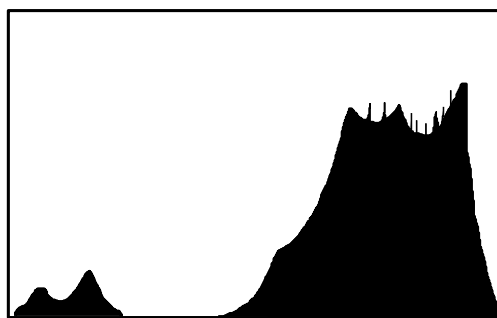
The result of the *segmentation of image* procedure described above is illustrated below



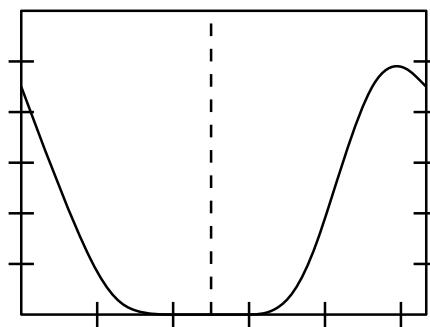
(a) the image



(b) the binary image



(c) the histogram



(d) within-group variance as function of threshold

B.4 Estimating Positions and Orientations

After an image is segmented, one can approach the problem of extracting the information on various properties of the objects.

As an example, suppose we are interested in estimating geometry of detected objects in terms of centers of masses (centroids) and orientations represented by points and lines in the next figure



Let a function $\mathcal{I}_i(\mathbf{r}, \mathbf{c})$ be an indicator of the object $\mathbf{i}$, i.e.

$$\mathcal{I}_i(\mathbf{r}, \mathbf{c}) = \begin{cases} 1 & : \text{pixel with coordinates } (\mathbf{r}, \mathbf{c}) \text{ belongs to an object } \mathbf{i} \\ 0 & : \text{otherwise} \end{cases}$$

which is obtained from segmentation.

Then, location of the **center of mass** of the object can be computed as follows

$$\bar{\mathbf{r}}_i = \frac{\sum_{\mathbf{r}, \mathbf{c}} \mathbf{r} \mathcal{I}_i(\mathbf{r}, \mathbf{c})}{\sum_{\mathbf{r}, \mathbf{c}} \mathcal{I}_i(\mathbf{r}, \mathbf{c})}, \quad \bar{\mathbf{c}}_i = \frac{\sum_{\mathbf{r}, \mathbf{c}} \mathbf{c} \mathcal{I}_i(\mathbf{r}, \mathbf{c})}{\sum_{\mathbf{r}, \mathbf{c}} \mathcal{I}_i(\mathbf{r}, \mathbf{c})}$$

On the other hand, the **orientation** of the object is defined by finding **a line**, along which the value of the function

$$\mathcal{L} = \sum_{\mathbf{r}, \mathbf{c}} d^2(\mathbf{r}, \mathbf{c}) \mathcal{I}_i(\mathbf{r}, \mathbf{c})$$

where $d(\mathbf{r}, \mathbf{c})$ is the distance from a pixel to the line, is **minimal**.

Appendix C

Elements of Vision Based Control

With a working computer vision system, it is possible to actively react on changes in the robot's environment. To use the information extracted from images taken by a camera, it is important to make an appropriate choice of camera configuration. Clearly, there are two reasonable choices for possible installation of a camera; it could be

- Fixed to some predefined position in the world frame;
- Attached to a robot at a particular place; this is called the “eye-in-hand” configuration.

In both situations, there are different ways to use digital images for feedback control.

C.1 Position *vs* Imaged-Based Control

There are two basic approaches for vision-based control:

- The first one is **Position-Based** and means that the system
 - Reconstruct **3-D** representation of the world from an image;
 - Control a robot to reach a desired configuration of the robot based on this **3-D** reconstruction with feedback based on the configuration error.
- The other one is **Image-Based** and means that the system
 - Does NOT reconstruct a **3-D** estimate of the world;
 - Control the robot based on the image data with feedback based on the mismatch with a desired configuration but as it would be originally described in terms of quantities on the image plane.

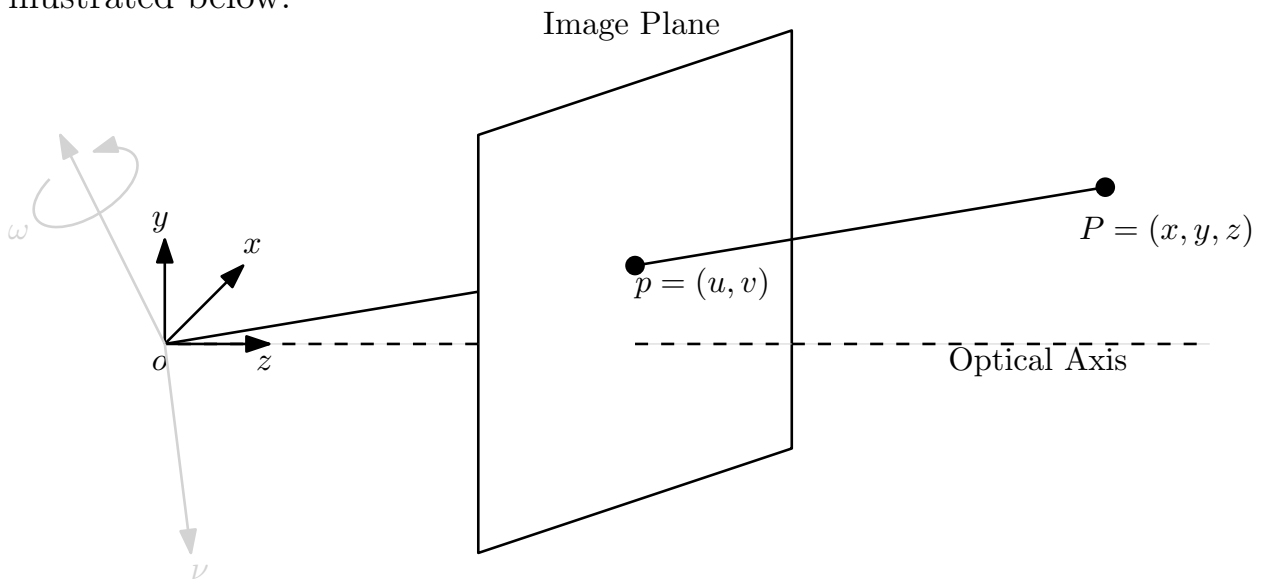
C.2 Camera Motion and Interaction Matrix

Let us consider the “eye-in-hand” configuration and position-based approach.

C.2.1 Camera Motion Description

To organize an appropriate feedback action, one needs to describe the changes of locations of various points of an image that are associated with a motion of the camera.

Suppose we know from computation of velocity kinematics for the camera frame that $\vec{v}$ is the velocity of its origin while $\vec{\omega}$ is its angular velocity as illustrated below.



Let $\mathbf{s}(t)$ be a point feature on the image with coordinates

$$\mathbf{s}(t) = [s_{x^c}(t), s_{y^c}(t), s_{z^c}(t)] = [u(t), v(t), \lambda]$$

that is associated with a fixed point of an object of interest $\mathbf{p}^0$ with coordinates $(\mathbf{x}, \mathbf{y}, \mathbf{z})$ in a fixed stationary frame.

We can directly estimate $\mathbf{s}(t)$ using certain image processing technique and can estimate its rate of change $\frac{d}{dt}\mathbf{s}(t)$ from a time series of digital images. This should allow us to describe the motion of the camera.

C.2.2 Definition of the Interaction Matrix

The question is whether based on $\mathbf{s}(t)$ and $\frac{d}{dt}\mathbf{s}(t)$ we can compute the velocity $\vec{v}(t)$ of the origin of the camera frame and its angular velocity $\vec{\omega}(t)$ in a particular frame?

In other words, the relation between the quantities $[\mathbf{s}(t), \frac{d}{dt}\mathbf{s}(t)]$ and $[\vec{v}(t), \vec{\omega}(t)]$ is of interest.

It is reasonable to guess that such a relation is linear

$$\frac{d}{dt}\mathbf{s}(t) = \mathbf{L}(\cdot) \begin{bmatrix} \mathbf{v}(t) \\ \boldsymbol{\omega}(t) \end{bmatrix}$$

The operator $\mathbf{L}(\cdot)$ is called the **interaction matrix**. To compute this matrix, one can proceed as follows:

- Relate coordinates of a fixed point $\mathbf{p}_c(t)$ and $\frac{d}{dt}\mathbf{p}_c(t)$ with the camera frame velocities;
- Relate $\mathbf{s}(t)$ and $\frac{d}{dt}\mathbf{s}(t)$ with $\mathbf{p}_c(t)$ and $\frac{d}{dt}\mathbf{p}_c(t)$;
- Combine these two relations.

C.2.3 Velocities of the Camera Frame

Note that when a point $\mathbf{p}$ with coordinates $\mathbf{p}^0$ is fixed in the inertial frame, but the camera moves:

- In the camera frame, the coordinates of $\mathbf{p}$ are changing with time: $\mathbf{p}^c = \mathbf{p}^c(t)$.
- The coordinates of $\mathbf{p}$ in the inertia and camera frames are related by a homogeneous transformation (see Chapter 2)

$$\mathbf{p}^0 = \mathbf{R}_c^0(t)\mathbf{p}^c(t) + \mathbf{o}^0(t)$$

Hence,

$$\mathbf{p}^c(t) = [\mathbf{R}_c^0(t)]^T (\mathbf{p}^0 - \mathbf{o}^0(t))$$

Differentiating this equation we obtain velocity of the point $\mathbf{p}$ written in the camera frame

$$\frac{d}{dt}\mathbf{p}^c(t) = \frac{d}{dt}([\mathbf{R}_c^0(t)]^T) (\mathbf{p}^0 - \mathbf{o}^0(t)) + [\mathbf{R}_c^0(t)]^T \frac{d}{dt}(\mathbf{p}^0 - \mathbf{o}^0(t))$$

Using the Poisson equation $\frac{d}{dt}(\mathbf{R}_c^0(t)) = \mathbf{S}(\boldsymbol{\omega}^0) \mathbf{R}_c^0(t)$ we obtain

$$\frac{d}{dt}([\mathbf{R}_c^0(t)]^T) = [\mathbf{R}_c^0(t)]^T \mathbf{S}(\boldsymbol{\omega}^0)^T = -[\mathbf{R}_c^0(t)]^T \mathbf{S}(\boldsymbol{\omega}^0)$$

and therefore

$$\begin{aligned} \frac{d}{dt}\mathbf{p}^c(t) &= -[\mathbf{R}_c^0(t)]^T \mathbf{S}(\boldsymbol{\omega}^0) (\mathbf{p}^0 - \mathbf{o}^0(t)) - [\mathbf{R}_c^0(t)]^T \dot{\mathbf{o}}^0(t) \\ &= -[\mathbf{R}_c^0(t)]^T \mathbf{S}(\boldsymbol{\omega}^0) \mathbf{R}_c^0(t) [\mathbf{R}_c^0(t)]^T (\mathbf{p}^0 - \mathbf{o}^0(t)) - [\mathbf{R}_c^0(t)]^T \dot{\mathbf{o}}^0(t) \end{aligned}$$

Now, using the standard change of coordinates identity

$$[R_c^0]^T S(\omega^0) R_c^0 = S\left([R_c^0]^T \omega^0\right) = \left([R_c^0]^T \omega^0\right) \times I$$

we obtain

$$\begin{aligned} \frac{d}{dt} p^c(t) &= - \underbrace{[R_c^0(t)]^T \omega^0}_{\omega^c} \times \underbrace{[R_c^0(t)]^T (p^0 - o^0(t))}_{p^c(t)} - \underbrace{[R_c^0(t)]^T \dot{o}^0(t)}_{\frac{d}{dt} o^c(t) = v^c} \\ &= -\omega^c \times p^c(t) - v^c \end{aligned}$$

Example

For illustrative purposes suppose for example that

- The optical axis z^c of a camera is parallel and opposite to the world z -axis;
- The camera motions are constrained to
 - rotation about the optical axis;
 - translation parallel to the image (x, y) plane.

It follows that

$$p^0 = \begin{bmatrix} \cos \theta(t) & -\sin \theta(t) & 0 \\ \sin \theta(t) & \cos \theta(t) & 0 \\ 0 & 0 & 1 \end{bmatrix} p^c(t) + \begin{bmatrix} x_{co}^0(t) \\ y_{co}^0(t) \\ z_{co}^0 \end{bmatrix}$$

and

$$\dot{p}^c = -\omega^c \times p^c - \dot{o}^c, \quad \omega^c = \begin{bmatrix} 0 \\ 0 \\ \dot{\theta} \end{bmatrix}, \quad \dot{o}^c = v^c = \begin{bmatrix} v_x^c \\ v_y^c \\ 0 \end{bmatrix}$$

C.2.4 Constructing the Interaction Matrix

The relation

$$\begin{aligned} \begin{bmatrix} \dot{x}^c \\ \dot{y}^c \\ \dot{z}^c \end{bmatrix} &= \dot{p}^c = -\omega^c \times p^c - v^c = - \begin{bmatrix} \omega_x^c \\ \omega_y^c \\ \omega_z^c \end{bmatrix} \times \begin{bmatrix} x^c \\ y^c \\ z^c \end{bmatrix} - \begin{bmatrix} v_x^c \\ v_y^c \\ v_z^c \end{bmatrix} \\ &= - \begin{bmatrix} \omega_y^c z_c - \omega_z^c y_c + v_x^c \\ \omega_z^c z_c - \omega_x^c z_c + v_y^c \\ \omega_x^c y_c - \omega_y^c z_c + v_z^c \end{bmatrix} \end{aligned}$$

combined with the projection equations

$$\mathbf{u} = \mathbf{k} x^c, \quad \mathbf{v} = \mathbf{k} y^c, \quad \lambda = \mathbf{k} z^c,$$

allow us to compute the operator $L(\cdot)$ in

$$\begin{bmatrix} \dot{\mathbf{u}}(t) \\ \dot{\mathbf{v}}(t) \end{bmatrix} = L(\cdot) \begin{bmatrix} \mathbf{v}(t) \\ \boldsymbol{\omega}(t) \end{bmatrix}$$

Indeed,

$$\begin{aligned} \dot{\mathbf{u}} &= \frac{d}{dt} \left[\frac{\lambda x^c}{z^c} \right] = \lambda \frac{z^c \dot{x}^c - x^c \dot{z}^c}{(z^c)^2} \\ &= -\frac{\lambda}{z^c} \mathbf{v}_x^c + \frac{\mathbf{u}}{z^c} \mathbf{v}_z^c + \frac{\mathbf{u} \mathbf{v}}{\lambda} \boldsymbol{\omega}_x^c - \frac{\lambda^2 + \mathbf{u}^2}{\lambda} \boldsymbol{\omega}_y^c + \mathbf{v} \boldsymbol{\omega}_z^c \end{aligned}$$

and

$$\dot{\mathbf{v}} = \frac{d}{dt} \left[\frac{\lambda y^c}{z^c} \right] = -\frac{\lambda}{z^c} \mathbf{v}_y^c + \frac{\mathbf{v}}{z^c} \mathbf{v}_z^c - \frac{\mathbf{u} \mathbf{v}}{\lambda} \boldsymbol{\omega}_y^c + \frac{\lambda^2 + \mathbf{v}^2}{\lambda} \boldsymbol{\omega}_x^c - \mathbf{u} \boldsymbol{\omega}_z^c$$

Combining these two equations we have

$$\begin{bmatrix} \dot{\mathbf{u}} \\ \dot{\mathbf{v}} \end{bmatrix} = \begin{bmatrix} -\frac{\lambda}{z^c} & 0 & \frac{\mathbf{u}}{z^c} & \frac{\mathbf{u} \mathbf{v}}{\lambda} & -\frac{\lambda^2 + \mathbf{u}^2}{\lambda} & \mathbf{v} \\ 0 & -\frac{\lambda}{z^c} & \frac{\mathbf{v}}{z^c} & \frac{\lambda^2 + \mathbf{v}^2}{\lambda} & -\frac{\mathbf{u} \mathbf{v}}{\lambda} & -\mathbf{u} \end{bmatrix} \begin{bmatrix} \mathbf{v}_x^c \\ \mathbf{v}_y^c \\ \mathbf{v}_z^c \\ \boldsymbol{\omega}_x^c \\ \boldsymbol{\omega}_y^c \\ \boldsymbol{\omega}_z^c \end{bmatrix}$$

Clearly, the interaction matrix L depends on $\mathbf{u}$, $\mathbf{v}$, and z^c , which is the distance to the point feature along the optical axis. The equation can be rewritten as

$$\begin{bmatrix} \dot{\mathbf{u}} \\ \dot{\mathbf{v}} \end{bmatrix} = L(\mathbf{u}, \mathbf{v}, z^c) \begin{bmatrix} \mathbf{v}^c \\ \boldsymbol{\omega}^c \end{bmatrix} = \frac{1}{z^c} L_v(\mathbf{u}, \mathbf{v}) \mathbf{v}^c + L_\omega(\mathbf{u}, \mathbf{v}) \boldsymbol{\omega}^c$$

to separate the dependence on z^c .

C.2.5 Properties of the Interaction Matrix

It is not hard to show that the equation

$$\begin{bmatrix} \dot{\mathbf{u}} \\ \dot{\mathbf{v}} \end{bmatrix} = L(\mathbf{u}, \mathbf{v}, z^c) \begin{bmatrix} \mathbf{v}(t) \\ \boldsymbol{\omega}(t) \end{bmatrix}$$

has a non-trivial kernel that is

$$\exists \xi \in \mathbb{R}^6 : \quad L(u, v, z^c) \xi = 0, \quad \xi \neq 0$$

Moreover, the basis of the kernel subspace is

$$\begin{bmatrix} u \\ v \\ \lambda \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} 0 \\ 0 \\ 0 \\ u \\ v \\ \lambda \end{bmatrix}, \quad \begin{bmatrix} uvz^c \\ -(u^2 + \lambda^2)z^c \\ \lambda vz^c \\ -\lambda^2 \\ 0 \\ u\lambda \end{bmatrix}, \quad \begin{bmatrix} \lambda(u^2 + v^2 + \lambda^2)z^c \\ 0 \\ -u(u^2 + v^2 + \lambda^2)z^c \\ uv\lambda \\ -(u^2 + \lambda^2)z^c \\ u\lambda^2 \end{bmatrix}$$

C.3 Reconstructing a Camera Motion

Let us assume that we have processed coordinates of n different feature points $P_1, \dots, P_n$.

Combining the interaction matrix based relations for them we have

$$\begin{bmatrix} \dot{u}_1 \\ \dot{v}_1 \end{bmatrix} = \begin{bmatrix} -\frac{\lambda}{z_1^c} & 0 & \frac{u_1}{z_1^c} & \frac{u_1 v_1}{\lambda} & -\frac{\lambda^2 + u_1^2}{\lambda} & v_1 \\ 0 & -\frac{\lambda}{z_1^c} & \frac{v_1}{z_1^c} & \frac{\lambda^2 + v_1^2}{\lambda} & -\frac{u_1 v_1}{\lambda} & -u_1 \end{bmatrix} \begin{bmatrix} v_x^c \\ v_y^c \\ v_z^c \\ \omega_x^c \\ \omega_y^c \\ \omega_z^c \end{bmatrix}$$

$$\vdots$$

$$\begin{bmatrix} \dot{u}_n \\ \dot{v}_n \end{bmatrix} = \begin{bmatrix} -\frac{\lambda}{z_n^c} & 0 & \frac{u_n}{z_n^c} & \frac{u_n v_n}{\lambda} & -\frac{\lambda^2 + u_n^2}{\lambda} & v_n \\ 0 & -\frac{\lambda}{z_n^c} & \frac{v_n}{z_n^c} & \frac{\lambda^2 + v_n^2}{\lambda} & -\frac{u_n v_n}{\lambda} & -u_n \end{bmatrix} \begin{bmatrix} v_x^c \\ v_y^c \\ v_z^c \\ \omega_x^c \\ \omega_y^c \\ \omega_z^c \end{bmatrix}$$

Clearly these combined equations for n -feature points

$$\begin{bmatrix} \dot{u}_1 \\ \dot{v}_1 \\ \vdots \\ \dot{u}_n \\ \dot{v}_n \end{bmatrix} = \begin{bmatrix} L(u_1, v_1, z_1^c) \\ \vdots \\ L(u_n, v_n, z_n^c) \end{bmatrix} \begin{bmatrix} v_x^c \\ v_y^c \\ v_z^c \\ \omega_x^c \\ \omega_y^c \\ \omega_z^c \end{bmatrix}$$

can be used for computation of the velocities v^c and ω^c !

For instance, if there are **3**-feature points, then

$$\begin{bmatrix} v_x^c \\ v_y^c \\ v_z^c \\ \omega_x^c \\ \omega_y^c \\ \omega_z^c \end{bmatrix} = \begin{bmatrix} L(u_1, v_1, z_1^c) \\ L(u_2, v_2, z_2^c) \\ L(u_3, v_3, z_3^c) \end{bmatrix}^{-1} \begin{bmatrix} \dot{u}_1 \\ \dot{v}_1 \\ \dot{u}_2 \\ \dot{v}_2 \\ \dot{u}_3 \\ \dot{v}_3 \end{bmatrix}$$

C.4 An Idea for Image-Based Control Design

Let us consider now the **Image-Based Control** design.

In this settings,

- the target configuration is not given in terms of the **3D World**;
- it is specified in terms of a location of the image features s^d on the image plane.

Suppose we can extract the current status of the feature $s(t)$ and compute the image error

$$e(t) = s(t) - s^d$$

A possible idea for feedback control design based on the value of the error $e(t)$ is the following. Let us produce a control signal to decrease the mismatch.

Ignoring dynamics, let us assume for simplicity that

- the control variables are the vectors of linear velocity $\vec{v}$ of the origin of the camera frame and
- its angular velocity $\vec{\omega}$.

We know that

$$\frac{d}{dt}e(t) = \frac{d}{dt}[s(t) - s^d] = \frac{d}{dt}s(t) = L(\cdot) \begin{bmatrix} v(t) \\ \omega(t) \end{bmatrix}$$

Hence, for example, the control variables $\vec{v}(t)$, $\vec{\omega}(t)$ can be chosen to ensure

$$L(\cdot) \begin{bmatrix} v(t) \\ \omega(t) \end{bmatrix} := -\gamma e(t), \quad \gamma > 0$$

so that the dynamics of the error variable becomes

$$\frac{d}{dt}e(t) = -\gamma e(t)$$

and exponentially stable tracking is achieved.

Index

- analytic Jacobian, 82
- angle-axis representation, 41
- angular velocity, 66, 71
- basic rotation, 31
- closed kinematic chain, 134
- constraint force, 108, 110, 132, 134
- coordinates and frames, 24
- cross product, 66
- DH convention, 50
- DH parameters, 52, 55
- Euler angles, 42
- Euler-Lagrange equations of first kind, 133
- Euler-Lagrange equations of second kind, 98, 115, 121, 122
- external force, 112, 114, 121
- forward kinematics, 47
- generalized coordinates, 105, 132
- generalized force, 111, 112
- holonomic constraints, 102, 104, 110, 111, 132, 133
- homogeneous transformation, 46, 47
- inertial frame, 49
- inverse kinematics, 58, 84
- joint, 47
- joint variable, 49
- kinematic chain, 47
- kinematic decoupling, 59
- kinetic energy, 98, 113, 119
- Lagrangian, 98, 114
- link, 47
- link angle, 52
- link length, 52
- link offset, 52
- link twist, 52
- manipulator Jacobian, 75, 121
- non-holonomic constraints, 105
- number of degrees of freedom, 102, 105
- planar bar, 106, 111, 114
- point, 26
- potential energy, 98, 114, 120
- prismatic joint, 48
- relative angular velocity, 72, 73
- revolute joint, 48
- rigid motion, 45, 49
- roll, pitch, yaw, 44
- rotation matrix, 28, 30
- singular configuration, 82
- spin matrix, 69, 70
- two-link planar articulated robot, 57, 80, 126, 129, 144
- vector, 26
- virtual displacements, 106, 110
- virtual generalized displacements, 110
- virtual work, 108